

Studentu informacine sistema
v3.0

Sugeneruota Doxygen 1.17.0

skyrius 1

v3.0 — Nuosavas Vector konteineris

Galutinė laboratorinių darbų versija · VU ISI Objektinio programavimo kursas

1.1 Santrauka

Sukurtas nuosavas `Vector` konteineris, realizuojantis ~99% `std::vector` interfeiso, ir juo pakeistas standartinis konteineris visoje programoje. Atlikta spartos analizė, parašyti unit testai, sugeneruota dokumentacija ir paruoštas diegimo failas.

1.2 Nauja šioje versijoje

1.2.1 Vector konteineris

Naujas failas `OOP_3/Vector.h` — apie 80 metodų.

Kategorija	Realizuota	<code>std::vector</code> turi
Member tipai	12	12
Konstruktoriai / destruktorius	8	8
Priskyrimas	6	6
Elementų prieiga	10	10
Iteratoriai	12	12
Talpa	6	6
Modifikavimas	17	18
Ne-nariai operatoriai	9	9
Iš viso	80	81 (~99%)

Nerealizuota tik `Vector<bool>` specializacija — sąmoningai, nes ji laikoma standarto klaida.

1.2.2 Programa perjungta į Vector

Pridėtas `konteineris.h` perjungiklis — programą galima perkompiliuoti su bet kuriuo konteineriu:

```
cmake .. -DNAUDOTI_STD_VECTOR=ON      # std::vector
cmake .. -DNAUDOTI_STD_VECTOR=OFF     # nuosava Vector (numatytoji)
```

Taip spartos palyginimas sąžiningas — abi versijos naudoja tą patį kodą.

1.2.3 Diegimo failas

setup.exe (CPack + NSIS):

Veiksmas	Statusas
Diegia į C:\Program Files\VU\Marius-Augustinas	
Nuoroda darbalaukyje	
Nuoroda Start meniu (VU → Marius-Augustinas)	
Prašo administratoriaus teisių	
Ištraukia testinius failus (10k, 100k)	
Pašalinimas per Windows programų sąrašą	

1.3 Tyrimų rezultatai

1.3.1 push_back sparta

Elementų	std::vector	Vector	Santykis
10 000	0.000141 s	0.000125 s	0.89×
100 000	0.000954 s	0.000691 s	0.72×
1 000 000	0.017269 s	0.003643 s	0.21×
10 000 000	0.089110 s	0.047492 s	0.53×
100 000 000	0.727454 s	0.315388 s	0.43×

Su gryniais `int` elementais nuosava realizacija 2–5× greitesnė — dėl mažesnio perskirstymų skaičiaus.

1.3.2 Atminties perskirstymai

Elementų	std::vector	Vector
10 000	24	15
1 000 000	35	21
100 000 000	47	28

`Vector` naudoja 2× augimo koeficientą, MSVC `std::vector` — 1.5×. Todėl perskirstymų ~1.7 karto mažiau.

1.3.3 Programos sparta su realiais duomenimis

Studentų	std::vector	Vector	Skirtumas
100 000	0.500 s	0.530 s	+6%
1 000 000	5.247 s	6.261 s	+19%
10 000 000	90.624 s	123.427 s	+36%

Su sudėtingais objektais (`studentas` turi du `std::string` ir `Vector<int>`) nuosava realizacija atsilieka. Priežastis — MSVC `std::vector` turi optimizacijų, kurių paprastesnė realizacija neturi.

Rikiavimas ir skirstymas praktiškai lygūs, nes juos atlieka STL algoritmai per iteratorius, o `Vector` iteratoriai yra paprastos rodyklės.

1.4 Unit testai

`tests/vectorTests.cpp` — 70 sekcijų, suskirstytų į 9 grupes. Kiekviena operacija atliekama su `std::vector` ir `Vector`, rezultatai lyginami.

Žymė	Sekcijų
↵ [konstruktoriai]↵	8
↵ [priskyrimas]↵	6
↵ [prieiga]↵	5
↵ [iteratoriai]↵	5
↵ [talpa]↵	8
↵ [modifikavimas]↵	24
↵ [operatoriai]↵	5
↵ [tipai]↵	3
↵ [krastiniai]↵	6

Testuose naudojama `Sekiklis` klasė, skaičiuojanti konstruktorių, kopijų ir perkėlimų kvietimus — taip patikrinama, ar perskirstymas tikrai naudoja `move`, ne `copy`.

```
./vectorTests          # visi testai
./vectorTests "[talpa]" # tik talpos testai
```

1.5 Dokumentacija

Visa `Vector` klasė pakomentuota Doxygen žymomis.

Formatas	Vieta
HTML	docs/doxygen/html/index.html
LaTeX	docs/doxygen/↵ latex/
PDF	docs/doxygen/refman.pdf

1.6 Pakeisti failai

Failas	Pakeitimas
OOP_3/Vector.h	Naujas — nuosavas konteineris
OOP_3/konteineris.h	Naujas — perjungiklis
tests/vectorTests.cpp	Naujas — Vector testai
tests/benchmark.cpp	Naujas — spartos analizė
OOP_3/studentas.h	<code>std::vector<int> nd_ → Vector<int> nd_</code>
OOP_3/OOP_3.cpp	Naudoja Konteineris < studentas >
CMakeLists.txt	+ benchmark, vectorTests, CPack

1.7 Diegimas

1.7.1 Iš setup.exe

Atsisiųsti prisegtą `setup.exe` → paleisti → programa įdiegiama į `C:\Program Files\VU\Marius-Augustinas` su nuorodomis darbalaukyje ir Start meniu.

Pašalinimas: **Settings** → **Apps** → **Studentu informacine sistema** → **Uninstall**.

1.7.2 Iš šaltinio

```
mkdir build && cd build
```

```
cmake ..
```

```
cmake --build . --config Release
```

Unit testams reikia `tests/catch.hpp`:

```
curl -L -o tests/catch.hpp \
  https://github.com/catchorg/Catch2/releases/download/v2.13.10/catch.hpp
```

1.8 Prisegti failai

Failas	Aprašymas
<code>setup.exe</code>	Diegimo failas su testiniais duomenimis (10k, 100k)
<code>refman.pdf</code>	Pilna API dokumentacija

1.9 Žinomos pastabos

- [Vector](#)<bool> nespecializuotas — veikia kaip įprastas `bool` masyvas
- Su 10 mln. studentų matavimai svyravo iki 18 s tarp paleidimų (disko kešas, sistemos apkrova) — pateikti vidurkiai iš dviejų bandymų
- 100 mln. elementų benchmark'as naudoja ~400 MB vienam konteineriui
- MSVC neturi `/O3` — maksimalus optimizavimo lygis yra `/O2`

Išsamus aprašymas su funkcijų pavyzdžiais, lentelėmis ir ekrano kopijomis — [README.md](#).

skyrius 2

Direktorių hierarchija

2.1 Direktorijos

OOP_3	??
calculate.cpp	??
calculate.h	??
enter.cpp	??
enter.h	??
file.cpp	??
file.h	??
generate.cpp	??
generate.h	??
konteineris.h	??
main.h	??
menu.cpp	??
menu.h	??
OOP_3.cpp	??
output.h	??
print.cpp	??
print.h	??
studentas.cpp	??
studentas.h	??
test.cpp	??
test.h	??
teststudentas.cpp	??
teststudentas.h	??
Vector.h	??
zmogus.cpp	??
zmogus.h	??
tests	??
benchmark.cpp	??
unitTests.cpp	??
Vectortests.cpp	??

skyrius 3

Hierarchijos Indeksas

3.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

splitResult< Container >	??
Vector< T, Allocator >	??
zmogus	??
studentas	??

skyrius 4

Klasės Indeksas

4.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

splitResult< Container >	??
studentas	
Studento duomenys — išvestinė klasė iš abstrakčios zmogus	??
Vector< T, Allocator >	
Dinaminis masyvas — pilnavertė std::vector alternatyva	??
zmogus	
ABSTRAKTI bazinė klasė — bendras bet kokio žmogaus aprašymas	??

skyrius 5

Failo Indeksas

5.1 Failai

Visų failų sąrašas su trumpais aprašymais:

OOP_3/	calculate.cpp	??
OOP_3/	calculate.h	??
OOP_3/	enter.cpp	??
OOP_3/	enter.h	??
OOP_3/	file.cpp	??
OOP_3/	file.h	??
OOP_3/	generate.cpp	??
OOP_3/	generate.h	??
OOP_3/	konteineris.h	
	Konteinerio tipo perjungimas tarp std::vector ir nuosavos Vector	??
OOP_3/	main.h	??
OOP_3/	menu.cpp	??
OOP_3/	menu.h	??
OOP_3/	OOP_3.cpp	??
OOP_3/	output.h	??
OOP_3/	print.cpp	??
OOP_3/	print.h	??
OOP_3/	studentas.cpp	??
OOP_3/	studentas.h	
	Išvestinė klasė studentas — studento duomenys ir galutinių balų skaičiavimas	??
OOP_3/	test.cpp	??
OOP_3/	test.h	??
OOP_3/	teststudentas.cpp	??
OOP_3/	teststudentas.h	??
OOP_3/	Vector.h	
	Nuosava std::vector alternatyva — dinaminis masyvas	??
OOP_3/	zmogus.cpp	??
OOP_3/	zmogus.h	
	Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis	??
tests/	benchmark.cpp	
	Spartos analizė: std::vector vs nuosava Vector klasė	??
tests/	unitTests.cpp	
	Unit testai klasėms zmogus ir studentas (Catch2 framework)	??
tests/	Vectortests.cpp	
	Unit testai nuosavai Vector klasei (Catch2)	??

skyrius 6

Direktorių dokumentacija

6.1 OOP_3 Direktorijos aprašas

Failai

- failas [calculate.cpp](#)
- failas [calculate.h](#)
- failas [enter.cpp](#)
- failas [enter.h](#)
- failas [file.cpp](#)
- failas [file.h](#)
- failas [generate.cpp](#)
- failas [generate.h](#)
- failas [konteineris.h](#)

Konteinerio tipo perjungimas tarp `std::vector` ir nuosavos [Vector](#).

- failas [main.h](#)
- failas [menu.cpp](#)
- failas [menu.h](#)
- failas [OOP_3.cpp](#)
- failas [output.h](#)
- failas [print.cpp](#)
- failas [print.h](#)
- failas [studentas.cpp](#)
- failas [studentas.h](#)

Išvestinė klasė `studentas` — studento duomenys ir galutinių balų skaičiavimas.

- failas [test.cpp](#)
- failas [test.h](#)
- failas [teststudentas.cpp](#)
- failas [teststudentas.h](#)
- failas [Vector.h](#)

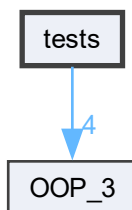
Nuosava `std::vector` alternatyva — dinaminis masyvas.

- failas [zmogus.cpp](#)
- failas [zmogus.h](#)

Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis.

6.2 tests Direktorijos aprašas

Directory dependency graph for tests:



Failai

- failas [benchmark.cpp](#)
Spartos analizė: std::vector vs nuosava [Vector](#) klasė.
- failas [unitTests.cpp](#)
Unit testai klasėms zmogus ir studentas (Catch2 framework).
- failas [Vectortests.cpp](#)
Unit testai nuosavai [Vector](#) klasei (Catch2).

skyrius 7

Klasės Dokumentacija

7.1 splitResult< Container > Struktūra Šablonas

```
#include <main.h>
```

Bendradarbiavimo diagrama splitResult< Container >:

splitResult< Container >	
+	kietiakai
+	vargsiukai

Vieši Atributai

- Container [kietiakai](#)
- Container [vargsiukai](#)

7.1.1 Smulkus aprašymas

```
template<typename Container>
```

```
struct splitResult< Container >
```

Apibrėžimas failo [main.h](#) eilutėje [25](#).

7.1.2 Atributų Dokumentacija

7.1.2.1 kietiakai

```
template<typename Container>
```

```
Container splitResult< Container >::kietiakai
```

Apibrėžimas failo [main.h](#) eilutėje [27](#).

Naudojamas [splitStudentai\(\)](#), ir [testData\(\)](#).

7.1.2.2 vargsiukai

```
template<typename Container>
```

```
Container splitResult< Container >::vargsiukai
```

Apibrėžimas failo [main.h](#) eilutėje 28.

Naudojamas [splitStudentai\(\)](#), ir [testData\(\)](#).

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- OOP_3/[main.h](#)

7.2 studentas Klasė

Studento duomenys — išvestinė klasė iš abstrakčios `zmogus`.

```
#include <studentas.h>
```

Paveldimumo diagrama studentas:



Bendradarbiavimo diagrama studentas:



Vieši Metodai

Konstruktoriai

- `studentas ()`
Numatytasis konstruktorius.
- `studentas (const std::string &vardas, const std::string &pavarde, const Vector< int > &nd, int egzaminas)`
Pilnas konstruktorius su validacija.
- `studentas (std::istream &is)`

Konstruktorius iš įvesties srauto.

Rule of Five

Visos penkios funkcijos realizuotos pilnai (*ne = default*).

Kiekviena **privalo** iškviešti atitinkamą bazinės klasės funkciją — be to `vardas_` ir `pavarde_` liktų tušti arba nenukopijuoti. Tai dažniausia paveldėjimo klaida, kurios kompiliatorius neaptinka.

- `~studentas ()` override
Destruktorius.
- `studentas (const studentas &other)`
Kopijavimo konstruktorius.
- `studentas & operator= (const studentas &other)`
Kopijavimo priskyrimo operatorius.
- `studentas (studentas &&other) noexcept`
Perkėlimo (move) konstruktorius.
- `studentas & operator= (studentas &&other) noexcept`
Perkėlimo (move) priskyrimo operatorius.

Grynai virtualių metodų realizacija

- `double galutinis () const` override
Grąžina galutinį įvertinimą (bazinės klasės interfeisas).
- `void print (std::ostream &os) const` override
Išveda studentą į srautą.
- `std::istream & read (std::istream &is) override`
Nuskaityto studentą iš srauto.
- `std::string tipas () const` override
Grąžina tipo pavadinimą.

Get'eriai

- `const Vector< int > & nd () const`
Grąžina namų darbų pažymius.
- `int egzaminas () const`
Grąžina egzamino pažymį.
- `double galutinisVid () const`
Grąžina galutinį balą pagal namų darbų vidurkį.
- `double galutinisMed () const`
Grąžina galutinį balą pagal namų darbų medianą.
- `bool tuscias () const`
Patikrina, ar objektas neturi vardo ir pavardės.

Set'eriai

- `void setNd (const Vector< int > &nd)`
Nustato namų darbų pažymius su validacija.
- `void setEgzaminas (int e)`
Nustato egzamino pažymį su validacija.
- `void addPazymys (int p)`
Prideda vieną namų darbo pažymį.
- `void isvalyk ()`
Ištuština objektą.

Skaiciavimas

- `void calculateGalutinis ()`
Apskaičiuoja abu galutinius balus.

Įvestis

- `std::istream & readStudentas (std::istream &is, int ndCount=0, int lineNumber=0)`
Nuskaityti studentą iš srauto.
- `void parseFromLine (const std::string &line, int ndCount, int lineNumber)`
Greitas parsinimas iš eilutės (failo skaitymui).

Išvestis

- `void appendListTo (std::string &out) const`
Prideda suformatuotą eilutę į buferį (failo rašymui).

Perdengti operatoriai

Programoje nenaudojami, bet realizuoti būsimiems klasės naudotojams — jie leidžia naudoti studentą su STL algoritmais ir asociatyviais konteneriais.

- `bool operator== (const studentas &other) const`
Lygina du studentus.
- `bool operator!= (const studentas &other) const`
Tikrina, ar studentai skiriasi.
- `bool operator< (const studentas &other) const`
Rikiavimo operatorius.
- `bool operator> (const studentas &other) const`
Atvirkštinis rikiavimo operatorius.
- `int operator[] (size_t i) const`
Prieiga prie namų darbo pažymio pagal indeksą.
- `operator bool () const`
Konvertavimas į `bool` — ar objektas turi duomenų.

Vieši Metodai inherited from `zmogus`

- `virtual ~zmogus ()`
Virtualus destruktorius.
- `const std::string & vardas () const`
Grąžina žmogaus vardą.
- `const std::string & pavarde () const`
Grąžina žmogaus pavardę.
- `std::string pilnasVardas () const`
Grąžina pilną vardą.
- `void setVardas (const std::string &v)`
Nustato vardą su validacija.
- `void setPavarde (const std::string &p)`
Nustato pavardę su validacija.

Statiniai Vieši Atributai

- `static int gyvuStudentu = 0`
Gyvuojančių studentų skaitiklis.

Konstantos

- `static constexpr int minPazymys = 1`
Mažiausias leistinas pažymys.
- `static constexpr int maxPazymys = 10`
Didžiausias leistinas pažymys.
- `static constexpr double namuDarbaiSvoris = 0.4`
Namų darbų svoris galutiname balse.
- `static constexpr double egzaminasSvoris = 0.6`
Egzamino svoris galutiname balse.

Statiniai Vieši Atributai inherited from **zmogus**

- static int **gyvuZmoniu** = 0
Gyvuojančių objektų skaitiklis.

Statiniai Privatūs Metodai

- static double **vidurkis** (const **Vector**< int > &nd)
Apskaičiuoja aritmetinį vidurkį.
- static double **mediana** (const **Vector**< int > &nd)
Apskaičiuoja medianą.
- static bool **pazymysTinkamas** (int p)
Patikrina, ar pažymys patenka į leistiną intervalą.

Privatūs Atributai

- **Vector**< int > **nd_**
Namų darbų pažymiai (kiekvienas 1–10).
- int **egzaminas_**
Egzamino pažymys (1–10).
- double **galutinisVid_**
Galutinis balas pagal namų darbų vidurkį
- double **galutinisMed_**
Galutinis balas pagal namų darbų medianą

Additional Inherited Members

Apsaugoti Metodai inherited from **zmogus**

- **zmogus** ()
Numatytasis konstruktorius.
- **zmogus** (const std::string &vardas, const std::string &pavarde)
Pilnas konstruktorius su validacija.
- **zmogus** (const **zmogus** &other)
Kopijavimo konstruktorius.
- **zmogus** (**zmogus** &&other) noexcept
Perkėlimo (move) konstruktorius.
- **zmogus** & **operator=** (const **zmogus** &other)
Kopijavimo priskyrimo operatorius.
- **zmogus** & **operator=** (**zmogus** &&other) noexcept
Perkėlimo (move) priskyrimo operatorius.

Apsaugoti Atributai inherited from **zmogus**

- std::string **vardas_**
Žmogaus vardas.
- std::string **pavarde_**
Žmogaus pavardė

7.2.1 Smulkus aprašymas

Studento duomenys — išvestinė klasė iš abstrakčios `zmogus`.

Prie paveldėto vardo ir pavardės prideda namų darbų pažymius, egzamino įvertinimą ir du galutinius balus (pagal vidurkį ir pagal medianą).

Paveldėjimas

Iš `zmogus` paveldi:

- `vardas_`, `pavarde_` (protected laukai)
- `vardas()`, `pavarde()`, `pilnasVardas()` get'eriaus
- `setVardas()`, `setPavarde()` set'eriaus su validacija
- virtualių destruktorių

Realizuoja visus keturis grynai virtualius bazinės klasės metodus: `galutinis()`, `print()`, `read()`, `tipas()`.

Galutinio balo formulė

$$\text{galutinis} = 0.4 * \text{namų_darbų_vidurkis_arba_mediana} + 0.6 * \text{egzaminas}$$

Svoriai apibrėžti konstantomis `namuDarbaiSvoris` ir `egzaminasSvoris`.

Naudojimo pavyzdys

```
// Sukūrimas su pilnu konstruktoriumi
studentas s("Jonas", "Jonaitis", {5, 6, 7, 8}, 9);
s.calculateGalutinis();
std::cout << s << "\n";

// Nuskaitymas iš srauto
std::istringstream iss("Ona Onaite 10 9 8 7");
studentas s2;
iss >> s2;

// Polimorfinis naudojimas
std::unique_ptr<zmogus> p = std::make_unique<studentas>(
    "Petras", "Petraitis", Vector<int>{5, 5}, 8);
std::cout << p->tipas() << ": " << p->galutinis() << "\n";
```

Įspėjimas

Galutiniai balai NĖRA skaičiuojami automatiškai kuriant objektą — po duomenų nuskaitymo ar sukūrimo reikia iškviešti `calculateGalutinis()`. Toks sprendimas leidžia matuoti nuskaitymo ir skaičiavimo spartą atskirai.

Taip pat žiūrėti

[zmogus](#)

Apibrėžimas failo [studentas.h](#) eilutėje 71.

7.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.2.2.1 studentas() [1/5]

`studentas::studentas ()`

Numatytasis konstruktorius.

Sukuria tuščią studentą: be vardo, pavardės ir pažymių, su nulinais galutiniais balais. Skirtas objektui, į kurį duomenys bus nuskaityti vėliau (pvz. per `readStudentas()`).

Apibrėžimas failo [studentas.cpp](#) eilutėje 60.

Ryšiai [egzaminas_](#), [galutinisMed_](#), [galutinisVid_](#), [gyvuStudentu](#), [nd_](#), ir [zmogus::zmogus\(\)](#).

Naudojamas [operator!=\(\)](#), [operator<\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [operator==\(\)](#), [operator>\(\)](#), [studentas\(\)](#), ir [studentas\(\)](#).

7.2.2.2 studentas() [2/5]

```
studentas::studentas (
    const std::string & vardas,
    const std::string & pavarde,
    const Vector< int > & nd,
    int egzaminas)
```

Pilnas konstruktorius su validacija.

Vardą ir pavardę validuoja bazinė klasė, pažymius — ši klasė.

Parametrai

<i>vardas</i>	Studento vardas (negali būti tuščias)
<i>pavarde</i>	Studento pavardė (negali būti tuščia)
<i>nd</i>	Namų darbų pažymiai (kiekvienas 1–10)
<i>egzaminas</i>	Egzamino pažymys (1–10)

Išimtys

<i>std::invalid_argument</i>	jei vardas arba pavardė tuščia
<i>std::runtime_error</i>	jei bent vienas pažymys už ribų

Pastaba

Galutiniai balai lieka 0.0 — reikia iškviesti [calculateGalutinis\(\)](#).

Apibrėžimas failo [studentas.cpp](#) eilutėje 68.

Ryšiai [egzaminas\(\)](#), [egzaminas_](#), [galutinisMed_](#), [galutinisVid_](#), [gyvuStudentu](#), [nd\(\)](#), [nd_](#), [zmogus::pavarde\(\)](#), [pazymysTinkamas\(\)](#), [zmogus::vardas\(\)](#), ir [zmogus::zmogus\(\)](#).

7.2.2.3 studentas() [3/5]

```
studentas::studentas (
    std::istream & is) [explicit]
```

Konstruktorius iš įvesties srauto.

Iškart nuskaito duomenis per [readStudentas\(\)](#). Pažymėtas `explicit`, kad neįvyktų netyčinis konvertavimas iš srauto į studentą.

Parametrai

<i>is</i>	Įvesties srautas
-----------	------------------

Išimtys

<i>std::runtime_error</i>	jei duomenys netinkami arba jų trūksta
---------------------------	--

Apibrėžimas failo [studentas.cpp](#) eilutėje 86.

Ryšiai [egzaminas_](#), [galutinisMed_](#), [galutinisVid_](#), [gyvuStudentu](#), [nd_](#), [readStudentas\(\)](#), ir [zmogus::zmogus\(\)](#).

7.2.2.4 ~studentas()

```
studentas::~~studentas () [override]
```

Destruktorius.

Išvalo studento duomenis ir sumažina `gyvuStudentu` skaitiklį. Bazinės klasės destruktorius `~zmogus()` išskviečiamas **automatiškai** po šio kūno — jo kviešti rankiniu būdu nereikia.

Naikinimo tvarka

`~studentas()` → `~zmogus()` (atvirkštinė kūrimo tvarkai)

Apibrėžimas failo `studentas.cpp` eilutėje 111.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `gyvuStudentu`, ir `nd_`.

7.2.2.5 studentas() [4/5]

```
studentas::studentas (
    const studentas & other)
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją (deep copy) — `std::string` ir `std::vector` laukai kopijuoja savo vidinius buferius, todėl objektai lieka visiškai nepriklausomi.

Parametrai

<i>other</i>	Kopijuojamas studentas
--------------	------------------------

Pastaba

Member initializer list prasideda `zmogus(other)` — be šio kvietimo būtų išskviestas `zmogus()` numatytasis konstruktorius, ir vardas su pavarde liktų tušti.

Apibrėžimas failo `studentas.cpp` eilutėje 130.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `gyvuStudentu`, `nd_`, `studentas()`, ir `zmogus::zmogus()`.

7.2.2.6 studentas() [5/5]

```
studentas::studentas (
    studentas && other) [noexcept]
```

Perkėlimo (move) konstruktorius.

Perima kito objekto resursus be kopijavimo. Šaltinis paliekamas galiojančioje, bet tuščioje būsenoje — jį saugu naikinti arba naudoti iš naujo.

Parametrai

<i>other</i>	Studentas, iš kurio perkeliama duomenys
--------------	---

Įspėjimas

Bazinė dalis perkeliama `zmogus(std::move(other))`. `std::move` čia **būtinas**: `other` yra pavardintas kintamasis, todėl pats savaime `lvalue` — be `std::move` būtų išskviestas `zmogus` **kopijavimo**, ne perkėlimo konstruktorius.

Pastaba

`noexcept` kritiškai svarbus spartai: `std::vector` persikirstymo metu naudoja `std::move_if_noexcept`, todėl be `noexcept` rinktųsi kopijavimo konstruktorių.

Apibrėžimas failo `studentas.cpp` eilutėje 172.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `gyvuStudentu`, `nd_`, `studentas()`, ir `zmogus::zmogus()`.

7.2.3 Metodų Dokumentacija

7.2.3.1 vidurkis()

```
double studentas::vidurkis (
    const Vector< int > & nd) [static], [private]
```

Apskaičiuoja aritmetinį vidurkį.

Parametrai

<i>nd</i>	Pažymių vektorius
-----------	-------------------

Gražina

Vidurkis; 0.0 jei vektorius tuščias

Apibrėžimas failo [studentas.cpp](#) eilutėje 12.

Ryšiai [nd\(\)](#).

Naudojamas [calculateGalutinis\(\)](#).

7.2.3.2 mediana()

```
double studentas::mediana (
    const Vector< int > & nd) [static], [private]
```

Apskaičiuoja medianą.

Nelyginiam elementų kiekiui gražina vidurinį elementą, lyginiam — dviejų vidurinių aritmetinį vidurkį. Įeinantis vektorius nekeičiamas (kopijuojamas ir rikiuojamas viduje).

Parametrai

<i>nd</i>	Pažymių vektorius (nebūtinai surikiuotas)
-----------	---

Gražina

Mediana; 0.0 jei vektorius tuščias

Apibrėžimas failo [studentas.cpp](#) eilutėje 19.

Ryšiai [Vector< T, Allocator >::begin\(\)](#), [Vector< T, Allocator >::end\(\)](#), [nd\(\)](#), ir [Vector< T, Allocator >::size\(\)](#).

Naudojamas [calculateGalutinis\(\)](#).

7.2.3.3 pazymysTinkamas()

```
bool studentas::pazymysTinkamas (
    int p) [inline], [static], [private]
```

Patikrina, ar pažymys patenka į leistiną intervalą.

Sąmoningai gražina `bool`, o ne meta išimtį: klaidos pranešimo formavimas (`std::string` konkatencija) yra brangus, o su 10 mln. studentų ši funkcija kviečiama ~160 mln. kartų. Pranešimas formuojamas tik ten, kur išimtis tikrai metama.

Parametrai

<i>p</i>	Tikrinamas pažymys
----------	--------------------

Gražina

`true` jei `minPazymys` p `maxPazymys`

Apibrėžimas failo [studentas.h](#) eilutėje 108.

Ryšiai [maxPazymys](#), ir [minPazymys](#).

Naudojamas [addPazymys\(\)](#), [parseFromLine\(\)](#), [readStudentas\(\)](#), [setEgzaminas\(\)](#), [setNd\(\)](#), ir [studentas\(\)](#).

7.2.3.4 operator=() [1/2]

```
studentas & studentas::operator= (
    const studentas & other)
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Priskiriamas studentas
--------------	------------------------

Gražina

Nuoroda į šį objektą (grandininiam priskyrimui $a = b = c$)

Pastaba

Turi apsaugą nuo priskyrimo sau (`this == &other`) — be jos `a = a` galėtų sugadinti duomenis.

Bazinė dalis priskiriama kvalifikuotu kvietimu `zmogus::operator=(other)`. Be kvalifikacijos kompiliatorius parinktų `studentas::operator=` ir įvyktų begalinė rekursija.

Apibrėžimas failo `studentas.cpp` eilutėje 147.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `nd_`, `zmogus::operator=()`, ir `studentas()`.

7.2.3.5 operator=() [2/2]

```
studentas & studentas::operator= (
    studentas && other) [noexcept]
```

Perkėlimo (move) priskyrimo operatorius.

Parametrai

<i>other</i>	Studentas, iš kurio perkeliami duomenys
--------------	---

Gražina

Nuoroda į šį objektą

Pastaba

Bazinė dalis perkeliama `zmogus::operator=(std::move(other))`.

Apibrėžimas failo `studentas.cpp` eilutėje 190.

Ryšiai `egzaminas_`, `galutinisMed_`, `galutinisVid_`, `nd_`, `zmogus::operator=()`, ir `studentas()`.

7.2.3.6 galutinis()

```
double studentas::galutinis () const [override], [virtual]
```

Gražina galutinį įvertinimą (bazinės klasės interfeisas).

Studentui tai galutinis balas pagal namų darbų **vidurkį**. Medianos variantas prieinamas per `galutinisMed()`.

Gražina

Galutinis balas pagal vidurkį

Įspėjimas

Gražina 0.0, jei nebuvo iškviesta `calculateGalutinis()`.

Realizuoja `zmogus`.

Apibrėžimas failo `studentas.cpp` eilutėje 217.

Ryšiai `galutinisVid_`.

7.2.3.7 print()

```
void studentas::print (
    std::ostream & os) const [override], [virtual]
```

Išveda studentą į srautą.

Formatas: vardas (20 simbolių), pavardė (20), galutinis pagal vidurkį (20), galutinis pagal medianą (20). Realizuoja polimorfinį operator<<.

Parametrai

<i>os</i>	Išvesties srautas
-----------	-------------------

Realizuoja [zmogus](#).

Apibrėžimas failo [studentas.cpp](#) eilutėje 225.

Ryšiai [galutinisMed_](#), [galutinisVid_](#), [zmogus::pavarde_](#), ir [zmogus::vardas_](#).

7.2.3.8 read()

```
std::istream & studentas::read (
    std::istream & is) [override], [virtual]
```

Nuskaito studentą iš srauto.

Realizuoja polimorfinį operator>>. Deleguoja į [readStudentas\(\)](#).

Parametrai

<i>is</i>	Išvesties srautas
-----------	-------------------

Gražina

Nuoroda į tą patį srautą

Realizuoja [zmogus](#).

Apibrėžimas failo [studentas.cpp](#) eilutėje 237.

Ryšiai [readStudentas\(\)](#).

7.2.3.9 tipas()

```
std::string studentas::tipas () const [override], [virtual]
```

Gražina tipo pavadinimą.

Gražina

"Studentas"

Realizuoja [zmogus](#).

Apibrėžimas failo [studentas.cpp](#) eilutėje 245.

7.2.3.10 nd()

```
const Vector< int > & studentas::nd () const [inline]
```

Gražina namų darbų pažymius.

Gražina

Konstanti nuoroda į vektorių (be kopijavimo)

Apibrėžimas failo [studentas.h](#) eilutėje 317.

Ryšiai [nd_](#).

Naudojamas [appendListTo\(\)](#), [mediana\(\)](#), [setNd\(\)](#), [studentas\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [testStudentasKlase\(\)](#), ir [vidurkis\(\)](#).

7.2.3.11 egzaminas()

```
int studentas::egzaminas () const [inline]
```

Grąžina egzamino pažymį.

Apibrėžimas failo [studentas.h](#) eilutėje 320.

Ryšiai [egzaminas_](#).

Naudojamas [comparePagalEgzamina\(\)](#), [studentas\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.12 galutinisVid()

```
double studentas::galutinisVid () const [inline]
```

Grąžina galutinį balą pagal namų darbų vidurkį.

Apibrėžimas failo [studentas.h](#) eilutėje 323.

Ryšiai [galutinisVid_](#).

Naudojamas [comparePagalVidurki\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.13 galutinisMed()

```
double studentas::galutinisMed () const [inline]
```

Grąžina galutinį balą pagal namų darbų medianą.

Apibrėžimas failo [studentas.h](#) eilutėje 326.

Ryšiai [galutinisMed_](#).

Naudojamas [comparePagalMediana\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.14 tuscias()

```
bool studentas::tuscias () const [inline]
```

Patikrina, ar objektas neturi vardo ir pavardės.

Gražina

```
true jei ir vardas, ir pavardė tušti
```

Apibrėžimas failo [studentas.h](#) eilutėje 332.

Ryšiai [zmogus::pavarde_](#), ir [zmogus::vardas_](#).

7.2.3.15 setNd()

```
void studentas::setNd (
    const Vector< int > & nd)
```

Nustato namų darbų pažymius su validacija.

Parametrai

<i>nd</i>	Pažymių vektorius
-----------	-------------------

Išimtys

<i>std::runtime_error</i>	jei bent vienas pažymys už ribų
---------------------------	---------------------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 254.

Ryšiai [nd\(\)](#), [nd_](#), ir [pazymysTinkamas\(\)](#).

Naudojamas [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.16 setEgzaminas()

```
void studentas::setEgzaminas (
    int e)
```

Nustato egzamino pažymį su validacija.

Parametrai

<i>e</i>	Egzamino pažymys
----------	------------------

Išimtys

<i>std::runtime_error</i>	jei pažymys už ribų
---------------------------	---------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 261.

Ryšiai [egzaminas_](#), ir [pazymysTinkamas\(\)](#).

Naudojamas [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.17 addPazymys()

```
void studentas::addPazymys (
    int p)
```

Prideda vieną namų darbo pažymį.

Parametrai

<i>p</i>	Pridedamas pažymys
----------	--------------------

Išimtys

<i>std::runtime_error</i>	jei pažymys už ribų
---------------------------	---------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 267.

Ryšiai [nd_](#), ir [pazymysTinkamas\(\)](#).

Naudojamas [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.18 isvalyk()

```
void studentas::isvalyk ()
```

Ištuština objektą.

Išvalo **ir bazinės, ir išvestinės** klasės laukus. Vektoriaus atmintis faktiškai atlaisvinama ([shrink_to_fit](#)).

Apibrėžimas failo [studentas.cpp](#) eilutėje 273.

Ryšiai [egzaminas_](#), [galutinisMed_](#), [galutinisVid_](#), [nd_](#), [zmogus::pavarde_](#), ir [zmogus::vardas_](#).

Naudojamas [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.2.3.19 calculateGalutinis()

```
void studentas::calculateGalutinis ()
```

Apskaičiuoja abu galutinius balus.

Užpildo [galutinisVid_](#) ir [galutinisMed_](#) pagal formulę:

```
galutinisVid = 0.4 * vidurkis(nd) + 0.6 * egzaminas
galutinisMed = 0.4 * mediana(nd) + 0.6 * egzaminas
```

Tuščiam namų darbų sąrašui vidurkis ir mediana laikomi 0.0, todėl galutinis balas yra $0.6 * \text{egzaminas}$.

Pastaba

Kviečiamas atskirai, ne konstruktoriuje — taip nuskaitymo ir skaičiavimo spartą galima matuoti nepriklausomai.

Apibrėžimas failo `studentas.cpp` eilutėje 287.

Ryšiai `egzaminas_`, `egzaminasSvoris`, `galutinisMed_`, `galutinisVid_`, `mediana()`, `namuDarbaiSvoris`, `nd_`, ir `vidurkis()`.

Naudojamas `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, ir `testStudentasKlase()`.

7.2.3.20 readStudentas()

```
std::istream & studentas::readStudentas (
    std::istream & is,
    int ndCount = 0,
    int lineNumber = 0)
```

Nuskaityto studentą iš srauto.

Veikia dviem režimais priklausomai nuo `ndCount`:

- **`ndCount > 0`** — nuskaityti tiek namų darbų pažymių, po jų — egzaminą. Naudojama, kai stulpelių skaičius žinomas iš failo antraštės.
- **`ndCount == 0`** — skaito skaičius iki eilutės galo, paskutinį laiko egzaminu. Naudojama interaktyviai įvesčiai.

Parametrai

<i>is</i>	Įvesties srautas
<i>ndCount</i>	Namų darbų skaičius (0 = skaityti iki galo)
<i>lineNumber</i>	Eilutės numeris klaidos pranešimui (0 = nenaudoti)

Gražina

Nuoroda į tą patį srautą

Išimtys

<code>std::runtime_error</code>	jei trūksta duomenų arba pažymys už ribų
---------------------------------	--

Pastaba

Failo skaitymui naudokite `parseFromLine()` — jis kelis kartus greitesnis, nes neturi srauto mechanizmo.

Apibrėžimas failo `studentas.cpp` eilutėje 355.

Ryšiai `egzaminas_`, `nd_`, `zmogus::pavarde_`, `pazymysTinkamas()`, ir `zmogus::vardas_`.

Naudojamas `read()`, ir `studentas()`.

7.2.3.21 parseFromLine()

```
void studentas::parseFromLine (
    const std::string & line,
    int ndCount,
    int lineNumber)
```

Greitas parsinimas iš eilutės (failo skaitymui).

Optimizuota alternatyva `readStudentas()` metodui masiniam duomenų apdorojimui:

- **nekuriamas** `std::istringstream` objektas kiekvienai eilutei

- skaičiai parsinami su `std::from_chars` (C++17) — be locale, be stream sentry, be būsenos tikrinimo
- klaidos pranešimai formuojami tik metant išimtį

Su 10 mln. studentų šie trys pakeitimai sutrumpina failo nuskaitymą nuo ~90 s iki ~30 s.

Parametrai

<i>line</i>	Nuskaityama eilutė
<i>ndCount</i>	Namų darbų stulpelių skaičius (iš failo antraštės)
<i>lineNumber</i>	Eilutės numeris klaidos pranešimui

Išimtys

<code>std::runtime_error</code>	jei trūksta vardo, pavardės ar pažymio, arba pažymys už ribų
---------------------------------	--

Taip pat žiūrėti

[appendListTo\(\)](#) — atvirkštinė operacija

Apibrėžimas failo `studentas.cpp` eilutėje 300.

Ryšiai `egzaminas_`, `nd_`, `zmogus::pavarde_`, `pazymysTinkamas()`, ir `zmogus::vardas_`.

Naudojamas `readStudentaiFromFile()`, `TEST_CASE()`, ir `testStudentasKlase()`.

7.2.3.22 appendListTo()

```
void studentas::appendListTo (
    std::string & out) const
```

Prideda suformatuotą eilutę į buferį (failo rašymui).

Optimizuota alternatyva `operator<<` masiniam rašymui:

- skaičiai formatuojami su `std::to_chars` — be `std::setw` manipuliatorių
- rašoma į bendrą `std::string` buferį, ne į srautą

Kviečianti funkcija kaupia eilutes ir iškviečia vieną `file.write()` kas ~1 MB. Buferis nedidėja proporcingai studentų skaičiui — `clear()` išlaiko `capacity`.

Formatas: vardas (25), pavardė (25), kiekvienas ND pažymys (10), egzaminas (10), naujos eilutės simbolis.

Parametrai

<i>out</i>	Buferis, į kurį pridedama eilutė
------------	----------------------------------

Taip pat žiūrėti

[parseFromLine\(\)](#) — atvirkštinė operacija; kartu jos sudaro round-trip: išvesta eilutė nuskaityta atgal be pakitimų

Apibrėžimas failo `studentas.cpp` eilutėje 407.

Ryšiai `egzaminas_`, `nd()`, `nd_`, `zmogus::pavarde_`, ir `zmogus::vardas_`.

Naudojamas `TEST_CASE()`, ir `testStudentasKlase()`.

7.2.3.23 operator==()

```
bool studentas::operator== (
    const studentas & other) const
```

Lygina du studentus.

Lyginami vardas, pavardė, namų darbų pažymiai ir egzaminas. Galutiniai balai **nelyginami**, nes jie išvedami iš kitų laukų.

Parametrai

<i>other</i>	Lyginamas studentas
--------------	---------------------

Gražina

`true` jei visi duomenys sutampa

Apibrėžimas failo [studentas.cpp](#) eilutėje 434.

Ryšiai [egzaminas_](#), [nd_](#), [zmogus::pavarde_](#), [studentas\(\)](#), ir [zmogus::vardas_](#).

7.2.3.24 operator!=(())

```
bool studentas::operator!=(
    const studentas & other) const
```

Tikrina, ar studentai skiriasi.

Parametrai

<i>other</i>	Lyginamas studentas
--------------	---------------------

Gražina

`!(*this == other)`

Apibrėžimas failo [studentas.cpp](#) eilutėje 441.

Ryšiai [studentas\(\)](#).

7.2.3.25 operator<()

```
bool studentas::operator< (
    const studentas & other) const
```

Rikiavimo operatorius.

Lygina pagal pavardę; esant vienodoms pavardėms — pagal vardą. Leidžia naudoti `std::sort` be komparatoriaus ir dėti studentus į `std::set` ar `std::map`.

Parametrai

<i>other</i>	Lyginamas studentas
--------------	---------------------

Gražina

`true` jei šis studentas eina anksčiau

Apibrėžimas failo [studentas.cpp](#) eilutėje 446.

Ryšiai [zmogus::pavarde_](#), [studentas\(\)](#), ir [zmogus::vardas_](#).

7.2.3.26 operator>()

```
bool studentas::operator> (
    const studentas & other) const
```

Atvirkštinis rikiavimo operatorius.

Parametrai

<i>other</i>	Lyginamas studentas
--------------	---------------------

Gražina

```
other < *this
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 452.
Ryšiai [studentas\(\)](#).

7.2.3.27 operator[]()

```
int studentas::operator[] (
    size_t i) const
```

Prieiga prie namų darbo pažymio pagal indeksą.

Parametrai

<i>i</i>	Pažymio indeksas (nuo 0)
----------	--------------------------

Gražina

Pažymys

Išimtys

<i>std::out_of_range</i>	jei indeksas už ribų
--------------------------	----------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 456.
Ryšiai [nd_](#).

7.2.3.28 operator bool()

```
studentas::operator bool () const [explicit]
```

Konvertavimas į `bool` — ar objektas turi duomenų.

Pažymėtas `explicit`, kad neįvyktų netyčinis konvertavimas į `int` (pvz. `s + 1` nesikompiliuos, kaip ir turi būti).

Gražina

```
true
```

jei vardas ir pavardė nėra tušti

Pavyzdys

```
studentas s;
if (!s) std::cout << "Objektas tuscias\n";
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 462.
Ryšiai [zmogus::pavarde_](#), ir [zmogus::vardas_](#).

7.2.4 Atributų Dokumentacija**7.2.4.1 nd_**

```
Vector<int> studentas::nd_ [private]
```

Namų darbų pažymiai (kiekvienas 1–10).

Apibrėžimas failo [studentas.h](#) eilutėje 73.

Naudojamas [addPazymys\(\)](#), [appendListTo\(\)](#), [calculateGalutinis\(\)](#), [isvalyk\(\)](#), [nd\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [operator==\(\)](#), [operator\[\]\(\)](#), [parseFromLine\(\)](#), [readStudentas\(\)](#), [setNd\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), ir [~studentas\(\)](#).

7.2.4.2 egzaminas_

```
int studentas::egzaminas_ [private]
```

Egzamino pažymys (1–10).

Apibrėžimas failo [studentas.h](#) eilutėje 74.

Naudojamas [appendListTo\(\)](#), [calculateGalutinis\(\)](#), [egzaminas\(\)](#), [isvalyk\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [operator==\(\)](#), [parseFromLine\(\)](#), [readStudentas\(\)](#), [setEgzaminas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), ir [~studentas\(\)](#).

7.2.4.3 galutinisVid_

```
double studentas::galutinisVid_ [private]
```

Galutinis balas pagal namų darbų vidurkį

Apibrėžimas failo [studentas.h](#) eilutėje 75.

Naudojamas [calculateGalutinis\(\)](#), [galutinis\(\)](#), [galutinisVid\(\)](#), [isvalyk\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [print\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), ir [~studentas\(\)](#).

7.2.4.4 galutinisMed_

```
double studentas::galutinisMed_ [private]
```

Galutinis balas pagal namų darbų medianą

Apibrėžimas failo [studentas.h](#) eilutėje 76.

Naudojamas [calculateGalutinis\(\)](#), [galutinisMed\(\)](#), [isvalyk\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [print\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), ir [~studentas\(\)](#).

7.2.4.5 minPazymys

```
int studentas::minPazymys = 1 [static], [constexpr]
```

Mažiausias leistinas pažymys.

Apibrėžimas failo [studentas.h](#) eilutėje 115.

Naudojamas [enterEgzaminas\(\)](#), [enterPazymys\(\)](#), [generatePazymiai\(\)](#), ir [pazymysTinkamas\(\)](#).

7.2.4.6 maxPazymys

```
int studentas::maxPazymys = 10 [static], [constexpr]
```

Didžiausias leistinas pažymys.

Apibrėžimas failo [studentas.h](#) eilutėje 116.

Naudojamas [enterEgzaminas\(\)](#), [enterPazymys\(\)](#), [generatePazymiai\(\)](#), ir [pazymysTinkamas\(\)](#).

7.2.4.7 namuDarbaiSvoris

```
double studentas::namuDarbaiSvoris = 0.4 [static], [constexpr]
```

Namų darbų svoris galutiniame balse.

Apibrėžimas failo [studentas.h](#) eilutėje 117.

Naudojamas [calculateGalutinis\(\)](#).

7.2.4.8 egzaminasSvoris

```
double studentas::egzaminasSvoris = 0.6 [static], [constexpr]
```

Egzamino svoris galutiniame balse.

Apibrėžimas failo [studentas.h](#) eilutėje 118.

Naudojamas [calculateGalutinis\(\)](#).

7.2.4.9 gyvuStudentu

```
int studentas::gyvuStudentu = 0 [static]
```

Gyvuojančių studentų skaitiklis.

Didinamas kiekviename konstruktoriuje, mažinamas destruktoriuje. Naudojamas testuose objektų nutekėjimui aptikti — atskiras nuo [zmogus::gyvuZmoniu](#), todėl galima patikrinti, ar iškviesti abu destruktoriai ([~studentas\(\)](#) ir [~zmogus\(\)](#)).

Apibrėžimas failo [studentas.h](#) eilutėje 129.

Naudojamas [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [studentas\(\)](#), [TEST_CASE\(\)](#), [testStudentasKlase\(\)](#), ir [~studentas\(\)](#).

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [OOP_3/studentas.h](#)
- [OOP_3/studentas.cpp](#)

7.3 `Vector< T, Allocator >` Klasė Šablonas

Dinaminis masyvas — pilnavertė `std::vector` alternatyva.

```
#include <Vector.h>
```

Paveldimumo diagrama $\text{Vector} \langle T, \text{Allocator} \rangle$:



- *Dydžio tipas.*
using `difference_type` = `std::ptrdiff_t`
- *Iteratorių skirtumo tipas.*
using `reference` = `value_type&`
- *Nuoroda į elementą*
using `const_reference` = `const value_type&`
- *Konstanti nuoroda.*
using `pointer` = `typename std::allocator_traits<Allocator>::pointer`
- using `const_pointer` = `typename std::allocator_traits<Allocator>::const_pointer`
- using `iterator` = `T*`
Iteratorius (raw pointer).
- using `const_iterator` = `const T*`
Konstantus iteratorius.
- using `reverse_iterator` = `std::reverse_iterator<iterator>`
- using `const_reverse_iterator` = `std::reverse_iterator<const_iterator>`

Vieši Metodai

Konstruktoriai ir destruktorius

- `Vector ()` `noexcept(noexcept(Allocator()))`
Numatytasis konstruktorius — sukuria tuščią vektorių.
- `Vector (const Allocator &alloc)` `noexcept`
Konstruktorius su skirstykle.
- `Vector (size_type n, const T &value, const Allocator &alloc=Allocator())`
Sukuria vektorių su n kopijų reikšmės value.
- `Vector (size_type n, const Allocator &alloc=Allocator())`
Sukuria vektorių su n numatytais sukonstruotais elementais.
- `template<typename InputIt, typename = typename std::enable_if<!std::is_integral<InputIt>::value>::type>`
`Vector (InputIt first, InputIt last, const Allocator &alloc=Allocator())`
Sukuria vektorių iš iteratorių intervalo [first, last).
- `Vector (const Vector &other)`
Kopijavimo konstruktorius.
- `Vector (Vector &&other)` `noexcept`
Perkėlimo (move) konstruktorius.
- `Vector (std::initializer_list< T > il, const Allocator &alloc=Allocator())`
Konstruktorius iš inicijavimo sąrašo.
- `~Vector ()`
Destruktorius — sunaikina elementus ir atlaisvina atmintį.

Priskyrimas

- `Vector & operator= (const Vector &other)`
Kopijavimo priskyrimo operatorius.
- `Vector & operator= (Vector &&other)` `noexcept`
Perkėlimo priskyrimo operatorius.
- `Vector & operator= (std::initializer_list< T > il)`
Priskyrimas iš inicijavimo sąrašo.
- `void assign (size_type n, const T &value)`
Pakeičia turinį n kopijų reikšmės value.
- `template<typename InputIt, typename = typename std::enable_if<!std::is_integral<InputIt>::value>::type>`
`void assign (InputIt first, InputIt last)`
Pakeičia turinį intervalo [first, last) elementais.
- `void assign (std::initializer_list< T > il)`
Pakeičia turinį inicijavimo sąrašo elementais.
- `allocator_type get_allocator ()` `const noexcept`
Grąžina naudojamą skirstyklę.

Elementų prieiga

- `reference at (size_type pos)`
Prieiga prie elemento su ribų tikrinimu.
- `const_reference at (size_type pos) const`
Prieiga prie elemento su ribų tikrinimu.
- `reference operator[] (size_type pos)`
*Prieiga prie elemento **be** ribų tikrinimo.*
- `const_reference operator[] (size_type pos) const`
*Prieiga prie elemento **be** ribų tikrinimo.*
- `reference front ()`
Grąžina pirmą elementą.
- `const_reference front () const`
Grąžina pirmą elementą.
- `reference back ()`
Grąžina paskutinį elementą.
- `const_reference back () const`
Grąžina paskutinį elementą.
- `T * data () noexcept`
Grąžina rodyklę į vidinį masyvą.
- `const T * data () const noexcept`
Grąžina rodyklę į vidinį masyvą.

Iteratoriai

- `iterator begin () noexcept`
Pradžia.
- `const_iterator begin () const noexcept`
Pradžia (const).
- `const_iterator cbegin () const noexcept`
Pradžia (const).
- `iterator end () noexcept`
Pabaiga.
- `const_iterator end () const noexcept`
Pabaiga (const).
- `const_iterator cend () const noexcept`
Pabaiga (const).
- `reverse_iterator rbegin () noexcept`
Atvirkštinė pradžia (paskutinis elementas).
- `const_reverse_iterator rbegin () const noexcept`
Atvirkštinė pradžia (paskutinis elementas).
- `const_reverse_iterator crbegin () const noexcept`
Atvirkštinė pradžia (paskutinis elementas).
- `reverse_iterator rend () noexcept`
Atvirkštinė pabaiga (prieš pirmą elementą).
- `const_reverse_iterator rend () const noexcept`
Atvirkštinė pabaiga (prieš pirmą elementą).
- `const_reverse_iterator crend () const noexcept`
Atvirkštinė pabaiga (prieš pirmą elementą).

Talpa

- `bool empty () const noexcept`
Patikrina, ar vektorius tuščias.
- `size_type size () const noexcept`
Grąžina elementų skaičių.
- `size_type max_size () const noexcept`
Grąžina didžiausią galimą elementų skaičių.
- `void reserve (size_type n)`
Paskiria atmintį bent n elementams.
- `size_type capacity () const noexcept`
Grąžina esamą talpą.

- void `shrink_to_fit` ()
Sumažina talpą iki dydžio.

Modifikavimas

- void `clear` () noexcept
Pašalina visus elementus.
- `iterator insert` (`const_iterator` pos, const T &value)
Įterpia elementą prieš pos.
- `iterator insert` (`const_iterator` pos, T &&value)
Įterpia elementą prieš pos (perkėlimu).
- `iterator insert` (`const_iterator` pos, `size_type` n, const T &value)
Įterpia n kopijų reikšmės prieš pos.
- template<typename InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
`iterator insert` (`const_iterator` pos, InputIt first, InputIt last)
Įterpia intervalo [first, last) elementus prieš pos.
- `iterator insert` (`const_iterator` pos, std::initializer_list< T > il)
Įterpia inicijavimo sąrašo elementus prieš pos.
- template<typename... Args>
`iterator emplace` (`const_iterator` pos, Args &&... args)
Sukuria elementą vietoje prieš pos.
- `iterator erase` (`const_iterator` pos)
Pašalina elementą pozicijoje pos.
- `iterator erase` (`const_iterator` first, `const_iterator` last)
Pašalina elementus intervale [first, last).
- void `push_back` (const T &value)
Prideda elementą į galą.
- void `push_back` (T &&value)
Prideda elementą į galą (perkėlimu).
- template<typename... Args>
`reference emplace_back` (Args &&... args)
Sukuria elementą vietoje gale.
- void `pop_back` ()
Pašalina paskutinį elementą.
- void `resize` (`size_type` n)
Pakeičia elementų skaičių.
- void `resize` (`size_type` n, const T &value)
Pakeičia elementų skaičių, naujus užpildant reikšme.
- void `swap` (`Vector` &other) noexcept
Sukeičia turinį su kitu vektoriumi.

Privatūs Tipai

- using `AllocTraits` = std::allocator_traits<Allocator>

Privatūs Metodai

- void `destroyRange` (pointer first, pointer last)
Sunaikina visus elementus intervale [first, last).
- void `deallocateAll` ()
Atlaisvina visą paskirtą atmintį ir sunaikina elementus.
- `size_type` `augimoTalpa` (`size_type` reikia) const
Apskaičiuoja naują talpą augimo metu.
- void `perskirstyk` (`size_type` naujaTalpa)
Perskirsto atmintį į naują talpą.
- `iterator insertImpl` (`const_iterator` pos, `size_type` n, const T &value)
Bendra insert realizacija n kopijų įterpimui.

Privatūs Atributai

- `pointer begin_`
Rodyklė į pirmą elementą
- `pointer end_`
Rodyklė po paskutinio elemento.
- `pointer cap_`
Rodyklė po paskirtos atminties.
- Allocator `alloc_`
Atminties skirstyklė

7.3.1 Smulkus aprašymas

`template<typename T, typename Allocator = std::allocator<T>>`
`class Vector< T, Allocator >`

Dinaminis masyvas — pilnavertė `std::vector` alternatyva.

Realizuoja beveik visą `std::vector` interfeisą: member tipus, konstruktorius, elementų prieigą, iteratorius, talpos valdymą, modifikavimo operacijas ir ne-narius operatorius.

Template Parameters

<i>T</i>	Elemento tipas
<i>Allocator</i>	Atminties skirstyklė (pagal nutylėjimą <code>std::allocator<T></code>)

Architektūra

Klasė laiko tris rodykles, kaip ir tipinė `std::vector` realizacija:

```
begin_ → pradžia
end_   → po paskutinio elemento (size = end_ - begin_)
cap_   → po paskirtos atminties (capacity = cap_ - begin_)
```

Atmintis **skiriama ir konstruojama atskirai**: `allocate()` tik rezervuoja baitus, o objektai kuriami `construct()` metodu. Todėl `reserve(1000)` nesukuria 1000 objektų — tik paskiria vietą.

Augimo strategija

Pritrūkus vietos, talpa **dvigubinama** (arba nustatoma į 1, jei buvo 0). Toks koeficientas duoda amortizuotą $O(1)$ `push_back()` sudėtingumą: n elementų įdėjimui reikia $\log(n)$ perskirstymų, o bendras kopijavimo darbas yra $O(n)$.

Naudojimo pavyzdys

```
Vector<int> v = {1, 2, 3};
v.push_back(4);
v.insert(v.begin() + 1, 10); // {1, 10, 2, 3, 4}

for (int x : v) std::cout << x << " ";

Vector<int> v2(v); // kopija
Vector<int> v3(std::move(v)); // perkėlimas
```

Pastaba

Šioje realizacijoje `Vector<bool>` nėra specializuotas. Standartinis `std::vector<bool>` pakuoja reikšmes į bitus, bet tai laikoma standarto klaida, nes toks konteineris nebeatitinka bendrųjų konteinerio reikalavimų. Čia `Vector<bool>` veikia kaip įprastas `bool` masyvas.

Apibrėžimas failo `Vector.h` eilutėje 69.

7.3.2 Tipų Aprašymo Dokumentacija

7.3.2.1 value_type

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::value_type = T
```

Elemento tipas.

Apibrėžimas failo [Vector.h](#) eilutėje 76.

7.3.2.2 allocator_type

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::allocator_type = Allocator
```

Skirstyklės tipas.

Apibrėžimas failo [Vector.h](#) eilutėje 77.

7.3.2.3 size_type

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::size_type = std::size_t
```

Dydžio tipas.

Apibrėžimas failo [Vector.h](#) eilutėje 78.

7.3.2.4 difference_type

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::difference_type = std::ptrdiff_t
```

Iteratorių skirtumo tipas.

Apibrėžimas failo [Vector.h](#) eilutėje 79.

7.3.2.5 reference

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::reference = value_type&
```

Nuoroda į elementą

Apibrėžimas failo [Vector.h](#) eilutėje 80.

7.3.2.6 const_reference

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::const_reference = const value_type&
```

Konstanti nuoroda.

Apibrėžimas failo [Vector.h](#) eilutėje 81.

7.3.2.7 pointer

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::pointer = typename std::allocator_traits<Allocator>::pointer
```

Apibrėžimas failo [Vector.h](#) eilutėje 82.

7.3.2.8 const_pointer

```
template<typename T, typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::const_pointer = typename std::allocator_traits<Allocator>::const_pointer
```

Apibrėžimas failo [Vector.h](#) eilutėje 83.

7.3.2.9 iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::iterator = T*
Iteratorius (raw pointer).
Apibrėžimas failo Vector.h eilutėje 84.
```

7.3.2.10 const_iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_iterator = const T*
Konstantus iteratorius.
Apibrėžimas failo Vector.h eilutėje 85.
```

7.3.2.11 reverse_iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::reverse_iterator = std::reverse_iterator<iterator>
Apibrėžimas failo Vector.h eilutėje 86.
```

7.3.2.12 const_reverse_iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reverse_iterator = std::reverse_iterator<const_iterator>
Apibrėžimas failo Vector.h eilutėje 87.
```

7.3.2.13 AllocTraits

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::AllocTraits = std::allocator_traits<Allocator> [private]
Apibrėžimas failo Vector.h eilutėje 97.
```

7.3.3 Konstruktorius ir Destruktorius Dokumentacija

7.3.3.1 Vector() [1/8]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector () [inline], [noexcept]
Numatytasis konstruktorius — sukuria tuščią vektorį.
```

Postcondition

```
size() == 0 && capacity() == 0
```

Apibrėžimas failo [Vector.h](#) eilutėje 185.

Ryšiai [alloc_](#), [begin_](#), [cap_](#), ir [end_](#).

Naudojamas [operator=\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [swap\(\)](#), [Vector\(\)](#), [Vector\(\)](#), ir [Vector\(\)](#).

7.3.3.2 Vector() [2/8]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Allocator & alloc) [inline], [explicit], [noexcept]
Konstruktorius su skirstykle.
```

Parametrai

<i>alloc</i>	Naudojama atminties skirstyklė
--------------	--------------------------------

Apibrėžimas failo [Vector.h](#) eilutėje 193.

7.3.3.3 Vector() [3/8]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type n,
    const T & value,
    const Allocator & alloc = Allocator()) [inline]
```

Sukuria vektorių su *n* kopijų reikšmės *value*.

Parametrai

<i>n</i>	Elementų skaičius
<i>value</i>	Kopijuojama reikšmė
<i>alloc</i>	Skirstyklė

Apibrėžimas failo [Vector.h](#) eilutėje 203.

Ryšiai [alloc_](#), [begin_](#), [cap_](#), [deallocateAll\(\)](#), ir [end_](#).

7.3.3.4 Vector() [4/8]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type n,
    const Allocator & alloc = Allocator()) [inline], [explicit]
```

Sukuria vektorių su *n* numatyta sukonstruotų elementų.

Parametrai

<i>n</i>	Elementų skaičius
<i>alloc</i>	Skirstyklė

Apibrėžimas failo [Vector.h](#) eilutėje 222.

Ryšiai [alloc_](#), [begin_](#), [cap_](#), [deallocateAll\(\)](#), ir [end_](#).

7.3.3.5 Vector() [5/8]

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
Vector< T, Allocator >::Vector (
    InputIt first,
    InputIt last,
    const Allocator & alloc = Allocator()) [inline]
```

Sukuria vektorių iš iteratorių intervalo [*first*, *last*).

`enable_if` sąlyga atskiria šį konstruktorių nuo `Vector(size_type, const T&)` — be jos `Vector<int>(5, 10)` būtų klaidingai interpretuotas kaip intervalas.

Template Parameters

<i>InputIt</i>	Įvesties iteratoriaus tipas
----------------	-----------------------------

Parametrai

<i>first</i>	Intervalo pradžia
--------------	-------------------

<i>last</i>	Intervalo pabaiga
<i>alloc</i>	Skirstyklė

Apibrėžimas failo [Vector.h](#) eilutėje 251.

Ryšiai [alloc_](#), [begin_](#), [cap_](#), [deallocateAll\(\)](#), ir [end_](#).

7.3.3.6 Vector() [6/8]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other) [inline]
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją. Naujo vektoriaus talpa lygi kopijuojamo **dydžiui**, ne talpai — nekopijuojama nepanaudota atmintis.

Parametrai

<i>other</i>	Kopijuojamas vektorius
--------------	------------------------

Apibrėžimas failo [Vector.h](#) eilutėje 274.

Ryšiai [alloc_](#), [begin_](#), [cap_](#), [deallocateAll\(\)](#), [end_](#), [size\(\)](#), ir [Vector\(\)](#).

7.3.3.7 Vector() [7/8]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Perkėlimo (move) konstruktorius.

Perima kito vektoriaus atmintį be kopijavimo — tik trys rodyklių priskyrimai. Šaltinis paliekamas tuščias.

Parametrai

<i>other</i>	Vektorius, iš kurio perkeliama
--------------	--------------------------------

Apibrėžimas failo [Vector.h](#) eilutėje 298.

Ryšiai [Vector\(\)](#).

7.3.3.8 Vector() [8/8]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    std::initializer_list< T > il,
    const Allocator & alloc = Allocator()) [inline]
```

Konstruktorius iš inicijavimo sąrašo.

Parametrai

<i>il</i>	Inicijavimo sąrašas, pvz. {1, 2, 3}
<i>alloc</i>	Skirstyklė

Apibrėžimas failo [Vector.h](#) eilutėje 310.

Ryšiai [begin\(\)](#), [end\(\)](#), ir [Vector\(\)](#).

7.3.3.9 ~Vector()

```
template<typename T, typename Allocator = std::allocator<T>>
```

`Vector< T, Allocator >::~~Vector () [inline]`
 Destruktorius — sunaikina elementus ir atlaisvina atmintį.
 Apibrėžimas failo `Vector.h` eilutėje 317.
 Ryšiai `deallocateAll()`.

7.3.4 Metodų Dokumentacija

7.3.4.1 `destroyRange()`

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::destroyRange (
    pointer first,
    pointer last) [inline], [private]
```

Sunaikina visus elementus intervale `[first, last)`.
 Kviečia destruktoriaus, bet **neatlaisvina** atminties.
 Apibrėžimas failo `Vector.h` eilutėje 104.
 Ryšiai `alloc_`.
 Naudojamas `clear()`, `deallocateAll()`, `erase()`, `resize()`, ir `resize()`.

7.3.4.2 `deallocateAll()`

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::deallocateAll () [inline], [private]
```

Atlaisvina visą paskirtą atmintį ir sunaikina elementus.
 Apibrėžimas failo `Vector.h` eilutėje 112.
 Ryšiai `alloc_`, `begin_`, `cap_`, `capacity()`, `destroyRange()`, ir `end_`.
 Naudojamas `assign()`, `assign()`, `operator=()`, `perskirstyk()`, `shrink_to_fit()`, `Vector()`, `Vector()`, `Vector()`, `Vector()`, ir `~Vector()`.

7.3.4.3 `augimoTalpa()`

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::augimoTalpa (
    size_type reikia) const [inline], [private]
```

Apskaičiuoja naują talpą augimo metu.
 Dvigubina esamą talpą; jei ji 0 — grąžina 1. Jei reikalinga talpa didesnė nei dvigubinta, grąžina reikalingą.

Parametrai

<i>reikia</i>	Minimali reikalinga talpa
---------------	---------------------------

Gražina

Nauja talpa

Išimtys

<code>std::length_error</code>	jei viršijamas <code>max_size()</code>
--------------------------------	--

Apibrėžimas failo `Vector.h` eilutėje 130.
 Ryšiai `capacity()`, ir `max_size()`.
 Naudojamas `emplace()`, `emplace_back()`, `insert()`, `insert()`, `insertImpl()`, `push_back()`, `push_back()`, `resize()`, ir `resize()`.

7.3.4.4 `perskirstyk()`

```
template<typename T, typename Allocator = std::allocator<T>>
```



```
void Vector< T, Allocator >::perskirstyk (
    size_type naujaTalpa) [inline], [private]
```

Perskirsto atmintį į naują talpą.

Elementai perkeliami (move) jei tipas turi `noexcept move` konstruktorių — kitaip kopijuojami, kad būtų išlaikyta stipri išimčių sauga.

Parametrai

<i>naujaTalpa</i>	Nauja talpa (turi būti <code>size()</code>)
-------------------	--

Apibrėžimas failo `Vector.h` eilutėje 152.

Ryšiai `alloc_`, `begin_`, `cap_`, `deallocateAll()`, `end_`, ir `size()`.

Naudojamas `emplace()`, `emplace_back()`, `insert()`, `insert()`, `insertImpl()`, `push_back()`, `push_back()`, `reserve()`, `resize()`, `resize()`, ir `shrink_to_fit()`.

7.3.4.5 operator=() [1/3]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    const Vector< T, Allocator > & other) [inline]
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Priskiriamas vektorius
--------------	------------------------

Gražina

Nuoroda į šį objektą

Apibrėžimas failo `Vector.h` eilutėje 331.

Ryšiai `assign()`, `begin()`, `end()`, ir `Vector()`.

7.3.4.6 operator=() [2/3]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Perkėlimo priskyrimo operatorius.

Parametrai

<i>other</i>	Vektorius, iš kurio perkeliama
--------------	--------------------------------

Gražina

Nuoroda į šį objektą

Apibrėžimas failo `Vector.h` eilutėje 342.

Ryšiai `alloc_`, `begin_`, `cap_`, `deallocateAll()`, `end_`, ir `Vector()`.

7.3.4.7 operator=() [3/3]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    std::initializer_list< T > il) [inline]
```

Priskyrimas iš inicijavimo sąrašo.

Parametrai

<i>il</i>	Inicijavimo sąrašas
-----------	---------------------

Gražina

Nuoroda į šį objektą

Apibrėžimas failo [Vector.h](#) eilutėje 358.

Ryšiai [assign\(\)](#), ir [Vector\(\)](#).

7.3.4.8 assign() [1/3]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    size_type n,
    const T & value) [inline]
```

Pakeičia turinį *n* kopijų reikšmės *value*.

Parametrai

<i>n</i>	Elementų skaičius
<i>value</i>	Kopijuojama reikšmė

Apibrėžimas failo [Vector.h](#) eilutėje 368.

Ryšiai [alloc_](#), [begin_](#), [cap_](#), [capacity\(\)](#), [clear\(\)](#), [deallocateAll\(\)](#), ir [end_](#).

Naudojamas [operator=\(\)](#), [operator=\(\)](#), ir [TEST_CASE\(\)](#).

7.3.4.9 assign() [2/3]

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
void Vector< T, Allocator >::assign (
    InputIt first,
    InputIt last) [inline]
```

Pakeičia turinį intervalo [*first*, *last*) elementais.

Template Parameters

<i>InputIt</i>	Įvesties iteratoriaus tipas
----------------	-----------------------------

Parametrai

<i>first</i>	Intervalo pradžia
<i>last</i>	Intervalo pabaiga

Apibrėžimas failo [Vector.h](#) eilutėje 389.

Ryšiai [alloc_](#), [begin_](#), [cap_](#), [capacity\(\)](#), [clear\(\)](#), [deallocateAll\(\)](#), ir [end_](#).

7.3.4.10 assign() [3/3]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    std::initializer_list< T > il) [inline]
```

Pakeičia turinį inicijavimo sąrašo elementais.

Parametrai

<i>il</i>	Inicijavimo sąrašas
-----------	---------------------

Apibrėžimas failo [Vector.h](#) eilutėje 406.

Ryšiai [assign\(\)](#).

Naudojamas [assign\(\)](#).

7.3.4.11 get_allocator()

```
template<typename T, typename Allocator = std::allocator<T>>
allocator_type Vector< T, Allocator >::get_allocator () const [inline], [noexcept]
```

Gražina naudojamą skirstyklę.

Gražina

Skirstyklės kopija

Apibrėžimas failo [Vector.h](#) eilutėje 412.

Ryšiai [alloc_](#).

7.3.4.12 at() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::at (
    size_type pos) [inline]
```

Prieiga prie elemento su ribų tikrinimu.

Parametrai

<i>pos</i>	Elemento indeksas
------------	-------------------

Gražina

Nuoroda į elementą

Išimtys

<i>std::out_of_range</i>	jei <i>pos</i> >= <i>size()</i>
--------------------------	---------------------------------

Apibrėžimas failo [Vector.h](#) eilutėje 427.

Ryšiai [begin_](#), ir [size\(\)](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.13 at() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::at (
    size_type pos) const [inline]
```

Prieiga prie elemento su ribų tikrinimu.

Parametrai

<i>pos</i>	Elemento indeksas
------------	-------------------

Gražina

Nuoroda į elementą

Išimtys

<code>std::out_of_range</code>	jei <code>pos >= size()</code>
--------------------------------	-----------------------------------

Apibrėžimas failo [Vector.h](#) eilutėje 434.

Ryšiai [begin_](#), ir [size\(\)](#).

7.3.4.14 operator[]() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::operator[] (
    size_type pos) [inline]
```

Prieiga prie elemento **be** ribų tikrinimo.

Parametrai

<i>pos</i>	Elemento indeksas
------------	-------------------

Gražina

Nuoroda į elementą

Išpėjimas

Su `pos >= size()` elgesys neapibrėžtas. Ribų tikrinimui naudokite [at\(\)](#).

Apibrėžimas failo [Vector.h](#) eilutėje 447.

Ryšiai [begin_](#).

7.3.4.15 operator[]() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::operator[] (
    size_type pos) const [inline]
```

Prieiga prie elemento **be** ribų tikrinimo.

Parametrai

<i>pos</i>	Elemento indeksas
------------	-------------------

Gražina

Nuoroda į elementą

Išpėjimas

Su `pos >= size()` elgesys neapibrėžtas. Ribų tikrinimui naudokite [at\(\)](#).

Apibrėžimas failo [Vector.h](#) eilutėje 450.

Ryšiai [begin_](#).

7.3.4.16 front() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::front () [inline]
```

Grąžina pirmą elementą.

Įspėjimas

Tuščiam vektoriui neapibrėžta.

Apibrėžimas failo [Vector.h](#) eilutėje 453.

Ryšiai [begin_](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.17 front() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::front () const [inline]
```

Grąžina pirmą elementą.

Įspėjimas

Tuščiam vektoriui neapibrėžta.

Apibrėžimas failo [Vector.h](#) eilutėje 455.

Ryšiai [begin_](#).

7.3.4.18 back() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::back () [inline]
```

Grąžina paskutinį elementą.

Įspėjimas

Tuščiam vektoriui neapibrėžta.

Apibrėžimas failo [Vector.h](#) eilutėje 458.

Ryšiai [end_](#).

Naudojamas [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.3.4.19 back() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::back () const [inline]
```

Grąžina paskutinį elementą.

Įspėjimas

Tuščiam vektoriui neapibrėžta.

Apibrėžimas failo [Vector.h](#) eilutėje 460.

Ryšiai [end_](#).

7.3.4.20 data() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
T * Vector< T, Allocator >::data () [inline], [noexcept]
```

Grąžina rodyklę į vidinį masyvą.

Gražina

Rodyklė į duomenis; nullptr jei tuščias

Apibrėžimas failo [Vector.h](#) eilutėje 466.

Ryšiai [begin_](#).

Naudojamas [readStudentaiFromFile\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [writeStudentaiToFile\(\)](#).

7.3.4.21 data() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const T * Vector< T, Allocator >::data () const [inline], [noexcept]
```

Gražina rodyklę į vidinį masyvą.

Gražina

Rodyklę į duomenis; nullptr jei tuščias

Apibrėžimas failo [Vector.h](#) eilutėje 468.

Ryšiai [begin_](#).

7.3.4.22 begin() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::begin () [inline], [noexcept]
```

Pradžia.

Apibrėžimas failo [Vector.h](#) eilutėje 477.

Ryšiai [begin_](#).

Naudojamas [erase\(\)](#), [erase_if\(\)](#), [mediana\(\)](#), [studentas::mediana\(\)](#), [operator<\(\)](#), [operator=\(\)](#), [operator==\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [testStudentasKlase\(\)](#), ir [Vector\(\)](#).

7.3.4.23 begin() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::begin () const [inline], [noexcept]
```

Pradžia (const).

Apibrėžimas failo [Vector.h](#) eilutėje 478.

Ryšiai [begin_](#).

7.3.4.24 cbegin()

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cbegin () const [inline], [noexcept]
```

Pradžia (const).

Apibrėžimas failo [Vector.h](#) eilutėje 479.

Ryšiai [begin_](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.25 end() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::end () [inline], [noexcept]
```

Pabaiga.

Apibrėžimas failo [Vector.h](#) eilutėje 481.

Ryšiai [end_](#).

Naudojamas [erase\(\)](#), [erase_if\(\)](#), [mediana\(\)](#), [studentas::mediana\(\)](#), [operator<\(\)](#), [operator=\(\)](#), [operator==\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [testStudentasKlase\(\)](#), ir [Vector\(\)](#).

7.3.4.26 end() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::end () const [inline], [noexcept]
```

Pabaiga (const).

Apibrėžimas failo [Vector.h](#) eilutėje 482.

Ryšiai [end_](#).

7.3.4.27 cend()

```
template<typename T, typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cend () const [inline], [noexcept]
```

Pabaiga (const).

Apibrėžimas failo [Vector.h](#) eilutėje 483.

Ryšiai [end_](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.28 rbegin() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rbegin () [inline], [noexcept]
```

Atvirkštinė pradžia (paskutinis elementas).

Apibrėžimas failo [Vector.h](#) eilutėje 486.

Ryšiai [end_](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.29 rbegin() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rbegin () const [inline], [noexcept]
```

Atvirkštinė pradžia (paskutinis elementas).

Apibrėžimas failo [Vector.h](#) eilutėje 488.

Ryšiai [end_](#).

7.3.4.30 crbegin()

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crbegin () const [inline], [noexcept]
```

Atvirkštinė pradžia (paskutinis elementas).

Apibrėžimas failo [Vector.h](#) eilutėje 490.

Ryšiai [end_](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.31 rend() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rend () [inline], [noexcept]
```

Atvirkštinė pabaiga (prieš pirmą elementą).

Apibrėžimas failo [Vector.h](#) eilutėje 493.

Ryšiai [begin_](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.32 rend() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rend () const [inline], [noexcept]
```

Atvirkštinė pabaiga (prieš pirmą elementą).

Apibrėžimas failo [Vector.h](#) eilutėje 495.

Ryšiai [begin_](#).

7.3.4.33 crend()

```
template<typename T, typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crend () const [inline], [noexcept]
```

Atvirkštinė pabaiga (prieš pirmą elementą).

Apibrėžimas failo [Vector.h](#) eilutėje 497.

Ryšiai [begin_](#).

7.3.4.34 empty()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::empty () const [inline], [noexcept]
```

Patikrina, ar vektorius tuščias.

Gražina

true jei `size()` == 0

Apibrėžimas failo `Vector.h` eilutėje 510.

Ryšiai `begin_`, ir `end_`.

Naudojamas `median()`, `shrink_to_fit()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, ir `testStudentasKlase()`.

7.3.4.35 size()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size () const [inline], [noexcept]
```

Gražina elementų skaičių.

Gražina

Elementų skaičius

Apibrėžimas failo `Vector.h` eilutėje 516.

Ryšiai `begin_`, ir `end_`.

Naudojamas `at()`, `at()`, `emplace()`, `emplace_back()`, `insert()`, `insert()`, `insertImpl()`, `median()`, `studentas::median()`, `operator==()`, `perskirstyk()`, `push_back()`, `push_back()`, `readStudentaiFromFile()`, `resize()`, `resize()`, `shrink_to_fit()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, `testStudentasKlase()`, `Vector()`, ir `writeStudentaiToFile()`.

7.3.4.36 max_size()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::max_size () const [inline], [noexcept]
```

Gražina didžiausią galimą elementų skaičių.

Gražina

Teorinė riba

Apibrėžimas failo `Vector.h` eilutėje 524.

Ryšiai `alloc_`.

Naudojamas `augimoTalpa()`, `reserve()`, ir `TEST_CASE()`.

7.3.4.37 reserve()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::reserve (
    size_type n) [inline]
```

Paskiria atmintį bent `n` elementams.

Jei `n` <= `capacity()`, nieko nedaro. Elementų **nekuria** — `size()` nepakinta.

Parametrai

<code>n</code>	Reikalinga talpa
----------------	------------------

Išimtys

<code>std::length_error</code>	jei <code>n > max_size()</code>
--------------------------------	------------------------------------

Pastaba

Naudinga, kai iš anksto žinomas elementų skaičius — išvengiama daugkartinių perskirstymų.

Apibrėžimas failo `Vector.h` eilutėje 540.

Ryšiai `capacity()`, `max_size()`, ir `perskirstyk()`.

Naudojamas `generatePazymiai()`, `TEST_CASE()`, `TEST_CASE()`, ir `testStudentasKlase()`.

7.3.4.38 capacity()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::capacity () const [inline], [noexcept]
```

Gražina esamą talpą.

Gražina

Kiek elementų telpa be perskirstymo

Apibrėžimas failo `Vector.h` eilutėje 551.

Ryšiai `begin_`, ir `cap_`.

Naudojamas `assign()`, `assign()`, `augimoTalpa()`, `deallocateAll()`, `emplace()`, `insert()`, `insert()`, `insertImpl()`, `reserve()`, `resize()`, `resize()`, `shrink_to_fit()`, `skaiciuokVector()`, `TEST_CASE()`, `TEST_CASE()`, `TEST_CASE()`, ir `tyrimas3()`.

7.3.4.39 shrink_to_fit()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::shrink_to_fit () [inline]
```

Sumažina talpą iki dydžio.

Atlaisvina nepanaudotą atmintį. Jei `capacity() == size()`, nieko nedaro.

Apibrėžimas failo `Vector.h` eilutėje 561.

Ryšiai `capacity()`, `deallocateAll()`, `empty()`, `perskirstyk()`, ir `size()`.

Naudojamas `TEST_CASE()`.

7.3.4.40 clear()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::clear () [inline], [noexcept]
```

Pašalina visus elementus.

`size()` tampa 0, bet **talpa nepakinta** — atmintis lieka paskirta pakartotiniam naudojimui.

Apibrėžimas failo `Vector.h` eilutėje 581.

Ryšiai `begin_`, `destroyRange()`, ir `end_`.

Naudojamas `assign()`, `assign()`, ir `TEST_CASE()`.

7.3.4.41 insert() [1/5]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    const T & value) [inline]
```

Įterpia elementą prieš `pos`.

Parametrai

<code>pos</code>	Pozicija, prieš kurią įterpiama
<code>value</code>	Įterpiama reikšmė

Gražina

Iteratorius į įterptą elementą

Apibrėžimas failo [Vector.h](#) eilutėje 592.

Ryšiai [insertImpl\(\)](#).

Naudojamas [insert\(\)](#), ir [TEST_CASE\(\)](#).

7.3.4.42 insert() [2/5]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    T && value) [inline]
```

Įterpia elementą prieš pos (perkėlimu).

Parametrai

<i>pos</i>	Pozicija
<i>value</i>	Įterpiama reikšmė

Gražina

Iteratorius į įterptą elementą

Apibrėžimas failo [Vector.h](#) eilutėje 602.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [begin_](#), [capacity\(\)](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

7.3.4.43 insert() [3/5]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    size_type n,
    const T & value) [inline]
```

Įterpia n kopijų reikšmės prieš pos.

Parametrai

<i>pos</i>	Pozicija
<i>n</i>	Kopijų skaičius
<i>value</i>	Įterpiama reikšmė

Gražina

Iteratorius į pirmą įterptą elementą

Apibrėžimas failo [Vector.h](#) eilutėje 626.

Ryšiai [insertImpl\(\)](#).

7.3.4.44 insert() [4/5]

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename InputIt, typename = typename std::enable_if< !std::is_integral<InputIt>::value>::type>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
```

```
InputIt first,
InputIt last) [inline]
```

Įterpia intervalo [first, last) elementus prieš pos.

Template Parameters

<i>InputIt</i>	Įvesties iteratoriaus tipas
----------------	-----------------------------

Parametrai

<i>pos</i>	Pozicija
<i>first</i>	Intervalo pradžia
<i>last</i>	Intervalo pabaiga

Gražina

Iteratorius į pirmą įterptą elementą

Apibrėžimas failo [Vector.h](#) eilutėje 641.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [begin_](#), [capacity\(\)](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

7.3.4.45 insert() [5/5]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    std::initializer_list< T > il) [inline]
```

Įterpia inicijavimo sąrašo elementus prieš pos.

Parametrai

<i>pos</i>	Pozicija
<i>il</i>	Inicijavimo sąrašas

Gražina

Iteratorius į pirmą įterptą elementą

Apibrėžimas failo [Vector.h](#) eilutėje 669.

Ryšiai [insert\(\)](#).

7.3.4.46 emplace()

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename... Args>
iterator Vector< T, Allocator >::emplace (
    const_iterator pos,
    Args &&... args) [inline]
```

Sukuria elementą vietoje prieš pos.

Elementas konstruojamas **tiesiogiai** konteinerio atmintyje — be laikino objekto ir be kopijavimo.

Template Parameters

<i>Args</i>	Konstruktoriaus argumentų tipai
-------------	---------------------------------

Parametrai

<i>pos</i>	Pozicija
<i>args</i>	Argumentai, perduodami elemento konstruktoriui

Gražina

Iteratorius į sukurta elementą

Apibrėžimas failo [Vector.h](#) eilutėje 685.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [begin_](#), [capacity\(\)](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

7.3.4.47 erase() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator pos) [inline]
```

Pašalina elementą pozicijoje *pos*.

Parametrai

<i>pos</i>	Šalinamo elemento pozicija
------------	----------------------------

Gražina

Iteratorius į kitą elementą po pašalinto

Apibrėžimas failo [Vector.h](#) eilutėje 707.

Ryšiai [alloc_](#), [begin_](#), ir [end_](#).

Naudojamas [erase\(\)](#), [erase_if\(\)](#), ir [TEST_CASE\(\)](#).

7.3.4.48 erase() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

Pašalina elementus intervale [*first*, *last*).

Parametrai

<i>first</i>	Intervalo pradžia
<i>last</i>	Intervalo pabaiga

Gražina

Iteratorius į kitą elementą po pašalintų

Apibrėžimas failo [Vector.h](#) eilutėje 721.

Ryšiai [begin_](#), [destroyRange\(\)](#), ir [end_](#).

7.3.4.49 push_back() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    const T & value) [inline]
```

Prideda elementą į galą.

Amortizuotas sudėtingumas $O(1)$. Pritrūkus vietos, talpa dvigubinama.

Parametrai

<i>value</i>	Pridedama reikšmė
--------------	-------------------

Apibrėžimas failo [Vector.h](#) eilutėje 739.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [cap_](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

Naudojamas [enterPazymiai\(\)](#), [generatePazymiai\(\)](#), [matuokVector\(\)](#), [skaiciuokVector\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [testStudentasKlase\(\)](#), ir [tyrimas3\(\)](#).

7.3.4.50 push_back() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    T && value) [inline]
```

Prideda elementą į galą (perkėlimu).

Parametrai

<i>value</i>	Pridedama reikšmė
--------------	-------------------

Apibrėžimas failo [Vector.h](#) eilutėje 750.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [cap_](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

7.3.4.51 emplace_back()

```
template<typename T, typename Allocator = std::allocator<T>>
template<typename... Args>
reference Vector< T, Allocator >::emplace_back (
    Args &&... args) [inline]
```

Sukuria elementą vietoje gale.

Efektyvesnis už `push_back (T (args...))` — objektas konstruojamas tiesiogiai konteinerio atmintyje, be laikino objekto.

Template Parameters

<i>Args</i>	Konstruktoriaus argumentų tipai
-------------	---------------------------------

Parametrai

<i>args</i>	Argumentai, perduodami elemento konstruktoriui
-------------	--

Gražina

Nuoroda į sukurtą elementą

Apibrėžimas failo [Vector.h](#) eilutėje 768.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [cap_](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

Naudojamas [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [TEST_CASE\(\)](#).

7.3.4.52 pop_back()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::pop_back () [inline]
```

Pašalina paskutinį elementą.

Įspėjimas

Tuščiam vektoriui elgesys neapibrėžtas.

Apibrėžimas failo [Vector.h](#) eilutėje 780.

Ryšiai [alloc_](#), ir [end_](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.53 `resize()` [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type n) [inline]
```

Pakeičia elementų skaičių.

Didinant — nauji elementai konstruojami numatyta. Mažinant — pertekliniai sunaikinami, talpa nepakinta.

Parametrai

<i>n</i>	Naujas dydis
----------	--------------

Apibrėžimas failo [Vector.h](#) eilutėje 793.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [begin_](#), [capacity\(\)](#), [destroyRange\(\)](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.54 `resize()` [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type n,
    const T & value) [inline]
```

Pakeičia elementų skaičių, naujus užpildant reikšme.

Parametrai

<i>n</i>	Naujas dydis
<i>value</i>	Reikšmė naujiems elementams

Apibrėžimas failo [Vector.h](#) eilutėje 810.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [begin_](#), [capacity\(\)](#), [destroyRange\(\)](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

7.3.4.55 `swap()`

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::swap (
    Vector< T, Allocator > & other) [inline], [noexcept]
```

Sukeičia turinį su kitu vektoriumi.

Sudėtingumas $O(1)$ — sukeičiamos tik rodyklės, elementai nejudinami.

Parametrai

<i>other</i>	Vektorius, su kuriuo keičiamasi
--------------	---------------------------------

Apibrėžimas failo [Vector.h](#) eilutėje 830.

Ryšiai [alloc_](#), [begin_](#), [cap_](#), [end_](#), ir [Vector\(\)](#).

Naudojamas [TEST_CASE\(\)](#).

7.3.4.56 `insertImpl()`

```
template<typename T, typename Allocator = std::allocator<T>>
```

```
iterator Vector< T, Allocator >::insertImpl (
    const_iterator pos,
    size_type n,
    const T & value) [inline], [private]
```

Bendra insert realizacija n kopijų įterpimui.

Apibrėžimas failo [Vector.h](#) eilutėje 843.

Ryšiai [alloc_](#), [augimoTalpa\(\)](#), [begin_](#), [capacity\(\)](#), [end_](#), [perskirstyk\(\)](#), ir [size\(\)](#).

Naudojamas [insert\(\)](#), ir [insert\(\)](#).

7.3.5 Atributų Dokumentacija

7.3.5.1 begin_

```
template<typename T, typename Allocator = std::allocator<T>>
pointer Vector< T, Allocator >::begin_ [private]
```

Rodyklė į pirmą elementą

Apibrėžimas failo [Vector.h](#) eilutėje 92.

Naudojamas [assign\(\)](#), [assign\(\)](#), [at\(\)](#), [at\(\)](#), [begin\(\)](#), [begin\(\)](#), [capacity\(\)](#), [cbegin\(\)](#), [clear\(\)](#), [crend\(\)](#), [data\(\)](#), [data\(\)](#), [deallocateAll\(\)](#), [emplace\(\)](#), [empty\(\)](#), [erase\(\)](#), [erase\(\)](#), [front\(\)](#), [front\(\)](#), [insert\(\)](#), [insert\(\)](#), [insertImpl\(\)](#), [operator=\(\)](#), [operator\[\]\(\)](#), [operator\[\]\(\)](#), [perskirstyk\(\)](#), [rend\(\)](#), [rend\(\)](#), [resize\(\)](#), [resize\(\)](#), [size\(\)](#), [swap\(\)](#), [Vector\(\)](#), [Vector\(\)](#), [Vector\(\)](#), [Vector\(\)](#), ir [Vector\(\)](#).

7.3.5.2 end_

```
template<typename T, typename Allocator = std::allocator<T>>
pointer Vector< T, Allocator >::end_ [private]
```

Rodyklė po paskutinio elemento.

Apibrėžimas failo [Vector.h](#) eilutėje 93.

Naudojamas [assign\(\)](#), [assign\(\)](#), [back\(\)](#), [back\(\)](#), [cend\(\)](#), [clear\(\)](#), [crbegin\(\)](#), [deallocateAll\(\)](#), [emplace\(\)](#), [emplace_back\(\)](#), [empty\(\)](#), [end\(\)](#), [end\(\)](#), [erase\(\)](#), [erase\(\)](#), [insert\(\)](#), [insert\(\)](#), [insertImpl\(\)](#), [operator=\(\)](#), [perskirstyk\(\)](#), [pop_back\(\)](#), [push_back\(\)](#), [push_back\(\)](#), [rbegin\(\)](#), [rbegin\(\)](#), [resize\(\)](#), [resize\(\)](#), [size\(\)](#), [swap\(\)](#), [Vector\(\)](#), [Vector\(\)](#), [Vector\(\)](#), [Vector\(\)](#), ir [Vector\(\)](#).

7.3.5.3 cap_

```
template<typename T, typename Allocator = std::allocator<T>>
pointer Vector< T, Allocator >::cap_ [private]
```

Rodyklė po paskirtos atminties.

Apibrėžimas failo [Vector.h](#) eilutėje 94.

Naudojamas [assign\(\)](#), [assign\(\)](#), [capacity\(\)](#), [deallocateAll\(\)](#), [emplace_back\(\)](#), [operator=\(\)](#), [perskirstyk\(\)](#), [push_back\(\)](#), [push_back\(\)](#), [swap\(\)](#), [Vector\(\)](#), [Vector\(\)](#), [Vector\(\)](#), [Vector\(\)](#), ir [Vector\(\)](#).

7.3.5.4 alloc_

```
template<typename T, typename Allocator = std::allocator<T>>
Allocator Vector< T, Allocator >::alloc_ [private]
```

Atminties skirstyklė

Apibrėžimas failo [Vector.h](#) eilutėje 95.

Naudojamas [assign\(\)](#), [assign\(\)](#), [deallocateAll\(\)](#), [destroyRange\(\)](#), [emplace\(\)](#), [emplace_back\(\)](#), [erase\(\)](#), [get_allocator\(\)](#), [insert\(\)](#), [insert\(\)](#), [insertImpl\(\)](#), [max_size\(\)](#), [operator=\(\)](#), [perskirstyk\(\)](#), [pop_back\(\)](#), [push_back\(\)](#), [push_back\(\)](#), [resize\(\)](#), [resize\(\)](#), [swap\(\)](#), [Vector\(\)](#), [Vector\(\)](#), [Vector\(\)](#), [Vector\(\)](#), ir [Vector\(\)](#).

Dokumentacija šiai klasei sugeneruota iš šio failo:

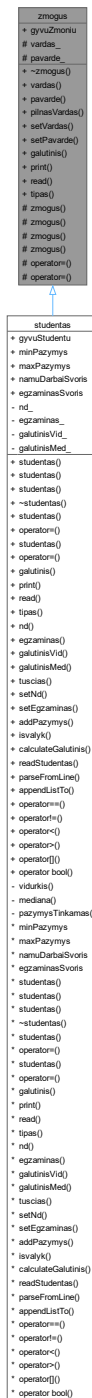
- [OOP_3/Vector.h](#)

7.4 zmogus Klasė

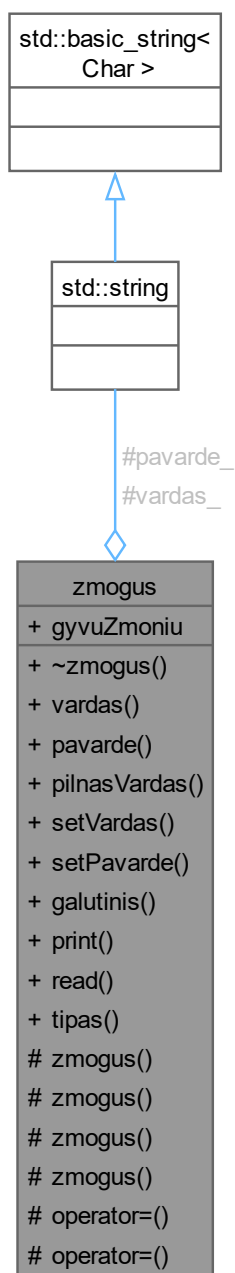
ABSTRAKTI bazinė klasė — bendras bet kokio žmogaus aprašymas.

```
#include <zmogus.h>
```

Paveldimumo diagrama žmogus:



Bendradarbiavimo diagrama zmogus:



Vieši Metodai

- virtual `~zmogus ()`
Virtualus destruktorius.
- const std::string & `vardas ()` const
Grąžina žmogaus vardą.
- const std::string & `pavarde ()` const
Grąžina žmogaus pavardę.

- `std::string pilnasVardas () const`
Grąžina pilną vardą.
- `void setVardas (const std::string &v)`
Nustato vardą su validacija.
- `void setPavarde (const std::string &p)`
Nustato pavardę su validacija.
- `virtual double galutinis () const =0`
Grąžina galutinį įvertinimą.
- `virtual void print (std::ostream &os) const =0`
Išveda objektą į srautą.
- `virtual std::istream & read (std::istream &is)=0`
Nuskaityto objektą iš srauto.
- `virtual std::string tipas () const =0`
Grąžina žmogaus tipo pavadinimą.

Statiniai Vieši Atributai

- `static int gyvuZmoniu = 0`
Gyvuojančių objektų skaitiklis.

Apsaugoti Metodai

- `zmogus ()`
Numatytasis konstruktorius.
- `zmogus (const std::string &vardas, const std::string &pavarde)`
Pilnas konstruktorius su validacija.
- `zmogus (const zmogus &other)`
Kopijavimo konstruktorius.
- `zmogus (zmogus &&other) noexcept`
Perkėlimo (move) konstruktorius.
- `zmogus & operator= (const zmogus &other)`
Kopijavimo priskyrimo operatorius.
- `zmogus & operator= (zmogus &&other) noexcept`
Perkėlimo (move) priskyrimo operatorius.

Apsaugoti Atributai

- `std::string vardas_`
Žmogaus vardas.
- `std::string pavarde_`
Žmogaus pavardė

7.4.1 Smulkus aprašymas

ABSTRAKTI bazinė klasė — bendras bet kokio žmogaus aprašymas.

Laiko vardą ir pavardę bei apibrėžia interfeisą, kurį privalo realizuoti kiekviena išvestinė klasė.

Kodėl klasė abstrakti

Klasė padaryta abstrakti **dviem nepriklausomais mechanizmais**:

1. **Grynai virtualūs metodai** (= 0) — `galutinis()`, `print()`, `read()`, `tipas()`. Kompiliatorius neleidžia kurti objektų iš klasės, kuri turi bent vieną neapibrėžtą virtualų metodą.
2. **protected konstruktoriai** — prieinami tik išvestinėms klasėms. Tai antras apsaugos sluoksnis.

Naudojimo pavyzdys

```
// zmogus z; // KLAIDA: abstrakti klasė
// zmogus* p = new zmogus(); // KLAIDA: abstrakti klasė

studentas s("Jonas", "Jonaitis", {5, 6, 7}, 8); // OK
zmogus& z = s; // OK -- nuoroda
std::cout << z.tipas() << "\n"; // "Studentas"

std::unique_ptr<zmogus> p =
    std::make_unique<studentas>("Ona", "Onaite", {10}, 10); // OK
```

Įspėjimas

Destruktorius PRIVALO būti virtualus — kitaip naikinant objektą per bazinės klasės rodyklę išvestinės klasės laukai liktų neatlaisvinti.

Taip pat žiūrėti

[studentas](#)

Apibrėžimas failo [zmogus.h](#) eilutėje 50.

7.4.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.4.2.1 zmogus() [1/4]

```
zmogus::zmogus () [protected]
```

Numatytasis konstruktorius.

Sukuria objektą su tuščiu vardu ir pavarde. `protected` — kviečiamas tik iš išvestinių klasių.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 12.

Ryšiai [gyvuZmoniu](#), [pavarde_](#), ir [vardas_](#).

Naudojamas [operator=\(\)](#), [operator=\(\)](#), [studentas::studentas\(\)](#), [studentas::studentas\(\)](#), [studentas::studentas\(\)](#), [studentas::studentas\(\)](#), [studentas::studentas\(\)](#), [zmogus\(\)](#), ir [zmogus\(\)](#).

7.4.2.2 zmogus() [2/4]

```
zmogus::zmogus (
    const std::string & vardas,
    const std::string & pavarde) [protected]
```

Pilnas konstruktorius su validacija.

Parametrai

<i>vardas</i>	Žmogaus vardas (negali būti tuščias)
<i>pavarde</i>	Žmogaus pavardė (negali būti tuščia)

Išimtys

<i>std::invalid_argument</i>	jei vardas arba pavardė tuščia
------------------------------	--------------------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 18.

Ryšiai [gyvuZmoniu](#), [pavarde\(\)](#), [pavarde_](#), [vardas\(\)](#), ir [vardas_](#).

7.4.2.3 zmogus() [3/4]

```
zmogus::zmogus (
    const zmogus & other) [protected]
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją — `std::string` laukai kopijuoja savo vidinius buferius.

Parametrai

<i>other</i>	Kopijuojamas objektas
--------------	-----------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 32.

Ryšiai [gyvuZmoniu](#), [pavarde_](#), [vardas_](#), ir [zmogus\(\)](#).

7.4.2.4 zmogus() [4/4]

```
zmogus::zmogus (
    zmogus && other) [protected], [noexcept]
```

Perkėlimo (move) konstruktorius.

Perima kito objekto resursus be kopijavimo. Šaltinis paliekamas galiojančioje, bet tuščioje būsenoje.

Parametrai

<i>other</i>	Objektas, iš kurio perkeliama duomenys
--------------	--

Pastaba

`noexcept` būtinai: be `std::vector` persikėlimo metu naudotų kopijavimo konstruktorių (`std::vector` naudoja `move_if_noexcept`).

Apibrėžimas failo [zmogus.cpp](#) eilutėje 42.

Ryšiai [gyvuZmoniu](#), [pavarde_](#), [vardas_](#), ir [zmogus\(\)](#).

7.4.2.5 ~zmogus()

```
zmogus::~zmogus () [virtual]
```

Virtualus destruktorius.

Išpėjimas

`virtual` kvalifikatorius kritiškai svarbus. Be jo:

```
zmogus* p = new studentas(...);
delete p;      // kviečiamas TIK ~zmogus()
               // studentas laukai NEATLAISVINAMI
```

Su `virtual` naikinimo tvarka teisinga: `~studentas()` → `~zmogus()`.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 82.

Ryšiai [gyvuZmoniu](#), [pavarde_](#), ir [vardas_](#).

7.4.3 Metodų Dokumentacija**7.4.3.1 operator=() [1/2]**

```
zmogus & zmogus::operator= (
    const zmogus & other) [protected]
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>other</i>	Priskiriamas objektas
--------------	-----------------------

Gražina

Nuoroda į šį objektą (grandininiam priskyrimui `a = b = c`)

Pastaba

Turi apsaugą nuo priskyrimo sau (`this == &other`).

Apibrėžimas failo [zmogus.cpp](#) eilutėje 51.

Ryšiai [pavarde_](#), [vardas_](#), ir [zmogus\(\)](#).

Naudojamas [studentas::operator=\(\)](#), ir [studentas::operator=\(\)](#).

7.4.3.2 operator=() [2/2]

```
zmogus & zmogus::operator= (
    zmogus && other) [protected], [noexcept]
```

Perkėlimo (move) priskyrimo operatorius.

Parametrai

<i>other</i>	Objektas, iš kurio perkeliama duomenys
--------------	--

Gražina

Nuoroda į šį objektą

Apibrėžimas failo [zmogus.cpp](#) eilutėje 65.

Ryšiai [pavarde_](#), [vardas_](#), ir [zmogus\(\)](#).

7.4.3.3 vardas()

```
const std::string & zmogus::vardas () const [inline]
```

Gražina žmogaus vardą.

Gražina

Konstanti nuoroda į vardą (be kopijavimo)

Apibrėžimas failo [zmogus.h](#) eilutėje 141.

Ryšiai [vardas_](#).

Naudojamas [comparePagalVarda\(\)](#), [studentas::studentas\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [testStudentasKlase\(\)](#), ir [zmogus\(\)](#).

7.4.3.4 pavarde()

```
const std::string & zmogus::pavarde () const [inline]
```

Gražina žmogaus pavardę.

Gražina

Konstanti nuoroda į pavardę (be kopijavimo)

Apibrėžimas failo [zmogus.h](#) eilutėje 147.

Ryšiai [pavarde_](#).

Naudojamas [comparePagalPavarde\(\)](#), [studentas::studentas\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [testStudentasKlase\(\)](#), ir [zmogus\(\)](#).

7.4.3.5 pilnasVardas()

```
std::string zmogus::pilnasVardas () const [inline]
```

Gražina pilną vardą.

Gražina

Vardas ir pavardė, atskirti tarpu

Apibrėžimas failo [zmogus.h](#) eilutėje 153.

Ryšiai [pavarde_](#), ir [vardas_](#).

Naudojamas [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.4.3.6 setVardas()

```
void zmogus::setVardas (
    const std::string & v)
```

Nustato vardą su validacija.

Parametrai

<i>v</i>	Naujas vardas
----------	---------------

Išimtys

<i>std::invalid_argument</i>	jei vardas tuščias
------------------------------	--------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 94.

Ryšiai [vardas_](#).

Naudojamas [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.4.3.7 setPavarde()

```
void zmogus::setPavarde (
    const std::string & p)
```

Nustato pavardę su validacija.

Parametrai

<i>p</i>	Nauja pavardė
----------	---------------

Išimtys

<i>std::invalid_argument</i>	jei pavardė tuščia
------------------------------	--------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 101.

Ryšiai [pavarde_](#).

Naudojamas [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.4.3.8 galutinis()

```
virtual double zmogus::galutinis () const [pure virtual]
```

Gražina galutinį įvertinimą.

Grynai virtualus — kiekvienas žmogaus tipas skaičiuoja jį savaip. Studentui tai galutinis balas pagal namų darbų vidurkį ir egzaminą.

Gražina

Galutinis įvertinimas

Realizuota [studentas](#).

Naudojamas [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.4.3.9 print()

```
virtual void zmogus::print (
    std::ostream & os) const [pure virtual]
```

Išveda objektą į srautą.

Grynai virtualus. Realizuoja polimorfinį operator<<.

Parametrai

<i>os</i>	Išvesties srautas
-----------	-------------------

Realizuota [studentas](#).

Naudojamas [operator<<\(\)](#).

7.4.3.10 read()

```
virtual std::istream & zmogus::read (
    std::istream & is) [pure virtual]
```

Nuskaityti objektą iš srauto.

Grynai virtualus. Realizuoja polimorfinį [operator>>](#).

Parametrai

<i>is</i>	Išvesties srautas
-----------	-------------------

Gražina

Nuoroda į tą patį srautą (grandininiam skaitymui)

Realizuota [studentas](#).

Naudojamas [operator>>\(\)](#).

7.4.3.11 tipas()

```
virtual std::string zmogus::tipas () const [pure virtual]
```

Gražina žmogaus tipo pavadinimą.

Grynai virtualus. Naudojamas diagnostikai ir logams.

Gražina

Tipo pavadinimas, pvz. "Studentas"

Realizuota [studentas](#).

Naudojamas [TEST_CASE\(\)](#), ir [testStudentasKlase\(\)](#).

7.4.4 Atributų Dokumentacija**7.4.4.1 vardas_**

```
std::string zmogus::vardas_ [protected]
```

Žmogaus vardas.

Apibrėžimas failo [zmogus.h](#) eilutėje 52.

Naudojamas [studentas::appendListTo\(\)](#), [studentas::isvalyk\(\)](#), [studentas::operator bool\(\)](#), [studentas::operator<\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [studentas::operator==\(\)](#), [studentas::parseFromLine\(\)](#), [pilnasVardas\(\)](#), [studentas::print\(\)](#), [studentas::readStudentas\(\)](#), [setVardas\(\)](#), [studentas::tuscias\(\)](#), [vardas\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), ir [~zmogus\(\)](#).

7.4.4.2 pavarde_

```
std::string zmogus::pavarde_ [protected]
```

Žmogaus pavardė

Apibrėžimas failo [zmogus.h](#) eilutėje 53.

Naudojamas [studentas::appendListTo\(\)](#), [studentas::isvalyk\(\)](#), [studentas::operator bool\(\)](#), [studentas::operator<\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [studentas::operator==\(\)](#), [studentas::parseFromLine\(\)](#), [pavarde\(\)](#), [pilnasVardas\(\)](#), [studentas::print\(\)](#), [studentas::readStudentas\(\)](#), [setPavarde\(\)](#), [studentas::tuscias\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), ir [~zmogus\(\)](#).

7.4.4.3 gyvuZmoniu

```
int zmogus::gyvuZmoniu = 0 [static]
```

Gyvuojančių objektų skaitiklis.

Didinamas kiekviename konstruktoriuje, mažinamas destruktoriuje. Naudojamas testuose objektų nutekėjimui aptikti.

Apibrėžimas failo [zmogus.h](#) eilutėje 121.

Naudojamas [TEST_CASE\(\)](#), [testStudentasKlase\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), [zmogus\(\)](#), ir [~zmogus\(\)](#).

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [OOP_3/zmogus.h](#)
- [OOP_3/zmogus.cpp](#)

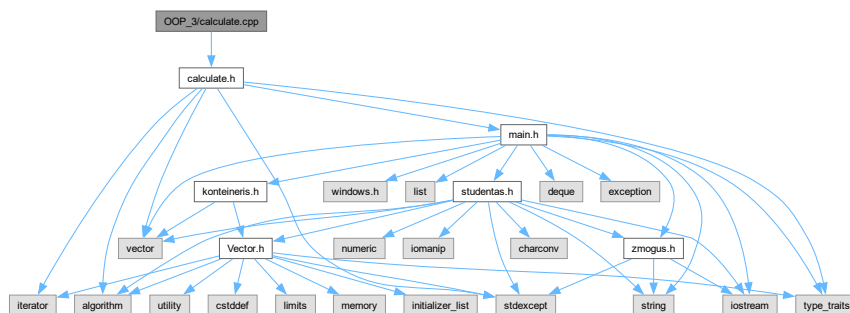
skyrius 8

Failo Dokumentacija

8.1 OOP_3/calculate.cpp Failo Nuoroda

```
#include "calculate.h"
```

Įtraukimo priklausomybių diagrama calculate.cpp:



Funkcijos

- double `mediana` (const `Vector`< int > &namuDarbai)

8.1.1 Funkcijos Dokumentacija

8.1.1.1 `mediana()`

```
double mediana (
    const Vector< int > & namuDarbai)
```

Apibrėžimas failo `calculate.cpp` eilutėje 3.

Ryšiai `Vector`< T, Allocator >::begin(), `Vector`< T, Allocator >::empty(), `Vector`< T, Allocator >::end(), `mediana()`, ir `Vector`< T, Allocator >::size().

Naudojamas `mediana()`.

8.2 calculate.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "calculate.h"
00002
00003 double mediana(const Vector<int>& namuDarbai)
00004 {
00005     double mediana{ 0.0 };
00006     if (!namuDarbai.empty())
00007     {
```

```

00008         Vector<int> sorted = namuDarbai;
00009         std::sort(sorted.begin(), sorted.end());
00010         size_t mid{ sorted.size() / 2 };
00011         mediana = (sorted.size() % 2 == 0) ? (sorted[mid - 1] + sorted[mid]) / 2.0 : sorted[mid];
00012         return mediana;
00013     }
00014     else
00015         return 0.0;
00016 }

```

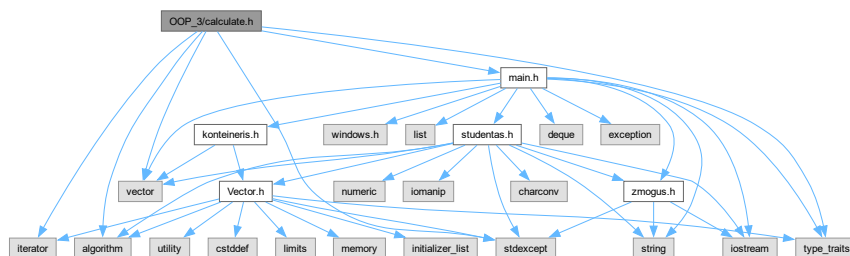
8.3 OOP_3/calculate.h Failo Nuoroda

```

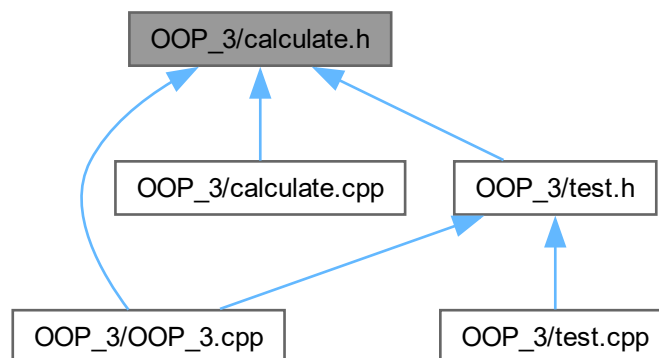
#include "main.h"
#include <vector>
#include <algorithm>
#include <iterator>
#include <type_traits>
#include <stdexcept>

```

Įtraukimo priklausomybių diagrama calculate.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- template<typename Container>
void **calculateGalutinis** (Container &studentai)
- template<typename Container>
void **sortStudentai** (Container &studentai, char sortBy)

- `template<typename Container>`
`splitResult< Container > splitStudentai (const Container &studentai)`
- `template<typename Container>`
`Container partitionStudentai (Container &studentai)`

8.3.1 Funkcijos Dokumentacija

8.3.1.1 calculateGalutinis()

```
template<typename Container>
void calculateGalutinis (
    Container & studentai)
```

Apibrėžimas failo `calculate.h` eilutėje 15.

Naudojamas `runProgram()`, `testContainer()`, ir `testData()`.

8.3.1.2 sortStudentai()

```
template<typename Container>
void sortStudentai (
    Container & studentai,
    char sortBy)
```

Apibrėžimas failo `calculate.h` eilutėje 28.

Ryšiai `comparePagalMediana()`, `comparePagalPavarde()`, `comparePagalVarda()`, ir `comparePagalVidurki()`.

Naudojamas `runProgram()`, `testContainer()`, ir `testData()`.

8.3.1.3 splitStudentai()

```
template<typename Container>
splitResult< Container > splitStudentai (
    const Container & studentai)
```

Apibrėžimas failo `calculate.h` eilutėje 54.

Ryšiai `splitResult< Container >::kietiakai`, ir `splitResult< Container >::vargsiukai`.

Naudojamas `testData()`.

8.3.1.4 partitionStudentai()

```
template<typename Container>
Container partitionStudentai (
    Container & studentai)
```

Apibrėžimas failo `calculate.h` eilutėje 73.

Naudojamas `runProgram()`, ir `testContainer()`.

8.4 calculate.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef CALCULATE_H_DEFINED
00002 #define CALCULATE_H_DEFINED
00003
00004 #include "main.h"
00005 #include <vector>           // vector
00006 #include <algorithm>       // sort, partition_copy, stable_partition
00007 #include <iterator>        // back_inserter, make_move_iterator
00008 #include <type_traits>     // is_same_v
00009 #include <stdexcept>       // runtime_error
00010
00011 // -----
00012 // Template: calculateGalutinis
00013 // -----
00014 template<typename Container>
00015 void calculateGalutinis(Container& studentai)
00016 {
00017     if (studentai.empty())
00018         throw std::runtime_error("Studentų sąrašas tuščias -- nėra ką skaičiuoti.");
00019 }
```

```

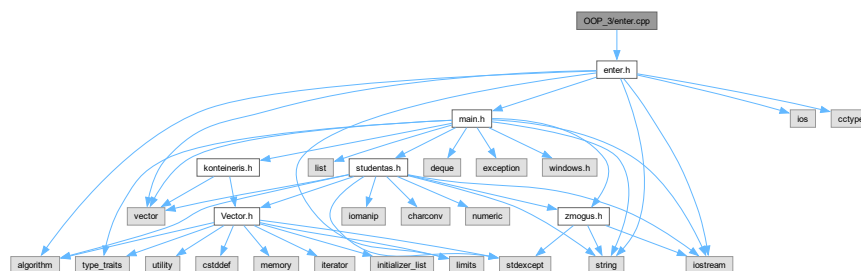
00020     for (auto& s : studentai)
00021         s.calculateGalutinis();
00022 }
00023
00024 // -----
00025 // Template: sortStudentai
00026 // -----
00027 template<typename Container>
00028 void sortStudentai(Container& studentai, char sortBy)
00029 {
00030     auto cmp = [sortBy](const studentas& a, const studentas& b) -> bool
00031     {
00032         switch (sortBy)
00033         {
00034             case '1': return comparePagalVarda(a, b);
00035             case '2': return comparePagalPavarde(a, b);
00036             case '3': return comparePagalVidurki(a, b);
00037             case '4': return comparePagalMediana(a, b);
00038             default: return false;
00039         }
00040     };
00041
00042     if constexpr (std::is_same_v<Container, std::list<studentas>>)
00043         studentai.sort(cmp); // list narys
00044     else
00045         std::sort(studentai.begin(), studentai.end(), cmp); // vector / deque
00046 }
00047
00048 // -----
00049 // Template: splitStudentai (1 strategija)
00050 // Skaido į DU naujus konteinerius, originalas nekeičiamas.
00051 // Neefektyvu atminties atžvilgiu -- duomenys dublikuojami.
00052 // -----
00053 template<typename Container>
00054 splitResult<Container> splitStudentai(const Container& studentai)
00055 {
00056     splitResult<Container> result;
00057
00058     std::partition_copy(studentai.begin(), studentai.end(),
00059         std::back_inserter(result.kietiakai),
00060         std::back_inserter(result.vargsiukai),
00061         [](const studentas& s) { return s.galutinisVid() >= 5.0; });
00062
00063     return result;
00064 }
00065
00066 // -----
00067 // Template: partitionStudentai (3 strategija -- optimaliausia)
00068 // Perskirsto originalų konteinerį vietoje, vargsiukus perkelia
00069 // į naują konteinerį ir blokiniu erase pašalina iš originalo.
00070 // Po iškvietimo studentai konteineryje lieka tik kietiakai.
00071 // -----
00072 template<typename Container>
00073 Container partitionStudentai(Container& studentai)
00074 {
00075     auto it = std::stable_partition(studentai.begin(), studentai.end(),
00076         [](const studentas& s) { return s.galutinisVid() >= 5.0; });
00077
00078     Container vargsiukai(std::make_move_iterator(it),
00079         std::make_move_iterator(studentai.end()));
00080     studentai.erase(it, studentai.end());
00081
00082     return vargsiukai;
00083 }
00084
00085 #endif // CALCULATE_H_DEFINED

```

8.5 OOP_3/enter.cpp Failo Nuoroda

```
#include "enter.h"
```

Įtraukimo priklausomybių diagrama enter.cpp:



Funkcijos

- `studentas enterStudentas (int n)`
- `std::string enterName (int n)`
- `std::string enterSurname (int n)`
- `int enterNumberOfPazymys (int n)`
- `int enterPazymys (int n)`
- `Vector< int > enterPazymiai (int n, int m)`
- `int enterEgzaminas (int n)`
- `bool isAllLetters (const std::string &input)`

8.5.1 Funkcijos Dokumentacija

8.5.1.1 enterStudentas()

```
studentas enterStudentas (
    int n)
```

Apibrėžimas failo `enter.cpp` eilutėje 7.

Ryšiai `enterEgzaminas()`, `enterName()`, `enterNumberOfPazymys()`, `enterPazymiai()`, ir `enterSurname()`.

Naudojamas `enterStudentai()`.

8.5.1.2 enterName()

```
std::string enterName (
    int n)
```

Apibrėžimas failo `enter.cpp` eilutėje 19.

Ryšiai `isAllLetters()`.

Naudojamas `enterStudentas()`, ir `generateOnlyPazymiai()`.

8.5.1.3 enterSurname()

```
std::string enterSurname (
    int n)
```

Apibrėžimas failo `enter.cpp` eilutėje 34.

Ryšiai `isAllLetters()`.

Naudojamas `enterStudentas()`, ir `generateOnlyPazymiai()`.

8.5.1.4 enterNumberOfPazymys()

```
int enterNumberOfPazymys (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 50.
Naudojamas [enterStudentas\(\)](#).

8.5.1.5 enterPazymys()

```
int enterPazymys (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 66.
Ryšiai [studentas::maxPazymys](#), ir [studentas::minPazymys](#).
Naudojamas [enterPazymiai\(\)](#).

8.5.1.6 enterPazymiai()

```
Vector< int > enterPazymiai (
    int n,
    int m)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 83.
Ryšiai [enterPazymys\(\)](#), ir [Vector< T, Allocator >::push_back\(\)](#).
Naudojamas [enterStudentas\(\)](#).

8.5.1.7 enterEgzaminas()

```
int enterEgzaminas (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 94.
Ryšiai [studentas::maxPazymys](#), ir [studentas::minPazymys](#).
Naudojamas [enterStudentas\(\)](#).

8.5.1.8 isAllLetters()

```
bool isAllLetters (
    const std::string & input)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 111.
Naudojamas [enterName\(\)](#), ir [enterSurname\(\)](#).

8.6 enter.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "enter.h"
00002
00003 // -----
00004 // enterStudentas -- surenka vieno studento duomenis ir sukuria
00005 // objektą per pilną konstruktorį (jis validuoja ir apskaičiuoja).
00006 // -----
00007 studentas enterStudentas(int n)
00008 {
00009     std::cout << '# ' << n << u8" studentas:\n";
00010
00011     std::string vardas = enterName(n);
00012     std::string pavarde = enterSurname(n);
00013     Vector<int> nd = enterPazymiai(n, enterNumberOfPazymys(n));
00014     int egzaminas = enterEgzaminas(n);
00015
00016     return studentas(vardas, pavarde, nd, egzaminas);
00017 }
00018
00019 std::string enterName(int n)
00020 {
00021     std::string v;
00022
00023     std::cout << u8"Iveskite studento #" << n << u8" vardą: ";
00024     while (!(std::cin > v) || !isAllLetters(v))
00025     {
```

```

00026         std::cin.clear();
00027         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00028         std::cout << u8"Naudokite raides. Iveskite studento vardą: ";
00029     }
00030
00031     return v;
00032 }
00033
00034 std::string enterSurname(int n)
00035 {
00036     std::string v;
00037
00038     std::cout << u8"Iveskite studento #" << n << u8" pavardę: ";
00039     while (!(std::cin > v) || !isAllLetters(v))
00040     {
00041         std::cin.clear();
00042         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00043         std::cout << u8"Naudokite raides.\n";
00044         std::cout << u8"Iveskite studento #" << n << u8" pavardę: ";
00045     }
00046
00047     return v;
00048 }
00049
00050 int enterNumberOfPazymys(int n)
00051 {
00052     int tmp{};
00053
00054     std::cout << u8"Iveskite kiek #" << n << u8" studentas turėjo namų darbų: ";
00055     while (!(std::cin > tmp) || tmp < 0)
00056     {
00057         std::cin.clear();
00058         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00059         std::cout << u8"Iveskite neneigiamą skaičių.\n";
00060         std::cout << u8"Iveskite kiek #" << n << u8" studentas turėjo namų darbų: ";
00061     }
00062
00063     return tmp;
00064 }
00065
00066 int enterPazymys(int n)
00067 {
00068     int tmp{};
00069
00070     std::cout << n << u8" pažymys: ";
00071     while (!(std::cin > tmp) || tmp < studentas::minPazymys || tmp > studentas::maxPazymys)
00072     {
00073         std::cin.clear();
00074         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00075         std::cout << u8"Iveskite skaičių nuo " << studentas::minPazymys
00076             << u8" iki " << studentas::maxPazymys << u8".\n";
00077         std::cout << n << u8" pažymys: ";
00078     }
00079
00080     return tmp;
00081 }
00082
00083 Vector<int> enterPazymiai(int n, int m) // n - studento numeris, m - pažymių skaičius
00084 {
00085     Vector<int> tmp;
00086     std::cout << u8"Iveskite #" << n << u8" studento namų darbų pažymius.\n";
00087     for (int i{ 0 }; i < m; i++)
00088     {
00089         tmp.push_back(enterPazymys(i + 1));
00090     }
00091     return tmp;
00092 }
00093
00094 int enterEgzaminas(int n)
00095 {
00096     int tmp{};
00097
00098     std::cout << u8"Iveskite kiek #" << n << u8" studentas gavo iš egzamino.\n";
00099     while (!(std::cin > tmp) || tmp < studentas::minPazymys || tmp > studentas::maxPazymys)
00100     {
00101         std::cin.clear();
00102         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00103         std::cout << u8"Iveskite skaičių nuo " << studentas::minPazymys
00104             << u8" iki " << studentas::maxPazymys << u8".\n";
00105         std::cout << u8"Iveskite kiek #" << n << u8" studentas gavo iš egzamino.\n";
00106     }
00107
00108     return tmp;
00109 }
00110
00111 bool isAllLetters(const std::string& input) {
00112     return std::all_of(input.begin(), input.end(), ::isalpha);

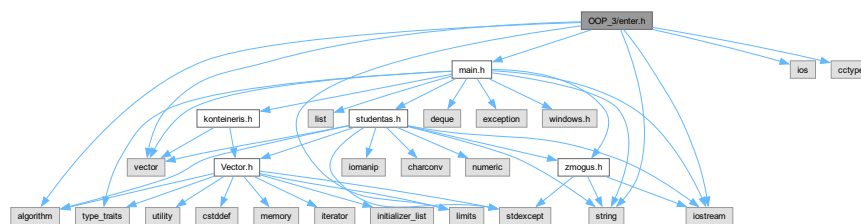
```

```
00113 }
```

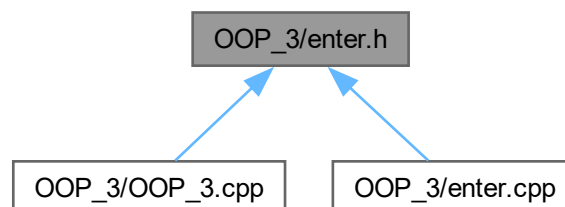
8.7 OOP_3/enter.h Failo Nuoroda

```
#include "main.h"
#include <iostream>
#include <limits>
#include <ios>
#include <vector>
#include <string>
#include <algorithm>
#include <cctype>
```

Įtraukimo priklausomybių diagrama enter.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
`Container enterStudentai ()`

8.7.1 Funkcijos Dokumentacija

8.7.1.1 enterStudentai()

```
template<typename Container>
```

```
Container enterStudentai ()
```

Apibrėžimas failo [enter.h](#) eilutėje 21.

Ryšiai [askIfMoreStudents\(\)](#), ir [enterStudentas\(\)](#).

Naudojamas [runProgram\(\)](#).

8.8 enter.h

Eiti į šio failo dokumentaciją.

```

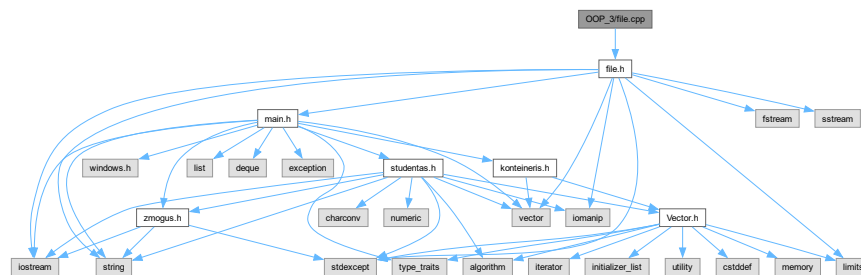
00001 #ifndef ENTER_H_DEFINED
00002 #define ENTER_H_DEFINED
00003
00004 #include "main.h"
00005 #undef max
00006 #include <iostream> // cout, cin
00007 #include <limits>   // numeric_limits
00008 #include <ios>      // streamsize
00009 #include <vector>   // vector
00010 #include <string>   // string
00011 #include <algorithm> // all_of
00012 #include <cctype>   // isalpha
00013
00014 // -----
00015 // Template: enterStudentai<Container>
00016 // Įveda studentus ranka į bet kokį konteinerį
00017 // (vector, list, deque) naudojant push_back.
00018 // -----
00019
00020 template<typename Container>
00021 Container enterStudentai()
00022 {
00023     Container tmp;
00024     int n{ 1 };
00025     do
00026     {
00027         tmp.push_back(enterStudentas(n));
00028         n++;
00029     } while (askIfMoreStudents());
00030
00031     return tmp;
00032 }
00033
00034 #endif // ENTER_H_DEFINED

```

8.9 OOP_3/file.cpp Failo Nuoroda

```
#include "file.h"
```

Įtraukimo priklausomybių diagrama file.cpp:



Funkcijos

- `std::string enterFileName ()`
- `std::string enterOutputFileName (std::string desc)`

8.9.1 Funkcijos Dokumentacija

8.9.1.1 enterFileName()

```
std::string enterFileName ()
```

Apibrėžimas failo `file.cpp` eilutėje 3.

Naudojamas `runProgram()`.

8.9.1.2 enterOutputFileName()

```
std::string enterOutputFileName (
    std::string desc)
```

Apibrėžimas failo [file.cpp](#) eilutėje 16.

Naudojamas [outputStudentai\(\)](#), ir [runProgram\(\)](#).

8.10 file.cpp

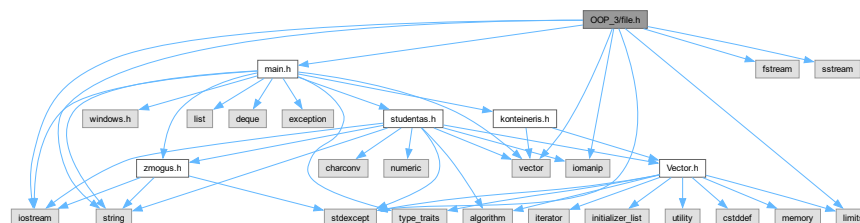
Eiti į šio failo dokumentaciją.

```
00001 #include "file.h"
00002
00003 std::string enterFileName()
00004 {
00005     std::string tmp;
00006     std::cout << u8"Iveskite failo pavadinimą (su .txt plėtiniu): ";
00007     while (!(std::cin >> tmp) || tmp.size() < 5 || tmp.substr(tmp.size() - 4) != ".txt")
00008     {
00009         std::cin.clear();
00010         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00011         std::cout << u8"Netinkama įvestis. Įveskite failo pavadinimą su .txt plėtiniu: ";
00012     }
00013     return tmp;
00014 }
00015
00016 std::string enterOutputFileName(std::string desc)
00017 {
00018     std::string tmp;
00019     std::cout << u8"Iveskite " << desc << u8" failo pavadinimą (su .txt plėtiniu): ";
00020     while (!(std::cin >> tmp) || tmp.size() < 5 || tmp.substr(tmp.size() - 4) != ".txt")
00021     {
00022         std::cin.clear();
00023         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00024         std::cout << u8"Netinkama įvestis. Įveskite išvesties failo pavadinimą su .txt plėtiniu: ";
00025     }
00026     return tmp;
00027 }
```

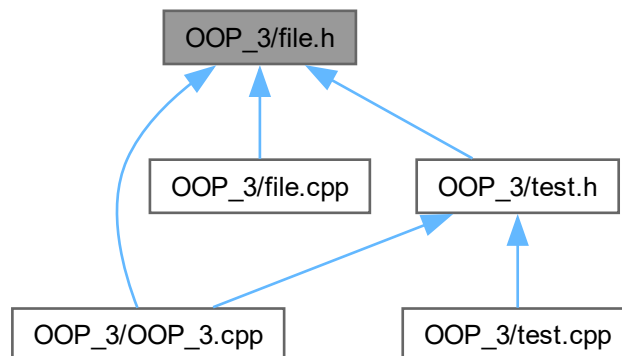
8.11 OOP_3/file.h Failo Nuoroda

```
#include "main.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <iomanip>
#include <limits>
#include <stdexcept>
```

Įtraukimo priklausomybių diagrama file.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
Container `readStudentaiFromFile` (const std::string &filename)
- `template<typename Container>`
void `writeStudentaiToFile` (const Container &studentai, const std::string &filename)
- `template<typename Container>`
void `writeStudentaiListToFile` (const Container &studentai, const std::string &filename)

Kintamieji

- `constexpr size_t FILE_BUFFER_SIZE = 64 * 1024`

8.11.1 Funkcijos Dokumentacija

8.11.1.1 readStudentaiFromFile()

```
template<typename Container>
Container readStudentaiFromFile (
    const std::string & filename)
```

Apibrėžimas failo `file.h` eilutėje 29.

Ryšiai `Vector< T, Allocator >::data()`, `FILE_BUFFER_SIZE`, `studentas::parseFromLine()`, ir `Vector< T, Allocator >::size()`.
Naudojamas `runProgram()`, `testContainer()`, ir `testData()`.

8.11.1.2 writeStudentaiToFile()

```
template<typename Container>
void writeStudentaiToFile (
    const Container & studentai,
    const std::string & filename)
```

Apibrėžimas failo `file.h` eilutėje 76.

Ryšiai `Vector< T, Allocator >::data()`, `FILE_BUFFER_SIZE`, ir `Vector< T, Allocator >::size()`.
Naudojamas `outputStudentai()`.

8.11.1.3 writeStudentaiListToFile()

```
template<typename Container>
void writeStudentaiListToFile (
    const Container & studentai,
    const std::string & filename)
```

Apibrėžimas failo [file.h](#) eilutėje 107.

Ryšiai [NUMBER_OF_PAZYMYS](#).

Naudojamas [runProgram\(\)](#), [testData\(\)](#), ir [testGenerateStudentai\(\)](#).

8.11.2 Kintamojo Dokumentacija

8.11.2.1 FILE_BUFFER_SIZE

```
size_t FILE_BUFFER_SIZE = 64 * 1024 [constexpr]
```

Apibrėžimas failo [file.h](#) eilutėje 16.

Naudojamas [readStudentaiFromFile\(\)](#), ir [writeStudentaiToFile\(\)](#).

8.12 file.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef FILE_H_DEFINED
00002 #define FILE_H_DEFINED
00003
00004 #include "main.h"
00005 #undef max
00006 #include <iostream> // cout, cin
00007 #include <fstream> // ifstream, ofstream
00008 #include <sstream> // istringstream
00009 #include <vector> // vector
00010 #include <string> // string
00011 #include <iomanip> // setprecision, setw
00012 #include <limits> // numeric_limits
00013 #include <stdexcept> // runtime_error
00014
00015 // Rašymo buferio dydis -- 64 KB.
00016 constexpr size_t FILE_BUFFER_SIZE = 64 * 1024;
00017
00018 // -----
00019 // Template: readStudentaiFromFile<Container>
00020 // Nuskaityti studentus iš .txt failo į bet kokį konteinerį.
00021 //
00022 // Optimizacijos:
00023 // - nenaudojamas std::istringstream (nekuriamas objektas eilutei)
00024 // - studentas::parseFromLine naudoja std::from_chars
00025 // - line buferis pernaudojamas visoms eilutėms
00026 // - studentas objektas kuriamas vietoje ir perkeliamas (move)
00027 // -----
00028 template<typename Container>
00029 Container readStudentaiFromFile(const std::string& filename)
00030 {
00031     Container studentai;
00032     std::ifstream file(filename);
00033
00034     if (!file.is_open())
00035         throw std::runtime_error("Nepavyko atidaryti failo: " + filename);
00036
00037     // Didesnis skaitymo buferis -- mažiau OS kvietimų
00038     Kontaineris<char> rdbuf(FILE_BUFFER_SIZE);
00039     file.rdbuf()->pubsetbuf(rdbuf.data(), rdbuf.size());
00040
00041     std::string line;
00042     line.reserve(256); // eilutės buferis pernaudojamas -- jokių perskirstymų
00043
00044     if (!std::getline(file, line))
00045         throw std::runtime_error("Failas tuščias arba sugadintas: " + filename);
00046
00047     // Apskaičiuojame ND stulpelių skaičių iš antraštės
00048     // (antraštė nuskaityta tik vieną kartą, todėl istringstream čia pigus)
00049     std::istringstream headerStream(line);
00050     std::string token;
00051     int ndCount = 0;
00052     while (headerStream >> token)
00053         if (token.find("ND") != std::string::npos) ndCount++;
00054
00055     studentas s; // objektas pernaudojamas -- nekuriamas naujas kiekvienai eilutei
00056     int lineNumber = 1;
```

```

00057
00058     while (std::getline(file, line))
00059     {
00060         lineNumber++;
00061         if (line.empty()) continue;
00062
00063         s.parseFromLine(line, ndCount, lineNumber);
00064         studentai.push_back(std::move(s));
00065     }
00066
00067     return studentai;
00068 }
00069
00070 // -----
00071 // Template: writeStudentaiToFile
00072 // Išveda studentus su galutiniais balais.
00073 // Formatavimą atlieka klasės operator«.
00074 // -----
00075 template<typename Container>
00076 void writeStudentaiToFile(const Container& studentai, const std::string& filename)
00077 {
00078     std::ofstream file(filename);
00079     if (!file.is_open())
00080         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00081
00082     Konteineris<char> wrbuf(FILE_BUFFER_SIZE);
00083     file.rdbuf()->pubsetbuf(wrbuf.data(), wrbuf.size());
00084
00085     file << u8"Vardas          Pavardė          Galutinis (Vid.)    Galutinis (Med.)\n";
00086     file << u8"-----\n";
00087
00088     for (const auto& s : studentai)
00089         file << s << "\n";
00090
00091     file.close();
00092 }
00093
00094 // -----
00095 // Template: writeStudentaiListToFile
00096 // Išveda studentus su visais namų darbų pažymiais.
00097 // Šis formatas suderinamas su readStudentaiFromFile skaitymui.
00098 //
00099 // Optimizacijos:
00100 // - eilutės formuojamos į std::string buferį (studentas::appendListTo)
00101 // - naudojant std::to_chars vietoj std::setw manipuliatorių
00102 // - buferis išvalomas kas ~1 MB -- atmintis NEauga proporcingai
00103 // - studentų skaičiui (skirtingai nei kaupiant viską į vieną string)
00104 // - vienas file.write() vietoj kelių « operacijų eilutei
00105 // -----
00106 template<typename Container>
00107 void writeStudentaiListToFile(const Container& studentai, const std::string& filename)
00108 {
00109     std::ofstream file(filename, std::ios::binary);
00110     if (!file.is_open())
00111         throw std::runtime_error("Nepavyko sukurti failo: " + filename);
00112
00113     // Antraštė
00114     std::string out;
00115     out.reserve(1024 * 1024 + 1024); // ~1 MB darbinis buferis
00116
00117     out.append("Vardas");
00118     out.append(25 - 6, ' ');
00119     out.append("Pavarde");
00120     out.append(25 - 7, ' ');
00121
00122     for (int i = 1; i <= NUMBER_OF_PAZYMYS; ++i)
00123     {
00124         std::string nd = "ND" + std::to_string(i);
00125         out.append(10 - nd.size(), ' ');
00126         out.append(nd);
00127     }
00128     out.append(10 - 9, ' ');
00129     out.append("Egzaminas");
00130     out.push_back('\n');
00131
00132     // Eilutės -- buferis išvalomas pasiekus ~1 MB ribą
00133     constexpr size_t FLUSH_LIMIT = 1024 * 1024;
00134
00135     for (const auto& s : studentai)
00136     {
00137         s.appendListTo(out);
00138
00139         if (out.size() >= FLUSH_LIMIT)
00140         {
00141             file.write(out.data(), static_cast<std::streamsize>(out.size()));
00142             out.clear(); // capacity išlieka -- jokių naujų alokacijų
00143         }
00144     }

```

```

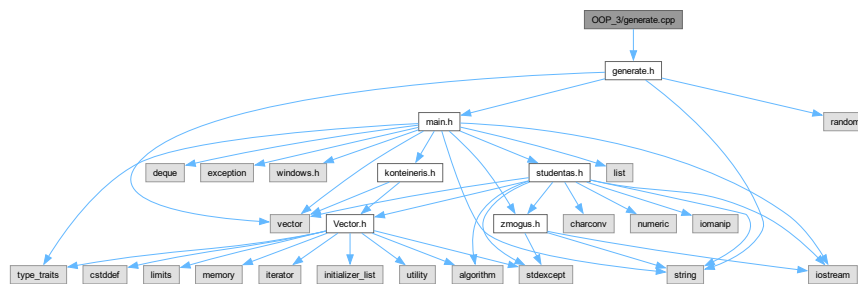
00144     }
00145
00146     // Likutis
00147     if (!out.empty())
00148         file.write(out.data(), static_cast<std::streamsize>(out.size()));
00149
00150     file.close();
00151 }
00152
00153 #endif // FILE_H_DEFINED

```

8.13 OOP_3/generate.cpp Failo Nuoroda

```
#include "generate.h"
```

Įtraukimo priklausomybių diagrama generate.cpp:



Funkcijos

- `std::mt19937 & getRng ()`
- `Vector< int > generatePazymiai (int m)`

8.13.1 Funkcijos Dokumentacija

8.13.1.1 getRng()

```
std::mt19937 & getRng ()
```

Apibrėžimas failo [generate.cpp](#) eilutėje 3.

Naudojamas [generatePazymiai\(\)](#).

8.13.1.2 generatePazymiai()

```
Vector< int > generatePazymiai (
    int m)
```

Apibrėžimas failo [generate.cpp](#) eilutėje 9.

Ryšiai [getRng\(\)](#), [studentas::maxPazymys](#), [studentas::minPazymys](#), [Vector< T, Allocator >::push_back\(\)](#), ir [Vector< T, Allocator >::reserve\(\)](#).

Naudojamas [generateOnlyPazymiai\(\)](#), ir [generateStudentai\(\)](#).

8.14 generate.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "generate.h"
00002
00003 std::mt19937& getRng()
00004 {
00005     static std::mt19937 rng(std::random_device{}());
00006     return rng;
00007 }
00008

```

```

00009 Vector<int> generatePazymiai(int m) // m - pažymių skaičius
00010 {
00011     Vector<int> tmp;
00012     tmp.reserve(m);
00013     std::uniform_int_distribution<int> dist(studentas::minPazymys, studentas::maxPazymys);
00014     std::mt19937& rng = getRng();
00015     for (int i{ 0 }; i < m; i++)
00016     {
00017         tmp.push_back(dist(rng));
00018     }
00019     return tmp;
00020 }

```

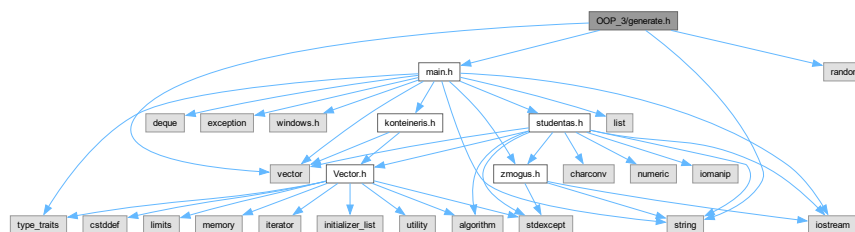
8.15 OOP_3/generate.h Failo Nuoroda

```

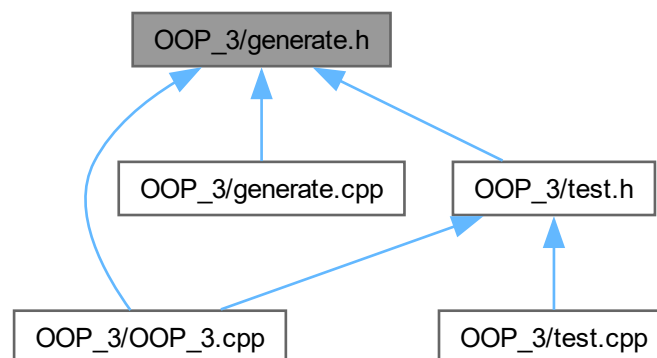
#include "main.h"
#include <vector>
#include <string>
#include <random>

```

Įtraukimo priklausomybių diagrama generate.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
Container `generateStudentai` (int n)
- `template<typename Container>`
Container `generateOnlyPazymiai` (int n)

8.15.1 Funkcijos Dokumentacija

8.15.1.1 generateStudentai()

```
template<typename Container>
Container generateStudentai (
    int n)
```

Apibrėžimas failo [generate.h](#) eilutėje 15.

Ryšiai [generatePazymiai\(\)](#), ir [NUMBER_OF_PAZYMYS](#).

Naudojamas [runProgram\(\)](#), ir [testGenerateStudentai\(\)](#).

8.15.1.2 generateOnlyPazymiai()

```
template<typename Container>
Container generateOnlyPazymiai (
    int n)
```

Apibrėžimas failo [generate.h](#) eilutėje 34.

Ryšiai [enterName\(\)](#), [enterSurname\(\)](#), [generatePazymiai\(\)](#), ir [NUMBER_OF_PAZYMYS](#).

Naudojamas [runProgram\(\)](#).

8.16 generate.h

Eiti į šio failo dokumentaciją.

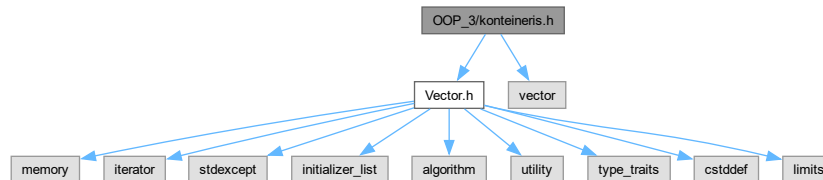
```
00001 #ifndef GENERATE_H_DEFINED
00002 #define GENERATE_H_DEFINED
00003
00004 #include "main.h"
00005 #include <vector> // vector
00006 #include <string> // string
00007 #include <random> // mt19937, random_device
00008
00009 // -----
00010 // Template: generateStudentai
00011 // Generuoja studentus su šabloniniais vardais (VardasNR1,
00012 // PavardėNR1, ...) ir atsitiktiniais pažymiais.
00013 // -----
00014 template<typename Container>
00015 Container generateStudentai(int n)
00016 {
00017     Container studentai;
00018     for (int i = 0; i < n; i++)
00019     {
00020         studentai.push_back(studentas(
00021             "VardasNR" + std::to_string(i + 1),
00022             "PavardėNR" + std::to_string(i + 1),
00023             generatePazymiai(NUMBER_OF_PAZYMYS),
00024             generatePazymiai(1)[0]));
00025     }
00026     return studentai;
00027 }
00028
00029 // -----
00030 // Template: generateOnlyPazymiai
00031 // Vardai ir pavardės įvedami rankiniu būdu, pažymiai generuojami.
00032 // -----
00033 template<typename Container>
00034 Container generateOnlyPazymiai(int n)
00035 {
00036     Container studentai;
00037     for (int i = 0; i < n; i++)
00038     {
00039         studentai.push_back(studentas(
00040             enterName(i + 1),
00041             enterSurname(i + 1),
00042             generatePazymiai(NUMBER_OF_PAZYMYS),
00043             generatePazymiai(1)[0]));
00044     }
00045     return studentai;
00046 }
00047
00048 #endif // GENERATE_H_DEFINED
```


8.17 OOP_3/konteineris.h Failo Nuoroda

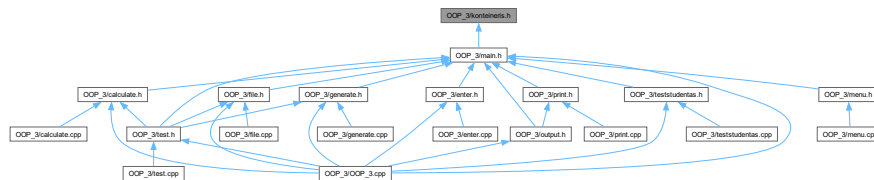
Konteinerio tipo perjungimas tarp `std::vector` ir nuosavos `Vector`.

```
#include "Vector.h"
#include <vector>
```

Įtraukimo priklausomybių diagrama `konteineris.h`:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Tipų apibrėžimai

- `template<typename T>`
`using Konteineris = Vector<T>`
Naudojamo konteinerio šablonas.

Funkcijos

- `const char * konteinerioPavadinimas ()`
Gražina naudojamo konteinerio pavadinimą (diagnostikai).

8.17.1 Smulkus aprašymas

Konteinerio tipo perjungimas tarp `std::vector` ir nuosavos `Vector`.

Leidžia visą programą perkompiliuoti su kitu konteineriu pakeitus vieną `#define` arba kompiliavimo parametą — taip galima palyginti abiejų realizacijų spartą tomis pačiomis sąlygomis.

Naudojimas

Pagal nutylėjimą naudojama nuosava `Vector`. Norint grįžti prie `std::vector`, kompiliuoti su:

```
g++ -DNAUDOTI_STD_VECTOR ...
```

arba `CMakeLists.txt` faile įjungti `NAUDOTI_STD_VECTOR` parametą.

Pavyzdys

```
#include "konteineris.h"

StudentuVektorius studentai; // Vector<studentas> arba std::vector<studentas>
studentai.push_back(s);

std::cout << konteinerioPavadinimas() << "\n"; // "Vector" arba "std::vector"
```

Apibrėžimas faile `konteineris.h`.

8.17.2 Tipų apibrėžimų Dokumentacija

8.17.2.1 Konteineris

```
template<typename T>
```

```
using Konteineris = Vector<T>
```

Naudojamo konteinerio šablonas.

Apibrėžimas failo [konteineris.h](#) eilutėje 47.

8.17.3 Funkcijos Dokumentacija

8.17.3.1 konteinerioPavadinimas()

```
const char * konteinerioPavadinimas () [inline]
```

Gražina naudojamo konteinerio pavadinimą (diagnostikai).

Apibrėžimas failo [konteineris.h](#) eilutėje 50.

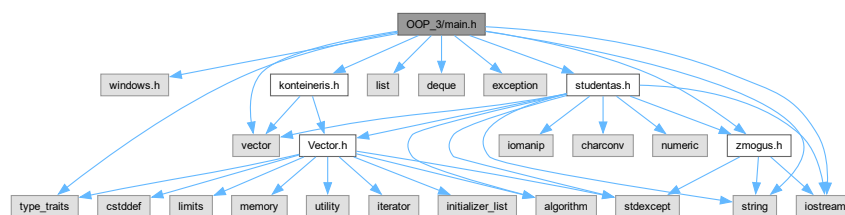
8.18 konteineris.h

Eiti į šio failo dokumentaciją.

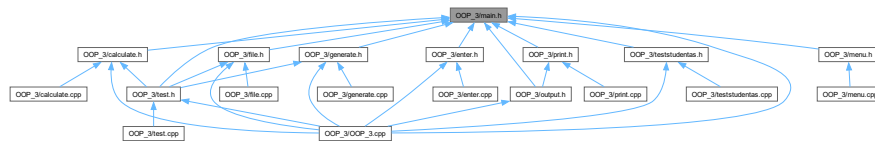
```
00001 #ifndef KONTEINERIS_H_DEFINED
00002 #define KONTEINERIS_H_DEFINED
00003
00004 #include "Vector.h"
00005 #include <vector>
00006
00007 /**
00008  * @file konteineris.h
00009  * @brief Konteinerio tipo perjungimas tarp std::vector ir nuosavos Vector.
00010  *
00011  * Leidžia visą programą perkompiliuoti su kitu konteineriu pakeitus
00012  * vieną '#define' arba kompiliavimo parametą -- taip galima palyginti
00013  * abiejų realizacijų spartą tomis pačiomis sąlygomis.
00014  *
00015  * @par Naudojimas
00016  * Pagal nutylėjimą naudojama nuosava Vector. Norint grįžti prie
00017  * std::vector, kompiliuoti su:
00018  * @code
00019  * g++ -DNAUDOTI_STD_VECTOR ...
00020  * @endcode
00021  * arba 'CMakeLists.txt' faile įjungti 'NAUDOTI_STD_VECTOR' parametą.
00022  *
00023  * @par Pavyzdys
00024  * @code
00025  * #include "konteineris.h"
00026  *
00027  * StudentuVektorius studentai; // Vector<studentas> arba std::vector<studentas>
00028  * studentai.push_back(s);
00029  *
00030  * std::cout << konteinerioPavadinimas() << "\n"; // "Vector" arba "std::vector"
00031  * @endcode
00032  */
00033
00034 #ifdef NAUDOTI_STD_VECTOR
00035
00036 /// Naudojamo konteinerio šablonas
00037 template <typename T>
00038 using Konteineris = std::vector<T>;
00039
00040 /// @brief Gražina naudojamo konteinerio pavadinimą (diagnostikai)
00041 inline const char* konteinerioPavadinimas() { return "std::vector"; }
00042
00043 #else
00044
00045 /// Naudojamo konteinerio šablonas
00046 template <typename T>
00047 using Konteineris = Vector<T>;
00048
00049 /// @brief Gražina naudojamo konteinerio pavadinimą (diagnostikai)
00050 inline const char* konteinerioPavadinimas() { return "Vector (nuosava)"; }
00051
00052 #endif
00053
00054 #endif // KONTEINERIS_H_DEFINED
```

8.19 OOP_3/main.h Failo Nuoroda

```
#include <windows.h>
#include <iostream>
#include <string>
#include <vector>
#include <list>
#include <deque>
#include <exception>
#include <type_traits>
#include "zmogus.h"
#include "studentas.h"
#include "konteineris.h"
Įtraukimo priklausomybių diagrama main.h:
```



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- struct `splitResult< Container >`

Apibrėžimai

- `#define` `NUMBER_OF_PAZYMYS` 15

Funkcijos

- char `askMenuChoice` ()
- int `enterNumberOfStudents` ()
- char `askSortBy` ()
- bool `askIfMoreStudents` ()
- char `askOutputChoice` ()
- char `askContainerChoice` ()
- `Vector< int >` `generatePazymiai` (int m)
- void `printWelcome` ()
- void `printNameAsk` ()
- `studentas` `enterStudentas` (int n)
- `std::string` `enterName` (int n)
- `std::string` `enterSurname` (int n)

- int `enterNumberOfPazymys` (int n)
- int `enterPazymys` (int n)
- `Vector< int > enterPazymiai` (int n, int m)
- int `enterEgzaminas` (int n)
- std::string `enterFileName` ()
- std::string `enterOutputFileName` (std::string desc="išvesties")
- void `testContainers` (int n)
- bool `testStudentasKlase` ()
- bool `isAllLetters` (const std::string &input)

8.19.1 Apibrėžimų Dokumentacija

8.19.1.1 NUMBER_OF_PAZYMYS

```
#define NUMBER_OF_PAZYMYS 15
```

Apibrėžimas failo `main.h` eilutėje 19.

Naudojamas `generateOnlyPazymiai()`, `generateStudentai()`, ir `writeStudentaiListToFile()`.

8.19.2 Funkcijos Dokumentacija

8.19.2.1 askMenuChoice()

```
char askMenuChoice ()
```

Apibrėžimas failo `menu.cpp` eilutėje 3.

Naudojamas `runProgram()`.

8.19.2.2 enterNumberOfStudents()

```
int enterNumberOfStudents ()
```

Apibrėžimas failo `menu.cpp` eilutėje 27.

Naudojamas `runProgram()`.

8.19.2.3 askSortBy()

```
char askSortBy ()
```

Apibrėžimas failo `menu.cpp` eilutėje 41.

Naudojamas `runProgram()`.

8.19.2.4 askIfMoreStudents()

```
bool askIfMoreStudents ()
```

Apibrėžimas failo `menu.cpp` eilutėje 73.

Naudojamas `enterStudentai()`.

8.19.2.5 askOutputChoice()

```
char askOutputChoice ()
```

Apibrėžimas failo `menu.cpp` eilutėje 87.

Naudojamas `runProgram()`.

8.19.2.6 askContainerChoice()

```
char askContainerChoice ()
```

Apibrėžimas failo `menu.cpp` eilutėje 103.

Naudojamas `main()`.

8.19.2.7 generatePazymiai()

```
Vector< int > generatePazymiai (
    int m)
```

Apibrėžimas failo [generate.cpp](#) eilutėje 9.

Ryšiai [getRng\(\)](#), [studentas::maxPazymys](#), [studentas::minPazymys](#), [Vector< T, Allocator >::push_back\(\)](#), ir [Vector< T, Allocator >::reserve\(\)](#).

Naudojamas [generateOnlyPazymiai\(\)](#), ir [generateStudentai\(\)](#).

8.19.2.8 printWelcome()

```
void printWelcome ()
```

Apibrėžimas failo [print.cpp](#) eilutėje 3.

Naudojamas [main\(\)](#).

8.19.2.9 printNameAsk()

```
void printNameAsk ()
```

Apibrėžimas failo [print.cpp](#) eilutėje 10.

8.19.2.10 enterStudentas()

```
studentas enterStudentas (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 7.

Ryšiai [enterEgzaminas\(\)](#), [enterName\(\)](#), [enterNumberOfPazymys\(\)](#), [enterPazymiai\(\)](#), ir [enterSurname\(\)](#).

Naudojamas [enterStudentai\(\)](#).

8.19.2.11 enterName()

```
std::string enterName (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 19.

Ryšiai [isAllLetters\(\)](#).

Naudojamas [enterStudentas\(\)](#), ir [generateOnlyPazymiai\(\)](#).

8.19.2.12 enterSurname()

```
std::string enterSurname (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 34.

Ryšiai [isAllLetters\(\)](#).

Naudojamas [enterStudentas\(\)](#), ir [generateOnlyPazymiai\(\)](#).

8.19.2.13 enterNumberOfPazymys()

```
int enterNumberOfPazymys (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 50.

Naudojamas [enterStudentas\(\)](#).

8.19.2.14 enterPazymys()

```
int enterPazymys (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 66.

Ryšiai [studentas::maxPazymys](#), ir [studentas::minPazymys](#).

Naudojamas [enterPazymiai\(\)](#).

8.19.2.15 enterPazymiai()

```
Vector< int > enterPazymiai (
    int n,
    int m)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 83.

Ryšiai [enterPazymys\(\)](#), ir [Vector< T, Allocator >::push_back\(\)](#).

Naudojamas [enterStudentas\(\)](#).

8.19.2.16 enterEgzaminas()

```
int enterEgzaminas (
    int n)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 94.

Ryšiai [studentas::maxPazymys](#), ir [studentas::minPazymys](#).

Naudojamas [enterStudentas\(\)](#).

8.19.2.17 enterFileName()

```
std::string enterFileName ()
```

Apibrėžimas failo [file.cpp](#) eilutėje 3.

Naudojamas [runProgram\(\)](#).

8.19.2.18 enterOutputFileName()

```
std::string enterOutputFileName (
    std::string desc = "išvesties")
```

Apibrėžimas failo [file.cpp](#) eilutėje 16.

Naudojamas [outputStudentai\(\)](#), ir [runProgram\(\)](#).

8.19.2.19 testContainers()

```
void testContainers (
    int n)
```

Apibrėžimas failo [test.cpp](#) eilutėje 50.

Ryšiai [testContainer\(\)](#).

Naudojamas [runProgram\(\)](#).

8.19.2.20 testStudentasKlase()

```
bool testStudentasKlase ()
```

Apibrėžimas failo [teststudentas.cpp](#) eilutėje 45.

Ryšiai [studentas::addPazymys\(\)](#), [studentas::appendListTo\(\)](#), [Vector< T, Allocator >::back\(\)](#), [Vector< T, Allocator >::begin\(\)](#), [studentas::calculateGalutinis\(\)](#), [studentas::egzaminas\(\)](#), [Vector< T, Allocator >::empty\(\)](#), [Vector< T, Allocator >::end\(\)](#), [zmogus::galutinis\(\)](#), [studentas::galutinisMed\(\)](#), [studentas::galutinisVid\(\)](#), [studentas::gyvuStudentu](#), [zmogus::gyvuZmoniu](#), [studentas::isvalyk\(\)](#), [studentas::nd\(\)](#), [studentas::parseFromLine\(\)](#), [zmogus::pavarde\(\)](#), [zmogus::pilnasVardas\(\)](#), [Vector< T, Allocator >::push_back\(\)](#), [Vector< T, Allocator >::reserve\(\)](#), [studentas::setEgzaminas\(\)](#), [studentas::setNd\(\)](#), [zmogus::setPavarde\(\)](#), [zmogus::setVardas\(\)](#), [Vector< T, Allocator >::size\(\)](#), [zmogus::tipas\(\)](#), ir [zmogus::vardas\(\)](#).

Naudojamas [runProgram\(\)](#).

8.19.2.21 isAllLetters()

```
bool isAllLetters (
    const std::string & input)
```

Apibrėžimas failo [enter.cpp](#) eilutėje 111.

Naudojamas [enterName\(\)](#), ir [enterSurname\(\)](#).

8.20 main.h

Eiti į šio failo dokumentaciją.

```

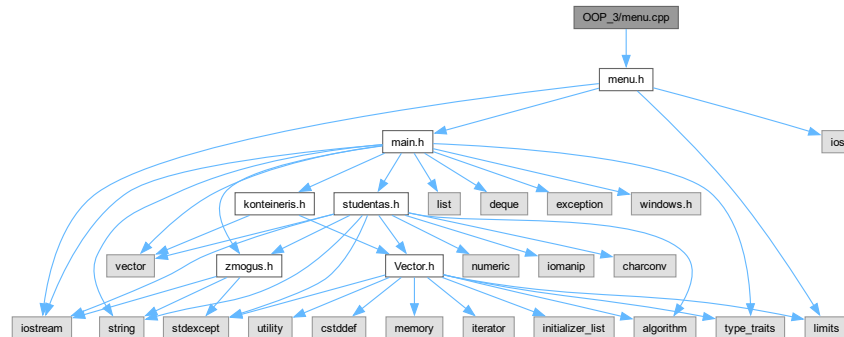
00001 #ifndef _WIN32
00002 #include <windows.h>
00003 #endif
00004
00005 #ifndef MAIN_H_DEFINED
00006 #define MAIN_H_DEFINED
00007
00008 #include <iostream>      // cout, cin
00009 #include <string>        // string
00010 #include <vector>        // vector
00011 #include <list>          // list
00012 #include <deque>         // deque
00013 #include <exception>     // exception
00014 #include <type_traits>    // is_abstract, is_base_of, is_nothrow_move_constructible
00015 #include "zmogus.h"      // abstrakti bazinė klasė
00016 #include "studentas.h"   // išvestinė klasė
00017 #include "konteineris.h"
00018
00019 #define NUMBER_OF_PAZYMYS 15
00020
00021 // -----
00022 // splitResult -- 1 strategijos rezultatas (du nauji konteineriai)
00023 // -----
00024 template<typename Container>
00025 struct splitResult
00026 {
00027     Container kietiakai;
00028     Container vargsiukai;
00029 };
00030
00031 // Function declarations
00032
00033 // Menu functions
00034 char askMenuChoice();
00035 int enterNumberOfStudents();
00036 char askSortBy();
00037 bool askIfMoreStudents();
00038 char askOutputChoice();
00039 char askContainerChoice();
00040
00041 // Generate functions
00042 Vector<int> generatePazymiai(int m);
00043
00044 // Print functions
00045 void printWelcome();
00046 void printNameAsk();
00047
00048 // User input functions
00049 studentas enterStudentas(int n);
00050 std::string enterName(int n);
00051 std::string enterSurname(int n);
00052 int enterNumberOfPazymys(int n);
00053 int enterPazymys(int n);
00054 Vector<int> enterPazymiai(int n, int m);
00055 int enterEgzaminas(int n);
00056
00057 // File handling functions
00058 std::string enterFileName();
00059 std::string enterOutputFileName(std::string desc = "išvesties");
00060
00061 // Test functions
00062 void testContainers(int n);
00063 bool testStudentasKlase();      // klasių metodų testas (v1.2 / v1.5)
00064
00065 // Utility functions
00066 bool isAllLetters(const std::string& input);
00067
00068 #endif

```

8.21 OOP_3/menu.cpp Failo Nuoroda

```
#include "menu.h"
```

Įtraukimo priklausomybių diagrama menu.cpp:



Funkcijos

- char [askMenuChoice](#) ()
- int [enterNumberOfStudents](#) ()
- char [askSortBy](#) ()
- char [askAverageOrMedian](#) ()
- bool [askIfMoreStudents](#) ()
- char [askOutputChoice](#) ()
- char [askContainerChoice](#) ()

8.21.1 Funkcijos Dokumentacija

8.21.1.1 askMenuChoice()

```
char askMenuChoice ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 3.

Naudojamas [runProgram\(\)](#).

8.21.1.2 enterNumberOfStudents()

```
int enterNumberOfStudents ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 27.

Naudojamas [runProgram\(\)](#).

8.21.1.3 askSortBy()

```
char askSortBy ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 41.

Naudojamas [runProgram\(\)](#).

8.21.1.4 askAverageOrMedian()

```
char askAverageOrMedian ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 59.

8.21.1.5 askIfMoreStudents()

```
bool askIfMoreStudents ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 73.

Naudojamas [enterStudentai\(\)](#).

8.21.1.6 askOutputChoice()

```
char askOutputChoice ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 87.

Naudojamas [runProgram\(\)](#).

8.21.1.7 askContainerChoice()

```
char askContainerChoice ()
```

Apibrėžimas failo [menu.cpp](#) eilutėje 103.

Naudojamas [main\(\)](#).

8.22 menu.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "menu.h"
00002
00003 char askMenuChoice()
00004 {
00005     char choice;
00006     std::cout << u8"Pasirinkite veiksmą:\n";
00007     std::cout << u8"1. Įvesti studentus ranka\n";
00008     std::cout << u8"2. Generuoti tik pažymius\n";
00009     std::cout << u8"3. Generuoti studentų vardus ir pažymius\n";
00010     std::cout << u8"4. Nuskaityti studentus iš failo\n";
00011     std::cout << u8"5. Generuoti studentų failą\n";
00012     std::cout << u8"6. Studentų failų generavimo testavimas\n";
00013     std::cout << u8"7. Duomenų apdorojimo testavimas\n";
00014     std::cout << u8"8. Konteinerių palyginimo testavimas\n";
00015     std::cout << u8"9. Klasės studentas metodų testavimas\n";
00016     std::cout << u8"0. Baigti darbą\n";
00017     std::cout << u8"Jūsų pasirinkimas: ";
00018     do
00019     {
00020         std::cin >> choice;
00021         if (choice < '0' || choice > '9')
00022             std::cout << u8"Netinkama įvestis, bandykite dar kartą: ";
00023     } while (choice < '0' || choice > '9');
00024     return choice;
00025 }
00026
00027 int enterNumberOfStudents()
00028 {
00029     int n;
00030     std::cout << u8"Įveskite studentų skaičių: ";
00031     while (!(std::cin >> n) || n <= 0)
00032     {
00033         std::cin.clear();
00034         std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00035         std::cout << u8"Įveskite teigiamą skaičių.\n";
00036         std::cout << u8"Įveskite studentų skaičių: ";
00037     }
00038     return n;
00039 }
00040
00041 char askSortBy()
00042 {
00043     char choice;
00044     std::cout << u8"Rikiavimas pagal:\n"
00045         << u8"1. Vardą\n"
00046         << u8"2. Pavardę\n"
00047         << u8"3. Galutinį įvertinimą pagal vidurkį\n"
00048         << u8"4. Galutinį įvertinimą pagal medianą\n"
00049         << u8"Jūsų pasirinkimas: ";
00050     do
00051     {
00052         std::cin >> choice;
00053         if (choice < '1' || choice > '4')
00054             std::cout << u8"Netinkama įvestis, bandykite dar kartą: ";
00055     } while (choice < '1' || choice > '4');
00056     return choice;
```

```

00057 }
00058
00059 char askAverageOrMedian()
00060 {
00061     char choice;
00062     std::cout << u8"Ar norite naudoti vidurkį (v) ar medianą (m) galutiniam įvertinimui apskaičiuoti?
(v/m): ";
00063     do
00064     {
00065         std::cin >> choice;
00066         choice = static_cast<char>(std::tolower(static_cast<unsigned char>(choice)));
00067         if (choice != 'v' && choice != 'm')
00068             std::cout << u8"Netinkama įvestis, bandykite 'v' arba 'm': ";
00069     } while (choice != 'v' && choice != 'm');
00070     return choice;
00071 }
00072
00073 bool askIfMoreStudents()
00074 {
00075     char choice;
00076     std::cout << u8"Ar norite įvesti dar vieną studentą? (y/n): ";
00077     do
00078     {
00079         std::cin >> choice;
00080         choice = static_cast<char>(std::tolower(static_cast<unsigned char>(choice)));
00081         if (choice != 'y' && choice != 'n')
00082             std::cout << u8"Netinkama įvestis, bandykite 'y' arba 'n': ";
00083     } while (choice != 'y' && choice != 'n');
00084     return choice == 'y';
00085 }
00086
00087 char askOutputChoice()
00088 {
00089     char choice;
00090     std::cout << u8"Pasirinkite išvesties formatą:\n"
00091     << u8"1. Išvesti į konsolę\n"
00092     << u8"2. Išvesti į failą\n"
00093     << u8"Jūsų pasirinkimas: ";
00094     do
00095     {
00096         std::cin >> choice;
00097         if (choice < '1' || choice > '2')
00098             std::cout << u8"Netinkama įvestis, bandykite dar kartą: ";
00099     } while (choice < '1' || choice > '2');
00100     return choice;
00101 }
00102
00103 char askContainerChoice()
00104 {
00105     char choice;
00106     std::cout << u8"Pasirinkite konteinerį:\n"
00107     << u8"1. vector\n"
00108     << u8"2. list\n"
00109     << u8"3. deque\n"
00110     << u8"Jūsų pasirinkimas: ";
00111     do
00112     {
00113         std::cin >> choice;
00114         if (choice < '1' || choice > '3')
00115             std::cout << u8"Netinkama įvestis, bandykite dar kartą: ";
00116     } while (choice < '1' || choice > '3');
00117     return choice;
00118 }

```

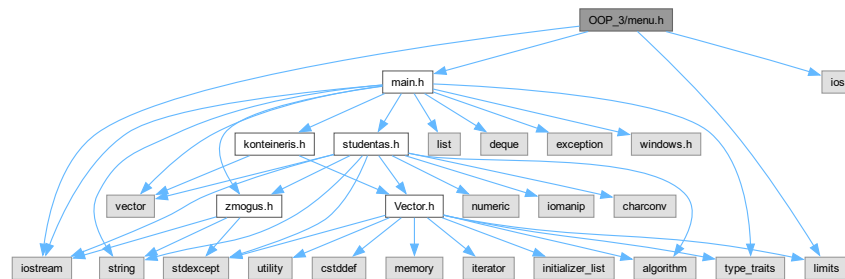
8.23 OOP_3/menu.h Failo Nuoroda

```

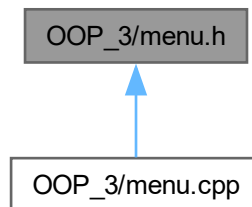
#include "main.h"
#include <iostream>
#include <limits>
#include <ios>

```

Įtraukimo priklausomybių diagrama menu.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



8.24 menu.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #ifndef MENU_H_DEFINED
00002 #define MENU_H_DEFINED
00003
00004 #include "main.h"
00005 #undef max
00006 #include <iostream> // cout, cin
00007 #include <limits>   // numeric_limits
00008 #include <ios>      // streamsize
00009
00010 #endif // MENU_H_DEFINED
  
```

8.25 OOP_3/OOP_3.cpp Failo Nuoroda

```

#include "main.h"
#include "calculate.h"
#include "file.h"
#include "enter.h"
#include "generate.h"
#include "output.h"
#include "test.h"
#include "testStudentas.h"
  
```



```

00028         std::cout << u8"Programa baigta.\n";
00029         return;
00030     case '1':
00031         studentai = enterStudentai<Container>();
00032         calculateGalutinis(studentai);
00033         sortChoice = askSortBy();
00034         sortStudentai<Container>(studentai, sortChoice);
00035         vargsiukai = partitionStudentai(studentai);
00036         outputChoice = askOutputChoice();
00037         outputStudentai<Container>(studentai, outputChoice, "kietiakai");
00038         outputStudentai<Container>(vargsiukai, outputChoice, "vargsiukai");
00039         break;
00040     case '2':
00041         studentai = generateOnlyPazymiai<Container>(enterNumberOfStudents());
00042         calculateGalutinis(studentai);
00043         sortChoice = askSortBy();
00044         sortStudentai<Container>(studentai, sortChoice);
00045         vargsiukai = partitionStudentai(studentai);
00046         outputChoice = askOutputChoice();
00047         outputStudentai<Container>(studentai, outputChoice, "kietiakai");
00048         outputStudentai<Container>(vargsiukai, outputChoice, "vargsiukai");
00049         break;
00050     case '3':
00051         studentai = generateStudentai<Container>(enterNumberOfStudents());
00052         calculateGalutinis(studentai);
00053         sortChoice = askSortBy();
00054         sortStudentai<Container>(studentai, sortChoice);
00055         vargsiukai = partitionStudentai(studentai);
00056         outputChoice = askOutputChoice();
00057         outputStudentai<Container>(studentai, outputChoice, "kietiakai");
00058         outputStudentai<Container>(vargsiukai, outputChoice, "vargsiukai");
00059         break;
00060     case '4':
00061         studentai = readStudentaiFromFile<Container>(enterFileName());
00062         calculateGalutinis(studentai);
00063         sortChoice = askSortBy();
00064         sortStudentai<Container>(studentai, sortChoice);
00065         vargsiukai = partitionStudentai(studentai);
00066         outputChoice = askOutputChoice();
00067         outputStudentai<Container>(studentai, outputChoice, "kietiakai");
00068         outputStudentai<Container>(vargsiukai, outputChoice, "vargsiukai");
00069         break;
00070     case '5':
00071         studentai = generateStudentai<Container>(enterNumberOfStudents());
00072         writeStudentaiListToFile<Container>(studentai, enterOutputFileName());
00073         break;
00074     case '6':
00075         testGenerateStudentai<Container>(1000);
00076         testGenerateStudentai<Container>(10000);
00077         testGenerateStudentai<Container>(100000);
00078         testGenerateStudentai<Container>(1000000);
00079         testGenerateStudentai<Container>(10000000);
00080         break;
00081     case '7':
00082         testData<Container>(100000);
00083         testData<Container>(1000000);
00084         testData<Container>(10000000);
00085         break;
00086     case '8':
00087         testContainers(1000);
00088         testContainers(10000);
00089         testContainers(100000);
00090         testContainers(1000000);
00091         testContainers(10000000);
00092         break;
00093     case '9':
00094         // Klasės studentas metodų testas (v1.2)
00095         if (testStudentasKlase())
00096             std::cout << u8"Visi klases testai praejo sekmingai.\n\n";
00097         else
00098             std::cout << u8"DEMESIO: kai kurie klases testai nepraejo!\n\n";
00099         break;
00100     default:
00101         std::cout << u8"Neteisingas pasirinkimas. Bandykite dar kartą.\n";
00102         break;
00103     }
00104 }
00105 }
00106
00107 int main()
00108 {
00109     #ifdef _WIN32
00110         SetConsoleOutputCP(CP_UTF8);
00111         SetConsoleCP(CP_UTF8);
00112     #endif
00113
00114     printWelcome();

```



```
void outputStudentai (const Container &studentai, char choice, std::string desc="išvesties")
```

8.27.1 Funkcijos Dokumentacija

8.27.1.1 outputStudentai()

```
template<typename Container>
void outputStudentai (
    const Container & studentai,
    char choice,
    std::string desc = "išvesties")
```

Apibrėžimas failo [output.h](#) eilutėje 14.

Ryšiai [enterOutputFileName\(\)](#), [printStudentai\(\)](#), ir [writeStudentaiToFile\(\)](#).

Naudojamas [runProgram\(\)](#).

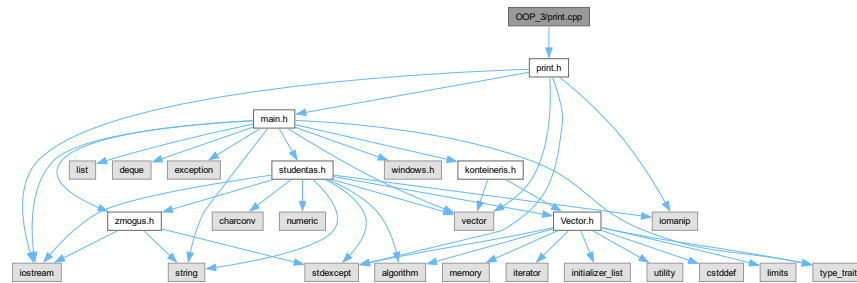
8.28 output.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef OUTPUT_H_DEFINED
00002 #define OUTPUT_H_DEFINED
00003
00004 #include "main.h"
00005 #include "print.h"
00006
00007 #include <stdexcept> // runtime_error
00008
00009 // -----
00010 // Template: outputStudentai
00011 // Išveda studentus į konsolę arba į failą, priklausomai nuo vartotojo pasirinkimo.
00012 // -----
00013 template<typename Container>
00014 void outputStudentai(const Container& studentai, char choice, std::string desc = "išvesties")
00015 {
00016     switch (choice)
00017     {
00018     case '1':
00019         printStudentai<Container>(studentai);
00020         break;
00021     case '2':
00022     {
00023         std::string outputFileName = enterOutputFileName(desc);
00024         writeStudentaiToFile<Container>(studentai, outputFileName);
00025     }
00026     break;
00027     default:
00028         throw std::runtime_error("Neteisingas pasirinkimas išvesties formatui.");
00029     }
00030 }
00031
00032 #endif // OUTPUT_H_DEFINED
```

8.29 OOP_3/print.cpp Failo Nuoroda

#include "print.h"
Įtraukimo priklausomybių diagrama print.cpp:



Funkcijos

- void [printWelcome\(\)](#)
- void [printNameAsk\(\)](#)

8.29.1 Funkcijos Dokumentacija

8.29.1.1 printWelcome()

void printWelcome ()
Apibrėžimas failo [print.cpp](#) eilutėje 3.
Naudojamas [main\(\)](#).

8.29.1.2 printNameAsk()

void printNameAsk ()
Apibrėžimas failo [print.cpp](#) eilutėje 10.

8.30 print.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "print.h"
00002
00003 void printWelcome()
00004 {
00005     std::cout << u8"\n-----\n";
00006     std::cout << u8" Studentų informacinė sistema \n";
00007     std::cout << u8"-----\n\n";
00008 }
00009
00010 void printNameAsk()
00011 {
00012     std::cout << u8"Įveskite naujus studentus.\n";
00013 }
  
```

8.31 OOP_3/print.h Failo Nuoroda

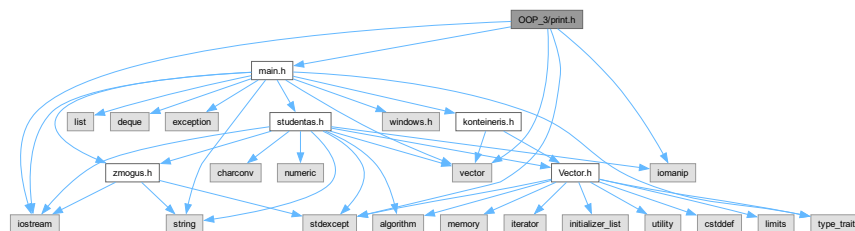
```

#include "main.h"
#include <iostream>
#include <iomanip>
#include <vector>
  
```

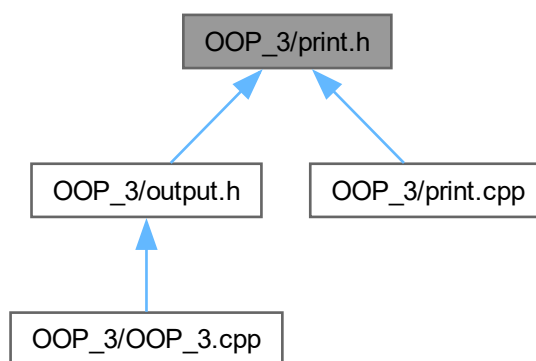


```
#include <stdexcept>
```

Įtraukimo priklausomybių diagrama print.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- `template<typename Container>`
void `printStudentai` (const Container &studentai)

8.31.1 Funkcijos Dokumentacija

8.31.1.1 printStudentai()

```
template<typename Container>
void printStudentai (
    const Container & studentai)
```

Apibrėžimas failo `print.h` eilutėje 14.

Naudojamas `outputStudentai()`.

8.32 print.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef PRINT_H_DEFINED
00002 #define PRINT_H_DEFINED
00003
00004 #include "main.h"
00005 #include <iostream> // cout, cin
```

```

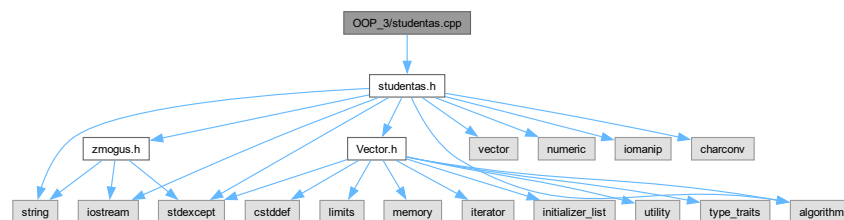
00006 #include <iomanip>    // setprecision, setw
00007 #include <vector>      // vector
00008 #include <stdexcept>   // runtime_error
00009
00010 // -----
00011 // printStudentai -- išveda studentus į konsolę su antrašte.
00012 // -----
00013 template<typename Container>
00014 void printStudentai(const Container& studentai)
00015 {
00016     if (studentai.empty())
00017         throw std::runtime_error("Studentų sąrašas tuščias -- nėra ko spausdinti.");
00018
00019     std::cout << std::left << std::setw(20) << u8"Vardas"
00020               << std::left << std::setw(20) << u8"Pavardė"
00021               << std::left << std::setw(20) << u8"Galutinis (Vid.)"
00022               << std::left << std::setw(20) << u8"Galutinis (Med.)" << "\n";
00023     std::cout << u8"-----\n";
00024
00025     for (const auto& s : studentai)
00026         std::cout << s << "\n";
00027 }
00028
00029 #endif // PRINT_H_DEFINED

```

8.33 OOP_3/studentas.cpp Failo Nuoroda

```
#include "studentas.h"
```

Įtraukimo priklausomybių diagrama studentas.cpp:



Funkcijos

- bool `comparePagalVarda` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal vardą (didėjančiai).
- bool `comparePagalPavarde` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal pavardę (didėjančiai).
- bool `comparePagalEgzamina` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal egzamino pažymį (mažėjančiai).
- bool `comparePagalVidurki` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal galutinį balą su vidurkiu (mažėjančiai).
- bool `comparePagalMediana` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal galutinį balą su mediana (mažėjančiai).

8.33.1 Funkcijos Dokumentacija

8.33.1.1 `comparePagalVarda()`

```
bool comparePagalVarda (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal vardą (didėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true
```

 jei *s1* vardas abėcėliškai anksčiau

Apibrėžimas failo [studentas.cpp](#) eilutėje 470.

Ryšiai [zmogus::vardas\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.33.1.2 comparePagalPavarde()

```
bool comparePagalPavarde (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal pavardę (didėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true
```

 jei *s1* pavardė abėcėliškai anksčiau

Apibrėžimas failo [studentas.cpp](#) eilutėje 474.

Ryšiai [zmogus::pavarde\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.33.1.3 comparePagalEgzamina()

```
bool comparePagalEgzamina (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal egzamino pažymį (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true
```

 jei *s1* egzaminas didesnis

Apibrėžimas failo [studentas.cpp](#) eilutėje 478.

Ryšiai [studentas::egzaminas\(\)](#).

8.33.1.4 comparePagalVidurki()

```
bool comparePagalVidurki (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal galutinį balą su vidurkiu (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

true jei *s1* galutinis balas didesnis

Apibrėžimas failo [studentas.cpp](#) eilutėje 482.

Ryšiai [studentas::galutinisVid\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.33.1.5 comparePagalMediana()

```
bool comparePagalMediana (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal galutinį balą su mediana (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

true jei *s1* galutinis balas didesnis

Apibrėžimas failo [studentas.cpp](#) eilutėje 486.

Ryšiai [studentas::galutinisMed\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.34 studentas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "studentas.h"
00002
00003 // -----
00004 // Statinis narys
00005 // -----
00006 int studentas::gyvuStudentu = 0;
00007
00008 // -----
00009 // Privatūs pagalbiniai metodai
00010 // -----
00011
00012 double studentas::vidurkis(const Vector<int>& nd) {
00013     if (nd.empty())
00014         return 0.0;
00015     double sum = std::accumulate(nd.begin(), nd.end(), 0.0);
00016     return sum / nd.size();
00017 }
00018
00019 double studentas::mediana(const Vector<int>& nd) {
00020     if (nd.empty())
00021         return 0.0;
00022     Vector<int> sortedNd = nd;
00023     std::sort(sortedNd.begin(), sortedNd.end());
00024     size_t size = sortedNd.size();
00025     if (size % 2 == 0)
00026         return (sortedNd[size / 2 - 1] + sortedNd[size / 2]) / 2.0;
00027     else
00028         return sortedNd[size / 2];
```

```

00029 }
00030
00031 // -----
00032 // Vidinės pagalbinės funkcijos (tik šiam failui)
00033 // -----
00034
00035 namespace {
00036
00037     inline std::string vietosTekstas(int lineNumber) {
00038         return (lineNumber > 0)
00039             ? "Eilutėje " + std::to_string(lineNumber)
00040             : std::string("Ivestyje");
00041     }
00042
00043     inline std::string ribuKlaida(int lineNumber, const char* ko, int p) {
00044         return vietosTekstas(lineNumber) + " " + ko + " pažymys už ribų ("
00045             + std::to_string(studentas::minPazymys) + "-"
00046             + std::to_string(studentas::maxPazymys) + "): "
00047             + std::to_string(p);
00048     }
00049
00050 } // anoniminis namespace
00051
00052 // =====
00053 // 1. KONSTRUKTORIAI
00054 //
00055 // Kiekvienas kviečia atitinkamą BAZINĖS klasės konstruktorių
00056 // per member initializer list -- zmogus() arba zmogus(v, p).
00057 // Kūrimo tvarka: zmogus konstruktorius → studentas konstruktorius
00058 // =====
00059
00060 studentas::studentas()
00061     : zmogus(), // bazinės klasės konstruktorius
00062     nd_(), egzaminas_(0),
00063     galutinisVid_(0.0), galutinisMed_(0.0)
00064 {
00065     ++gyvuStudentu;
00066 }
00067
00068 studentas::studentas(const std::string& vardas,
00069     const std::string& pavarde,
00070     const Vector<int>& nd,
00071     int egzaminas)
00072     : zmogus(vardas, pavarde), // bazinė validuoja vardą ir pavardę
00073     nd_(nd), egzaminas_(egzaminas),
00074     galutinisVid_(0.0), galutinisMed_(0.0)
00075 {
00076     // Studentui specifinė validacija
00077     for (int p : nd_)
00078         if (!pazymysTinkamas(p))
00079             throw std::runtime_error("Namų darbo pažymys už ribų: " + std::to_string(p));
00080     if (!pazymysTinkamas(egzaminas_))
00081         throw std::runtime_error("Egzamino pažymys už ribų: " + std::to_string(egzaminas_));
00082
00083     ++gyvuStudentu;
00084 }
00085
00086 studentas::studentas(std::istream& is)
00087     : zmogus(),
00088     nd_(), egzaminas_(0),
00089     galutinisVid_(0.0), galutinisMed_(0.0)
00090 {
00091     ++gyvuStudentu;
00092     readStudentas(is);
00093 }
00094
00095 // =====
00096 // 2. RULE OF FIVE
00097 //
00098 // Visos penkios funkcijos kviečia atitinkamas BAZINĖS klasės
00099 // funkcijas -- kitaip vardas_ ir pavarde_ nebūtų nukopijuoti
00100 // arba perkelti (dažna klaida paveldėjime!).
00101 // =====
00102
00103 // -----
00104 // 2.1 DESTRUKTORIUS (override)
00105 //
00106 // Naikinimo tvarka atvirkštinė kūrimui:
00107 // ~studentas() → ~zmogus()
00108 // Bazinės klasės destruktorius kviečiamas AUTOMATIŠKAI --
00109 // jo rankiniu būdu kviesti nereikia ir negalima.
00110 // -----
00111 studentas::~studentas()
00112 {
00113     nd_.clear();
00114     nd_.shrink_to_fit();
00115     egzaminas_ = 0;

```

```

00116     galutinisVid_ = 0.0;
00117     galutinisMed_ = 0.0;
00118
00119     --gyvuStudentu;
00120     // ~zmogus() iškviečiamas automatiškai po šio kūno
00121 }
00122
00123 // -----
00124 // 2.2 KOPIJAVIMO KONSTRUKTORIUS
00125 //
00126 // zmogus(other) -- bazinės dalies kopijavimas.
00127 // Be šio kvietimo būtų iškviestas zmogus() numatytasis
00128 // konstruktorius, ir vardas_/pavarde_ liktų tušti.
00129 // -----
00130 studentas::studentas(const studentas& other)
00131     : zmogus(other), // bazinės dalies kopija
00132     nd_(other.nd_),
00133     egzaminas_(other egzaminas_),
00134     galutinisVid_(other.galutinisVid_),
00135     galutinisMed_(other.galutinisMed_)
00136 {
00137     ++gyvuStudentu;
00138 }
00139
00140 // -----
00141 // 2.3 KOPIJAVIMO PRISKYRIMO OPERATORIUS
00142 //
00143 // zmogus::operator=(other) -- bazinės dalies priskyrimas.
00144 // Kvalifikuotas kvietimas būtinas, nes be jo įvyktų
00145 // begalinė rekursija (kvietų save patį).
00146 // -----
00147 studentas& studentas::operator=(const studentas& other)
00148 {
00149     if (this == &other) // apsauga nuo priskyrimo sau
00150         return *this;
00151
00152     zmogus::operator=(other); // bazinės dalies priskyrimas
00153
00154     nd_ = other.nd_;
00155     egzaminas_ = other egzaminas_;
00156     galutinisVid_ = other.galutinisVid_;
00157     galutinisMed_ = other.galutinisMed_;
00158
00159     return *this;
00160 }
00161
00162 // -----
00163 // 2.4 PERKĖLIMO (MOVE) KONSTRUKTORIUS
00164 //
00165 // std::move(other) būtinas: other yra pavadintas kintamasis,
00166 // todėl pats savaime yra lvalue -- be std::move būtų iškviestas
00167 // zmogus KOPIJAVIMO, ne perkėlimo konstruktorius.
00168 //
00169 // noexcept -- kad std::vector perskirstymo metu naudotų
00170 // perkėlimą, ne kopijavimą (std::move_if_noexcept).
00171 // -----
00172 studentas::studentas(studentas&& other) noexcept
00173     : zmogus(std::move(other)), // bazinės dalies perkėlimas
00174     nd_(std::move(other.nd_)),
00175     egzaminas_(other egzaminas_),
00176     galutinisVid_(other.galutinisVid_),
00177     galutinisMed_(other.galutinisMed_)
00178 {
00179     // Šaltinis paliekamas galiojančioje, bet tuščioje būsenoje
00180     other egzaminas_ = 0;
00181     other.galutinisVid_ = 0.0;
00182     other.galutinisMed_ = 0.0;
00183
00184     ++gyvuStudentu;
00185 }
00186
00187 // -----
00188 // 2.5 PERKĖLIMO (MOVE) PRISKYRIMO OPERATORIUS
00189 // -----
00190 studentas& studentas::operator=(studentas&& other) noexcept
00191 {
00192     if (this == &other) // apsauga nuo priskyrimo sau
00193         return *this;
00194
00195     zmogus::operator=(std::move(other)); // bazinės dalies perkėlimas
00196
00197     nd_ = std::move(other.nd_);
00198     egzaminas_ = other egzaminas_;
00199     galutinisVid_ = other.galutinisVid_;
00200     galutinisMed_ = other.galutinisMed_;
00201
00202     other egzaminas_ = 0;

```

```

00203     other.galutinisVid_ = 0.0;
00204     other.galutinisMed_ = 0.0;
00205
00206     return *this;
00207 }
00208
00209 // =====
00210 // 3. GRYNAI VIRTUALIŲ METODŲ REALIZACIJA (override)
00211 // =====
00212
00213 // -----
00214 // galutinis -- bazinės klasės reikalaujamas įvertinimas.
00215 // Studentui tai galutinis balas pagal vidurkį.
00216 // -----
00217 double studentas::galutinis() const
00218 {
00219     return galutinisVid_;
00220 }
00221
00222 // -----
00223 // print -- realizuoja polimorfinį operatorą
00224 // -----
00225 void studentas::print(std::ostream& os) const
00226 {
00227     os << std::left << std::setw(20) << vardas_
00228         << std::left << std::setw(20) << pavarde_
00229         << std::fixed << std::setprecision(2)
00230         << std::left << std::setw(20) << galutinisVid_
00231         << std::left << std::setw(20) << galutinisMed_;
00232 }
00233
00234 // -----
00235 // read -- realizuoja polimorfinį operatorą
00236 // -----
00237 std::istream& studentas::read(std::istream& is)
00238 {
00239     return readStudentas(is);
00240 }
00241
00242 // -----
00243 // tipas -- žmogaus tipo pavadinimas
00244 // -----
00245 std::string studentas::tipas() const
00246 {
00247     return "Studentas";
00248 }
00249
00250 // =====
00251 // 4. SET'ERIAI
00252 // =====
00253
00254 void studentas::setNd(const Vector<int>& nd) {
00255     for (int p : nd)
00256         if (!pazymysTinkamas(p))
00257             throw std::runtime_error("Namų darbo pažymys už ribų: " + std::to_string(p));
00258     nd_ = nd;
00259 }
00260
00261 void studentas::setEgzaminas(int e) {
00262     if (!pazymysTinkamas(e))
00263         throw std::runtime_error("Egzamino pažymys už ribų: " + std::to_string(e));
00264     egzaminas_ = e;
00265 }
00266
00267 void studentas::addPazymys(int p) {
00268     if (!pazymysTinkamas(p))
00269         throw std::runtime_error("Namų darbo pažymys už ribų: " + std::to_string(p));
00270     nd_.push_back(p);
00271 }
00272
00273 void studentas::isvalyk() {
00274     vardas_.clear(); // protected laukas iš zmogus
00275     pavarde_.clear(); // protected laukas iš zmogus
00276     nd_.clear();
00277     nd_.shrink_to_fit();
00278     egzaminas_ = 0;
00279     galutinisVid_ = 0.0;
00280     galutinisMed_ = 0.0;
00281 }
00282
00283 // =====
00284 // 5. SKAIČIAVIMAS
00285 // =====
00286
00287 void studentas::calculateGalutinis() {
00288     galutinisVid_ = namuDarbaiSvoris * vidurkis(nd_) + egzaminasSvoris * egzaminas_;
00289     galutinisMed_ = namuDarbaiSvoris * mediana(nd_) + egzaminasSvoris * egzaminas_;

```

```

00290 }
00291
00292 // =====
00293 // 6. ĮVESTIS
00294 // =====
00295
00296 // -----
00297 // parseFromLine -- GREITAS parsinimas failo skaitymui
00298 // Nenaudoja std::istream, naudoja std::from_chars
00299 // -----
00300 void studentas::parseFromLine(const std::string& line, int ndCount, int lineNumber)
00301 {
00302     const char* p = line.data();
00303     const char* end = p + line.size();
00304
00305     auto skipSpace = [&]() {
00306         while (p < end && (*p == ' ' || *p == '\t' || *p == '\r')) ++p;
00307     };
00308
00309     // --- Vardas ---
00310     skipSpace();
00311     const char* start = p;
00312     while (p < end && *p != ' ' && *p != '\t' && *p != '\r') ++p;
00313     if (start == p)
00314         throw std::runtime_error(vietosTekstas(lineNumber) + " trūksta vardo.");
00315     vardas_.assign(start, p - start);
00316
00317     // --- Pavardė ---
00318     skipSpace();
00319     start = p;
00320     while (p < end && *p != ' ' && *p != '\t' && *p != '\r') ++p;
00321     if (start == p)
00322         throw std::runtime_error(vietosTekstas(lineNumber) + " trūksta pavardės.");
00323     pavarde_.assign(start, p - start);
00324
00325     // --- Pažymiai ---
00326     auto readInt = [&](const char* ko) -> int {
00327         skipSpace();
00328         int v = 0;
00329         auto res = std::from_chars(p, end, v);
00330         if (res.ec != std::errc())
00331             throw std::runtime_error(vietosTekstas(lineNumber)
00332                                     + " trūksta " + ko + " pažymio.");
00333         p = res.ptr;
00334         return v;
00335     };
00336
00337     nd_.clear();
00338     nd_.reserve(ndCount);
00339
00340     for (int i = 0; i < ndCount; ++i) {
00341         int v = readInt("namų darbo");
00342         if (!pazymysTinkamas(v))
00343             throw std::runtime_error(ribuKlaida(lineNumber, "namų darbo", v));
00344         nd_.push_back(v);
00345     }
00346
00347     egzaminas_ = readInt("egzamino");
00348     if (!pazymysTinkamas(egzaminas_))
00349         throw std::runtime_error(ribuKlaida(lineNumber, "egzamino", egzaminas_));
00350 }
00351
00352 // -----
00353 // readStudentas -- įvestis per std::istream
00354 // -----
00355 std::istream& studentas::readStudentas(std::istream& is, int ndCount, int lineNumber) {
00356     if (!(is >> vardas_ >> pavarde_))
00357         throw std::runtime_error(vietosTekstas(lineNumber)
00358                                 + " trūksta vardo arba pavardės.");
00359
00360     nd_.clear();
00361
00362     if (ndCount > 0) {
00363         nd_.reserve(ndCount);
00364         for (int i = 0; i < ndCount; ++i) {
00365             int p;
00366             if (!(is >> p))
00367                 throw std::runtime_error(vietosTekstas(lineNumber)
00368                                         + " trūksta namų darbo pažymio.");
00369             if (!pazymysTinkamas(p))
00370                 throw std::runtime_error(ribuKlaida(lineNumber, "namų darbo", p));
00371             nd_.push_back(p);
00372         }
00373         if (!(is >> egzaminas_))
00374             throw std::runtime_error(vietosTekstas(lineNumber)
00375                                     + " trūksta egzamino pažymio.");
00376     }

```



```

00377     else {
00378         // Nežinomas skaičius -- skaitome iki galo, paskutinis yra egzaminas
00379         int p;
00380         while (is » p) {
00381             if (!pazymysTinkamas(p))
00382                 throw std::runtime_error(ribuKlaida(lineNumber, "namų darbo", p));
00383             nd_.push_back(p);
00384         }
00385         if (nd_.empty())
00386             throw std::runtime_error(vietosTekstas(lineNumber)
00387                                     + " trūksta egzamino pažymio.");
00388
00389         egzaminas_ = nd_.back();
00390         nd_.pop_back();
00391         is.clear(); // eilutės pabaiga nėra klaida šiame režime
00392     }
00393
00394     if (!pazymysTinkamas(egzaminas_))
00395         throw std::runtime_error(ribuKlaida(lineNumber, "egzamino", egzaminas_));
00396
00397     return is;
00398 }
00399
00400 // =====
00401 // 7. IŠVESTIS
00402 // =====
00403
00404 // -----
00405 // appendListTo -- GREITAS formatavimas failo rašymui (std::to_chars)
00406 // -----
00407 void studentas::appendListTo(std::string& out) const
00408 {
00409     out.append(vardas_);
00410     if (vardas_.size() < 25) out.append(25 - vardas_.size(), ' ');
00411
00412     out.append(pavarde_);
00413     if (pavarde_.size() < 25) out.append(25 - pavarde_.size(), ' ');
00414
00415     char buf[16];
00416     auto appendInt = [&](int v) {
00417         auto res = std::to_chars(buf, buf + sizeof(buf), v);
00418         size_t len = static_cast<size_t>(res.ptr - buf);
00419         if (len < 10) out.append(10 - len, ' ');
00420         out.append(buf, len);
00421     };
00422
00423     for (int nd : nd_)
00424         appendInt(nd);
00425     appendInt(egzaminas_);
00426
00427     out.push_back('\n');
00428 }
00429
00430 // =====
00431 // 8. PERDENGTI OPERATORIAI
00432 // =====
00433
00434 bool studentas::operator==(const studentas& other) const {
00435     return vardas_ == other.vardas_
00436         && pavarde_ == other.pavarde_
00437         && nd_ == other.nd_
00438         && egzaminas_ == other.egzaminas_;
00439 }
00440
00441 bool studentas::operator!=(const studentas& other) const {
00442     return !(*this == other);
00443 }
00444
00445 // Rikiavimas pagal pavarde, esant vienodoms -- pagal vardą
00446 bool studentas::operator<(const studentas& other) const {
00447     if (pavarde_ != other.pavarde_)
00448         return pavarde_ < other.pavarde_;
00449     return vardas_ < other.vardas_;
00450 }
00451
00452 bool studentas::operator>(const studentas& other) const {
00453     return other < *this;
00454 }
00455
00456 int studentas::operator[](size_t i) const {
00457     if (i >= nd_.size())
00458         throw std::out_of_range("Namų darbo indeksas už ribų: " + std::to_string(i));
00459     return nd_[i];
00460 }
00461
00462 studentas::operator bool() const {
00463     return !vardas_.empty() && !pavarde_.empty();

```

```

00464 }
00465
00466 // =====
00467 // 9. NE-NARIAI LYGINIMO FUNKCIJOS
00468 // =====
00469
00470 bool comparePagalVarda(const studentas& s1, const studentas& s2) {
00471     return s1.vardas() < s2.vardas();
00472 }
00473
00474 bool comparePagalPavarde(const studentas& s1, const studentas& s2) {
00475     return s1.pavarde() < s2.pavarde();
00476 }
00477
00478 bool comparePagalEgzamina(const studentas& s1, const studentas& s2) {
00479     return s1.egzaminas() > s2.egzaminas();
00480 }
00481
00482 bool comparePagalVidurki(const studentas& s1, const studentas& s2) {
00483     return s1.galutinisVid() > s2.galutinisVid();
00484 }
00485
00486 bool comparePagalMediana(const studentas& s1, const studentas& s2) {
00487     return s1.galutinisMed() > s2.galutinisMed();
00488 }

```

8.35 OOP_3/studentas.h Failo Nuoroda

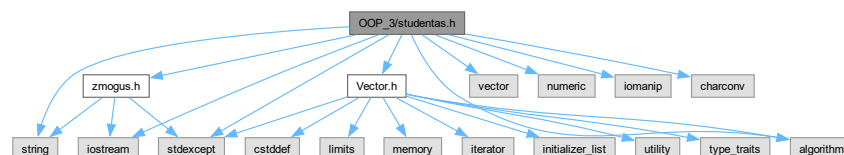
Išvestinė klasė studentas — studento duomenys ir galutinių balų skaičiavimas.

```

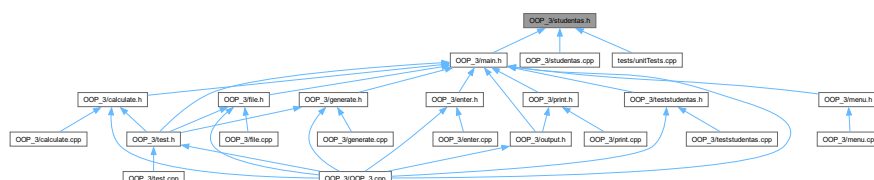
#include "zmogus.h"
#include "Vector.h"
#include <string>
#include <vector>
#include <iostream>
#include <numeric>
#include <algorithm>
#include <iomanip>
#include <stdexcept>
#include <charconv>

```

Įtraukimo priklausomybių diagrama studentas.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class **studentas**

Studento duomenys — išvestinė klasė iš abstrakčios zmogus.

Funkcijos

Ne-nariai lyginimo funkcijos

Naudojamos `sortStudentai()` šablone kaip komparatoriai.

Nėra klasės nariai, nes naudoja tik viešą interfeisą — taip laikomasi principo, kad klasės narių turi būti kuo mažiau.

- bool `comparePagalVarda` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal vardą (didėjančiai).
- bool `comparePagalPavarde` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal pavardę (didėjančiai).
- bool `comparePagalEgzamina` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal egzamino pažymį (mažėjančiai).
- bool `comparePagalVidurki` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal galutinį balą su vidurkiu (mažėjančiai).
- bool `comparePagalMediana` (const `studentas` &s1, const `studentas` &s2)
Lygina pagal galutinį balą su mediana (mažėjančiai).

8.35.1 Smulkus aprašymas

Išvestinė klasė `studentas` — studento duomenys ir galutinių balų skaičiavimas.

Autorius

Marius Augustinas

Data

2026

Apibrėžimas faile `studentas.h`.

8.35.2 Funkcijos Dokumentacija

8.35.2.1 `comparePagalVarda()`

```
bool comparePagalVarda (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal vardą (didėjančiai).

Parametrai

<code>s1</code>	Pirmas studentas
<code>s2</code>	Antras studentas

Gražina

`true` jei `s1` vardas abėcėliškai anksčiau

Apibrėžimas failo `studentas.cpp` eilutėje 470.

Ryšiai `zmogus::vardas()`.

Naudojamas `sortStudentai()`.

8.35.2.2 `comparePagalPavarde()`

```
bool comparePagalPavarde (
    const studentas & s1,
    const studentas & s2)
```

Lygina pagal pavardę (didėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true
```

 jei *s1* pavardė abėcėliškai anksčiau

Apibrėžimas failo [studentas.cpp](#) eilutėje 474.

Ryšiai [zmogus::pavarde\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.35.2.3 comparePagalEgzamina()

```
bool comparePagalEgzamina (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal egzamino pažymį (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true
```

 jei *s1* egzaminas didesnis

Apibrėžimas failo [studentas.cpp](#) eilutėje 478.

Ryšiai [studentas::egzaminas\(\)](#).

8.35.2.4 comparePagalVidurki()

```
bool comparePagalVidurki (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal galutinį balą su vidurkiu (mažėjančiai).

Parametrai

<i>s1</i>	Pirmas studentas
<i>s2</i>	Antras studentas

Gražina

```
true
```

 jei *s1* galutinis balas didesnis

Apibrėžimas failo [studentas.cpp](#) eilutėje 482.

Ryšiai [studentas::galutinisVid\(\)](#).

Naudojamas [sortStudentai\(\)](#).

8.35.2.5 comparePagalMediana()

```
bool comparePagalMediana (  
    const studentas & s1,  
    const studentas & s2)
```

Lygina pagal galutinį balą su mediana (mažėjančiai).

Parametrai

s1	Pirmas studentas
s2	Antras studentas

Gražina

true jei s1 galutinis balas didesnis

Apibrėžimas failo `studentas.cpp` eilutėje 486.

Ryšiai `studentas::galutinisMed()`.

Naudojamas `sortStudentai()`.

8.36 studentas.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef STUDENTAS_H_DEFINED
00002 #define STUDENTAS_H_DEFINED
00003
00004 #include "zmogus.h"
00005 #include "Vector.h"
00006
00007 #include <string>
00008 #include <vector>
00009 #include <iostream>
00010 #include <numeric>
00011 #include <algorithm>
00012 #include <iomanip>
00013 #include <stdexcept>
00014 #include <charconv>
00015
00016 /**
00017  * @file studentas.h
00018  * @brief Išvestinė klasė studentas -- studento duomenys ir galutinių balų skaičiavimas.
00019  * @author Marius Augustinas
00020  * @date 2026
00021  */
00022
00023 /**
00024  * @class studentas
00025  * @brief Studento duomenys -- išvestinė klasė iš abstrakčios zmogus.
00026  *
00027  * Prie paveldėto vardo ir pavardės prideda namų darbų pažymius,
00028  * egzamino įvertinimą ir du galutinius balus (pagal vidurkį ir pagal mediana).
00029  *
00030  * @par Paveldėjimas
00031  * Iš zmogus paveldi:
00032  * - `vardas_`, `pavarde_` ('protected' laukai)
00033  * - vardas(), pavarde(), pilnasVardas() get'eries
00034  * - setVardas(), setPavarde() set'eries su validacija
00035  * - virtualių destruktorių
00036  *
00037  * Realizuoja visus keturis grynai virtualius bazinės klasės metodus:
00038  * galutinis(), print(), read(), tipas().
00039  *
00040  * @par Galutinio balo formulė
00041  * @code
00042  * galutinis = 0.4 * namų_darbu_vidurkis_arba_mediana + 0.6 * egzaminas
00043  * @endcode
00044  * Svoriai apibrėžti konstantomis #namuDarbaiSvoris ir #egzaminasSvoris.
00045  *
00046  * @par Naudojimo pavyzdys
00047  * @code
00048  * // Sukūrimas su pilnu konstruktoriumi
00049  * studentas s("Jonas", "Jonaitis", {5, 6, 7, 8}, 9);
00050  * s.calculateGalutinis();
00051  * std::cout << s << "\n";
00052  *
00053  * // Nuskaitymas iš srauto
00054  * std::istringstream iss("Ona Onaite 10 9 8 7");
00055  * studentas s2;
00056  * iss >> s2;
00057  *
00058  * // Polimorfinis naudojimas
```

```

00059 * std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00060 *     "Petras", "Petraitis", Vector<int>(5, 5), 8);
00061 * std::cout << p->tipas() << ": " << p->galutinis() << "\n";
00062 * @endcode
00063 *
00064 * @warning Galutiniai balai NĖRA skaičiuojami automatiškai kuriant objektą --
00065 *     po duomenų nuskaitymo ar sukūrimo reikia iškviešti
00066 *     calculateGalutinis(). Toks sprendimas leidžia matuoti nuskaitymo
00067 *     ir skaičiavimo spartą atskirai.
00068 *
00069 * @see zmogus
00070 */
00071 class studentas : public zmogus {
00072 private:
00073     Vector<int> nd_;          ///< Namų darbų pažymiai (kiekvienas 1-10)
00074     int egzaminas_;          ///< Egzamino pažymys (1-10)
00075     double galutinisVid_;     ///< Galutinis balas pagal namų darbų vidurkį
00076     double galutinisMed_;     ///< Galutinis balas pagal namų darbų medianą
00077
00078     /**
00079      * @brief Apskaičiuoja aritmetinį vidurkį.
00080      * @param nd Pažymių vektorius
00081      * @return Vidurkis; 0.0 jei vektorius tuščias
00082      */
00083     static double vidurkis(const Vector<int>& nd);
00084
00085     /**
00086      * @brief Apskaičiuoja medianą.
00087      *
00088      * Nelyginiam elementų kiekiui gražina vidurinį elementą,
00089      * lyginiam -- dviejų vidurinių aritmetinį vidurkį.
00090      * Įeinantis vektorius nekeičiamas (kopijuojamas ir rikiuojamas viduje).
00091      *
00092      * @param nd Pažymių vektorius (nebūtinai surikiuotas)
00093      * @return Mediana; 0.0 jei vektorius tuščias
00094      */
00095     static double mediana(const Vector<int>& nd);
00096
00097     /**
00098      * @brief Patikrina, ar pažymys patenka į leistiną intervalą.
00099      *
00100      * Šauningai gražina `bool`, o ne meta išimtį: klaidos pranešimo
00101      * formavimas (`std::string` konkatencija) yra brangus, o su 10 mln.
00102      * studentų ši funkcija kviečiama ~160 mln. kartų. Pranešimas
00103      * formuojamas tik ten, kur išimtis tikrai metama.
00104      *
00105      * @param p Tikrinamas pažymys
00106      * @return `true` jei #minPazymys p #maxPazymys
00107      */
00108     static inline bool pazymysTinkamas(int p) {
00109         return p >= minPazymys && p <= maxPazymys;
00110     }
00111 public:
00112     /// @name Konstantos
00113     /// @{
00114     static constexpr int minPazymys = 1;          ///< Mažiausias leistinas pažymys
00115     static constexpr int maxPazymys = 10;         ///< Didžiausias leistinas pažymys
00116     static constexpr double namuDarbaiSvoris = 0.4; ///< Namų darbų svoris galutiniame balse
00117     static constexpr double egzaminasSvoris = 0.6; ///< Egzamino svoris galutiniame balse
00118     /// @}
00119
00120     /**
00121      * @brief Gyvuojančių studentų skaitiklis.
00122      *
00123      * Didinamas kiekviename konstruktoriuje, mažinamas destruktoriuje.
00124      * Naudojamas testuose objektų nutekėjimui aptikti -- atskiras nuo
00125      * zmogus::gyvuZmoniu, todėl galima patikrinti, ar iškviešti abu
00126      * destruktoriai (~studentas() ir ~zmogus()).
00127      */
00128     static int gyvuStudentu;
00129
00130     // =====
00131     /// @name Konstruktoriai
00132     /// @{
00133     // =====
00134
00135     /**
00136      * @brief Numatytasis konstruktorius.
00137      *
00138      * Sukuria tuščią studentą: be vardo, pavardės ir pažymių,
00139      * su nuliniiais galutiniais balais. Skirtas objektui, į kurį
00140      * duomenys bus nuskaityti vėliau (pvz. per readStudentas()).
00141      */
00142     studentas();
00143
00144     /**
00145

```

```

00146     * @brief Pilnas konstruktorius su validacija.
00147     *
00148     * Vardą ir pavardę validuoja bazinė klasė, pažymius -- ši klasė.
00149     *
00150     * @param vardas Studento vardas (negali būti tuščias)
00151     * @param pavarde Studento pavardė (negali būti tuščia)
00152     * @param nd Namų darbų pažymiai (kiekvienas 1-10)
00153     * @param egzaminas Egzamino pažymys (1-10)
00154     *
00155     * @throws std::invalid_argument jei vardas arba pavardė tuščia
00156     * @throws std::runtime_error jei bent vienas pažymys už ribų
00157     *
00158     * @note Galutiniai balai lieka 0.0 -- reikia iškvieisti calculateGalutinis().
00159     */
00160     studentas(const std::string& vardas,
00161               const std::string& pavarde,
00162               const Vector<int>& nd,
00163               int egzaminas);
00164
00165     /**
00166     * @brief Konstruktorius iš įvesties srauto.
00167     *
00168     * Iškart nuskaityto duomenis per readStudentas().
00169     * Pažymėtas `explicit`, kad neįvyktų netyčinis konvertavimas
00170     * iš srauto į studentą.
00171     *
00172     * @param is Įvesties srautas
00173     * @throws std::runtime_error jei duomenys netinkami arba jų trūksta
00174     */
00175     explicit studentas(std::istream& is);
00176
00177     /// @}
00178
00179     // =====
00180     /// @name Rule of Five
00181     ///
00182     /// Visos penkios funkcijos realizuotos pilnai (ne `= default`).
00183     /// Kiekviena **privalo** iškvieisti atitinkamą bazinės klasės funkciją --
00184     /// be to `vardas_` ir `pavarde_` liktų tušti arba nenukopijuoti.
00185     /// Tai dažniausia paveldėjimo klaida, kurios kompiliatorius neaptinka.
00186     /// @{
00187     // =====
00188
00189     /**
00190     * @brief Destruktorius.
00191     *
00192     * Išvalo studento duomenis ir sumažina #gyvuStudentu skaitiklį.
00193     * Bazinės klasės destruktoriaus `~zmogus()` iškviečiamas
00194     * **automatiškai** po šio kūno -- jo kvieisti rankiniu būdu nereikia.
00195     *
00196     * @par Naikinimo tvarka
00197     * `~studentas()` → `~zmogus()` (atvirkštinė kūrimo tvarkai)
00198     */
00199     ~studentas() override;
00200
00201     /**
00202     * @brief Kopijavimo konstruktorius.
00203     *
00204     * Sukuria gilia kopiją (deep copy) -- `std::string` ir `std::vector`
00205     * laukai kopijuoja savo vidinius buferius, todėl objektai lieka
00206     * visiškai nepriklausomi.
00207     *
00208     * @param other Kopijuojamas studentas
00209     *
00210     * @note Member initializer list prasideda `zmogus(other)` -- be šio
00211     * kvietimo būtų iškvieistas `zmogus()` numatytasis konstruktorius,
00212     * ir vardas su pavarde liktų tušti.
00213     */
00214     studentas(const studentas& other);
00215
00216     /**
00217     * @brief Kopijavimo priskyrimo operatorius.
00218     *
00219     * @param other Priskiriamas studentas
00220     * @return Nuoroda į šį objektą (grandininiam priskyrimui `a = b = c`)
00221     *
00222     * @note Turi apsaugą nuo priskyrimo sau (`this == &other`) -- be jos
00223     * `a = a` galėtų sugadinti duomenis.
00224     * @note Bazinė dalis priskiriama kvalifikuotu kvietimu
00225     * `zmogus::operator=(other)`. Be kvalifikacijos kompiliatorius
00226     * parinktų `studentas::operator=` ir įvyktų begalinė rekursija.
00227     */
00228     studentas& operator=(const studentas& other);
00229
00230     /**
00231     * @brief Perkėlimo (move) konstruktorius.
00232     *

```

```

00233     * Perima kito objekto resursus be kopijavimo. Šaltinis paliekamas
00234     * galiojančioje, bet tuščioje būsenoje -- jį saugu naikinti arba
00235     * naudoti iš naujo.
00236     *
00237     * @param other Studentas, iš kurio perkeliami duomenys
00238     *
00239     * @warning Bazinė dalis perkeliama `zmogus(std::move(other))`.
00240     *         `std::move` čia **būtinas**: `other` yra pavadintas
00241     *         kintamasis, todėl pats savaime lvalue -- be `std::move`
00242     *         būtų iškviestas `zmogus` **kopijavimo**, ne perkėlimo
00243     *         konstruktorius.
00244     *
00245     * @note `noexcept` kritiškai svarbus spartai: `std::vector`
00246     *        perskirstymo metu naudoja `std::move_if_noexcept`, todėl
00247     *        be `noexcept` rinktųsi kopijavimo konstruktorių.
00248     */
00249     studentas(studentas&& other) noexcept;
00250
00251     /**
00252     * @brief Perkėlimo (move) priskyrimo operatorius.
00253     *
00254     * @param other Studentas, iš kurio perkeliami duomenys
00255     * @return Nuoroda į šį objektą
00256     *
00257     * @note Bazinė dalis perkeliama `zmogus::operator=(std::move(other))`.
00258     */
00259     studentas& operator=(studentas&& other) noexcept;
00260
00261     /// @}
00262
00263     // =====
00264     /// @name Grynai virtualių metodų realizacija
00265     /// @{
00266     // =====
00267
00268     /**
00269     * @brief Gražina galutinį įvertinimą (bazinės klasės interfeisas).
00270     *
00271     * Studentui tai galutinis balas pagal namų darbų **vidurkį**.
00272     * Mediano variantas prieinamas per galutinisMed().
00273     *
00274     * @return Galutinis balas pagal vidurkį
00275     * @warning Gražina 0.0, jei nebuvo iškviesta calculateGalutinis().
00276     */
00277     double galutinis() const override;
00278
00279     /**
00280     * @brief Išveda studentą į srautą.
00281     *
00282     * Formatas: vardas (20 simbolių), pavardė (20), galutinis pagal
00283     * vidurkį (20), galutinis pagal medianą (20).
00284     * Realizuoja polimorfinį `operator«`.
00285     *
00286     * @param os Išvesties srautas
00287     */
00288     void print(std::ostream& os) const override;
00289
00290     /**
00291     * @brief Nuskaito studentą iš srauto.
00292     *
00293     * Realizuoja polimorfinį `operator»`. Deleguoja į readStudentas().
00294     *
00295     * @param is Įvesties srautas
00296     * @return Nuoroda į tą patį srautą
00297     */
00298     std::istream& read(std::istream& is) override;
00299
00300     /**
00301     * @brief Gražina tipo pavadinimą.
00302     * @return `"Studentas"`
00303     */
00304     std::string tipas() const override;
00305
00306     /// @}
00307
00308     // =====
00309     /// @name Get'eriai
00310     /// @{
00311     // =====
00312
00313     /**
00314     * @brief Gražina namų darbų pažymius.
00315     * @return Konstanti nuoroda į vektorių (be kopijavimo)
00316     */
00317     inline const Vector<int>& nd() const { return nd_; }
00318
00319     /// @brief Gražina egzamino pažymį.

```



```

00320     inline int egzaminas() const { return egzaminas_; }
00321
00322     /// @brief Gražina galutinį balą pagal namų darbų vidurkį.
00323     inline double galutinisVid() const { return galutinisVid_; }
00324
00325     /// @brief Gražina galutinį balą pagal namų darbų medianą.
00326     inline double galutinisMed() const { return galutinisMed_; }
00327
00328     /**
00329      * @brief Patikrina, ar objektas neturi vardo ir pavardės.
00330      * @return `true` jei ir vardas, ir pavardė tušti
00331      */
00332     inline bool tuscias() const { return vardas_.empty() && pavarde_.empty(); }
00333
00334     /// @}
00335
00336     // =====
00337     /// @name Set'eriai
00338     /// @{
00339     // =====
00340
00341     /**
00342      * @brief Nustato namų darbų pažymius su validacija.
00343      * @param nd Pažymių vektorius
00344      * @throws std::runtime_error jei bent vienas pažymys už ribų
00345      */
00346     void setNd(const Vector<int>& nd);
00347
00348     /**
00349      * @brief Nustato egzamino pažymį su validacija.
00350      * @param e Egzamino pažymys
00351      * @throws std::runtime_error jei pažymys už ribų
00352      */
00353     void setEgzaminas(int e);
00354
00355     /**
00356      * @brief Prideda vieną namų darbo pažymį.
00357      * @param p Pridedamas pažymys
00358      * @throws std::runtime_error jei pažymys už ribų
00359      */
00360     void addPazymys(int p);
00361
00362     /**
00363      * @brief Ištuština objektą.
00364      *
00365      * Išvalo **ir bazinės, ir išvestinės** klasės laukus. Vektoriaus
00366      * atmintis faktiškai atlaisvinama ('shrink_to_fit').
00367      */
00368     void isvalyk();
00369
00370     /// @}
00371
00372     // =====
00373     /// @name Skaičiavimas
00374     /// @{
00375     // =====
00376
00377     /**
00378      * @brief Apskaičiuoja abu galutinius balus.
00379      *
00380      * Užpildo `galutinisVid_` ir `galutinisMed_` pagal formulę:
00381      * @code
00382      * galutinisVid = 0.4 * vidurkis(nd) + 0.6 * egzaminas
00383      * galutinisMed = 0.4 * mediana(nd) + 0.6 * egzaminas
00384      * @endcode
00385      *
00386      * Tuščiam namų darbų sąrašui vidurkis ir mediana laikomi 0.0,
00387      * todėl galutinis balas yra `0.6 * egzaminas`.
00388      *
00389      * @note Kviečiamas atskirai, ne konstruktoriuje -- taip nuskaitymo
00390      *       ir skaičiavimo spartą galima matuoti nepriklausomai.
00391      */
00392     void calculateGalutinis();
00393
00394     /// @}
00395
00396     // =====
00397     /// @name Įvestis
00398     /// @{
00399     // =====
00400
00401     /**
00402      * @brief Nuskaito studentą iš srauto.
00403      *
00404      * Veikia dviem režimais priklausomai nuo @p ndCount:
00405      * - **ndCount > 0** -- nuskaito tiksliai tiek namų darbų pažymių,
00406      *   po jų -- egzaminą. Naudojama, kai stulpelių skaičius žinomas

```

```

00407      * iš failo antraštės.
00408      * - **ndCount == 0** -- skaito skaičius iki eilutės galo, paskutinį
00409      *   laiko egzaminu. Naudojama interaktyviai įvesčiai.
00410      *
00411      * @param is Įvesties srautas
00412      * @param ndCount Namų darbų skaičius (0 = skaityti iki galo)
00413      * @param lineNumber Eilutės numeris klaidos pranešimui (0 = nenaudoti)
00414      * @return Nuoroda į tą patį srautą
00415      *
00416      * @throws std::runtime_error jei trūksta duomenų arba pažymys už ribų
00417      *
00418      * @note Failo skaitymui naudokite parseFromLine() -- jis kelis kartus
00419      *   greitesnis, nes neturi srauto mechanizmo.
00420      */
00421 std::istream& readStudentas(std::istream& is, int ndCount = 0, int lineNumber = 0);
00422
00423 /**
00424  * @brief Greitas parsinimas iš eilutės (failo skaitymui).
00425  *
00426  * Optimizuota alternatyva readStudentas() metodui masiniam
00427  * duomenų apdorojimui:
00428  * - **nekuriamas** `std::istream` objektas kiekvienai eilutei
00429  * - skaičiai parsinami su `std::from_chars` (C++17) -- be locale,
00430  *   be stream sentry, be būsenos tikrinimo
00431  * - klaidos pranešimai formuojami tik metant išimtį
00432  *
00433  * Su 10 mln. studentų šie trys pakeitimai sutrumpina failo
00434  * nuskaitymą nuo ~90 s iki ~30 s.
00435  *
00436  * @param line Nuskaitoma eilutė
00437  * @param ndCount Namų darbų stulpelių skaičius (iš failo antraštės)
00438  * @param lineNumber Eilutės numeris klaidos pranešimui
00439  *
00440  * @throws std::runtime_error jei trūksta vardo, pavardės ar pažymio,
00441  *   arba pažymys už ribų
00442  *
00443  * @see appendListTo() -- atvirkštinė operacija
00444  */
00445 void parseFromLine(const std::string& line, int ndCount, int lineNumber);
00446
00447 /// @}
00448
00449 // =====
00450 /// @name Išvestis
00451 /// @{
00452 // =====
00453
00454 /**
00455  * @brief Prideda suformatuotą eilutę į buferį (failo rašymui).
00456  *
00457  * Optimizuota alternatyva `operator<<` masiniam rašymui:
00458  * - skaičiai formatuojami su `std::to_chars` -- be `std::setw`
00459  *   manipuliatorių
00460  * - rašoma į bendrą `std::string` buferį, ne į srautą
00461  *
00462  * Kviečianti funkcija kaupia eilutes ir išskviečia vieną
00463  * `file.write()` kas ~1 MB. Buferis nedidėja proporcingai studentų
00464  * skaičiui -- `clear()` išlaiko `capacity`.
00465  *
00466  * Formatas: vardas (25), pavardė (25), kiekvienas ND pažymys (10),
00467  * egzaminas (10), naujos eilutės simbolis.
00468  *
00469  * @param out Buferis, į kurį pridedama eilutė
00470  *
00471  * @see parseFromLine() -- atvirkštinė operacija; kartu jos sudaro
00472  *   round-trip: išvesta eilutė nuskaitoma atgal be pakitimų
00473  */
00474 void appendListTo(std::string& out) const;
00475
00476 /// @}
00477
00478 // =====
00479 /// @name Perdengti operatoriai
00480 ///
00481 /// Programoje nenaudojami, bet realizuoti būsimiems klasės
00482 /// naudotojams -- jie leidžia naudoti studentą su STL algoritmais
00483 /// ir asociatyviais kontaineriais.
00484 /// @{
00485 // =====
00486
00487 /**
00488  * @brief Lygina du studentus.
00489  *
00490  * Lyginami vardas, pavardė, namų darbų pažymiai ir egzaminas.
00491  * Galutiniai balai **nelyginami**, nes jie išvedami iš kitų laukų.
00492  *
00493  * @param other Lyginamas studentas

```

```

00494     * @return `true` jei visi duomenys sutampa
00495     */
00496     bool operator==(const studentas& other) const;
00497
00498     /**
00499     * @brief Tikrina, ar studentai skiriasi.
00500     * @param other Lyginamas studentas
00501     * @return `!( *this == other)`
00502     */
00503     bool operator!=(const studentas& other) const;
00504
00505     /**
00506     * @brief Rikiavimo operatorius.
00507     *
00508     * Lygina pagal pavardę; esant vienodoms pavardėms -- pagal vardą.
00509     * Leidžia naudoti `std::sort` be komparatoriaus ir dėti studentus
00510     * į `std::set` ar `std::map`.
00511     *
00512     * @param other Lyginamas studentas
00513     * @return `true` jei šis studentas eina anksčiau
00514     */
00515     bool operator<(const studentas& other) const;
00516
00517     /**
00518     * @brief Atvirkštinis rikiavimo operatorius.
00519     * @param other Lyginamas studentas
00520     * @return `other < *this`
00521     */
00522     bool operator>(const studentas& other) const;
00523
00524     /**
00525     * @brief Prieiga prie namų darbo pažymio pagal indeksą.
00526     *
00527     * @param i Pažymio indeksas (nuo 0)
00528     * @return Pažymys
00529     * @throws std::out_of_range jei indeksas už ribų
00530     */
00531     int operator[](size_t i) const;
00532
00533     /**
00534     * @brief Konvertavimas į `bool` -- ar objektas turi duomenų.
00535     *
00536     * Pažymėtas `explicit`, kad neįvyktų netyčinis konvertavimas į `int`
00537     * (pvz. `s + 1` nesikompiliuos, kaip ir turi būti).
00538     *
00539     * @return `true` jei vardas ir pavardė nėra tušti
00540     *
00541     * @par Pavyzdys
00542     * @code
00543     * studentas s;
00544     * if (!s) std::cout << "Objektas tuscias\n";
00545     * @endcode
00546     */
00547     explicit operator bool() const;
00548
00549     /// @}
00550 };
00551
00552 // =====
00553 /// @name Ne-nariai lyginimo funkcijos
00554 ///
00555 /// Naudojamos `sortStudentai()` šablone kaip komparatoriai. Nėra klasės
00556 /// nariai, nes naudoja tik viešą interfeisą -- taip laikomasi principo,
00557 /// kad klasės narių turi būti kuo mažiau.
00558 /// @{
00559 // =====
00560
00561 /**
00562 * @brief Lygina pagal vardą (didėjančiai).
00563 * @param s1 Pirmas studentas
00564 * @param s2 Antras studentas
00565 * @return `true` jei `s1` vardas abėcėliškai anksčiau
00566 */
00567 bool comparePagalVarda(const studentas& s1, const studentas& s2);
00568
00569 /**
00570 * @brief Lygina pagal pavardę (didėjančiai).
00571 * @param s1 Pirmas studentas
00572 * @param s2 Antras studentas
00573 * @return `true` jei `s1` pavardė abėcėliškai anksčiau
00574 */
00575 bool comparePagalPavarde(const studentas& s1, const studentas& s2);
00576
00577 /**
00578 * @brief Lygina pagal egzamino pažymį (mažėjančiai).
00579 * @param s1 Pirmas studentas
00580 * @param s2 Antras studentas

```

```

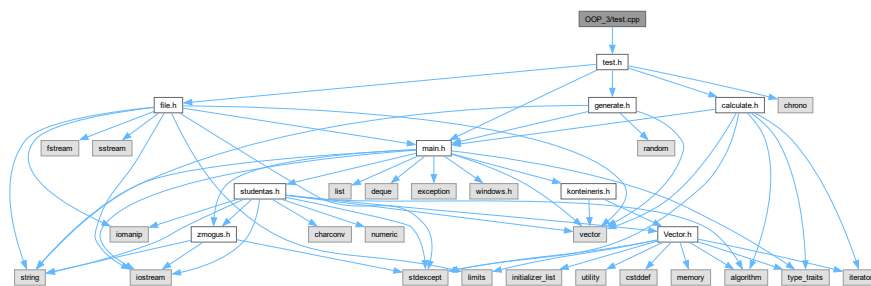
00581 * @return `true` jei `s1` egzaminas didesnis
00582 */
00583 bool comparePagalEgzamina(const studentas& s1, const studentas& s2);
00584
00585 /**
00586 * @brief Lygina pagal galutinį balą su vidurkiu (mažėjančiai).
00587 * @param s1 Pirmas studentas
00588 * @param s2 Antras studentas
00589 * @return `true` jei `s1` galutinis balas didesnis
00590 */
00591 bool comparePagalVidurki(const studentas& s1, const studentas& s2);
00592
00593 /**
00594 * @brief Lygina pagal galutinį balą su mediana (mažėjančiai).
00595 * @param s1 Pirmas studentas
00596 * @param s2 Antras studentas
00597 * @return `true` jei `s1` galutinis balas didesnis
00598 */
00599 bool comparePagalMediana(const studentas& s1, const studentas& s2);
00600
00601 /// @}
00602
00603 #endif // STUDENTAS_H_DEFINED

```

8.37 OOP_3/test.cpp Failo Nuoroda

```
#include "test.h"
```

Įtraukimo priklausomybių diagrama test.cpp:



Funkcijos

- `template<typename Container>`
`static void testContainer (const std::string &label, const std::string &filename)`
- `void testContainers (int n)`

8.37.1 Funkcijos Dokumentacija

8.37.1.1 testContainer()

```

template<typename Container>
void testContainer (
    const std::string & label,
    const std::string & filename) [static]

```

Apibrėžimas failo [test.cpp](#) eilutėje 12.

Ryšiai [calculateGalutinis\(\)](#), [partitionStudentai\(\)](#), [readStudentaiFromFile\(\)](#), ir [sortStudentai\(\)](#).

Naudojamas [testContainers\(\)](#).

8.37.1.2 testContainers()

```

void testContainers (
    int n)

```

Apibrėžimas failo [test.cpp](#) eilutėje 50.

Ryšiai [testContainer\(\)](#).

Naudojamas [runProgram\(\)](#).

8.38 test.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "test.h"
00002
00003 // -----
00004 // Vidinis šablonas -- naudojamas tik šiame faile.
00005 // Matuoja žingsnius su pasirinktu konteineriu:
00006 // 1. Nuskaitymas iš failo
00007 // 2. Galutinių balų skaičiavimas
00008 // 3. Rikiavimas
00009 // 4. Skirstymas į dvi grupes
00010 // -----
00011 template<typename Container>
00012 static void testContainer(const std::string& label, const std::string& filename)
00013 {
00014     std::cout << u8"    [" << label << u8"]\n";
00015
00016     // 1. Nuskaitymas (be skaičiavimo)
00017     auto start = std::chrono::high_resolution_clock::now();
00018     Container studentai = readStudentaiFromFile<Container>(filename);
00019     auto end = std::chrono::high_resolution_clock::now();
00020     std::chrono::duration<double> elapsed = end - start;
00021     std::cout << u8"    Nuskaitymas: " << elapsed.count() << u8" s\n";
00022
00023     // 2. Galutinių balų skaičiavimas (atskiras žingsnis)
00024     auto start2 = std::chrono::high_resolution_clock::now();
00025     calculateGalutinis(studentai);
00026     auto end2 = std::chrono::high_resolution_clock::now();
00027     std::chrono::duration<double> elapsed2 = end2 - start2;
00028     std::cout << u8"    Skaičiavimas: " << elapsed2.count() << u8" s\n";
00029
00030     // 3. Rikiavimas
00031     // vector/deque: std::sort | list: .sort() (sprendžiama if constexpr calculate.h)
00032     auto start3 = std::chrono::high_resolution_clock::now();
00033     sortStudentai(studentai, '3'); // rikiuojama pagal galutinisVid
00034     auto end3 = std::chrono::high_resolution_clock::now();
00035     std::chrono::duration<double> elapsed3 = end3 - start3;
00036     std::cout << u8"    Rikiavimas: " << elapsed3.count() << u8" s\n";
00037
00038     // 4. Skirstymas į dvi grupes (3 strategija)
00039     auto start4 = std::chrono::high_resolution_clock::now();
00040     Container vargsiukai = partitionStudentai(studentai);
00041     auto end4 = std::chrono::high_resolution_clock::now();
00042     std::chrono::duration<double> elapsed4 = end4 - start4;
00043     std::cout << u8"    Skirstymas: " << elapsed4.count() << u8" s\n";
00044 }
00045
00046 // -----
00047 // testContainers -- iškviečiama iš meniu (pasirinkimas 8)
00048 // Testuoja tą patį failą su visais trimis konteineriais
00049 // -----
00050 void testContainers(int n)
00051 {
00052     const std::string filename = "data/studentai" + std::to_string(n) + ".txt";
00053     std::cout << u8"=== Konteinerių palyginimas: " << n << u8" studentų ==\n";
00054     testContainer<Konteineris<studentas>>("vector", filename);
00055     testContainer<std::list<studentas>>("list", filename);
00056     testContainer<std::deque<studentas>>("deque", filename);
00057     std::cout << "\n";
00058 }
```

8.39 OOP_3/test.h Failo Nuoroda

```
#include "main.h"
#include "file.h"
#include "generate.h"
#include "calculate.h"
#include <chrono>
```


8.40 test.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef TEST_H_DEFINED
00002 #define TEST_H_DEFINED
00003
00004 #include "main.h"
00005 #include "file.h"
00006 #include "generate.h"
00007 #include "calculate.h"
00008 #include <chrono>
00009
00010 // -----
00011 // testGenerateStudentai -- iškviečiama iš meniu (pasirinkimas 6)
00012 // Testuoja studentų failo generavimą su n studentų.
00013 // -----
00014 template<typename Container>
00015 void testGenerateStudentai(int n)
00016 {
00017     std::cout << u8"=== Studentų failo generavimo testavimas: " << n << u8" studentų ===\n";
00018
00019     auto start = std::chrono::high_resolution_clock::now();
00020     Container studentai = generateStudentai<Container>(n);
00021     auto end = std::chrono::high_resolution_clock::now();
00022     std::chrono::duration<double> elapsed = end - start;
00023     std::cout << u8" Generavimas: " << elapsed.count() << u8" s\n";
00024
00025     start = std::chrono::high_resolution_clock::now();
00026     writeStudentaiListToFile<Container>(studentai, "teststudentai" + std::to_string(n) + ".txt");
00027     end = std::chrono::high_resolution_clock::now();
00028     elapsed = end - start;
00029     std::cout << u8" Išvedimas į failą: " << elapsed.count() << u8" s\n";
00030 }
00031
00032 // -----
00033 // testData -- iškviečiama iš meniu (pasirinkimas 7)
00034 // Testuoja studentų duomenų apdorojimą su n studentų.
00035 // -----
00036 template<typename Container>
00037 void testData(int n)
00038 {
00039     auto startTotal = std::chrono::high_resolution_clock::now();
00040     std::cout << u8"Pradedamas testavimas su " << n << u8" studentų duomenimis.\n";
00041     std::cout << u8"-----\n";
00042
00043     auto start = std::chrono::high_resolution_clock::now();
00044     Container studentai = readStudentaiFromFile<Container>(
00045         "data/studentai" + std::to_string(n) + ".txt");
00046     auto end = std::chrono::high_resolution_clock::now();
00047     std::chrono::duration<double> elapsed = end - start;
00048     std::cout << n << u8" failo skaitymas užtruko: " << elapsed.count() << u8" sekundžių.\n";
00049
00050     auto start2 = std::chrono::high_resolution_clock::now();
00051     calculateGalutinis(studentai);
00052     auto end2 = std::chrono::high_resolution_clock::now();
00053     std::chrono::duration<double> elapsed2 = end2 - start2;
00054     std::cout << n << u8" galutinio balo skaičiavimas užtruko: "
00055         << elapsed2.count() << u8" sekundžių.\n";
00056
00057     auto start3 = std::chrono::high_resolution_clock::now();
00058     sortStudentai(studentai, '4');
00059     auto end3 = std::chrono::high_resolution_clock::now();
00060     std::chrono::duration<double> elapsed3 = end3 - start3;
00061     std::cout << n << u8" rikiavimas pagal medianą užtruko: "
00062         << elapsed3.count() << u8" sekundžių.\n";
00063
00064     auto start4 = std::chrono::high_resolution_clock::now();
00065     splitResult<Container> result = splitStudentai<Container>(studentai);
00066     auto end4 = std::chrono::high_resolution_clock::now();
00067     std::chrono::duration<double> elapsed4 = end4 - start4;
00068     std::cout << n << u8" skirstymas į kietiakai ir vargsiukai užtruko: "
00069         << elapsed4.count() << u8" sekundžių.\n";
00070
00071     auto start5 = std::chrono::high_resolution_clock::now();
00072     writeStudentaiListToFile<Container>(result.kietiakai, "testkietiakai" + std::to_string(n) +
00073         ".txt");
00074     auto end5 = std::chrono::high_resolution_clock::now();
00075     std::chrono::duration<double> elapsed5 = end5 - start5;
00076     std::cout << n << u8" Kietiakai failo generavimas užtruko: "
00077         << elapsed5.count() << u8" sekundžių.\n";
00078
00079     auto start6 = std::chrono::high_resolution_clock::now();
00080     writeStudentaiListToFile<Container>(result.vargsiukai, "testvargsiukai" + std::to_string(n) +
00081         ".txt");
00082     auto end6 = std::chrono::high_resolution_clock::now();
00083     std::chrono::duration<double> elapsed6 = end6 - start6;

```

```

00082
00083     auto endTotal = std::chrono::high_resolution_clock::now();
00084     std::chrono::duration<double> elapsedTotal = endTotal - startTotal;
00085     std::cout << n << u8" Vargsiukai failo generavimas užtruko: "
00086             << elapsed6.count() << u8" sekundžių.\n";
00087     std::cout << n << u8" Visas testavimas užtruko: "
00088             << elapsedTotal.count() << u8" sekundžių.\n\n";
00089 }
00090
00091 #endif

```

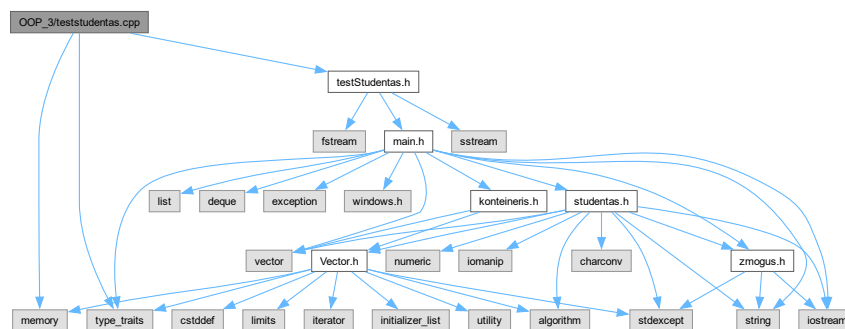
8.41 OOP_3/teststudentas.cpp Failo Nuoroda

```

#include "testStudentas.h"
#include <memory>
#include <type_traits>

```

Įtraukimo priklausomybių diagrama teststudentas.cpp:



Funkcijos

- bool `testStudentasKlase` ()

8.41.1 Funkcijos Dokumentacija

8.41.1.1 testStudentasKlase()

```
bool testStudentasKlase ()
```

Apibrėžimas failo `teststudentas.cpp` eilutėje 45.

Ryšiai `studentas::addPazymys()`, `studentas::appendListTo()`, `Vector< T, Allocator >::back()`, `Vector< T, Allocator >::begin()`, `studentas::calculateGalutinis()`, `studentas::egzaminas()`, `Vector< T, Allocator >::empty()`, `Vector< T, Allocator >::end()`, `zmogus::galutinis()`, `studentas::galutinisMed()`, `studentas::galutinisVid()`, `studentas::gyvuStudentu`, `zmogus::gyvuZmoniu`, `studentas::isvalyk()`, `studentas::nd()`, `studentas::parseFromLine()`, `zmogus::pavarde()`, `zmogus::pilnasVardas()`, `Vector< T, Allocator >::push_back()`, `Vector< T, Allocator >::reserve()`, `studentas::setEgzaminas()`, `studentas::setNd()`, `zmogus::setPavarde()`, `zmogus::setVardas()`, `Vector< T, Allocator >::size()`, `zmogus::tipas()`, ir `zmogus::vardas()`.

Naudojamas `runProgram()`.

8.42 teststudentas.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "testStudentas.h"
00002 #include <memory>           // unique_ptr
00003 #include <type_traits>      // is_abstract, is_base_of
00004
00005 // =====
00006 // Testavimo infrastruktūra
00007 // =====
00008

```



```

00009 namespace {
00010
00011     int testuPraejo = 0;
00012     int testuKrito = 0;
00013
00014     void tikrink(bool salyga, const std::string& aprasymas)
00015     {
00016         if (salyga) {
00017             ++testuPraejo;
00018             std::cout << " [ OK ] " << aprasymas << "\n";
00019         }
00020         else {
00021             ++testuKrito;
00022             std::cout << " [KLAIDA] " << aprasymas << "\n";
00023         }
00024     }
00025
00026     void skyrius(const std::string& pavadinimas)
00027     {
00028         std::cout << "\n--- " << pavadinimas << " ---\n";
00029     }
00030
00031     // Testinis studentas su žinomomis reikšmėmis
00032     // ND: 5,5,5,5,5 -> vidurkis 5.0, mediana 5.0
00033     // Egzaminas 10 -> galutinis = 0.4*5 + 0.6*10 = 8.0
00034     studentas kurkTestini()
00035     {
00036         return studentas("Jonas", "Jonaitis", { 5, 5, 5, 5, 5 }, 10);
00037     }
00038
00039 } // anoniminis namespace
00040
00041 // =====
00042 // Pagrindinė testavimo funkcija
00043 // =====
00044
00045 bool testStudentasKlase()
00046 {
00047     testuPraejo = 0;
00048     testuKrito = 0;
00049
00050     std::cout << "\n===== \n";
00051     std::cout << " KLASIU zmogus IR studentas TESTAVIMAS (v1.5) \n";
00052     std::cout << "===== \n";
00053
00054     const int pradinisStudentu = studentas::gyvuStudentu;
00055     const int pradinisZmoniu = zmogus::gyvuZmoniu;
00056
00057     // =====
00058     // A. ABSTRAKCIJOJI KLAISE IR PAVELDEJIMAS (nauja v1.5)
00059     // =====
00060     skyrius("A1. Klase zmogus yra ABSTRAKTI");
00061     {
00062         tikrink(std::is_abstract<zmogus>::value,
00063             "std::is_abstract<zmogus> = true (turi grynai virtualiu metodu)");
00064         tikrink(!std::is_abstract<studentas>::value,
00065             "std::is_abstract<studentas> = false (visi metodai realizuoti)");
00066
00067         // Sie bandymai NESIKOMPILIUOTU -- todel patikra atliekama statiskai:
00068         // zmogus z; // klaida: abstrakti klase
00069         // zmogus* p = new zmogus(); // klaida: abstrakti klase
00070         tikrink(!std::is_default_constructible<zmogus>::value,
00071             "zmogus objekto sukurti NEIMANOMA (is_default_constructible = false)");
00072         tikrink(std::is_default_constructible<studentas>::value,
00073             "studentas objekta sukurti galima");
00074     }
00075
00076     skyrius("A2. Paveldejimo rysys");
00077     {
00078         tikrink((std::is_base_of<zmogus, studentas>::value),
00079             "studentas yra isvestine is zmogus");
00080         tikrink((std::is_convertible<studentas*, zmogus*>::value),
00081             "studentas* konvertuojasi i zmogus* (public paveldejimas)");
00082         tikrink(std::has_virtual_destructor<zmogus>::value,
00083             "zmogus turi VIRTUALU destruktoriu");
00084     }
00085
00086     skyrius("A3. Paveldeti bazines klases metodai");
00087     {
00088         studentas s = kurkTestini();
00089
00090         tikrink(s.vardas() == "Jonas", "paveldetas vardas() veikia");
00091         tikrink(s.pavarde() == "Jonaitis", "paveldetas pavarde() veikia");
00092         tikrink(s.pilnasVardas() == "Jonas Jonaitis", "paveldetas pilnasVardas() veikia");
00093
00094         s.setVardas("Petras");
00095         tikrink(s.vardas() == "Petras", "paveldetas setVardas() veikia");

```

```

00096
00097     bool metaTuscia = false;
00098     try { s.setPavarde(""); }
00099     catch (const std::invalid_argument&) { metaTuscia = true; }
00100     tikrink(metaTuscia, "paveldetas setPavarde() validuoja tuscia reiksme");
00101 }
00102
00103 skyrius("A4. POLIMORFIZMAS per bazines klases rodykle");
00104 {
00105     studentas s = kurkTestini();
00106     s.calculateGalutinis();
00107
00108     // Nuoroda i bazine klase
00109     zmogus& z = s;
00110     tikrink(z.vardas() == "Jonas", "prieiga prie vardo per zmogus& nuoroda");
00111     tikrink(z.tipas() == "Studentas",
00112             "virtualus tipas() grazina isvestines klases reiksme");
00113     tikrink(std::abs(z.galutinis() - 8.0) < 1e-9,
00114             "virtualus galutinis() = 8.0 (isvestines klases realizacija)");
00115
00116     // Rodykle i bazine klase
00117     std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00118         "Ona", "Onaite", Vector<int>{ 10, 10 }, 10);
00119     tikrink(p->tipas() == "Studentas", "virtualus tipas() per unique_ptr<zmogus>");
00120     tikrink(p->pilnasVardas() == "Ona Onaite", "paveldetas metodos per rodykle");
00121
00122     // dynamic_cast atgal i isvestine klase
00123     studentas* sp = dynamic_cast<studentas*>(p.get());
00124     tikrink(sp != nullptr, "dynamic_cast<studentas*> pavyko");
00125     if (sp) tikrink(sp->nd().size() == 2, "prieiga prie isvestines klases duomenu");
00126 }
00127
00128 skyrius("A5. VIRTUALUS destruktorius");
00129 {
00130     const int priesZmoniu = zmogus::gyvuZmoniu;
00131     const int priesStudentu = studentas::gyvuStudentu;
00132
00133     {
00134         // Objektas kuriamas kaip studentas, naikinamas per zmogus rodykle.
00135         // Be virtual destruktoriaus studentas laukai liktu neatlaisvinti.
00136         std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00137             "Testas", "Testaitis", Vector<int>{ 5, 5 }, 5);
00138
00139         tikrink(zmogus::gyvuZmoniu == priesZmoniu + 1,
00140                 "sukurus objekta zmogus skaitiklis padidejo");
00141         tikrink(studentas::gyvuStudentu == priesStudentu + 1,
00142                 "sukurus objekta studentas skaitiklis padidejo");
00143     }
00144
00145     tikrink(studentas::gyvuStudentu == priesStudentu,
00146             "~studentas() iskviestas naikinant per zmogus rodykle");
00147     tikrink(zmogus::gyvuZmoniu == priesZmoniu,
00148             "~zmogus() iskviestas po ~studentas()");
00149 }
00150
00151 skyrius("A6. Polimorfiniai operatoriai « ir »");
00152 {
00153     studentas s = kurkTestini();
00154     s.calculateGalutinis();
00155
00156     // operator« priima zmogus& ir kviecia virtualu print()
00157     zmogus& z = s;
00158     std::ostringstream oss;
00159     oss << z;
00160     tikrink(oss.str().find("Jonas") != std::string::npos,
00161             "operator« per zmogus& nuoroda kviecia studentas::print()");
00162
00163     // operator« priima zmogus& ir kviecia virtualu read()
00164     studentas s2;
00165     zmogus& z2 = s2;
00166     std::istringstream iss("Ona Onaite 9 8 7");
00167     iss >> z2;
00168     tikrink(s2.vardas() == "Ona",
00169             "operator« per zmogus& nuoroda kviecia studentas::read()");
00170     tikrink(s2.egzaminas() == 7, "nuskaityti duomenys teisingi");
00171 }
00172
00173 // =====
00174 // B. KONSTRUKTORIAI (patikra is vl.2)
00175 // =====
00176 skyrius("B1. Numatytasis konstruktorius");
00177 {
00178     studentas s;
00179     tikrink(s.vardas().empty(), "vardas tuscias");
00180     tikrink(s.pavarde().empty(), "pavarde tuscia");
00181     tikrink(s.nd().empty(), "nd vektorius tuscias");
00182     tikrink(s.egzaminas() == 0, "egzaminas = 0");

```

```

00183         tikrink(s.galutinisVid() == 0.0, "galutinisVid = 0.0");
00184         tikrink(s.galutinisMed() == 0.0, "galutinisMed = 0.0");
00185         tikrink(!static_cast<bool>(s), "operator bool grazina false tusciam objektui");
00186     }
00187
00188     skyrius("B2. Pilnas konstruktorius");
00189     {
00190         studentas s("Jonas", "Jonaitis", { 5, 5, 5, 5, 5 }, 10);
00191         tikrink(s.vardas() == "Jonas", "vardas issaugotas (per bazine klase)");
00192         tikrink(s.pavarde() == "Jonaitis", "pavarde issaugota (per bazine klase)");
00193         tikrink(s.nd().size() == 5, "nd dydis = 5");
00194         tikrink(s.egzaminas() == 10, "egzaminas = 10");
00195         tikrink(static_cast<bool>(s), "operator bool grazina true");
00196     }
00197
00198     skyrius("B3. Validacija konstruktoriuje");
00199     {
00200         bool metaTusciaVarda = false;
00201         try { studentas s("", "Jonaitis", { 5 }, 10); }
00202         catch (const std::invalid_argument&) { metaTusciaVarda = true; }
00203         tikrink(metaTusciaVarda,
00204             "BAZINE klase meta invalid_argument, kai vardas tuscias");
00205
00206         bool metaTusciaPavarde = false;
00207         try { studentas s("Jonas", "", { 5 }, 10); }
00208         catch (const std::invalid_argument&) { metaTusciaPavarde = true; }
00209         tikrink(metaTusciaPavarde,
00210             "BAZINE klase meta invalid_argument, kai pavarde tuscia");
00211
00212         bool metaBlogoND = false;
00213         try { studentas s("Jonas", "Jonaitis", { 5, 99 }, 10); }
00214         catch (const std::runtime_error&) { metaBlogoND = true; }
00215         tikrink(metaBlogoND,
00216             "ISVESTINE klase meta runtime_error, kai ND pazymys = 99");
00217
00218         bool metaBlogoEgz = false;
00219         try { studentas s("Jonas", "Jonaitis", { 5 }, 0); }
00220         catch (const std::runtime_error&) { metaBlogoEgz = true; }
00221         tikrink(metaBlogoEgz,
00222             "ISVESTINE klase meta runtime_error, kai egzaminas = 0");
00223     }
00224
00225     skyrius("B4. Konstruktorius is srauto");
00226     {
00227         std::istringstream iss("Petras Petraitis 8 9 10 7");
00228         studentas s(iss);
00229         tikrink(s.vardas() == "Petras", "vardas nuskaitytas");
00230         tikrink(s.pavarde() == "Petraitis", "pavarde nuskaityta");
00231         tikrink(s.nd().size() == 3, "nd dydis = 3 (paskutinis skaicius = egzaminas)");
00232         tikrink(s.egzaminas() == 7, "egzaminas = 7");
00233     }
00234
00235     // =====
00236     // C. RULE OF FIVE (patikra is v1.2 su paveldejimu)
00237     // =====
00238     skyrius("C1. Kopijavimo konstruktorius");
00239     {
00240         studentas originalas = kurkTestini();
00241         originalas.calculateGalutinis();
00242
00243         studentas kopija(originalas);
00244
00245         // Svarbiausia paveldejime: ar BAZINES klases laukai nukopijuoti?
00246         tikrink(kopija.vardas() == originalas.vardas(),
00247             "BAZINES klases vardas nukopijuotas (zmogus(other) kvietimas)");
00248         tikrink(kopija.pavarde() == originalas.pavarde(),
00249             "BAZINES klases pavarde nukopijuota");
00250         tikrink(kopija.nd() == originalas.nd(), "nd vektorius nukopijuotas");
00251         tikrink(kopija.egzaminas() == originalas.egzaminas(), "egzaminas nukopijuotas");
00252         tikrink(kopija.galutinisVid() == originalas.galutinisVid(),
00253             "galutinisVid nukopijuotas");
00254
00255         // Deep copy patikra
00256         kopija.setVardas("Pakeistas");
00257         tikrink(originalas.vardas() == "Jonas",
00258             "originalas NEPAKISTA, kai keiciama kopija (deep copy)");
00259     }
00260
00261     skyrius("C2. Kopijavimo priskyrimo operatorius");
00262     {
00263         studentas a = kurkTestini();
00264         a.calculateGalutinis();
00265         studentas b;
00266
00267         b = a;
00268
00269         tikrink(b.vardas() == a.vardas(),

```

```

00270         "BAZINES klases vardas priskirtas (zmogus::operator= kvietimas)");
00271     tikrink(b.nd() == a.nd(), "nd vektorius priskirtas");
00272     tikrink(b.galutinisVid() == a.galutinisVid(), "galutinisVid priskirtas");
00273
00274     a = a;
00275     tikrink(a.vardas() == "Jonas" && a.nd().size() == 5,
00276         "priskyrimas sau (a = a) nesugadina objekto");
00277
00278     studentas c, d;
00279     c = d = a;
00280     tikrink(c.vardas() == "Jonas" && d.vardas() == "Jonas",
00281         "grandinis priskyrimas (c = d = a) veikia");
00282 }
00283
00284 skyrius("C3. Perkeliimo (move) konstruktorius");
00285 {
00286     studentas saltinis = kurkTestini();
00287     saltinis.calculateGalutinis();
00288     const double tiketinasVid = saltinis.galutinisVid();
00289
00290     studentas tikslas(std::move(saltinis));
00291
00292     tikrink(tikslas.vardas() == "Jonas",
00293         "BAZINES klases vardas perkeltas (zmogus(std::move(other)))");
00294     tikrink(tikslas.nd().size() == 5, "nd vektorius perkeltas");
00295     tikrink(tikslas.galutinisVid() == tiketinasVid, "galutinisVid perkeltas");
00296
00297     tikrink(saltinis.vardas().empty(),
00298         "saltinio BAZINES klases vardas tuscias po perkeliimo");
00299     tikrink(saltinis.nd().empty(), "saltinio nd tuscias po perkeliimo");
00300     tikrink(saltinis.egzaminas() == 0, "saltinio egzaminas = 0 po perkeliimo");
00301     tikrink(!static_cast<bool>(saltinis), "saltinis konvertuojasi i false");
00302 }
00303
00304 skyrius("C4. Perkeliimo (move) priskyrimo operatorius");
00305 {
00306     studentas saltinis = kurkTestini();
00307     saltinis.calculateGalutinis();
00308     studentas tikslas("Senas", "Senaitis", { 1, 1 }, 1);
00309
00310     tikslas = std::move(saltinis);
00311
00312     tikrink(tikslas.vardas() == "Jonas",
00313         "BAZINES klases vardas perkeltas (zmogus::operator=(std::move))");
00314     tikrink(tikslas.nd().size() == 5, "nd perkeltas priskyrimu");
00315     tikrink(saltinis.vardas().empty(), "saltinis istustintas");
00316
00317     studentas x = kurkTestini();
00318     x = std::move(x);
00319     tikrink(x.vardas() == "Jonas" || x.vardas().empty(),
00320         "move priskyrimas sau nesukelia luzimo");
00321 }
00322
00323 skyrius("C5. noexcept specifikatorius");
00324 {
00325     tikrink(std::is_nothrow_move_constructible<studentas>::value,
00326         "studentas move konstruktorius yra noexcept");
00327     tikrink(std::is_nothrow_move_assignable<studentas>::value,
00328         "studentas move priskyrimas yra noexcept");
00329 }
00330
00331 skyrius("C6. Destruktorius ir naikinimo tvarka");
00332 {
00333     const int priesStudentu = studentas::gyvuStudentu;
00334     const int priesZmoniu = zmogus::gyvuZmoniu;
00335     {
00336         studentas a = kurkTestini();
00337         studentas b = kurkTestini();
00338         studentas c(a);
00339         tikrink(studentas::gyvuStudentu == priesStudentu + 3,
00340             "sukurti 3 studentai -- studentu skaitiklis padidejo 3");
00341         tikrink(zmogus::gyvuZmoniu == priesZmoniu + 3,
00342             "kartu sukurtos 3 zmogus bazines dalys");
00343     }
00344     tikrink(studentas::gyvuStudentu == priesStudentu,
00345         "~studentas() sumazino studentu skaitikli iki pradinio");
00346     tikrink(zmogus::gyvuZmoniu == priesZmoniu,
00347         "~zmogus() sumazino zmoniu skaitikli iki pradinio");
00348 }
00349
00350 // =====
00351 // D. IVESTIES / ISVESTIES OPERATORIAI (patikra is v1.2)
00352 // =====
00353 skyrius("D1. Investies operatorius »");
00354 {
00355     std::istringstream iss("Ona Onaite 10 9 8 6");
00356     studentas s;

```

```

00357         iss » s;
00358
00359         tikrink(s.vardas() == "Ona", "vardas nuskaitytas per »");
00360         tikrink(s.pavarde() == "Onaite", "pavarde nuskaityta per »");
00361         tikrink(s.nd().size() == 3, "nuskaityti 3 ND pazymiai");
00362         tikrink(s.egzaminas() == 6, "egzaminas = 6 (paskutinis skaicius)");
00363     }
00364
00365     skyrius("D2. Ivesties operatorius «");
00366     {
00367         studentas s = kurkTestini();
00368         s.calculateGalutinis();
00369
00370         std::ostringstream oss;
00371         oss « s;
00372         const std::string rez = oss.str();
00373
00374         tikrink(rez.find("Jonas") != std::string::npos, "isvestyje yra vardas");
00375         tikrink(rez.find("Jonaitis") != std::string::npos, "isvestyje yra pavarde");
00376         tikrink(rez.find("8.00") != std::string::npos,
00377             "isvestyje yra galutinis balas 8.00 (0.4*5 + 0.6*10)");
00378         tikrink(rez.size() >= 60, "isvestis suformatuota stulpeliais");
00379     }
00380
00381     skyrius("D3. Ivestis is failo (parseFromLine)");
00382     {
00383         studentas s;
00384         s.parseFromLine("Antanas Antanaitis 7 8 9 10 5", 4, 1);
00385
00386         tikrink(s.vardas() == "Antanas", "vardas parsintas");
00387         tikrink(s.pavarde() == "Antanaitis", "pavarde parsinta");
00388         tikrink(s.nd().size() == 4, "parsinti 4 ND pazymiai");
00389         tikrink(s.egzaminas() == 5, "egzaminas = 5");
00390
00391         bool metaBloga = false;
00392         try { studentas bad; bad.parseFromLine("Vardas Pavarde 99 5", 2, 7); }
00393         catch (const std::runtime_error&) { metaBloga = true; }
00394         tikrink(metaBloga, "meta runtime_error, kai pazymys uz ribu");
00395
00396         bool metaTrukstama = false;
00397         try { studentas bad; bad.parseFromLine("TikVardas", 2, 3); }
00398         catch (const std::runtime_error&) { metaTrukstama = true; }
00399         tikrink(metaTrukstama, "meta runtime_error, kai truksta duomenu");
00400     }
00401
00402     skyrius("D4. Ivestis i faila (appendListTo)");
00403     {
00404         studentas s("Rasa", "Rasaite", { 6, 7, 8 }, 9);
00405
00406         std::string buferis;
00407         s.appendListTo(buferis);
00408
00409         tikrink(buferis.find("Rasa") != std::string::npos, "buferyje yra vardas");
00410         tikrink(buferis.find("Rasaite") != std::string::npos, "buferyje yra pavarde");
00411         tikrink(buferis.back() == '\n', "eilute baigiasi nauja eilute");
00412         tikrink(buferis.size() == 25 + 25 + 4 * 10 + 1,
00413             "eilutes ilgis atitinka formatavima (25+25+4*10+1)");
00414
00415         studentas s2;
00416         s2.parseFromLine(buferis, 3, 1);
00417         tikrink(s2.vardas() == s.vardas() && s2.nd() == s.nd() &&
00418             s2.egzaminas() == s.egzaminas(),
00419             "isvesta eilute korektiskai nuskaitoma atgal (round-trip)");
00420     }
00421
00422     // =====
00423     // E. PAPILDOMI OPERATORIAI (patikra is vl.2)
00424     // =====
00425     skyrius("E1. Lyginimo operatoriai == ir !=");
00426     {
00427         studentas a("Jonas", "Jonaitis", { 5, 6 }, 7);
00428         studentas b("Jonas", "Jonaitis", { 5, 6 }, 7);
00429         studentas c("Petras", "Petraitis", { 5, 6 }, 7);
00430
00431         tikrink(a == b, "vienodi objektai lygus (==)");
00432         tikrink(a != c, "skirtingi objektai nelygus (!=)");
00433         tikrink(!(a == c), "== grazina false skirtingiems");
00434     }
00435
00436     skyrius("E2. Rikiavimo operatoriai < ir >");
00437     {
00438         studentas a("Jonas", "Adamkus", { 5 }, 7);
00439         studentas b("Jonas", "Zukauskas", { 5 }, 7);
00440
00441         tikrink(a < b, "Adamkus < Zukauskas (pagal pavarde)");
00442         tikrink(b > a, "Zukauskas > Adamkus");
00443     }

```

```

00444     studentas c("Antanas", "Petraitis", { 5 }, 7);
00445     studentas d("Zigmas", "Petraitis", { 5 }, 7);
00446     tikrink(c < d, "esant vienodoms pavidems lyginama pagal vardą");
00447
00448     Konteineris<studentas> v = { b, a };
00449     std::sort(v.begin(), v.end());
00450     tikrink(v[0].pavarde() == "Adamkus", "std::sort su operator< veikia");
00451 }
00452
00453 skyrius("E3. Indeksavimo operatorius []");
00454 {
00455     studentas s("Jonas", "Jonaitis", { 3, 6, 9 }, 10);
00456
00457     tikrink(s[0] == 3, "s[0] = 3");
00458     tikrink(s[1] == 6, "s[1] = 6");
00459     tikrink(s[2] == 9, "s[2] = 9");
00460
00461     bool metaUzRibu = false;
00462     try { int x = s[99]; (void)x; }
00463     catch (const std::out_of_range&) { metaUzRibu = true; }
00464     tikrink(metaUzRibu, "meta out_of_range, kai indeksas uz ribu");
00465 }
00466
00467 skyrius("E4. Skaiciavimo metodas calculateGalutinis");
00468 {
00469     // ND: 2,4,6,8,10 -> vidurkis 6.0, mediana 6.0
00470     studentas s("Testas", "Testaitis", { 2, 4, 6, 8, 10 }, 5);
00471     s.calculateGalutinis();
00472
00473     tikrink(std::abs(s.galutinisVid() - 5.4) < 1e-9,
00474             "galutinisVid = 5.4 (0.4*6.0 + 0.6*5)");
00475     tikrink(std::abs(s.galutinisMed() - 5.4) < 1e-9,
00476             "galutinisMed = 5.4 (mediana 6.0)");
00477
00478     // Lyginis medianos atvejis: 1,2,3,4 -> (2+3)/2 = 2.5
00479     studentas s2("Testas", "Testaitis", { 1, 2, 3, 4 }, 10);
00480     s2.calculateGalutinis();
00481     tikrink(std::abs(s2.galutinisMed() - (0.4 * 2.5 + 0.6 * 10)) < 1e-9,
00482             "mediana lyginiam kiekiui = (2+3)/2 = 2.5");
00483
00484     // Tuscias ND sarasas
00485     studentas s3;
00486     s3.setVardas("Tusciai");
00487     s3.setPavarde("Tusciaitis");
00488     s3.setEgzaminas(8);
00489     s3.calculateGalutinis();
00490     tikrink(std::abs(s3.galutinisVid() - 4.8) < 1e-9,
00491             "tuscias ND sarasas: galutinis = 0.6*8 = 4.8");
00492 }
00493
00494 skyrius("E5. Set'eriai ir isvalyk");
00495 {
00496     studentas s;
00497     s.setVardas("Naujas");           // paveldetas is zmogus
00498     s.setPavarde("Naujaitis");       // paveldetas is zmogus
00499     s.setNd({ 5, 6, 7 });           // studentas metodas
00500     s.setEgzaminas(9);               // studentas metodas
00501
00502     tikrink(s.vardas() == "Naujas", "paveldetas setVardas veikia");
00503     tikrink(s.nd().size() == 3, "setNd veikia");
00504     tikrink(s.egzaminas() == 9, "setEgzaminas veikia");
00505
00506     s.addPazymys(10);
00507     tikrink(s.nd().size() == 4 && s.nd().back() == 10, "addPazymys veikia");
00508
00509     bool metaBloga = false;
00510     try { s.setEgzaminas(50); }
00511     catch (const std::runtime_error&) { metaBloga = true; }
00512     tikrink(metaBloga, "setEgzaminas meta klaida, kai reiksme uz ribu");
00513
00514     s.isvalyk();
00515     tikrink(s.vardas().empty() && s.nd().empty() && s.egzaminas() == 0,
00516             "isvalyk() istustina ir bazines, ir isvestines klases laukus");
00517 }
00518
00519 skyrius("E6. Veikimas konteineriuose (move semantika)");
00520 {
00521     Konteineris<studentas> v;
00522     v.reserve(4);
00523     for (int i = 0; i < 4; ++i)
00524         v.push_back(studentas("Vardas" + std::to_string(i),
00525                               "Pavarde" + std::to_string(i),
00526                               { 5, 6, 7 }, 8));
00527
00528     tikrink(v.size() == 4, "4 objektai idėti i vektoriu");
00529
00530     v.reserve(100);

```

```

00531         tikrink(v[0].vardas() == "Vardas0" && v[3].vardas() == "Vardas3",
00532                 "duomenys islieka po vektoriaus perskirstymo");
00533
00534         Konteineris<studentas> v2 = std::move(v);
00535         tikrink(v2.size() == 4, "vektorius perkeltas su std::move");
00536         tikrink(v2[2].vardas() == "Vardas2", "duomenys teisingi po perkavimo");
00537     }
00538
00539     skyrius("E7. Polimorfinis konteineris (nauja v1.5)");
00540     {
00541         // Konteineris bazines klases rodykliu -- leidzia laikyti
00542         // skirtingu isvestiniu klasiu objektus vienoje vietoje
00543         Konteineris<std::unique_ptr<zmogus>> zmones;
00544         zmones.push_back(std::make_unique<studentas>(
00545             "Pirmas", "Pirmaitis", Vector<int>{ 10, 10 }, 10));
00546         zmones.push_back(std::make_unique<studentas>(
00547             "Antras", "Antraitis", Vector<int>{ 1, 1 }, 1));
00548
00549         for (auto& z : zmones) {
00550             studentas* sp = dynamic_cast<studentas*>(z.get());
00551             if (sp) sp->calculateGalutinis();
00552         }
00553
00554         tikrink(zmones.size() == 2, "polimorfiniame konteineryje 2 objektai");
00555         tikrink(zmones[0]->tipas() == "Studentas", "virtualus tipas() veikia");
00556         tikrink(std::abs(zmones[0]->galutinis() - 10.0) < 1e-9,
00557                 "pirmojo galutinis() = 10.0");
00558         tikrink(std::abs(zmones[1]->galutinis() - 1.0) < 1e-9,
00559                 "antrojo galutinis() = 1.0");
00560
00561         // Isvedimas per polimorfini operator«
00562         std::ostringstream oss;
00563         for (const auto& z : zmones)
00564             oss << *z << "\n";
00565         tikrink(oss.str().find("Pirmas") != std::string::npos &&
00566                 oss.str().find("Antras") != std::string::npos,
00567                 "polimorfinis operator« isveda visus objektus");
00568     }
00569
00570     // =====
00571     // Rezultatai
00572     // =====
00573     const int galutinisStudentu = studentas::gyvuStudentu;
00574     const int galutinisZmoniu = zmogus::gyvuZmoniu;
00575
00576     std::cout << "\n=====\\n";
00577     std::cout << " REZULTATAI\\n";
00578     std::cout << "-----\\n";
00579     std::cout << " Praejo: " << testuPraejo << "\\n";
00580     std::cout << " Krito: " << testuKrito << "\\n";
00581     std::cout << " Is viso: " << (testuPraejo + testuKrito) << "\\n";
00582     std::cout << "-----\\n";
00583     std::cout << " studentas objektu nutekejimas: "
00584             << (galutinisStudentu == pradinisStudentu ? "OK" : "KLaida") << "\\n";
00585     std::cout << " zmogus objektu nutekejimas: "
00586             << (galutinisZmoniu == pradinisZmoniu ? "OK" : "KLaida") << "\\n";
00587     std::cout << "=====\\n\\n";
00588
00589     return testuKrito == 0
00590         && galutinisStudentu == pradinisStudentu
00591         && galutinisZmoniu == pradinisZmoniu;
00592 }

```

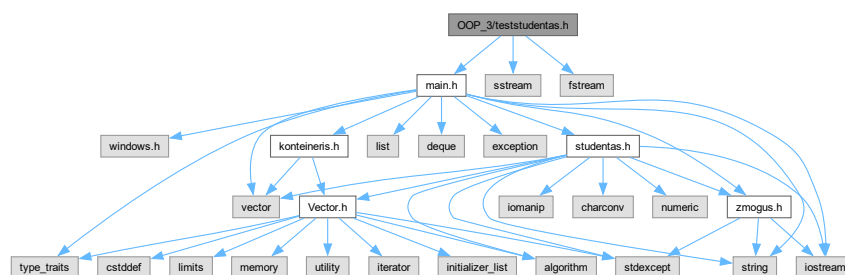
8.43 OOP_3/teststudentas.h Failo Nuoroda

```

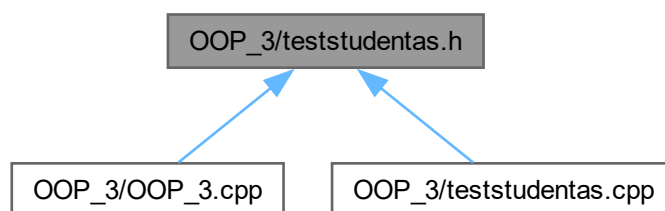
#include "main.h"
#include <sstream>
#include <fstream>

```

Įtraukimo priklausomybių diagrama teststudentas.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Funkcijos

- bool [testStudentasKlase](#) ()

8.43.1 Funkcijos Dokumentacija

8.43.1.1 testStudentasKlase()

```
bool testStudentasKlase ()
```

Apibrėžimas failo [teststudentas.cpp](#) eilutėje 45.

8.44 teststudentas.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef TESTSTUDENTAS_H_DEFINED
00002 #define TESTSTUDENTAS_H_DEFINED
00003
00004 #include "main.h"
00005 #include <sstream>
00006 #include <fstream>
00007
00008 // -----
00009 // testStudentasKlase -- rankinis klasių metodų testas.
00010 //
00011 // v1.5: papildyta abstrakčios bazinės klasės zmogus ir
00012 // paveldėjimo testais (A skyrius), išlaikant visus v1.2
00013 // testus (B-E skyriai).
00014 //
00015 // Iškviečiama iš meniu (pasirinkimas 9).
00016 // Gražina true, jei visi testai praėjo.
00017 // -----
  
```



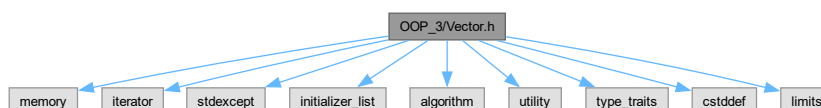
```
00018 bool testStudentasKlase();
00019
00020 #endif // TESTSTUDENTAS_H_DEFINED
```

8.45 OOP 3/Vector.h Failo Nuoroda

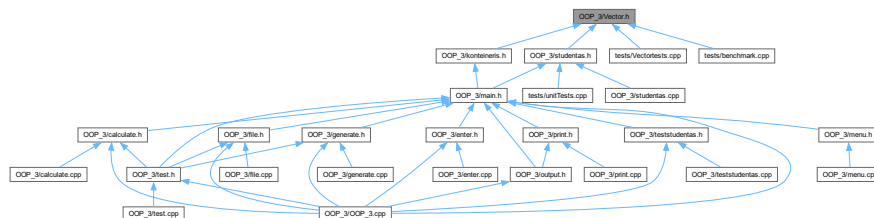
Nuosava std::vector alternatyva — dinaminis masyvas.

```
#include <memory>
#include <iterator>
#include <stdexcept>
#include <initializer_list>
#include <algorithm>
#include <utility>
#include <type_traits>
#include <cstdint>
#include <limits>
```

Įtraukimo priklausomybių diagrama Vector.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class `Vector< T, Allocator >`

Dinaminis masyvas — pilnavertė std::vector alternatyva.

Funkcijos

Ne-nariai operatoriai

- `template<typename T, typename A>`
`bool operator== (const Vector< T, A > &a, const Vector< T, A > &b)`
Lygina du vektorius.
- `template<typename T, typename A>`
`bool operator!= (const Vector< T, A > &a, const Vector< T, A > &b)`
Tikrina, ar vektoriai skiriasi.
- `template<typename T, typename A>`
`bool operator< (const Vector< T, A > &a, const Vector< T, A > &b)`
Leksikografinis palyginimas.
- `template<typename T, typename A>`
`bool operator<= (const Vector< T, A > &a, const Vector< T, A > &b)`

- Leksikografinis palyginimas ().*
 • `template<typename T, typename A>`
 `bool operator> (const Vector< T, A > &a, const Vector< T, A > &b)`
Leksikografinis palyginimas (>).
- `template<typename T, typename A>`
 `bool operator>= (const Vector< T, A > &a, const Vector< T, A > &b)`
Leksikografinis palyginimas ().
- `template<typename T, typename A>`
 `void swap (Vector< T, A > &a, Vector< T, A > &b) noexcept`
Sukeičia dviejų vektorių turinį.
- `template<typename T, typename A, typename U>`
 `Vector< T, A >::size_type erase (Vector< T, A > &c, const U &value)`
Pašalina visus elementus, lygius value.
- `template<typename T, typename A, typename Pred>`
 `Vector< T, A >::size_type erase_if (Vector< T, A > &c, Pred pred)`
Pašalina visus elementus, tenkinančius predikatą.

8.45.1 Smulkus aprašymas

Nuosava `std::vector` alternatyva — dinaminis masyvas.

Autorius

Marius Augustinas

Data

2025

Apibrėžimas faile [Vector.h](#).

8.45.2 Funkcijos Dokumentacija

8.45.2.1 operator==()

```
template<typename T, typename A>
bool operator== (
    const Vector< T, A > & a,
    const Vector< T, A > & b)
```

Lygina du vektorius.

Gražina

`true` jei dydžiai ir visi elementai sutampa

Apibrėžimas failo [Vector.h](#) eilutėje 872.

Ryšiai [Vector< T, Allocator >::begin\(\)](#), [Vector< T, Allocator >::end\(\)](#), ir [Vector< T, Allocator >::size\(\)](#).

8.45.2.2 operator!=()

```
template<typename T, typename A>
bool operator!= (
    const Vector< T, A > & a,
    const Vector< T, A > & b)
```

Tikrina, ar vektoriai skiriasi.

Apibrėžimas failo [Vector.h](#) eilutėje 878.

8.45.2.3 operator<()

```
template<typename T, typename A>
bool operator< (
    const Vector< T, A > & a,
    const Vector< T, A > & b)
```

Leksikografinis palyginimas.

Gražina

true jei a eina anksčiau už b

Apibrėžimas failo [Vector.h](#) eilutėje 887.

Ryšiai [Vector< T, Allocator >::begin\(\)](#), ir [Vector< T, Allocator >::end\(\)](#).

8.45.2.4 operator<=()

```
template<typename T, typename A>
bool operator<= (
    const Vector< T, A > & a,
    const Vector< T, A > & b)
```

Leksikografinis palyginimas (<).

Apibrėžimas failo [Vector.h](#) eilutėje 893.

8.45.2.5 operator>()

```
template<typename T, typename A>
bool operator> (
    const Vector< T, A > & a,
    const Vector< T, A > & b)
```

Leksikografinis palyginimas (>).

Apibrėžimas failo [Vector.h](#) eilutėje 897.

8.45.2.6 operator>=()

```
template<typename T, typename A>
bool operator>= (
    const Vector< T, A > & a,
    const Vector< T, A > & b)
```

Leksikografinis palyginimas (>=).

Apibrėžimas failo [Vector.h](#) eilutėje 901.

8.45.2.7 swap()

```
template<typename T, typename A>
void swap (
    Vector< T, A > & a,
    Vector< T, A > & b) [noexcept]
```

Sukeičia dviejų vektorių turinį.

Ne-narė versija, kad veiktų su `std::swap` per ADL.

Apibrėžimas failo [Vector.h](#) eilutėje 909.

Naudojamas [TEST_CASE\(\)](#).

8.45.2.8 erase()

```
template<typename T, typename A, typename U>
Vector< T, A >::size_type erase (
    Vector< T, A > & c,
    const U & value)
```

Pašalina visus elementus, lygius value.

Gražina

Pašalintų elementų skaičius

Pastaba

Atitinka C++20 `std::erase`.

Apibrėžimas failo [Vector.h](#) eilutėje 917.

Ryšiai [Vector< T, Allocator >::begin\(\)](#), [Vector< T, Allocator >::end\(\)](#), ir [Vector< T, Allocator >::erase\(\)](#).

8.45.2.9 erase_if()

```
template<typename T, typename A, typename Pred>
Vector< T, A >::size_type erase_if (
    Vector< T, A > & c,
    Pred pred)
```

Pašalina visus elementus, tenkinančius predikatą.

Gražina

Pašalintų elementų skaičius

Pastaba

Atitinka C++20 `std::erase_if`.

Apibrėžimas failo [Vector.h](#) eilutėje 930.

Ryšiai [Vector< T, Allocator >::begin\(\)](#), [Vector< T, Allocator >::end\(\)](#), ir [Vector< T, Allocator >::erase\(\)](#).

8.46 Vector.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef VECTOR_H_DEFINED
00002 #define VECTOR_H_DEFINED
00003
00004 #include <memory>           // allocator, allocator_traits
00005 #include <iterator>         // reverse_iterator, iterator_traits, distance
00006 #include <stdexcept>        // out_of_range, length_error
00007 #include <initializer_list> // initializer_list
00008 #include <algorithm>         // equal, lexicographical_compare, copy, move
00009 #include <utility>           // move, forward, swap
00010 #include <type_traits>        // is_integral, enable_if
00011 #include <cstdint>           // size_t, ptrdiff_t
00012 #include <limits>            // numeric_limits
00013
00014 /**
00015  * @file Vector.h
00016  * @brief Nuosava std::vector alternatyva -- dinaminis masyvas.
00017  * @author Marius Augustinas
00018  * @date 2025
00019  */
00020
00021 /**
00022  * @class Vector
00023  * @brief Dinaminis masyvas -- pilnavertė std::vector alternatyva.
00024  *
00025  * Realizuoja beveik visą `std::vector` interfeisą: member tipus,
00026  * konstruktorius, elementų prieigą, iteratorius, talpos valdymą,
00027  * modifikavimo operacijas ir ne-narius operatorius.
00028  *
00029  * @tparam T Elemento tipas
00030  * @tparam Allocator Atminties skirstyklė (pagal nutylėjimą `std::allocator<T>`)
00031  *
00032  * @par Architektūra
00033  * Klasė laiko tris rodykles, kaip ir tipinė `std::vector` realizacija:
00034  * @code
00035  * begin_ → pradžia
00036  * end_   → po paskutinio elemento (size = end_ - begin_)
00037  * cap_   → po paskirtos atminties (capacity = cap_ - begin_)
00038  * @endcode
00039  *
00040  * Atmintis **skiriama ir konstruojama atskirai**: `allocate()` tik
00041  * rezervuoja baitus, o objektai kuriami `construct()` metodu. Todėl
00042  * `reserve(1000)` nesukuria 1000 objektų -- tik paskiria vietą.
00043  *
00044  * @par Augimo strategija
00045  * Pritrūkus vietos, talpa **dvigubinama** (arba nustatoma į 1, jei buvo 0).
00046  * Toks koeficientas duoda amortizuotą O(1) `push_back()` sudėtingumą:
00047  * n elementų idėjimui reikia log(n) perskirstymų, o bendras kopijavimo
00048  * darbas yra O(n).
00049  *
00050  * @par Naudojimo pavyzdys
00051  * @code
00052  * Vector<int> v = {1, 2, 3};
00053  * v.push_back(4);
00054  * v.insert(v.begin() + 1, 10); // {1, 10, 2, 3, 4}
00055  *
00056  * for (int x : v) std::cout << x << " ";
```

```

00057 *
00058 * Vector<int> v2(v);           // kopija
00059 * Vector<int> v3(std::move(v)); // perkélimas
00060 * @endcode
00061 *
00062 * @note Šioje realizacijoje `Vector<bool>` **nėra specializuotas**.
00063 * Standartinis `std::vector<bool>` pakuoja reikšmes į bitus, bet
00064 * tai laikoma standarto klaida, nes toks konteineris nebeatitinka
00065 * bendrųjų konteinerio reikalavimų. Čia `Vector<bool>` veikia
00066 * kaip įprastas `bool` masyvas.
00067 */
00068 template <typename T, typename Allocator = std::allocator<T>
00069 class Vector {
00070 public:
00071     // =====
00072     /// @name Member tipai
00073     /// @{
00074     // =====
00075
00076     using value_type = T;           ///< Elemento tipas
00077     using allocator_type = Allocator; ///< Skirstyklės tipas
00078     using size_type = std::size_t;  ///< Dyžio tipas
00079     using difference_type = std::ptrdiff_t; ///< Iteratorių skirtumo tipas
00080     using reference = value_type&;  ///< Nuoroda į elementą
00081     using const_reference = const value_type&; ///< Konstanti nuoroda
00082     using pointer = typename std::allocator_traits<Allocator>::pointer;
00083     using const_pointer = typename std::allocator_traits<Allocator>::const_pointer;
00084     using iterator = T*;             ///< Iteratorius (raw pointer)
00085     using const_iterator = const T*;  ///< Konstantus iteratorius
00086     using reverse_iterator = std::reverse_iterator<iterator>;
00087     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00088
00089     /// @}
00090
00091 private:
00092     pointer begin_;           ///< Rodyklė į pirmą elementą
00093     pointer end_;             ///< Rodyklė po paskutinio elemento
00094     pointer cap_;             ///< Rodyklė po paskirtos atminties
00095     Allocator alloc_;         ///< Atminties skirstyklė
00096
00097     using AllocTraits = std::allocator_traits<Allocator>;
00098
00099     /**
00100      * @brief Sunaikina visus elementus intervale [first, last).
00101      *
00102      * Kviečia destruktorius, bet **neatlaisvina** atminties.
00103      */
00104     void destroyRange(pointer first, pointer last) {
00105         for (pointer p = first; p != last; ++p)
00106             AllocTraits::destroy(alloc_, p);
00107     }
00108
00109     /**
00110      * @brief Atlaisvina visą paskirtą atmintį ir sunaikina elementus.
00111      */
00112     void deallocateAll() {
00113         if (begin_) {
00114             destroyRange(begin_, end_);
00115             AllocTraits::deallocate(alloc_, begin_, capacity());
00116         }
00117         begin_ = end_ = cap_ = nullptr;
00118     }
00119
00120     /**
00121      * @brief Apskaičiuoja naują talpą augimo metu.
00122      *
00123      * Dvigubina esamą talpą; jei ji 0 -- grąžina 1. Jei reikalinga
00124      * talpa didesnė nei dvigubinta, grąžina reikalingą.
00125      *
00126      * @param reikia Minimali reikalinga talpa
00127      * @return Nauja talpa
00128      * @throws std::length_error jei viršijamas max_size()
00129      */
00130     size_type augimoTalpa(size_type reikia) const {
00131         const size_type maxSz = max_size();
00132         if (reikia > maxSz)
00133             throw std::length_error("Vector: viršytas max_size()");
00134
00135         const size_type dabar = capacity();
00136         if (dabar >= maxSz / 2)
00137             return maxSz;
00138
00139         const size_type dviguba = dabar == 0 ? 1 : dabar * 2;
00140         return dviguba < reikia ? reikia : dviguba;
00141     }
00142
00143     /**

```

```

00144     * @brief Perskirsto atmintį į naują talpą.
00145     *
00146     * Elementai perkeliama (move) jei tipas turi `noexcept` move
00147     * konstruktorių -- kitaip kopijuojami, kad būtų išlaikyta
00148     * stipri išimčių sauga.
00149     *
00150     * @param naujaTalpa Nauja talpa (turi būti size())
00151     */
00152 void perskirstyk(size_type naujaTalpa) {
00153     pointer naujas = AllocTraits::allocate(alloc_, naujaTalpa);
00154     const size_type senasDydis = size();
00155
00156     size_type i = 0;
00157     try {
00158         for (; i < senasDydis; ++i)
00159             AllocTraits::construct(alloc_, naujas + i,
00160                                   std::move_if_noexcept(begin_[i]));
00161     }
00162     catch (...) {
00163         for (size_type j = 0; j < i; ++j)
00164             AllocTraits::destroy(alloc_, naujas + j);
00165         AllocTraits::deallocate(alloc_, naujas, naujaTalpa);
00166         throw;
00167     }
00168
00169     deallocateAll();
00170     begin_ = naujas;
00171     end_ = naujas + senasDydis;
00172     cap_ = naujas + naujaTalpa;
00173 }
00174
00175 public:
00176     // =====
00177     /// @name Konstruktoriai ir destruktorius
00178     /// @{
00179     // =====
00180
00181     /**
00182     * @brief Numatytasis konstruktorius -- sukuria tuščią vektorių.
00183     * @post `size() == 0` && `capacity() == 0`
00184     */
00185     Vector() noexcept(noexcept(Allocator()))
00186         : begin_(nullptr), end_(nullptr), cap_(nullptr), alloc_() {}
00187
00188
00189     /**
00190     * @brief Konstruktorius su skirstykle.
00191     * @param alloc Naudojama atminties skirstyklė
00192     */
00193     explicit Vector(const Allocator& alloc) noexcept
00194         : begin_(nullptr), end_(nullptr), cap_(nullptr), alloc_(alloc) {}
00195
00196
00197     /**
00198     * @brief Sukuria vektorių su n kopijų reikšmės @p value.
00199     * @param n Elementų skaičius
00200     * @param value Kopijuojama reikšmė
00201     * @param alloc Skirstyklė
00202     */
00203     Vector(size_type n, const T& value, const Allocator& alloc = Allocator())
00204         : begin_(nullptr), end_(nullptr), cap_(nullptr), alloc_(alloc)
00205     {
00206         if (n == 0) return;
00207         begin_ = AllocTraits::allocate(alloc_, n);
00208         cap_ = begin_ + n;
00209         end_ = begin_;
00210         try {
00211             for (size_type i = 0; i < n; ++i, ++end_)
00212                 AllocTraits::construct(alloc_, end_, value);
00213         }
00214         catch (...) { deallocateAll(); throw; }
00215     }
00216
00217     /**
00218     * @brief Sukuria vektorių su n numatytais sukonstruotų elementų.
00219     * @param n Elementų skaičius
00220     * @param alloc Skirstyklė
00221     */
00222     explicit Vector(size_type n, const Allocator& alloc = Allocator())
00223         : begin_(nullptr), end_(nullptr), cap_(nullptr), alloc_(alloc)
00224     {
00225         if (n == 0) return;
00226         begin_ = AllocTraits::allocate(alloc_, n);
00227         cap_ = begin_ + n;
00228         end_ = begin_;
00229         try {
00230             for (size_type i = 0; i < n; ++i, ++end_)

```

```

00231         AllocTraits::construct(alloc_, end_);
00232     }
00233     catch (...) { deallocateAll(); throw; }
00234 }
00235
00236 /**
00237  * @brief Sukuria vektorių iš iteratorių intervalo [first, last).
00238  *
00239  * `enable_if` sąlyga atskiria šį konstruktorių nuo
00240  * `Vector(size_type, const T&)` -- be jos `Vector<int>(5, 10)`
00241  * būtų klaidingai interpretuotas kaip intervalas.
00242  *
00243  * @tparam InputIt Įvesties iteratoriaus tipas
00244  * @param first Intervalo pradžia
00245  * @param last Intervalo pabaiga
00246  * @param alloc Skirstyklė
00247  */
00248 template <typename InputIt,
00249           typename = typename std::enable_if<
00250             !std::is_integral<InputIt>::value::type>
00251 Vector(InputIt first, InputIt last, const Allocator& alloc = Allocator())
00252     : begin_(nullptr), end_(nullptr), cap_(nullptr), alloc_(alloc)
00253 {
00254     const auto n = static_cast<size_type>(std::distance(first, last));
00255     if (n == 0) return;
00256     begin_ = AllocTraits::allocate(alloc_, n);
00257     cap_ = begin_ + n;
00258     end_ = begin_;
00259     try {
00260         for (; first != last; ++first, ++end_)
00261             AllocTraits::construct(alloc_, end_, *first);
00262     }
00263     catch (...) { deallocateAll(); throw; }
00264 }
00265
00266 /**
00267  * @brief Kopijavimo konstruktorius.
00268  *
00269  * Sukuria gilia kopiją. Naujo vektoriaus talpa lygi kopijuojamo
00270  * **dydžiui**, ne talpai -- nekopijuojama nepanaudota atmintis.
00271  *
00272  * @param other Kopijuojamas vektorius
00273  */
00274 Vector(const Vector& other)
00275     : begin_(nullptr), end_(nullptr), cap_(nullptr),
00276       alloc_(AllocTraits::select_on_container_copy_construction(other.alloc_))
00277 {
00278     const size_type n = other.size();
00279     if (n == 0) return;
00280     begin_ = AllocTraits::allocate(alloc_, n);
00281     cap_ = begin_ + n;
00282     end_ = begin_;
00283     try {
00284         for (size_type i = 0; i < n; ++i, ++end_)
00285             AllocTraits::construct(alloc_, end_, other.begin_[i]);
00286     }
00287     catch (...) { deallocateAll(); throw; }
00288 }
00289
00290 /**
00291  * @brief Perkėlimo (move) konstruktorius.
00292  *
00293  * Perima kito vektoriaus atmintį be kopijavimo -- tik trys
00294  * rodyklių priskyrimai. Šaltinis paliekamas tuščias.
00295  *
00296  * @param other Vektorius, iš kurio perkeliama
00297  */
00298 Vector(Vector&& other) noexcept
00299     : begin_(other.begin_), end_(other.end_), cap_(other.cap_),
00300       alloc_(std::move(other.alloc_))
00301 {
00302     other.begin_ = other.end_ = other.cap_ = nullptr;
00303 }
00304
00305 /**
00306  * @brief Konstruktorius iš inicijavimo sąrašo.
00307  * @param il Inicijavimo sąrašas, pvz. `{1, 2, 3}`
00308  * @param alloc Skirstyklė
00309  */
00310 Vector(std::initializer_list<T> il, const Allocator& alloc = Allocator())
00311     : Vector(il.begin(), il.end(), alloc) {}
00312 }
00313
00314 /**
00315  * @brief Destruktorius -- sunaikina elementus ir atlaisvina atmintį.
00316  */
00317 ~Vector() { deallocateAll(); }

```

```

00318
00319     /// @}
00320
00321     // =====
00322     /// @name Priskyrimas
00323     /// @{
00324     // =====
00325
00326     /**
00327      * @brief Kopijavimo priskyrimo operatorius.
00328      * @param other Priskiriamas vektorius
00329      * @return Nuoroda į šį objektą
00330      */
00331     Vector& operator=(const Vector& other) {
00332         if (this == &other) return *this;
00333         assign(other.begin(), other.end());
00334         return *this;
00335     }
00336
00337     /**
00338      * @brief Perkėlimo priskyrimo operatorius.
00339      * @param other Vektorius, iš kurio perkeliama
00340      * @return Nuoroda į šį objektą
00341      */
00342     Vector& operator=(Vector&& other) noexcept {
00343         if (this == &other) return *this;
00344         deallocateAll();
00345         begin_ = other.begin_;
00346         end_ = other.end_;
00347         cap_ = other.cap_;
00348         alloc_ = std::move(other.alloc_);
00349         other.begin_ = other.end_ = other.cap_ = nullptr;
00350         return *this;
00351     }
00352
00353     /**
00354      * @brief Priskyrimas iš inicijavimo sąrašo.
00355      * @param il Inicijavimo sąrašas
00356      * @return Nuoroda į šį objektą
00357      */
00358     Vector& operator=(std::initializer_list<T> il) {
00359         assign(il.begin(), il.end());
00360         return *this;
00361     }
00362
00363     /**
00364      * @brief Pakeičia turinį n kopijų reikšmės @p value.
00365      * @param n Elementų skaičius
00366      * @param value Kopijuojama reikšmė
00367      */
00368     void assign(size_type n, const T& value) {
00369         clear();
00370         if (n > capacity()) {
00371             deallocateAll();
00372             begin_ = AllocTraits::allocate(alloc_, n);
00373             cap_ = begin_ + n;
00374             end_ = begin_;
00375         }
00376         for (size_type i = 0; i < n; ++i, ++end_)
00377             AllocTraits::construct(alloc_, end_, value);
00378     }
00379
00380     /**
00381      * @brief Pakeičia turinį intervalo [first, last) elementais.
00382      * @tparam InputIt Įvesties iteratoriaus tipas
00383      * @param first Intervalo pradžia
00384      * @param last Intervalo pabaiga
00385      */
00386     template <typename InputIt,
00387              typename = typename std::enable_if<
00388                  !std::is_integral<InputIt>::value>::type>
00389     void assign(InputIt first, InputIt last) {
00390         clear();
00391         const auto n = static_cast<size_type>(std::distance(first, last));
00392         if (n > capacity()) {
00393             deallocateAll();
00394             begin_ = AllocTraits::allocate(alloc_, n);
00395             cap_ = begin_ + n;
00396             end_ = begin_;
00397         }
00398         for (; first != last; ++first, ++end_)
00399             AllocTraits::construct(alloc_, end_, *first);
00400     }
00401
00402     /**
00403      * @brief Pakeičia turinį inicijavimo sąrašo elementais.
00404      * @param il Inicijavimo sąrašas

```



```

00405     */
00406 void assign(std::initializer_list<T> il) { assign(il.begin(), il.end()); }
00407
00408 /**
00409  * @brief Gražina naudojamą skirstyklę.
00410  * @return Skirstyklės kopija
00411  */
00412 allocator_type get_allocator() const noexcept { return alloc_; }
00413
00414 /// @}
00415
00416 // =====
00417 /// @name Elementų prieiga
00418 /// @{
00419 // =====
00420
00421 /**
00422  * @brief Prieiga prie elemento su ribų tikrinimu.
00423  * @param pos Elemento indeksas
00424  * @return Nuoroda į elementą
00425  * @throws std::out_of_range jei `pos >= size()`
00426  */
00427 reference at(size_type pos) {
00428     if (pos >= size())
00429         throw std::out_of_range("Vector::at: indeksas uz ribu");
00430     return begin_[pos];
00431 }
00432
00433 /// @copydoc at(size_type)
00434 const_reference at(size_type pos) const {
00435     if (pos >= size())
00436         throw std::out_of_range("Vector::at: indeksas uz ribu");
00437     return begin_[pos];
00438 }
00439
00440 /**
00441  * @brief Prieiga prie elemento **be** ribų tikrinimo.
00442  * @param pos Elemento indeksas
00443  * @return Nuoroda į elementą
00444  * @warning Su `pos >= size()` elgesys neapibrėžtas. Ribų tikrinimui
00445  *         naudokite at().
00446  */
00447 reference operator[](size_type pos) { return begin_[pos]; }
00448
00449 /// @copydoc operator[](size_type)
00450 const_reference operator[](size_type pos) const { return begin_[pos]; }
00451
00452 /// @brief Gražina pirmą elementą. @warning Tuščiam vektoriui neapibrėžta.
00453 reference front() { return *begin_; }
00454 /// @copydoc front()
00455 const_reference front() const { return *begin_; }
00456
00457 /// @brief Gražina paskutinį elementą. @warning Tuščiam vektoriui neapibrėžta.
00458 reference back() { return *(end_ - 1); }
00459 /// @copydoc back()
00460 const_reference back() const { return *(end_ - 1); }
00461
00462 /**
00463  * @brief Gražina rodyklę į vidinį masyvą.
00464  * @return Rodyklė į duomenis; `nullptr` jei tuščias
00465  */
00466 T* data() noexcept { return begin_; }
00467 /// @copydoc data()
00468 const T* data() const noexcept { return begin_; }
00469
00470 /// @}
00471
00472 // =====
00473 /// @name Iteratoriai
00474 /// @{
00475 // =====
00476
00477 iterator begin() noexcept { return begin_; }          ///< Pradžia
00478 const_iterator begin() const noexcept { return begin_; }  ///< Pradžia (const)
00479 const_iterator cbegin() const noexcept { return begin_; }  ///< Pradžia (const)
00480
00481 iterator end() noexcept { return end_; }              ///< Pabaiga
00482 const_iterator end() const noexcept { return end_; }    ///< Pabaiga (const)
00483 const_iterator cend() const noexcept { return end_; }    ///< Pabaiga (const)
00484
00485 /// @brief Atvirkštinė pradžia (paskutinis elementas)
00486 reverse_iterator rbegin() noexcept { return reverse_iterator(end_); }
00487 /// @copydoc rbegin()
00488 const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end_); }
00489 /// @copydoc rbegin()
00490 const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(end_); }
00491

```

```

00492     /// @brief Atvirkštinė pabaiga (prieš pirmą elementą)
00493     reverse_iterator rend() noexcept { return reverse_iterator(begin_); }
00494     /// @copydoc rend()
00495     const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin_); }
00496     /// @copydoc rend()
00497     const_reverse_iterator crend() const noexcept { return const_reverse_iterator(begin_); }
00498
00499     /// @}
00500
00501     // =====
00502     /// @name Talpa
00503     /// @{
00504     // =====
00505
00506     /**
00507      * @brief Patikrina, ar vektorius tuščias.
00508      * @return `true` jei `size() == 0`
00509      */
00510     bool empty() const noexcept { return begin_ == end_; }
00511
00512     /**
00513      * @brief Gražina elementų skaičių.
00514      * @return Elementų skaičius
00515      */
00516     size_type size() const noexcept {
00517         return static_cast<size_type>(end_ - begin_);
00518     }
00519
00520     /**
00521      * @brief Gražina didžiausią galimą elementų skaičių.
00522      * @return Teorinė riba
00523      */
00524     size_type max_size() const noexcept {
00525         return AllocTraits::max_size(alloc_);
00526     }
00527
00528     /**
00529      * @brief Paskiria atmintį bent @p n elementams.
00530      *
00531      * Jei `n <= capacity()`, nieko nedaro. Elementų **nekuria** --
00532      * `size()` nepakinta.
00533      *
00534      * @param n Reikalinga talpa
00535      * @throws std::length_error jei `n > max_size()`
00536      *
00537      * @note Naudinga, kai iš anksto žinomas elementų skaičius --
00538      * išvengiama daugkartinių perskirstymų.
00539      */
00540     void reserve(size_type n) {
00541         if (n > max_size())
00542             throw std::length_error("Vector::reserve: viršytas max_size()");
00543         if (n > capacity())
00544             perskirstyk(n);
00545     }
00546
00547     /**
00548      * @brief Gražina esamą talpą.
00549      * @return Kiek elementų telpa be perskirstymo
00550      */
00551     size_type capacity() const noexcept {
00552         return static_cast<size_type>(cap_ - begin_);
00553     }
00554
00555     /**
00556      * @brief Sumažina talpą iki dydžio.
00557      *
00558      * Atlaisvina nepanaudotą atmintį. Jei `capacity() == size()`,
00559      * nieko nedaro.
00560      */
00561     void shrink_to_fit() {
00562         if (capacity() > size()) {
00563             if (empty()) deallocateAll();
00564             else perskirstyk(size());
00565         }
00566     }
00567
00568     /// @}
00569
00570     // =====
00571     /// @name Modifikavimas
00572     /// @{
00573     // =====
00574
00575     /**
00576      * @brief Pašalina visus elementus.
00577      *
00578      * `size()` tampa 0, bet **talpa nepakinta** -- atmintis lieka

```

```

00579     * paskirta pakartotiniam naudojimui.
00580     */
00581 void clear() noexcept {
00582     destroyRange(begin_, end_);
00583     end_ = begin_;
00584 }
00585
00586 /**
00587  * @brief Įterpia elementą prieš @p pos.
00588  * @param pos Pozicija, prieš kurią įterpiama
00589  * @param value Įterpiama reikšmė
00590  * @return Iteratorius į įterptą elementą
00591  */
00592 iterator insert(const_iterator pos, const T& value) {
00593     return insertImpl(pos, 1, value);
00594 }
00595
00596 /**
00597  * @brief Įterpia elementą prieš @p pos (perkėlimu).
00598  * @param pos Pozicija
00599  * @param value Įterpiama reikšmė
00600  * @return Iteratorius į įterptą elementą
00601  */
00602 iterator insert(const_iterator pos, T&& value) {
00603     const size_type idx = static_cast<size_type>(pos - begin_);
00604     if (size() == capacity())
00605         perskirstyk(augimoTalpa(size() + 1));
00606     pointer p = begin_ + idx;
00607     if (p != end_) {
00608         AllocTraits::construct(alloc_, end_, std::move(*(end_ - 1)));
00609         std::move_backward(p, end_ - 1, end_);
00610         *p = std::move(value);
00611     }
00612     else {
00613         AllocTraits::construct(alloc_, end_, std::move(value));
00614     }
00615     ++end_;
00616     return p;
00617 }
00618
00619 /**
00620  * @brief Įterpia n kopijų reikšmės prieš @p pos.
00621  * @param pos Pozicija
00622  * @param n Kopijų skaičius
00623  * @param value Įterpiama reikšmė
00624  * @return Iteratorius į pirmą įterptą elementą
00625  */
00626 iterator insert(const_iterator pos, size_type n, const T& value) {
00627     return insertImpl(pos, n, value);
00628 }
00629
00630 /**
00631  * @brief Įterpia intervalo [first, last) elementus prieš @p pos.
00632  * @tparam InputIt Įvesties iteratoriaus tipas
00633  * @param pos Pozicija
00634  * @param first Intervalo pradžia
00635  * @param last Intervalo pabaiga
00636  * @return Iteratorius į pirmą įterptą elementą
00637  */
00638 template <typename InputIt,
00639          typename = typename std::enable_if<
00640              !std::is_integral<InputIt>::value>::type>
00641 iterator insert(const_iterator pos, InputIt first, InputIt last) {
00642     const size_type idx = static_cast<size_type>(pos - begin_);
00643     const auto n = static_cast<size_type>(std::distance(first, last));
00644     if (n == 0) return begin_ + idx;
00645
00646     if (size() + n > capacity())
00647         perskirstyk(augimoTalpa(size() + n));
00648
00649     pointer p = begin_ + idx;
00650     // Perkeliame esamus elementus į galą
00651     for (pointer q = end_; q != p; --q)
00652         AllocTraits::construct(alloc_, q + n - 1, std::move(*(q - 1)));
00653     // Įrašome naujus
00654     pointer w = p;
00655     for (; first != last; ++first, ++w) {
00656         if (w < end_) *w = *first;
00657         else AllocTraits::construct(alloc_, w, *first);
00658     }
00659     end_ += n;
00660     return begin_ + idx;
00661 }
00662
00663 /**
00664  * @brief Įterpia inicijavimo sąrašo elementus prieš @p pos.
00665  * @param pos Pozicija

```

```

00666     * @param il Inicijavimo sąrašas
00667     * @return Iteratorius į pirmą įterptą elementą
00668     */
00669 iterator insert(const_iterator pos, std::initializer_list<T> il) {
00670     return insert(pos, il.begin(), il.end());
00671 }
00672
00673 /**
00674  * @brief Sukuria elementą vietoje prieš @p pos.
00675  *
00676  * Elementas konstruojamas **tiesiogiai** konteinerio atmintyje --
00677  * be laikino objekto ir be kopijavimo.
00678  *
00679  * @tparam Args Konstruktoriaus argumentų tipai
00680  * @param pos Pozicija
00681  * @param args Argumentai, perduodami elemento konstruktoriui
00682  * @return Iteratorius į sukurtą elementą
00683  */
00684 template <typename... Args>
00685 iterator emplace(const_iterator pos, Args&&... args) {
00686     const size_type idx = static_cast<size_type>(pos - begin_);
00687     if (size() == capacity())
00688         perskirstyk(augimoTalpa(size() + 1));
00689     pointer p = begin_ + idx;
00690     if (p != end_) {
00691         AllocTraits::construct(alloc_, end_, std::move(*(end_ - 1)));
00692         std::move_backward(p, end_ - 1, end_);
00693         *p = T(std::forward<Args>(args)...);
00694     }
00695     else {
00696         AllocTraits::construct(alloc_, end_, std::forward<Args>(args)...);
00697     }
00698     ++end_;
00699     return p;
00700 }
00701
00702 /**
00703  * @brief Pašalina elementą pozicijoje @p pos.
00704  * @param pos Šalinamo elemento pozicija
00705  * @return Iteratorius į kitą elementą po pašalinto
00706  */
00707 iterator erase(const_iterator pos) {
00708     pointer p = begin_ + (pos - begin_);
00709     std::move(p + 1, end_, p);
00710     --end_;
00711     AllocTraits::destroy(alloc_, end_);
00712     return p;
00713 }
00714
00715 /**
00716  * @brief Pašalina elementus intervale [first, last).
00717  * @param first Intervalo pradžia
00718  * @param last Intervalo pabaiga
00719  * @return Iteratorius į kitą elementą po pašalintų
00720  */
00721 iterator erase(const_iterator first, const_iterator last) {
00722     pointer f = begin_ + (first - begin_);
00723     pointer l = begin_ + (last - begin_);
00724     if (f == l) return f;
00725     pointer naujasEnd = std::move(l, end_, f);
00726     destroyRange(naujasEnd, end_);
00727     end_ = naujasEnd;
00728     return f;
00729 }
00730
00731 /**
00732  * @brief Prideda elementą į galą.
00733  *
00734  * Amortizuotas sudėtingumas O(1). Pritrūkus vietos, talpa
00735  * dvigubinama.
00736  *
00737  * @param value Pridedama reikšmė
00738  */
00739 void push_back(const T& value) {
00740     if (end_ == cap_)
00741         perskirstyk(augimoTalpa(size() + 1));
00742     AllocTraits::construct(alloc_, end_, value);
00743     ++end_;
00744 }
00745
00746 /**
00747  * @brief Prideda elementą į galą (perkėlimu).
00748  * @param value Pridedama reikšmė
00749  */
00750 void push_back(T&& value) {
00751     if (end_ == cap_)
00752         perskirstyk(augimoTalpa(size() + 1));

```

```

00753         AllocTraits::construct(alloc_, end_, std::move(value));
00754         ++end_;
00755     }
00756
00757     /**
00758     * @brief Sukuria elementą vietoje gale.
00759     *
00760     * Efektyvesnis už `push_back(T(args...))` -- objektas konstruojamas
00761     * tiesiogiai konteinerio atmintyje, be laikino objekto.
00762     *
00763     * @tparam Args Konstruktoriaus argumentų tipai
00764     * @param args Argumentai, perduodami elemento konstruktoriui
00765     * @return Nuoroda į sukurtą elementą
00766     */
00767     template <typename... Args>
00768     reference.emplace_back(Args&&... args) {
00769         if (end_ == cap_)
00770             perskirstyk(augimoTalpa(size() + 1));
00771         AllocTraits::construct(alloc_, end_, std::forward<Args>(args)...);
00772         ++end_;
00773         return *(end_ - 1);
00774     }
00775
00776     /**
00777     * @brief Pašalina paskutinį elementą.
00778     * @warning Tuščiam vektoriui elgesys neapibrėžtas.
00779     */
00780     void pop_back() {
00781         --end_;
00782         AllocTraits::destroy(alloc_, end_);
00783     }
00784
00785     /**
00786     * @brief Pakeičia elementų skaičių.
00787     *
00788     * Didinant -- nauji elementai konstruojami numatyta.
00789     * Mažinant -- pertekliniai sunaikinami, talpa nepakinta.
00790     *
00791     * @param n Naujas dydis
00792     */
00793     void resize(size_type n) {
00794         if (n < size()) {
00795             destroyRange(begin_ + n, end_);
00796             end_ = begin_ + n;
00797         }
00798         else if (n > size()) {
00799             if (n > capacity()) perskirstyk(augimoTalpa(n));
00800             for (size_type i = size(); i < n; ++i, ++end_)
00801                 AllocTraits::construct(alloc_, end_);
00802         }
00803     }
00804
00805     /**
00806     * @brief Pakeičia elementų skaičių, naujus užpildant reikšme.
00807     * @param n Naujas dydis
00808     * @param value Reikšmė naujiems elementams
00809     */
00810     void resize(size_type n, const T& value) {
00811         if (n < size()) {
00812             destroyRange(begin_ + n, end_);
00813             end_ = begin_ + n;
00814         }
00815         else if (n > size()) {
00816             if (n > capacity()) perskirstyk(augimoTalpa(n));
00817             for (size_type i = size(); i < n; ++i, ++end_)
00818                 AllocTraits::construct(alloc_, end_, value);
00819         }
00820     }
00821
00822     /**
00823     * @brief Sukeičia turinį su kitu vektoriumi.
00824     *
00825     * Sudėtingumas O(1) -- sukeičiamos tik rodyklės, elementai
00826     * nejudinami.
00827     *
00828     * @param other Vektorius, su kuriuo keičiamasi
00829     */
00830     void swap(Vector& other) noexcept {
00831         std::swap(begin_, other.begin_);
00832         std::swap(end_, other.end_);
00833         std::swap(cap_, other.cap_);
00834         std::swap(alloc_, other.alloc_);
00835     }
00836
00837     /// @}
00838
00839 private:

```

```

00840     /**
00841     * @brief Bendra insert realizacija n kopijų įterpimui.
00842     */
00843     iterator insertImpl(const_iterator pos, size_type n, const T& value) {
00844         const size_type idx = static_cast<size_type>(pos - begin_);
00845         if (n == 0) return begin_ + idx;
00846
00847         if (size() + n > capacity())
00848             perskirstyk(augimoTalpa(size() + n));
00849
00850         pointer p = begin_ + idx;
00851         for (pointer q = end_; q != p; --q)
00852             AllocTraits::construct(alloc_, q + n - 1, std::move(*(q - 1)));
00853         for (size_type i = 0; i < n; ++i) {
00854             if (p + i < end_) *(p + i) = value;
00855             else AllocTraits::construct(alloc_, p + i, value);
00856         }
00857         end_ += n;
00858         return begin_ + idx;
00859     }
00860 };
00861
00862 // =====
00863 /// @name Ne-nariai operatoriai
00864 /// @{
00865 // =====
00866
00867 /**
00868 * @brief Lygina du vektorius.
00869 * @return `true` jei dydžiai ir visi elementai sutampa
00870 */
00871 template <typename T, typename A>
00872 bool operator==(const Vector<T, A>& a, const Vector<T, A>& b) {
00873     return a.size() == b.size() && std::equal(a.begin(), a.end(), b.begin());
00874 }
00875
00876 /// @brief Tikrina, ar vektoriai skiriasi.
00877 template <typename T, typename A>
00878 bool operator!=(const Vector<T, A>& a, const Vector<T, A>& b) {
00879     return !(a == b);
00880 }
00881
00882 /**
00883 * @brief Leksikografinis palyginimas.
00884 * @return `true` jei @p a eina anksčiau už @p b
00885 */
00886 template <typename T, typename A>
00887 bool operator<(const Vector<T, A>& a, const Vector<T, A>& b) {
00888     return std::lexicographical_compare(a.begin(), a.end(), b.begin(), b.end());
00889 }
00890
00891 /// @brief Leksikografinis palyginimas ().
00892 template <typename T, typename A>
00893 bool operator<=(const Vector<T, A>& a, const Vector<T, A>& b) { return !(b < a); }
00894
00895 /// @brief Leksikografinis palyginimas (>).
00896 template <typename T, typename A>
00897 bool operator>(const Vector<T, A>& a, const Vector<T, A>& b) { return b < a; }
00898
00899 /// @brief Leksikografinis palyginimas ().
00900 template <typename T, typename A>
00901 bool operator>=(const Vector<T, A>& a, const Vector<T, A>& b) { return !(a < b); }
00902
00903 /**
00904 * @brief Sukeičia dviejų vektorių turinį.
00905 *
00906 * Ne-narė versija, kad veiktų su `std::swap` per ADL.
00907 */
00908 template <typename T, typename A>
00909 void swap(Vector<T, A>& a, Vector<T, A>& b) noexcept { a.swap(b); }
00910
00911 /**
00912 * @brief Pašalina visus elementus, lygius @p value.
00913 * @return Pašalintų elementų skaičius
00914 * @note Atitinka C++20 `std::erase`.
00915 */
00916 template <typename T, typename A, typename U>
00917 typename Vector<T, A>::size_type erase(Vector<T, A>& c, const U& value) {
00918     auto it = std::remove(c.begin(), c.end(), value);
00919     const auto n = static_cast<typename Vector<T, A>::size_type>(c.end() - it);
00920     c.erase(it, c.end());
00921     return n;
00922 }
00923
00924 /**
00925 * @brief Pašalina visus elementus, tenkinančius predikatą.
00926 * @return Pašalintų elementų skaičius

```

```

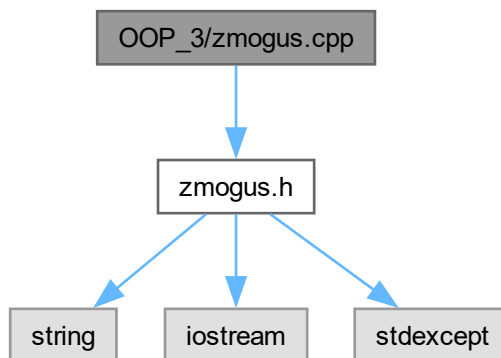
00927 * @note Atitinka C++20 `std::erase_if`.
00928 */
00929 template <typename T, typename A, typename Pred>
00930 typename Vector<T, A>::size_type erase_if(Vector<T, A>& c, Pred pred) {
00931     auto it = std::remove_if(c.begin(), c.end(), pred);
00932     const auto n = static_cast<typename Vector<T, A>::size_type>(c.end() - it);
00933     c.erase(it, c.end());
00934     return n;
00935 }
00936
00937 /// @}
00938
00939 #endif // VECTOR_H_DEFINED

```

8.47 OOP_3/zmogus.cpp Failo Nuoroda

```
#include "zmogus.h"
```

Įtraukimo priklausomybių diagrama zmogus.cpp:



Funkcijos

- `std::ostream & operator<< (std::ostream &os, const zmogus &z)`
Polimorfinis išvesties operatorius.
- `std::istream & operator>> (std::istream &is, zmogus &z)`
Polimorfinis įvesties operatorius.

8.47.1 Funkcijos Dokumentacija

8.47.1.1 operator<<()

```

std::ostream & operator<< (
    std::ostream & os,
    const zmogus & z)

```

Polimorfinis išvesties operatorius.

Priima `zmogus` & nuorodą, bet kviečia virtualų `print()` metodą. Todėl veikia su bet kuria išvestine klase — dabartine ir būsimomis.

Parametrai

<code>os</code>	Išvesties srautas
-----------------	-------------------

z	Išvedamas objektas
---	--------------------

Gražina

Nuoroda į srautą

Pavyzdys

```
studentas s("Jonas", "Jonaitis", {5, 6}, 7);
s.calculateGalutinis();

std::cout << s << "\n";           // į ekraną

std::ofstream f("rezultatai.txt");
f << s << "\n";                   // į failą

zmogus& z = s;
std::cout << z << "\n";           // per bazinės klasės nuorodą
```

Apibrėžimas failo [zmogus.cpp](#) eilutėje 112.

Ryšiai [zmogus::print\(\)](#).

8.47.1.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    zmogus & z)
```

Polimorfinis įvesties operatorius.

Priima [zmogus](#) & nuorodą, bet kviečia virtualų read() metodą.

Parametrai

is	Įvesties srautas
z	Objektas, į kurį nuskaitomi duomenys

Gražina

Nuoroda į srautą

Išimtys

<code>std::runtime_error</code>	jei duomenys netinkami
---------------------------------	------------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 118.

Ryšiai [zmogus::read\(\)](#).

8.48 zmogus.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "zmogus.h"
00002
00003 // -----
00004 // Statinis narys -- gyvuojančių objektų skaitiklis
00005 // -----
00006 int zmogus::gyvuZmoniu = 0;
00007
00008 // =====
00009 // Konstruktoriai (protected)
00010 // =====
00011
00012 zmogus::zmogus()
00013     : vardas_(), pavarde_()
```



```

00014 {
00015     ++gyvuZmoniu;
00016 }
00017
00018 zmogus::zmogus(const std::string& vardas, const std::string& pavarde)
00019     : vardas_(vardas), pavarde_(pavarde)
00020 {
00021     if (vardas_.empty())
00022         throw std::invalid_argument("Vardas negali būti tuščias");
00023     if (pavarde_.empty())
00024         throw std::invalid_argument("Pavardė negali būti tuščia");
00025
00026     ++gyvuZmoniu;
00027 }
00028
00029 // -----
00030 // Kopijavimo konstruktorius
00031 // -----
00032 zmogus::zmogus(const zmogus& other)
00033     : vardas_(other.vardas_), pavarde_(other.pavarde_)
00034 {
00035     ++gyvuZmoniu;
00036 }
00037
00038 // -----
00039 // Perkėlimo (move) konstruktorius
00040 // noexcept -- kad std::vector naudotų perkėlimą, ne kopijavimą
00041 // -----
00042 zmogus::zmogus(zmogus&& other) noexcept
00043     : vardas_(std::move(other.vardas_)), pavarde_(std::move(other.pavarde_))
00044 {
00045     ++gyvuZmoniu;
00046 }
00047
00048 // -----
00049 // Kopijavimo priskyrimo operatorius
00050 // -----
00051 zmogus& zmogus::operator=(const zmogus& other)
00052 {
00053     if (this == &other) // apsauga nuo priskyrimo sau
00054         return *this;
00055
00056     vardas_ = other.vardas_;
00057     pavarde_ = other.pavarde_;
00058
00059     return *this;
00060 }
00061
00062 // -----
00063 // Perkėlimo (move) priskyrimo operatorius
00064 // -----
00065 zmogus& zmogus::operator=(zmogus&& other) noexcept
00066 {
00067     if (this == &other) // apsauga nuo priskyrimo sau
00068         return *this;
00069
00070     vardas_ = std::move(other.vardas_);
00071     pavarde_ = std::move(other.pavarde_);
00072
00073     return *this;
00074 }
00075
00076 // =====
00077 // VIRTUALUS destruktorius
00078 //
00079 // Kviečiamas PO išvestinės klasės destruktoriaus.
00080 // Naikinimo tvarka: ~studentas() → ~zmogus()
00081 // =====
00082 zmogus::~zmogus()
00083 {
00084     vardas_.clear();
00085     pavarde_.clear();
00086
00087     --gyvuZmoniu;
00088 }
00089
00090 // =====
00091 // Set'eriai su validacija
00092 // =====
00093
00094 void zmogus::setVardas(const std::string& v)
00095 {
00096     if (v.empty())
00097         throw std::invalid_argument("Vardas negali būti tuščias");
00098     vardas_ = v;
00099 }
00100

```


Funkcijos

- `std::ostream & operator<<` (`std::ostream &os, const zmogus &z`)
Polimorfinis išvesties operatorius.
- `std::istream & operator>>` (`std::istream &is, zmogus &z`)
Polimorfinis įvesties operatorius.

8.49.1 Smulkus aprašymas

Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis.

Autorius

Marius Augustinas

Data

2026

Apibrėžimas faile [zmogus.h](#).

8.49.2 Funkcijos Dokumentacija

8.49.2.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const zmogus & z)
```

Polimorfinis išvesties operatorius.

Priima [zmogus](#) & nuorodą, bet kviečia virtualų `print()` metodą. Todėl veikia su bet kokia išvestine klase — dabartine ir būsimomis.

Parametrai

<code>os</code>	Išvesties srautas
<code>z</code>	Išvedamas objektas

Gražina

Nuoroda į srautą

Pavyzdys

```
studentas s("Jonas", "Jonaitis", {5, 6}, 7);
s.calculateGalutinis();

std::cout << s << "\n";           // į ekraną

std::ofstream f("rezultatai.txt");
f << s << "\n";                   // į failą

zmogus& z = s;
std::cout << z << "\n";           // per bazinės klasės nuorodą
```

Apibrėžimas failo [zmogus.cpp](#) eilutėje 112.

Ryšiai [zmogus::print\(\)](#).

8.49.2.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    zmogus & z)
```

Polimorfinis įvesties operatorius.

Priima [zmogus](#) & nuorodą, bet kviečia virtualų `read()` metodą.

Parametrai

<i>is</i>	Įvesties srautas
<i>z</i>	Objektas, į kurį nuskaitomi duomenys

Gražina

Nuoroda į srautą

Išimtys

<code>std::runtime_error</code>	jei duomenys netinkami
---------------------------------	------------------------

Apibrėžimas failo [zmogus.cpp](#) eilutėje 118.

Ryšiai [zmogus::read\(\)](#).

8.50 `zmogus.h`

Eiti į šio failo dokumentaciją.

```
00001 #ifndef ZMOGUS_H_DEFINED
00002 #define ZMOGUS_H_DEFINED
00003
00004 #include <string>
00005 #include <iostream>
00006 #include <stdexcept>
00007
00008 /**
00009  * @file zmogus.h
00010  * @brief Abstrakti bazinė klasė, aprašanti bendrus žmogaus duomenis.
00011  * @author Marius Augustinas
00012  * @date 2026
00013  */
00014
00015 /**
00016  * @class zmogus
00017  * @brief ABSTRAKTI bazinė klasė -- bendras bet kokio žmogaus aprašymas.
00018  *
00019  * Laiko vardą ir pavardę bei apibrėžia interfeisą, kurį privalo
00020  * realizuoti kiekviena išvestinė klasė.
00021  *
00022  * @par Kodėl klasė abstrakti
00023  * Klasė padaryta abstrakti **dviem nepriklausomais mechanizmais**:
00024  *
00025  * 1. **Grynai virtualūs metodai** ('= 0') -- galutinis(), print(),
00026  *    read(), tipas(). Kompiliatorius neleidžia kurti objektų iš klasės,
00027  *    kuri turi bent vieną neapibrėžtą virtualų metodą.
00028  * 2. **'protected' konstruktoriai** -- prieinami tik išvestinėms klasėms.
00029  *    Tai antras apsaugos sluoksnis.
00030  *
00031  * @par Naudojimo pavyzdys
00032  * @code
00033  * // zmogus z; // KLAIDA: abstrakti klasė
00034  * // zmogus* p = new zmogus(); // KLAIDA: abstrakti klasė
00035  *
00036  * studentas s("Jonas", "Jonaitis", {5, 6, 7}, 8); // OK
00037  * zmogus& z = s; // OK -- nuoroda
00038  * std::cout << z.tipas() << "\n"; // "Studentas"
00039  *
00040  * std::unique_ptr<zmogus> p =
00041  *    std::make_unique<studentas>("Ona", "Onaite", {10}, 10); // OK
00042  * @endcode
00043  *
00044  * @warning Destruktorius PRIVALO būti virtualus -- kitaip naikinant
00045  *    objektą per bazinės klasės rodyklę išvestinės klasės
00046  *    laukai liktų neatlaisvinti.
00047  *
00048  * @see studentas
00049  */
00050 class zmogus {
00051 protected:
```

```

00052     std::string vardas_;    ///< Žmogaus vardas
00053     std::string pavarde_;  ///< Žmogaus pavardė
00054
00055     /**
00056      * @brief Numatytasis konstruktorius.
00057      *
00058      * Sukuria objektą su tuščiu vardu ir pavarde.
00059      * `protected` -- kviečiamas tik iš išvestinių klasių.
00060      */
00061     zmogus();
00062
00063     /**
00064      * @brief Pilnas konstruktorius su validacija.
00065      *
00066      * @param vardas Žmogaus vardas (negali būti tuščias)
00067      * @param pavarde Žmogaus pavardė (negali būti tuščia)
00068      *
00069      * @throws std::invalid_argument jei vardas arba pavardė tuščia
00070      */
00071     zmogus(const std::string& vardas, const std::string& pavarde);
00072
00073     /**
00074      * @brief Kopijavimo konstruktorius.
00075      *
00076      * Sukuria gilia kopiją -- `std::string` laukai kopijuoja
00077      * savo vidinius buferius.
00078      *
00079      * @param other Kopijuojamas objektas
00080      */
00081     zmogus(const zmogus& other);
00082
00083     /**
00084      * @brief Perkėlimo (move) konstruktorius.
00085      *
00086      * Perima kito objekto resursus be kopijavimo. Šaltinis paliekamas
00087      * galiojančioje, bet tuščioje būsenoje.
00088      *
00089      * @param other Objektas, iš kurio perkeliami duomenys
00090      *
00091      * @note `noexcept` būtinas: be jo `std::vector` persikirstymo metu
00092      *        naudotų kopijavimo konstruktorių (`std::move_if_noexcept`).
00093      */
00094     zmogus(zmogus&& other) noexcept;
00095
00096     /**
00097      * @brief Kopijavimo priskyrimo operatorius.
00098      *
00099      * @param other Priskiriamas objektas
00100      * @return Nuoroda į šį objektą (grandininiam priskyrimui `a = b = c`)
00101      *
00102      * @note Turi apsaugą nuo priskyrimo sau (`this == &other`).
00103      */
00104     zmogus& operator=(const zmogus& other);
00105
00106     /**
00107      * @brief Perkėlimo (move) priskyrimo operatorius.
00108      *
00109      * @param other Objektas, iš kurio perkeliami duomenys
00110      * @return Nuoroda į šį objektą
00111      */
00112     zmogus& operator=(zmogus&& other) noexcept;
00113
00114 public:
00115     /**
00116      * @brief Gyvuojančių objektų skaitiklis.
00117      *
00118      * Didinamas kiekviename konstruktoriuje, mažinamas destruktoriuje.
00119      * Naudojamas testuose objektų nutekėjimui aptikti.
00120      */
00121     static int gyvuZmoniu;
00122
00123     /**
00124      * @brief Virtualus destruktorius.
00125      *
00126      * @warning `virtual` kvalifikatorius kritiškai svarbus. Be jo:
00127      * @code
00128      * zmogus* p = new studentas(...);
00129      * delete p;    // kviečiamas TIK ~zmogus()
00130      *             // studentas laukai NEATLAISVINAMI
00131      * @endcode
00132      *
00133      * Su `virtual` naikinimo tvarka teisinga: `~studentas()` + `~zmogus()`.
00134      */
00135     virtual ~zmogus();
00136
00137     /**
00138      * @brief Gražina žmogaus vardą.

```

```

00139     * @return Konstanti nuoroda į vardą (be kopijavimo)
00140     */
00141     inline const std::string& vardas() const { return vardas_; }
00142
00143     /**
00144     * @brief Gražina žmogaus pavardę.
00145     * @return Konstanti nuoroda į pavardę (be kopijavimo)
00146     */
00147     inline const std::string& pavarde() const { return pavarde_; }
00148
00149     /**
00150     * @brief Gražina pilną vardą.
00151     * @return Vardas ir pavardė, atskirti tarpu
00152     */
00153     inline std::string pilnasVardas() const { return vardas_ + " " + pavarde_; }
00154
00155     /**
00156     * @brief Nustato vardą su validacija.
00157     * @param v Naujas vardas
00158     * @throws std::invalid_argument jei vardas tuščias
00159     */
00160     void setVardas(const std::string& v);
00161
00162     /**
00163     * @brief Nustato pavardę su validacija.
00164     * @param p Nauja pavardė
00165     * @throws std::invalid_argument jei pavardė tuščia
00166     */
00167     void setPavarde(const std::string& p);
00168
00169     /**
00170     * @brief Gražina galutinį įvertinimą.
00171     *
00172     * Grynai virtualus -- kiekvienas žmogaus tipas skaičiuoja jį savaip.
00173     * Studentui tai galutinis balas pagal namų darbų vidurkį ir egzaminą.
00174     *
00175     * @return Galutinis įvertinimas
00176     */
00177     virtual double galutinis() const = 0;
00178
00179     /**
00180     * @brief Išveda objektą į srautą.
00181     *
00182     * Grynai virtualus. Realizuoja polimorfinį `operator«`.
00183     *
00184     * @param os Išvesties srautas
00185     */
00186     virtual void print(std::ostream& os) const = 0;
00187
00188     /**
00189     * @brief Nuskaito objektą iš srauto.
00190     *
00191     * Grynai virtualus. Realizuoja polimorfinį `operator»`.
00192     *
00193     * @param is Įvesties srautas
00194     * @return Nuoroda į tą patį srautą (grandininiam skaitymui)
00195     */
00196     virtual std::istream& read(std::istream& is) = 0;
00197
00198     /**
00199     * @brief Gražina žmogaus tipo pavadinimą.
00200     *
00201     * Grynai virtualus. Naudojamas diagnostikai ir logams.
00202     *
00203     * @return Tipo pavadinimas, pvz. "Studentas"
00204     */
00205     virtual std::string tipas() const = 0;
00206 };
00207
00208 /**
00209 * @brief Polimorfinis išvesties operatorius.
00210 *
00211 * Priima `zmogus` nuorodą, bet kviečia virtualų print() metodą.
00212 * Todėl veikia su bet kuria išvestine klase -- dabartine ir būsimomis.
00213 *
00214 * @param os Išvesties srautas
00215 * @param z Išvedamas objektas
00216 * @return Nuoroda į srautą
00217 *
00218 * @par Pavyzdys
00219 * @code
00220 * studentas s("Jonas", "Jonaitis", {5, 6}, 7);
00221 * s.calculateGalutinis();
00222 *
00223 * std::cout << s << "\n";           // į ekraną
00224 *
00225 * std::ofstream f("rezultatai.txt");

```

```

00226 * f « s « "\n";                // į failą
00227 *
00228 * zmogus& z = s;
00229 * std::cout « z « "\n";        // per bazinės klasės nuorodą
00230 * @endcode
00231 */
00232 std::ostream& operator<<(std::ostream& os, const zmogus& z);
00233
00234 /**
00235 * @brief Polimorfinis įvesties operatorius.
00236 *
00237 * Priima `zmogus&` nuorodą, bet kviečia virtualų read() metodą.
00238 *
00239 * @param is Įvesties srautas
00240 * @param z Objektas, į kurį nuskaitomi duomenys
00241 * @return Nuoroda į srautą
00242 *
00243 * @throws std::runtime_error jei duomenys netinkami
00244 */
00245 std::istream& operator>>(std::istream& is, zmogus& z);
00246
00247 #endif // ZMOGUS_H_DEFINED

```

8.51 README.md Failo Nuoroda

8.52 tests/benchmark.cpp Failo Nuoroda

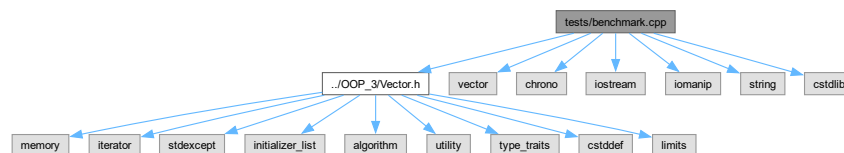
Spartos analizė: std::vector vs nuosava [Vector](#) klasė.

```

#include "../OOP_3/Vector.h"
#include <vector>
#include <chrono>
#include <iostream>
#include <iomanip>
#include <string>
#include <cstdlib>

```

Įtraukimo priklausomybių diagrama benchmark.cpp:



Tipų apibrėžimai

- using [Laikrodis](#) = std::chrono::high_resolution_clock
- using [Sekundes](#) = std::chrono::duration<double>

Funkcijos

- double [matuokStdVector](#) (unsigned int sz)
Matuoja std::vector užpildymo laiką.
- double [matuokVector](#) (unsigned int sz)
Matuoja nuosavos [Vector](#) užpildymo laiką.
- void [tyrimas1](#) ()
- unsigned long long [skaiciuokStdVector](#) (unsigned int sz)
Skaičiuoja std::vector perskirstymus.
- unsigned long long [skaiciuokVector](#) (unsigned int sz)
Skaičiuoja nuosavos [Vector](#) perskirstymus.
- void [tyrimas2](#) ()

- void [tyrimas3](#) ()
- int [main](#) (int argc, char *argv[])

8.52.1 Smulkus aprašymas

Spartos analizė: std::vector vs nuosava [Vector](#) klasė.

Atlieka du tyrimus:

1. push_back() spartos palyginimas su 10 ... 10 elementų
2. Atminties perskirstymų skaičiavimas

Kompiliavimas: g++ -std=c++17 -O2 [benchmark.cpp](#) -o benchmark

Paleidimas: ./benchmark — visi tyrimai ./benchmark 1 — tik push_back tyrimas ./benchmark 2 — tik perskirstymų tyrimas

Įspėjimas

100 000 000 elementų testas naudoja ~400 MB kiekvienam konteneriui. Su 8 GB RAM tai priimtina, bet mažesnėse sistemose gali prireikti praleisti paskutinį dydį.

Apibrėžimas faile [benchmark.cpp](#).

8.52.2 Tipų apibrėžimų Dokumentacija

8.52.2.1 Laikrodys

using [Laikrodys](#) = std::chrono::high_resolution_clock

Apibrėžimas failo [benchmark.cpp](#) eilutėje 31.

8.52.2.2 Sekundes

using [Sekundes](#) = std::chrono::duration<double>

Apibrėžimas failo [benchmark.cpp](#) eilutėje 32.

8.52.3 Funkcijos Dokumentacija

8.52.3.1 matuokStdVector()

```
double matuokStdVector (
    unsigned int sz)
```

Matuoja std::vector užpildymo laiką.

Parametrai

sz	Elementų skaičius
----	-------------------

Gražina

Trukmė sekundėmis

Apibrėžimas failo [benchmark.cpp](#) eilutėje 43.

Naudojamas [tyrimas1\(\)](#).

8.52.3.2 matuokVector()

```
double matuokVector (
    unsigned int sz)
```

Matuoja nuosavos [Vector](#) užpildymo laiką.

Parametrai

sz	Elementų skaičius
----	-------------------

Gražina

Trukmė sekundėmis

Apibrėžimas failo [benchmark.cpp](#) eilutėje 60.
Ryšiai [Vector< T, Allocator >::push_back\(\)](#).
Naudojamas [tyrimas1\(\)](#).

8.52.3.3 tyrimas1()

```
void tyrimas1 ()
```

Apibrėžimas failo [benchmark.cpp](#) eilutėje 72.
Ryšiai [matuokStdVector\(\)](#), ir [matuokVector\(\)](#).
Naudojamas [main\(\)](#).

8.52.3.4 skaiciuokStdVector()

```
unsigned long long skaiciuokStdVector (  
    unsigned int sz)
```

Skaičiuoja std::vector perskirstymus.

Perskirstymas aptinkamas stebint capacity() pokytį — jis įvyksta tada, kai capacity() == size() ir pridedamas naujas elementas.

Parametrai

sz	Elementų skaičius
----	-------------------

Gražina

Perskirstymų skaičius

Apibrėžimas failo [benchmark.cpp](#) eilutėje 117.
Naudojamas [tyrimas2\(\)](#).

8.52.3.5 skaiciuokVector()

```
unsigned long long skaiciuokVector (  
    unsigned int sz)
```

Skaičiuoja nuosavos [Vector](#) perskirstymus.

Parametrai

sz	Elementų skaičius
----	-------------------

Gražina

Perskirstymų skaičius

Apibrėžimas failo [benchmark.cpp](#) eilutėje 138.
Ryšiai [Vector< T, Allocator >::capacity\(\)](#), ir [Vector< T, Allocator >::push_back\(\)](#).
Naudojamas [tyrimas2\(\)](#).

8.52.3.6 tyrimas2()

```
void tyrimas2 ()
```

Apibrėžimas failo [benchmark.cpp](#) eilutėje 154.

Ryšiai [skaiciuokStdVector\(\)](#), ir [skaiciuokVector\(\)](#).

Naudojamas [main\(\)](#).

8.52.3.7 tyrimas3()

```
void tyrimas3 ()
```

Apibrėžimas failo [benchmark.cpp](#) eilutėje 187.

Ryšiai [Vector< T, Allocator >::capacity\(\)](#), ir [Vector< T, Allocator >::push_back\(\)](#).

Naudojamas [main\(\)](#).

8.52.3.8 main()

```
int main (
    int argc,
    char * argv[])
```

Apibrėžimas failo [benchmark.cpp](#) eilutėje 217.

Ryšiai [tyrimas1\(\)](#), [tyrimas2\(\)](#), ir [tyrimas3\(\)](#).

8.53 benchmark.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file benchmark.cpp
00003  * @brief Spartos analizė: std::vector vs nuosava Vector klasė.
00004  *
00005  * Atlieka du tyrimus:
00006  * 1. push_back() spartos palyginimas su 10 ... 10 elementų
00007  * 2. Atminties perskirstymų skaičiavimas
00008  *
00009  * Kompiliavimas:
00010  * g++ -std=c++17 -O2 benchmark.cpp -o benchmark
00011  *
00012  * Paleidimas:
00013  * ./benchmark -- visi tyrimai
00014  * ./benchmark 1 -- tik push_back tyrimas
00015  * ./benchmark 2 -- tik perskirstymų tyrimas
00016  *
00017  * @warning 100 000 000 elementų testas naudoja ~400 MB kiekvienam
00018  * konteineriui. Su 8 GB RAM tai priimtina, bet mažesnėse
00019  * sistemose gali prireikti praleisti paskutinį dydį.
00020  */
00021
00022 #include "../OOP_3/Vector.h"
00023
00024 #include <vector>
00025 #include <chrono>
00026 #include <iostream>
00027 #include <iomanip>
00028 #include <string>
00029 #include <cstdlib>
00030
00031 using Laikrodis = std::chrono::high_resolution_clock;
00032 using Sekundes = std::chrono::duration<double>;
00033
00034 // =====
00035 // 1 TYRIMAS -- push_back sparta
00036 // =====
00037
00038 /**
00039  * @brief Matuoja std::vector užpildymo laiką.
00040  * @param sz Elementų skaičius
00041  * @return Trukmė sekundėmis
00042  */
00043 double matuokStdVector(unsigned int sz)
00044 {
00045     auto pradzia = Laikrodis::now();
00046
00047     std::vector<int> v1;
00048     for (unsigned int i = 1; i <= sz; ++i)
00049         v1.push_back(static_cast<int>(i));
00050 }
```

```

00051     auto pabaiga = Laikrodis::now();
00052     return Sekundes(pabaiga - pradzia).count();
00053 }
00054
00055 /**
00056  * @brief Matuoja nuosavos Vector užpildymo laiką.
00057  * @param sz Elementų skaičius
00058  * @return Trukmė sekundėmis
00059  */
00060 double matuokVector(unsigned int sz)
00061 {
00062     auto pradzia = Laikrodis::now();
00063
00064     Vector<int> v2;
00065     for (unsigned int i = 1; i <= sz; ++i)
00066         v2.push_back(static_cast<int>(i));
00067
00068     auto pabaiga = Laikrodis::now();
00069     return Sekundes(pabaiga - pradzia).count();
00070 }
00071
00072 void tyrimas1()
00073 {
00074     const unsigned int dydziai[] = {
00075         10000u, 100000u, 1000000u, 10000000u, 100000000u
00076     };
00077
00078     std::cout << "\n=====\\n";
00079     std::cout << " 1 TYRIMAS - push_back() sparta\\n";
00080     std::cout << "=====\\n\\n";
00081
00082     std::cout << std::left << std::setw(14) << "Elementu"
00083         << std::right << std::setw(14) << "std::vector"
00084         << std::setw(14) << "Vector"
00085         << std::setw(12) << "Santykis" << "\\n";
00086     std::cout << std::string(54, '-') << "\\n";
00087
00088     for (unsigned int sz : dydziai) {
00089         const double t1 = matuokStdVector(sz);
00090         const double t2 = matuokVector(sz);
00091         const double santykis = t1 > 0.0 ? t2 / t1 : 0.0;
00092
00093         std::cout << std::left << std::setw(14) << sz
00094             << std::right << std::fixed << std::setprecision(6)
00095             << std::setw(14) << t1
00096             << std::setw(14) << t2
00097             << std::setprecision(2) << std::setw(11) << santykis << "x"
00098             << "\\n";
00099     }
00100
00101     std::cout << "\\nSantykis > 1.0 reiskia, kad Vector letesnis uz std::vector.\\n\\n";
00102 }
00103
00104 // =====
00105 // 2 TYRIMAS -- atminties perskirstymai
00106 // =====
00107
00108 /**
00109  * @brief Skaičiuoja std::vector perskirstymus.
00110  *
00111  * Perskirstymas aptinkamas stebint capacity() pokytį -- jis įvyksta
00112  * tada, kai capacity() == size() ir pridedamas naujas elementas.
00113  *
00114  * @param sz Elementų skaičius
00115  * @return Perskirstymų skaičius
00116  */
00117 unsigned long long skaiciuokStdVector(unsigned int sz)
00118 {
00119     std::vector<int> v;
00120     unsigned long long perskirstymu = 0;
00121     std::size_t senaTalpa = v.capacity();
00122
00123     for (unsigned int i = 1; i <= sz; ++i) {
00124         v.push_back(static_cast<int>(i));
00125         if (v.capacity() != senaTalpa) {
00126             ++perskirstymu;
00127             senaTalpa = v.capacity();
00128         }
00129     }
00130     return perskirstymu;
00131 }
00132
00133 /**
00134  * @brief Skaičiuoja nuosavos Vector perskirstymus.
00135  * @param sz Elementų skaičius
00136  * @return Perskirstymų skaičius
00137  */

```

```

00138 unsigned long long skaiciuokVector(unsigned int sz)
00139 {
00140     Vector<int> v;
00141     unsigned long long perskirstymu = 0;
00142     std::size_t senaTalpa = v.capacity();
00143
00144     for (unsigned int i = 1; i <= sz; ++i) {
00145         v.push_back(static_cast<int>(i));
00146         if (v.capacity() != senaTalpa) {
00147             ++perskirstymu;
00148             senaTalpa = v.capacity();
00149         }
00150     }
00151     return perskirstymu;
00152 }
00153
00154 void tyrimas2()
00155 {
00156     const unsigned int dydziai[] = {
00157         10000u, 100000u, 1000000u, 10000000u, 100000000u
00158     };
00159
00160     std::cout << "\n=====\\n";
00161     std::cout << " 2 TYRIMAS - atminties perskirstymu skaicius\\n";
00162     std::cout << "=====\\n\\n";
00163
00164     std::cout << std::left << std::setw(14) << "Elementu"
00165         << std::right << std::setw(14) << "std::vector"
00166         << std::setw(14) << "Vector" << "\\n";
00167     std::cout << std::string(42, '-') << "\\n";
00168
00169     for (unsigned int sz : dydziai) {
00170         const auto p1 = skaiciuokStdVector(sz);
00171         const auto p2 = skaiciuokVector(sz);
00172
00173         std::cout << std::left << std::setw(14) << sz
00174             << std::right << std::setw(14) << p1
00175             << std::setw(14) << p2 << "\\n";
00176     }
00177
00178     std::cout << "\\nPerskirstymas ivyksta, kai capacity() == size() ir\\n"
00179         << "pridedamas naujas elementas. Su dvigubinimo strategija\\n"
00180         << "perskirstymu skaicius artimas log2(n).\\n\\n";
00181 }
00182
00183 // =====
00184 // Papildomas tyrimas -- talpos augimo seka
00185 // =====
00186
00187 void tyrimas3()
00188 {
00189     std::cout << "\n=====\\n";
00190     std::cout << " 3 TYRIMAS - talpos augimo seka (pirmi 20 elementu)\\n";
00191     std::cout << "=====\\n\\n";
00192
00193     std::cout << std::left << std::setw(10) << "size()"
00194         << std::setw(20) << "std::vector cap"
00195         << std::setw(20) << "Vector cap" << "\\n";
00196     std::cout << std::string(50, '-') << "\\n";
00197
00198     std::vector<int> sv;
00199     Vector<int> v;
00200
00201     for (int i = 1; i <= 20; ++i) {
00202         sv.push_back(i);
00203         v.push_back(i);
00204         std::cout << std::left << std::setw(10) << i
00205             << std::setw(20) << sv.capacity()
00206             << std::setw(20) << v.capacity() << "\\n";
00207     }
00208
00209     std::cout << "\\nMSVC naudoja 1.5x augimo koeficienta, GCC/Clang - 2x.\\n"
00210         << "Nuosava Vector naudoja 2x.\\n\\n";
00211 }
00212
00213 // =====
00214 // main
00215 // =====
00216
00217 int main(int argc, char* argv[])
00218 {
00219     std::cout << "\\n";
00220     std::cout << "#####\\n";
00221     std::cout << "# std::vector vs Vector - spartos analize #\\n";
00222     std::cout << "#####\\n";
00223
00224     if (argc > 1) {

```

```

00225         const int nr = std::atoi(argv[1]);
00226         switch (nr) {
00227             case 1: tyrimas1(); break;
00228             case 2: tyrimas2(); break;
00229             case 3: tyrimas3(); break;
00230             default:
00231                 std::cout << "\nNezinomas tyrimo numeris. Galimi: 1, 2, 3\n";
00232                 return 1;
00233         }
00234     }
00235     else {
00236         tyrimas3();
00237         tyrimas1();
00238         tyrimas2();
00239     }
00240
00241     return 0;
00242 }

```

8.54 tests/unitTests.cpp Failo Nuoroda

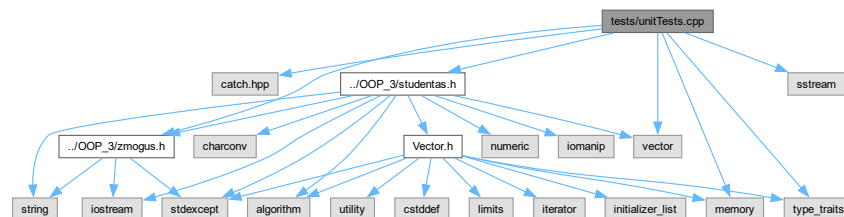
Unit testai klasėms zmogus ir studentas (Catch2 framework).

```

#include "catch.hpp"
#include "../OOP_3/zmogus.h"
#include "../OOP_3/studentas.h"
#include <sstream>
#include <vector>
#include <memory>
#include <type_traits>

```

Įtraukimo priklausomybių diagrama unitTests.cpp:



Apibrėžimai

- #define `CATCH_CONFIG_MAIN`

Funkcijos

- `TEST_CASE` ("zmogus yra abstrakti klase", "[abstrakti]")
- `TEST_CASE` ("Konstruktoriai", "[konstruktoriai]")
- `TEST_CASE` ("Konstruktoriaus validacija", "[konstruktoriai][validacija]")
- `TEST_CASE` ("Rule of Five: kopijavimo konstruktorius", "[rule5][kopijavimas]")
- `TEST_CASE` ("Rule of Five: kopijavimo priskyrimo operatorius", "[rule5][kopijavimas]")
- `TEST_CASE` ("Rule of Five: perkeliavimo konstruktorius", "[rule5][perkelimas]")
- `TEST_CASE` ("Rule of Five: perkeliavimo priskyrimo operatorius", "[rule5][perkelimas]")
- `TEST_CASE` ("Rule of Five: noexcept specifikatorius", "[rule5][noexcept]")
- `TEST_CASE` ("Rule of Five: destruktorius", "[rule5][destruktorius]")
- `TEST_CASE` ("Polimorfizmas", "[polimorfizmas]")
- `TEST_CASE` ("Ivesties operatorius >>", "[io]")
- `TEST_CASE` ("Išvesties operatorius <<", "[io]")
- `TEST_CASE` ("Failo I/O: parseFromLine ir appendListTo", "[io][failai]")
- `TEST_CASE` ("Lyginimo operatoriai", "[operatoriai]")

- `TEST_CASE` ("Indeksavimo operatorius []", "[operatoriai]")
- `TEST_CASE` ("calculateGalutinis", "[skaiciavimas]")
- `TEST_CASE` ("Seteriai", "[seteriai]")
- `TEST_CASE` ("Veikimas su STL konteineriais", "[konteineriai]")

8.54.1 Smulkus aprašymas

Unit testai klasėms žmogus ir studentas (Catch2 framework).

Testai suskirstyti į sekcijas pagal testuojamą klasės aspektą. Pagrindinis dėmesys — penkių metodų taisyklė (Rule of Five), nes būtent ji lengviausiai sugenda tyliai: pamiršus bazinės klasės kvietimą arba noexcept, programa vis tiek kompiliuos ir veiks, tik lėčiau arba su prarastais duomenimis.

Paleidimas: `./unitTests` — visi testai `./unitTests -s` — su detaliu išvedimu `./unitTests "[rule5]"` — tik Rule of Five testai `./unitTests -l` — testų sąrašas

Apibrėžimas faile [unitTests.cpp](#).

8.54.2 Apibrėžimų Dokumentacija

8.54.2.1 CATCH_CONFIG_MAIN

```
#define CATCH_CONFIG_MAIN
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 18.

8.54.3 Funkcijos Dokumentacija

8.54.3.1 TEST_CASE() [1/18]

```
TEST_CASE (
    "zmogus yra abstrakti klase" ,
    "" [abstrakti])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 52.

8.54.3.2 TEST_CASE() [2/18]

```
TEST_CASE (
    "Konstruktoriai" ,
    "" [konstruktoriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 82.

Ryšiai [studentas::egzaminas\(\)](#), [Vector< T, Allocator >::empty\(\)](#), [studentas::galutinisMed\(\)](#), [studentas::galutinisVid\(\)](#), [studentas::nd\(\)](#), [zmogus::pavarde\(\)](#), [Vector< T, Allocator >::size\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.3 TEST_CASE() [3/18]

```
TEST_CASE (
    "Konstruktoriaus validacija" ,
    "" [konstruktoriai][validacija])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 114.

8.54.3.4 TEST_CASE() [4/18]

```
TEST_CASE (
    "Rule of Five: kopijavimo konstruktorius" ,
    "" [rule5][kopijavimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 150.

Ryšiai [studentas::addPazymys\(\)](#), [studentas::calculateGalutinis\(\)](#), [studentas::egzaminas\(\)](#), [studentas::galutinisMed\(\)](#), [studentas::galutinisVid\(\)](#), [studentas::nd\(\)](#), [zmogus::pavarde\(\)](#), [zmogus::setVardas\(\)](#), [Vector< T, Allocator >::size\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.5 TEST_CASE() [5/18]

```
TEST_CASE (
    "Rule of Five: kopijavimo priskyrimo operatorius" ,
    "" [rule5][kopijavimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 181.

Ryšiai [studentas::calculateGalutinis\(\)](#), [studentas::egzaminas\(\)](#), [studentas::galutinisVid\(\)](#), [studentas::nd\(\)](#), [zmogus::pavarde\(\)](#), [Vector< T, Allocator >::size\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.6 TEST_CASE() [6/18]

```
TEST_CASE (
    "Rule of Five: perkeliimo konstruktorius" ,
    "" [rule5][perkelimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 221.

Ryšiai [studentas::calculateGalutinis\(\)](#), [studentas::egzaminas\(\)](#), [Vector< T, Allocator >::empty\(\)](#), [studentas::galutinisVid\(\)](#), [studentas::nd\(\)](#), [zmogus::pavarde\(\)](#), [studentas::setEgzaminas\(\)](#), [zmogus::setPavarde\(\)](#), [zmogus::setVardas\(\)](#), [Vector< T, Allocator >::size\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.7 TEST_CASE() [7/18]

```
TEST_CASE (
    "Rule of Five: perkeliimo priskyrimo operatorius" ,
    "" [rule5][perkelimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 259.

Ryšiai [studentas::calculateGalutinis\(\)](#), [Vector< T, Allocator >::empty\(\)](#), [studentas::nd\(\)](#), [Vector< T, Allocator >::size\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.8 TEST_CASE() [8/18]

```
TEST_CASE (
    "Rule of Five: noexcept specifikatorius" ,
    "" [rule5][noexcept])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 285.

8.54.3.9 TEST_CASE() [9/18]

```
TEST_CASE (
    "Rule of Five: destruktorius" ,
    "" [rule5][destruktorius])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 303.

Ryšiai [studentas::gyvuStudentu](#), ir [zmogus::gyvuZmoniu](#).

8.54.3.10 TEST_CASE() [10/18]

```
TEST_CASE (
    "Polimorfizmas" ,
    "" [polimorfizmas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 347.

Ryšiai [studentas::calculateGalutinis\(\)](#), [zmogus::galutinis\(\)](#), [zmogus::pilnasVardas\(\)](#), [zmogus::tipas\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.11 TEST_CASE() [11/18]

```
TEST_CASE (
    "Ivesties operatorius >>" ,
    "" [io])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 395.

Ryšiai [studentas::egzaminas\(\)](#), [studentas::nd\(\)](#), [zmogus::pavarde\(\)](#), [Vector< T, Allocator >::size\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.12 TEST_CASE() [12/18]

```
TEST_CASE (
    "Išvesties operatorius <<" ,
    "" [io])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 417.

Ryšiai [studentas::calculateGalutinis\(\)](#).

8.54.3.13 TEST_CASE() [13/18]

```
TEST_CASE (
    "Failo I/O: parseFromLine ir appendListTo" ,
    "" [io][failai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 439.

Ryšiai [studentas::appendListTo\(\)](#), [studentas::egzaminas\(\)](#), [studentas::nd\(\)](#), [studentas::parseFromLine\(\)](#), [zmogus::pavarde\(\)](#), [Vector< T, Allocator >::size\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.14 TEST_CASE() [14/18]

```
TEST_CASE (
    "Lyginimo operatoriai" ,
    "" [operatoriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 493.

8.54.3.15 TEST_CASE() [15/18]

```
TEST_CASE (
    "Indeksavimo operatorius " [],
    "" [operatoriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 525.

8.54.3.16 TEST_CASE() [16/18]

```
TEST_CASE (
    "calculateGalutinis" ,
    "" [skaiciavimas])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 545.

Ryšiai [studentas::calculateGalutinis\(\)](#), [studentas::galutinisMed\(\)](#), [studentas::galutinisVid\(\)](#), [studentas::setEgzaminas\(\)](#), [zmogus::setPavarde\(\)](#), ir [zmogus::setVardas\(\)](#).

8.54.3.17 TEST_CASE() [17/18]

```
TEST_CASE (
    "Seteriai" ,
    "" [seteriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 593.

Ryšiai [studentas::addPazymys\(\)](#), [Vector< T, Allocator >::back\(\)](#), [studentas::calculateGalutinis\(\)](#), [studentas::egzaminas\(\)](#), [Vector< T, Allocator >::empty\(\)](#), [studentas::galutinisVid\(\)](#), [studentas::isvalyk\(\)](#), [studentas::nd\(\)](#), [zmogus::pavarde\(\)](#), [studentas::setEgzaminas\(\)](#), [studentas::setNd\(\)](#), [zmogus::setPavarde\(\)](#), [zmogus::setVardas\(\)](#), [Vector< T, Allocator >::size\(\)](#), ir [zmogus::vardas\(\)](#).

8.54.3.18 TEST_CASE() [18/18]

```
TEST_CASE (
    "Veikimas su STL konteineriais" ,
    "" [konteineriai])
```

Apibrėžimas failo [unitTests.cpp](#) eilutėje 642.

8.55 unitTests.cpp

Eiti į šio failo dokumentaciją.

```

00001 /**
00002  * @file unitTests.cpp
00003  * @brief Unit testai klasėms zmogus ir studentas (Catch2 framework).
00004  *
00005  * Testai suskirstyti į sekcijas pagal testuojamą klasės aspektą.
00006  * Pagrindinis dėmesys -- penkių metodų taisyklė (Rule of Five),
00007  * nes būtent ji lengviausiai sugenda tyliai: pamiršus bazinės klasės
00008  * kvietimą arba noexcept, programa vis tiek kompiliuos ir veiks,
00009  * tik lėčiau arba su prarastais duomenimis.
00010  *
00011  * Paleidimas:
00012  *      ./unitTests           -- visi testai
00013  *      ./unitTests -s       -- su detaliu išvedimu
00014  *      ./unitTests "[rule5]" -- tik Rule of Five testai
00015  *      ./unitTests -l       -- testų sąrašas
00016  */
00017
00018 #define CATCH_CONFIG_MAIN
00019 #include "catch.hpp"
00020
00021 #include "../OOP_3/zmogus.h"
00022 #include "../OOP_3/studentas.h"
00023
00024 #include <sstream>
00025 #include <vector>
00026 #include <memory>
00027 #include <type_traits>
00028
00029 // =====
00030 // Pagalbinės funkcijos
00031 // =====
00032
00033 namespace {
00034
00035     /**
00036     * @brief Sukuria testinį studentą su žinomomis reikšmėmis.
00037     *
00038     * ND: 5,5,5,5,5 → vidurkis 5.0, mediana 5.0
00039     * Egzaminas 10 → galutinis = 0.4*5.0 + 0.6*10 = 8.0
00040     */
00041     studentas kurkTestini()
00042     {
00043         return studentas("Jonas", "Jonaitis", { 5, 5, 5, 5, 5 }, 10);
00044     }
00045
00046 } // anoniminis namespace
00047
00048 // =====
00049 // 1. ABSTRAKČIOJI KLASĖ
00050 // =====
00051
00052 TEST_CASE("zmogus yra abstrakti klase", "[abstrakti]")
00053 {
00054     SECTION("is_abstract patikra") {
00055         REQUIRE(std::is_abstract<zmogus>::value);
00056         REQUIRE_FALSE(std::is_abstract<studentas>::value);
00057     }
00058
00059     SECTION("zmogus objekto sukurti neįmanoma") {
00060         // Sis kodas NESIKOMPILIUOTU:
00061         //     zmogus z;
00062         //     zmogus* p = new zmogus();
00063         // Todel patikra atliekama statiskai:
00064         REQUIRE_FALSE(std::is_default_constructible<zmogus>::value);
00065         REQUIRE(std::is_default_constructible<studentas>::value);
00066     }
00067
00068     SECTION("paveldejimo rysys") {
00069         REQUIRE(std::is_base_of<zmogus, studentas>::value);
00070         REQUIRE(std::is_convertible<studentas*, zmogus*>::value);
00071     }
00072
00073     SECTION("virtualus destruktorius") {
00074         REQUIRE(std::has_virtual_destructor<zmogus>::value);
00075     }
00076 }
00077
00078 // =====
00079 // 2. KONSTRUKTORIAI
00080 // =====
00081
00082 TEST_CASE("Konstruktoriai", "[konstruktoriai]")
00083 {

```

```

00084 SECTION("numatytais konstruktorius inicijuojama visus laukus") {
00085     studentas s;
00086     REQUIRE(s.vardas().empty());
00087     REQUIRE(s.pavarde().empty());
00088     REQUIRE(s.nd().empty());
00089     REQUIRE(s.egzaminas() == 0);
00090     REQUIRE(s.galutinisVid() == Approx(0.0));
00091     REQUIRE(s.galutinisMed() == Approx(0.0));
00092     REQUIRE_FALSE(static_cast<bool>(s));
00093 }
00094
00095 SECTION("pilnas konstruktorius issaugo duomenis") {
00096     studentas s("Jonas", "Jonaitis", { 5, 6, 7 }, 8);
00097     REQUIRE(s.vardas() == "Jonas");
00098     REQUIRE(s.pavarde() == "Jonaitis");
00099     REQUIRE(s.nd().size() == 3);
00100     REQUIRE(s.egzaminas() == 8);
00101     REQUIRE(static_cast<bool>(s));
00102 }
00103
00104 SECTION("konstruktorius is srauto") {
00105     std::istringstream iss("Petras Petratis 8 9 10 7");
00106     studentas s(iss);
00107     REQUIRE(s.vardas() == "Petras");
00108     REQUIRE(s.pavarde() == "Petratis");
00109     REQUIRE(s.nd().size() == 3);
00110     REQUIRE(s.egzaminas() == 7);
00111 }
00112 }
00113
00114 TEST_CASE("Konstruktoriaus validacija", "[konstruktoriai][validacija]")
00115 {
00116     SECTION("bazine klase meta invalid_argument tusciam vardui") {
00117         REQUIRE_THROWS_AS(studentas("", "Jonaitis", { 5 }, 10),
00118             std::invalid_argument);
00119     }
00120
00121     SECTION("bazine klase meta invalid_argument tusciai pavardei") {
00122         REQUIRE_THROWS_AS(studentas("Jonas", "", { 5 }, 10),
00123             std::invalid_argument);
00124     }
00125
00126     SECTION("isvestine klase meta runtime_error blogam ND pazymiu") {
00127         REQUIRE_THROWS_AS(studentas("Jonas", "Jonaitis", { 5, 99 }, 10),
00128             std::runtime_error);
00129         REQUIRE_THROWS_AS(studentas("Jonas", "Jonaitis", { 0 }, 10),
00130             std::runtime_error);
00131     }
00132
00133     SECTION("isvestine klase meta runtime_error blogam egzaminui") {
00134         REQUIRE_THROWS_AS(studentas("Jonas", "Jonaitis", { 5 }, 0),
00135             std::runtime_error);
00136         REQUIRE_THROWS_AS(studentas("Jonas", "Jonaitis", { 5 }, 11),
00137             std::runtime_error);
00138     }
00139
00140     SECTION("ribines reiksmes 1 ir 10 yra leidziamos") {
00141         REQUIRE_NOTHROW(studentas("Jonas", "Jonaitis", { 1, 10 }, 1));
00142         REQUIRE_NOTHROW(studentas("Jonas", "Jonaitis", { 1, 10 }, 10));
00143     }
00144 }
00145
00146 // =====
00147 // 3. RULE OF FIVE -- pagrindinis testu blokas
00148 // =====
00149
00150 TEST_CASE("Rule of Five: kopijavimo konstruktorius", "[rule5][kopijavimas]")
00151 {
00152     studentas originalas = kurkTestini();
00153     originalas.calculateGalutinis();
00154
00155     studentas kopija(originalas);
00156
00157     SECTION("bazines klases laukai nukopijuoti") {
00158         // Daznausia paveldejimo klaida: pamirsus zmogus(other) kvietima,
00159         // vardas ir pavarde liktu tusti
00160         REQUIRE(kopija.vardas() == "Jonas");
00161         REQUIRE(kopija.pavarde() == "Jonaitis");
00162     }
00163
00164     SECTION("isvestines klases laukai nukopijuoti") {
00165         REQUIRE(kopija.nd() == originalas.nd());
00166         REQUIRE(kopija.egzaminas() == originalas.egzaminas());
00167         REQUIRE(kopija.galutinisVid() == Approx(originalas.galutinisVid()));
00168         REQUIRE(kopija.galutinisMed() == Approx(originalas.galutinisMed()));
00169     }
00170 }

```

```

00171     SECTION("kopiija yra GILI (deep copy) -- objektai nepriklausomi") {
00172         kopiija.setVardas("Pakeistas");
00173         REQUIRE(originalas.vardas() == "Jonas");
00174
00175         kopiija.addPazymys(1);
00176         REQUIRE(originalas.nd().size() == 5);
00177         REQUIRE(kopiija.nd().size() == 6);
00178     }
00179 }
00180
00181 TEST_CASE("Rule of Five: kopijavimo priskyrimo operatorius", "[rule5][kopijavimas]")
00182 {
00183     studentas a = kurkTestini();
00184     a.calculateGalutinis();
00185
00186     SECTION("priskyrimas nukopijuoja visus laukus") {
00187         studentas b;
00188         b = a;
00189         REQUIRE(b.vardas() == a.vardas());
00190         REQUIRE(b.pavarde() == a.pavarde());
00191         REQUIRE(b.nd() == a.nd());
00192         REQUIRE(b.galutinisVid() == Approx(a.galutinisVid()));
00193     }
00194
00195     SECTION("priskyrimas sau nesugadina objekto") {
00196         // Be patikros this == &other objektas galetu buti sugadintas
00197         a = a;
00198         REQUIRE(a.vardas() == "Jonas");
00199         REQUIRE(a.nd().size() == 5);
00200         REQUIRE(a.egzaminas() == 10);
00201     }
00202
00203     SECTION("grandinis priskyrimas veikia") {
00204         // Reikalauja, kad operator= grazintu *this
00205         studentas b, c;
00206         c = b = a;
00207         REQUIRE(b.vardas() == "Jonas");
00208         REQUIRE(c.vardas() == "Jonas");
00209         REQUIRE(c.nd() == a.nd());
00210     }
00211
00212     SECTION("priskyrimas perrasos senus duomenis") {
00213         studentas b("Senas", "Senaitis", { 1, 1, 1 }, 1);
00214         b = a;
00215         REQUIRE(b.vardas() == "Jonas");
00216         REQUIRE(b.nd().size() == 5);
00217         REQUIRE(b.egzaminas() == 10);
00218     }
00219 }
00220
00221 TEST_CASE("Rule of Five: perkeliimo konstruktorius", "[rule5][perkelimas]")
00222 {
00223     studentas saltinis = kurkTestini();
00224     saltinis.calculateGalutinis();
00225     const double tiketinasVid = saltinis.galutinisVid();
00226
00227     studentas tikslas(std::move(saltinis));
00228
00229     SECTION("bazines klases laukai perkelti") {
00230         // Reikalauja zmogus(std::move(other)) -- be std::move butu
00231         // iskviestas KOPIJAVIMO, ne perkeliimo konstruktorius
00232         REQUIRE(tikslas.vardas() == "Jonas");
00233         REQUIRE(tikslas.pavarde() == "Jonaitis");
00234     }
00235
00236     SECTION("isvestines klases laukai perkelti") {
00237         REQUIRE(tikslas.nd().size() == 5);
00238         REQUIRE(tikslas.egzaminas() == 10);
00239         REQUIRE(tikslas.galutinisVid() == Approx(tiketinasVid));
00240     }
00241
00242     SECTION("saltinis lieka galiojancioje tuscioje busenoje") {
00243         REQUIRE(saltinis.vardas().empty());
00244         REQUIRE(saltinis.pavarde().empty());
00245         REQUIRE(saltinis.nd().empty());
00246         REQUIRE(saltinis.egzaminas() == 0);
00247         REQUIRE_FALSE(static_cast<bool>(saltinis));
00248     }
00249
00250     SECTION("istustinta saltini saugu naudoti is naujo") {
00251         saltinis.setVardas("Naujas");
00252         saltinis.setPavarde("Naujaitis");
00253         saltinis.setEgzaminas(5);
00254         REQUIRE(saltinis.vardas() == "Naujas");
00255         REQUIRE(static_cast<bool>(saltinis));
00256     }
00257 }

```

```

00258
00259 TEST_CASE("Rule of Five: perkelimo priskyrimo operatorius", "[rule5][perkelimas]")
00260 {
00261     SECTION("perkelimas i esama objekta") {
00262         studentas saltinis = kurkTestini();
00263         saltinis.calculateGalutinis();
00264         studentas tikslas("Senas", "Senaitis", { 1, 1 }, 1);
00265
00266         tikslas = std::move(saltinis);
00267
00268         REQUIRE(tikslas.vardas() == "Jonas");
00269         REQUIRE(tikslas.nd().size() == 5);
00270         REQUIRE(tikslas.egzaminas() == 10);
00271         REQUIRE(saltinis.vardas().empty());
00272         REQUIRE(saltinis.nd().empty());
00273     }
00274
00275     SECTION("perkelimo priskyrimas sau nesukelia luzimo") {
00276         studentas x = kurkTestini();
00277         x = std::move(x);
00278         // Standartas negarantuoja konkrečios reikšmės po self-move,
00279         // tik kad objektas liktu galiojancioje busenoje
00280         REQUIRE_NOTHROW(x.vardas());
00281         REQUIRE_NOTHROW(x.nd().size());
00282     }
00283 }
00284
00285 TEST_CASE("Rule of Five: noexcept specifikatorius", "[rule5][noexcept]")
00286 {
00287     // Be noexcept std::vector persikirstymo metu naudotu KOPIJAVIMO,
00288     // ne perkelimo konstruktoriu (std::move_if_noexcept)
00289     SECTION("move konstruktorius yra noexcept") {
00290         REQUIRE(std::is_nothrow_move_constructible<studentas>::value);
00291     }
00292
00293     SECTION("move priskyrimas yra noexcept") {
00294         REQUIRE(std::is_nothrow_move_assignable<studentas>::value);
00295     }
00296
00297     SECTION("bazine klase taip pat noexcept") {
00298         REQUIRE(std::is_nothrow_move_constructible<zmogus>::value);
00299         REQUIRE(std::is_nothrow_move_assignable<zmogus>::value);
00300     }
00301 }
00302
00303 TEST_CASE("Rule of Five: destruktorius", "[rule5][destruktorius]")
00304 {
00305     const int priesStudentu = studentas::gyvuStudentu;
00306     const int priesZmoniu = zmogus::gyvuZmoniu;
00307
00308     SECTION("objektai sunaikinami isejus is srities") {
00309         {
00310             studentas a = kurkTestini();
00311             studentas b = kurkTestini();
00312             studentas c(a);
00313             REQUIRE(studentas::gyvuStudentu == priesStudentu + 3);
00314             REQUIRE(zmogus::gyvuZmoniu == priesZmoniu + 3);
00315         }
00316         REQUIRE(studentas::gyvuStudentu == priesStudentu);
00317         REQUIRE(zmogus::gyvuZmoniu == priesZmoniu);
00318     }
00319
00320     SECTION("virtualus destruktorius naikinant per bazines klases rodykle") {
00321         {
00322             // Be virtual destruktoriaus ~studentas() NEBUTU iskviestas,
00323             // ir nd_ vektorius liktu neatlaisvintas
00324             std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00325                 "Testas", "Testaitis", Vector<int>{ 5, 5 }, 5);
00326             REQUIRE(studentas::gyvuStudentu == priesStudentu + 1);
00327         }
00328         REQUIRE(studentas::gyvuStudentu == priesStudentu);
00329         REQUIRE(zmogus::gyvuZmoniu == priesZmoniu);
00330     }
00331
00332     SECTION("nera objektu nutekejimo naudojant konteinerius") {
00333         {
00334             std::vector<studentas> v;
00335             for (int i = 0; i < 10; ++i)
00336                 v.push_back(kurkTestini());
00337             REQUIRE(studentas::gyvuStudentu == priesStudentu + 10);
00338         }
00339         REQUIRE(studentas::gyvuStudentu == priesStudentu);
00340     }
00341 }
00342
00343 // =====
00344 // 4. POLIMORFIZMAS

```

```

00345 // =====
00346
00347 TEST_CASE("Polimorfizmas", "[polimorfizmas]")
00348 {
00349     studentas s = kurkTestini();
00350     s.calculateGalutinis();
00351
00352     SECTION("virtualus metodai per bazines klases nuoroda") {
00353         zmogus& z = s;
00354         REQUIRE(z.tipas() == "Studentas");
00355         REQUIRE(z.galutinis() == Approx(8.0));
00356         REQUIRE(z.vardas() == "Jonas");
00357         REQUIRE(z.pilnasVardas() == "Jonas Jonaitis");
00358     }
00359
00360     SECTION("virtualus metodai per unique_ptr") {
00361         std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00362             "Ona", "Onaite", Vector<int>{ 10, 10 }, 10);
00363         REQUIRE(p->tipas() == "Studentas");
00364         REQUIRE(p->pilnasVardas() == "Ona Onaite");
00365     }
00366
00367     SECTION("dynamic_cast atgal i isvestine klase") {
00368         std::unique_ptr<zmogus> p = std::make_unique<studentas>(
00369             "Ona", "Onaite", Vector<int>{ 10, 10 }, 10);
00370         studentas* sp = dynamic_cast<studentas*>(p.get());
00371         REQUIRE(sp != nullptr);
00372         REQUIRE(sp->nd().size() == 2);
00373     }
00374
00375     SECTION("polimorfinis konteineris") {
00376         std::vector<std::unique_ptr<zmogus> > zmones;
00377         zmones.push_back(std::make_unique<studentas>(
00378             "Pirmas", "Pirmaitis", Vector<int>{ 10, 10 }, 10));
00379         zmones.push_back(std::make_unique<studentas>(
00380             "Antras", "Antraitis", Vector<int>{ 1, 1 }, 1));
00381
00382         for (auto& z : zmones)
00383             if (auto* sp = dynamic_cast<studentas*>(z.get()))
00384                 sp->calculateGalutinis();
00385
00386         REQUIRE(zmones[0]->galutinis() == Approx(10.0));
00387         REQUIRE(zmones[1]->galutinis() == Approx(1.0));
00388     }
00389 }
00390
00391 // =====
00392 // 5. ĮVESTIES / IŠVESTIES OPERATORIAI
00393 // =====
00394
00395 TEST_CASE("Ivesties operatorius »", "[io]")
00396 {
00397     SECTION("nuskaito is istringstream") {
00398         std::istringstream iss("Ona Onaite 10 9 8 6");
00399         studentas s;
00400         iss >> s;
00401         REQUIRE(s.vardas() == "Ona");
00402         REQUIRE(s.pavarde() == "Onaite");
00403         REQUIRE(s.nd().size() == 3);
00404         REQUIRE(s.egzaminas() == 6);
00405     }
00406
00407     SECTION("polimorfinis » per zmogus nuoroda") {
00408         studentas s;
00409         zmogus& z = s;
00410         std::istringstream iss("Ona Onaite 9 8 7");
00411         iss >> z;
00412         REQUIRE(s.vardas() == "Ona");
00413         REQUIRE(s.egzaminas() == 7);
00414     }
00415 }
00416
00417 TEST_CASE("Ivesties operatorius «", "[io]")
00418 {
00419     studentas s = kurkTestini();
00420     s.calculateGalutinis();
00421
00422     SECTION("isveda varda, pavarde ir balus") {
00423         std::ostringstream oss;
00424         oss << s;
00425         const std::string rez = oss.str();
00426         REQUIRE(rez.find("Jonas") != std::string::npos);
00427         REQUIRE(rez.find("Jonaitis") != std::string::npos);
00428         REQUIRE(rez.find("8.00") != std::string::npos);
00429     }
00430
00431     SECTION("polimorfinis « per zmogus nuoroda") {

```

```

00432         zmogus& z = s;
00433         std::ostringstream oss;
00434         oss << z;
00435         REQUIRE(oss.str().find("Jonas") != std::string::npos);
00436     }
00437 }
00438
00439 TEST_CASE("Failo I/O: parseFromLine ir appendListTo", "[io][failai]")
00440 {
00441     SECTION("parseFromLine nuskaito eilute") {
00442         studentas s;
00443         s.parseFromLine("Antanas Antanaitis 7 8 9 10 5", 4, 1);
00444         REQUIRE(s.vardas() == "Antanas");
00445         REQUIRE(s.pavarde() == "Antanaitis");
00446         REQUIRE(s.nd().size() == 4);
00447         REQUIRE(s.egzaminas() == 5);
00448     }
00449
00450     SECTION("parseFromLine meta klaida blogam pazymiui") {
00451         studentas s;
00452         REQUIRE_THROWS_AS(s.parseFromLine("Vardas Pavarde 99 5", 2, 7),
00453             std::runtime_error);
00454     }
00455
00456     SECTION("parseFromLine meta klaida trukstant duomeniu") {
00457         studentas s;
00458         REQUIRE_THROWS_AS(s.parseFromLine("TikVardas", 2, 3),
00459             std::runtime_error);
00460     }
00461
00462     SECTION("appendListTo suformatuoja eilute") {
00463         studentas s("Rasa", "Rasaite", { 6, 7, 8 }, 9);
00464         std::string buferis;
00465         s.appendListTo(buferis);
00466         REQUIRE(buferis.find("Rasa") != std::string::npos);
00467         REQUIRE(buferis.back() == '\n');
00468         REQUIRE(buferis.size() == 25 + 25 + 4 * 10 + 1);
00469     }
00470
00471     SECTION("round-trip: isvesta eilute nuskaitoma atgal") {
00472         // Sis testas garantuoja, kad writeStudentaiListToFile sukurta
00473         // faila teisingai nuskaitys readStudentaiFromFile
00474         studentas originalas("Rasa", "Rasaite", { 6, 7, 8 }, 9);
00475         std::string buferis;
00476         originalas.appendListTo(buferis);
00477
00478         studentas nuskaitytas;
00479         nuskaitytas.parseFromLine(buferis, 3, 1);
00480
00481         REQUIRE(nuskaitytas.vardas() == originalas.vardas());
00482         REQUIRE(nuskaitytas.pavarde() == originalas.pavarde());
00483         REQUIRE(nuskaitytas.nd() == originalas.nd());
00484         REQUIRE(nuskaitytas.egzaminas() == originalas.egzaminas());
00485         REQUIRE(nuskaitytas == originalas);
00486     }
00487 }
00488
00489 // =====
00490 // 6. PERDENGTI OPERATORIAI
00491 // =====
00492
00493 TEST_CASE("Lyginimo operatoriai", "[operatoriai]")
00494 {
00495     studentas a("Jonas", "Jonaitis", { 5, 6 }, 7);
00496     studentas b("Jonas", "Jonaitis", { 5, 6 }, 7);
00497     studentas c("Petras", "Petraitis", { 5, 6 }, 7);
00498
00499     SECTION("== ir !=") {
00500         REQUIRE(a == b);
00501         REQUIRE(a != c);
00502         REQUIRE_FALSE(a == c);
00503     }
00504
00505     SECTION("< ir > rikiuoja pagal pavarde") {
00506         studentas x("Jonas", "Adamkus", { 5 }, 7);
00507         studentas y("Jonas", "Zukauskas", { 5 }, 7);
00508         REQUIRE(x < y);
00509         REQUIRE(y > x);
00510     }
00511
00512     SECTION("esant vienodoms pavardems lyginama pagal varda") {
00513         studentas x("Antanas", "Petraitis", { 5 }, 7);
00514         studentas y("Zigmas", "Petraitis", { 5 }, 7);
00515         REQUIRE(x < y);
00516     }
00517
00518     SECTION("veikia su std::sort") {

```

```

00519         std::vector<studentas> v = { c, a };
00520         std::sort(v.begin(), v.end());
00521         REQUIRE(v[0].pavarde() == "Jonaitis");
00522     }
00523 }
00524
00525 TEST_CASE("Indeksavimo operatorius []", "[operatoriai]")
00526 {
00527     studentas s("Jonas", "Jonaitis", { 3, 6, 9 }, 10);
00528
00529     SECTION("grazina teisinga pazymi") {
00530         REQUIRE(s[0] == 3);
00531         REQUIRE(s[1] == 6);
00532         REQUIRE(s[2] == 9);
00533     }
00534
00535     SECTION("meta out_of_range uz ribu") {
00536         REQUIRE_THROWS_AS(s[3], std::out_of_range);
00537         REQUIRE_THROWS_AS(s[99], std::out_of_range);
00538     }
00539 }
00540
00541 // =====
00542 // 7. SKAIČIAVIMO LOGIKA
00543 // =====
00544
00545 TEST_CASE("calculateGalutinis", "[skaiciavimas]")
00546 {
00547     SECTION("nelyginis ND kiekis") {
00548         // ND: 2,4,6,8,10 → vidurkis 6.0, mediana 6.0
00549         studentas s("Testas", "Testaitis", { 2, 4, 6, 8, 10 }, 5);
00550         s.calculateGalutinis();
00551         REQUIRE(s.galutinisVid() == Approx(0.4 * 6.0 + 0.6 * 5));
00552         REQUIRE(s.galutinisMed() == Approx(0.4 * 6.0 + 0.6 * 5));
00553     }
00554
00555     SECTION("lyginis ND kiekis -- mediana yra dviejų vidurio reikšmių vidurkis") {
00556         // ND: 1,2,3,4 → vidurkis 2.5, mediana (2+3)/2 = 2.5
00557         studentas s("Testas", "Testaitis", { 1, 2, 3, 4 }, 10);
00558         s.calculateGalutinis();
00559         REQUIRE(s.galutinisMed() == Approx(0.4 * 2.5 + 0.6 * 10));
00560     }
00561
00562     SECTION("vidurkis ir mediana skiriasi esant issibarsčiusiems duomenims") {
00563         // ND: 1,1,1,1,10 → vidurkis 2.8, mediana 1.0
00564         studentas s("Testas", "Testaitis", { 1, 1, 1, 1, 10 }, 5);
00565         s.calculateGalutinis();
00566         REQUIRE(s.galutinisVid() == Approx(0.4 * 2.8 + 0.6 * 5));
00567         REQUIRE(s.galutinisMed() == Approx(0.4 * 1.0 + 0.6 * 5));
00568         REQUIRE(s.galutinisVid() != Approx(s.galutinisMed()));
00569     }
00570
00571     SECTION("tuscias ND sarasas") {
00572         studentas s;
00573         s.setVardas("Tuscias");
00574         s.setPavarde("Tusciaitis");
00575         s.setEgzaminas(8);
00576         s.calculateGalutinis();
00577         REQUIRE(s.galutinisVid() == Approx(0.6 * 8));
00578         REQUIRE(s.galutinisMed() == Approx(0.6 * 8));
00579     }
00580
00581     SECTION("nesurikiuoti ND -- mediana skaiciuojama teisingai") {
00582         studentas s("Testas", "Testaitis", { 9, 1, 5 }, 10);
00583         s.calculateGalutinis();
00584         // Surikiavus: 1,5,9 → mediana 5
00585         REQUIRE(s.galutinisMed() == Approx(0.4 * 5.0 + 0.6 * 10));
00586     }
00587 }
00588
00589 // =====
00590 // 8. SET'ERIAI
00591 // =====
00592
00593 TEST_CASE("Seteriai", "[seteriai]")
00594 {
00595     studentas s;
00596
00597     SECTION("paveldeti bazines klases seteriai") {
00598         s.setVardas("Naujas");
00599         s.setPavarde("Naujaitis");
00600         REQUIRE(s.vardas() == "Naujas");
00601         REQUIRE(s.pavarde() == "Naujaitis");
00602     }
00603
00604     SECTION("bazines klases seteriai validuoja") {
00605         REQUIRE_THROWS_AS(s.setVardas(""), std::invalid_argument);
00606     }
00607 }

```

```

00606         REQUIRE_THROWS_AS(s.setPavarde(""), std::invalid_argument);
00607     }
00608
00609     SECTION("isvestines klases seteriai") {
00610         s.setNd({ 5, 6, 7 });
00611         s.setEgzaminas(9);
00612         REQUIRE(s.nd().size() == 3);
00613         REQUIRE(s.egzaminas() == 9);
00614
00615         s.addPazymys(10);
00616         REQUIRE(s.nd().size() == 4);
00617         REQUIRE(s.nd().back() == 10);
00618     }
00619
00620     SECTION("isvestines klases seteriai validuoja") {
00621         REQUIRE_THROWS_AS(s.setEgzaminas(50), std::runtime_error);
00622         REQUIRE_THROWS_AS(s.setNd({ 5, 99 }), std::runtime_error);
00623         REQUIRE_THROWS_AS(s.addPazymys(0), std::runtime_error);
00624     }
00625
00626     SECTION("isvalyk istustina abieju klasiu laukus") {
00627         studentas t = kurkTestini();
00628         t.calculateGalutinis();
00629         t.isvalyk();
00630         REQUIRE(t.vardas().empty());
00631         REQUIRE(t.pavarde().empty());
00632         REQUIRE(t.nd().empty());
00633         REQUIRE(t.egzaminas() == 0);
00634         REQUIRE(t.galutinisVid() == Approx(0.0));
00635     }
00636 }
00637
00638 // =====
00639 // 9. VEIKIMAS KONTEINERIUOSE
00640 // =====
00641
00642 TEST_CASE("Veikimas su STL konteineriais", "[konteineriai]")
00643 {
00644     SECTION("vector perskirstymas islaiko duomenis") {
00645         std::vector<studentas> v;
00646         for (int i = 0; i < 4; ++i)
00647             v.push_back(studentas("Vardas" + std::to_string(i),
00648                                   "Pavarde" + std::to_string(i),
00649                                   { 5, 6, 7 }, 8));
00650         v.reserve(100); // priverstinis perskirstymas
00651         REQUIRE(v.size() == 4);
00652         REQUIRE(v[0].vardas() == "Vardas0");
00653         REQUIRE(v[3].vardas() == "Vardas3");
00654     }
00655
00656     SECTION("vektorius perkeltas su std::move") {
00657         std::vector<studentas> v;
00658         v.push_back(kurkTestini());
00659         v.push_back(kurkTestini());
00660
00661         std::vector<studentas> v2 = std::move(v);
00662         REQUIRE(v2.size() == 2);
00663         REQUIRE(v2[0].vardas() == "Jonas");
00664     }
00665
00666     SECTION("veikia su std::list ir std::deque") {
00667         std::list<studentas> l;
00668         std::deque<studentas> d;
00669         l.push_back(kurkTestini());
00670         d.push_back(kurkTestini());
00671         REQUIRE(l.front().vardas() == "Jonas");
00672         REQUIRE(d.front().vardas() == "Jonas");
00673     }
00674 }

```

8.56 tests/Vectortests.cpp Failo Nuoroda

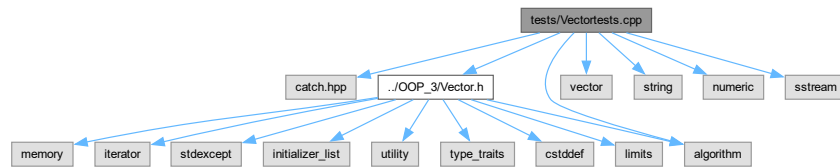
Unit testai nuosavai **Vector** klasei (Catch2).

```

#include "catch.hpp"
#include "../OOP_3/Vector.h"
#include <vector>
#include <string>
#include <numeric>
#include <algorithm>
#include <sstream>

```


Įtraukimo priklausomybių diagrama Vectortests.cpp:



Apibrėžimai

- #define `CATCH_CONFIG_MAIN`

Funkcijos

- `TEST_CASE` ("Konstruktoriai", "[konstruktoriai]")
- `TEST_CASE` ("Priskyrimas", "[priskyrimas]")
- `TEST_CASE` ("Elementu prieiga", "[prieiga]")
- `TEST_CASE` ("Iteratoriai", "[iteratoriai]")
- `TEST_CASE` ("Talpa", "[talpa]")
- `TEST_CASE` ("push_back ir pop_back", "[modifikavimas]")
- `TEST_CASE` ("emplace_back", "[modifikavimas]")
- `TEST_CASE` ("insert", "[modifikavimas]")
- `TEST_CASE` ("erase", "[modifikavimas]")
- `TEST_CASE` ("clear ir resize", "[modifikavimas]")
- `TEST_CASE` ("swap", "[modifikavimas]")
- `TEST_CASE` ("Lyginimo operatoriai", "[operatoriai]")

8.56.1 Smulkus aprašymas

Unit testai nuosavai `Vector` klasei (Catch2).

Kiekvienam testui taikomas tas pats principas: ta pati operacija atliekama su `std::vector` ir su `Vector`, o rezultatai lyginami. Taip tikrinama ne tik ar `Vector` veikia, bet ar veikia **taip pat** kaip standartinis konteineris.

Paleidimas: `./vectorTests` — visi testai `./vectorTests "[talpa]"` — tik talpos testai `./vectorTests -l` — testų sąrašas

Apibrėžimas faile `Vectortests.cpp`.

8.56.2 Apibrėžimų Dokumentacija

8.56.2.1 CATCH_CONFIG_MAIN

```
#define CATCH_CONFIG_MAIN
```

Apibrėžimas failo `Vectortests.cpp` eilutėje 16.

8.56.3 Funkcijos Dokumentacija

8.56.3.1 TEST_CASE() [1/12]

```
TEST_CASE (
    "Konstruktoriai" ,
    "" [konstruktoriai])
```

Apibrėžimas failo `Vectortests.cpp` eilutėje 73.

Ryšiai `Vector< T, Allocator >::capacity()`, `Vector< T, Allocator >::data()`, `Vector< T, Allocator >::empty()`, ir `Vector< T, Allocator >::size()`.

8.56.3.2 TEST_CASE() [2/12]

```
TEST_CASE (
    "Priskyrimas" ,
    "" [priskyrimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 157.

Ryšiai [Vector< T, Allocator >::assign\(\)](#), [Vector< T, Allocator >::data\(\)](#), ir [Vector< T, Allocator >::size\(\)](#).

8.56.3.3 TEST_CASE() [3/12]

```
TEST_CASE (
    "Elementu prieiga" ,
    "" [prieiga])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 209.

Ryšiai [Vector< T, Allocator >::at\(\)](#), [Vector< T, Allocator >::back\(\)](#), [Vector< T, Allocator >::data\(\)](#), ir [Vector< T, Allocator >::front\(\)](#).

8.56.3.4 TEST_CASE() [4/12]

```
TEST_CASE (
    "Iteratoriai" ,
    "" [iteratoriai])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 259.

Ryšiai [Vector< T, Allocator >::begin\(\)](#), [Vector< T, Allocator >::cbegin\(\)](#), [Vector< T, Allocator >::cend\(\)](#), [Vector< T, Allocator >::crbegin\(\)](#), [Vector< T, Allocator >::end\(\)](#), [Vector< T, Allocator >::rbegin\(\)](#), ir [Vector< T, Allocator >::rend\(\)](#).

8.56.3.5 TEST_CASE() [5/12]

```
TEST_CASE (
    "Talpa" ,
    "" [talpa])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 306.

Ryšiai [Vector< T, Allocator >::capacity\(\)](#), [Vector< T, Allocator >::data\(\)](#), [Vector< T, Allocator >::empty\(\)](#), [Vector< T, Allocator >::max_size\(\)](#), [Vector< T, Allocator >::push_back\(\)](#), [Vector< T, Allocator >::reserve\(\)](#), [Vector< T, Allocator >::shrink_to_fit\(\)](#), ir [Vector< T, Allocator >::size\(\)](#).

8.56.3.6 TEST_CASE() [6/12]

```
TEST_CASE (
    "push_back ir pop_back" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 403.

Ryšiai [Vector< T, Allocator >::back\(\)](#), [Vector< T, Allocator >::pop_back\(\)](#), [Vector< T, Allocator >::push_back\(\)](#), ir [Vector< T, Allocator >::size\(\)](#).

8.56.3.7 TEST_CASE() [7/12]

```
TEST_CASE (
    "emplace_back" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 438.

Ryšiai [Vector< T, Allocator >::emplace_back\(\)](#), ir [Vector< T, Allocator >::reserve\(\)](#).

8.56.3.8 TEST_CASE() [8/12]

```
TEST_CASE (
    "insert" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 463.

Ryšiai [Vector< T, Allocator >::back\(\)](#), [Vector< T, Allocator >::begin\(\)](#), [Vector< T, Allocator >::end\(\)](#), [Vector< T, Allocator >::insert\(\)](#), ir [Vector< T, Allocator >::size\(\)](#).

8.56.3.9 TEST_CASE() [9/12]

```
TEST_CASE (
    "erase" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 521.

Ryšiai [Vector< T, Allocator >::begin\(\)](#), [Vector< T, Allocator >::emplace_back\(\)](#), ir [Vector< T, Allocator >::erase\(\)](#).

8.56.3.10 TEST_CASE() [10/12]

```
TEST_CASE (
    "clear ir resize" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 555.

Ryšiai [Vector< T, Allocator >::back\(\)](#), [Vector< T, Allocator >::capacity\(\)](#), [Vector< T, Allocator >::clear\(\)](#), [Vector< T, Allocator >::empty\(\)](#), [Vector< T, Allocator >::empty\(\)](#), [Vector< T, Allocator >::resize\(\)](#), ir [Vector< T, Allocator >::size\(\)](#).

8.56.3.11 TEST_CASE() [11/12]

```
TEST_CASE (
    "swap" ,
    "" [modifikavimas])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 600.

Ryšiai [Vector< T, Allocator >::data\(\)](#), [Vector< T, Allocator >::size\(\)](#), [swap\(\)](#), ir [Vector< T, Allocator >::swap\(\)](#).

8.56.3.12 TEST_CASE() [12/12]

```
TEST_CASE (
    "Lyginimo operatoriai" ,
    "" [operatoriai])
```

Apibrėžimas failo [Vectortests.cpp](#) eilutėje 629.

8.57 Vectortests.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file vectorTests.cpp
00003  * @brief Unit testai nuosavai Vector klasei (Catch2).
00004  *
00005  * Kiekvienam testui taikomas tas pats principas: ta pati operacija
00006  * atliekama su `std::vector` ir su `Vector`, o rezultatai lyginami.
00007  * Taip tikrinama ne tik ar `Vector` veikia, bet ar veikia **taip pat**
00008  * kaip standartinis konteineris.
00009  *
00010  * Paleidimas:
00011  *     ./vectorTests                -- visi testai
00012  *     ./vectorTests "[talpa]"      -- tik talpos testai
00013  *     ./vectorTests -l             -- testų sąrašas
00014  */
00015
00016 #define CATCH_CONFIG_MAIN
00017 #include "catch.hpp"
00018 #include "../OOP_3/Vector.h"
00019
00020 #include <vector>
00021 #include <string>
00022 #include <numeric>
00023 #include <algorithm>
00024 #include <sstream>
00025
00026 // =====
00027 // Pagalbinė klasė objektų gyvavimui sekti
00028 // =====
00029
00030 namespace {
00031
00032     /**
00033      * @brief Testinė klasė, skaičiuojanti konstruktorių ir destruktorių kvietimus.
00034      *
00035      * Leidžia patikrinti, ar Vector teisingai konstruoja ir naikina
```

```

00036     * elementus -- ypač svarbu persikirstymo metu.
00037     */
00038     struct Sekiklis {
00039         static int gyvu;
00040         static int kopiju;
00041         static int perkelimu;
00042         int reiksme;
00043
00044         Sekiklis(int v = 0) : reiksme(v) { ++gyvu; }
00045         Sekiklis(const Sekiklis& o) : reiksme(o.reiksme) { ++gyvu; ++kopiju; }
00046         Sekiklis(Sekiklis&& o) noexcept : reiksme(o.reiksme) { ++gyvu; ++perkelimu; }
00047         Sekiklis& operator=(const Sekiklis& o) { reiksme = o.reiksme; ++kopiju; return *this; }
00048         Sekiklis& operator=(Sekiklis&& o) noexcept { reiksme = o.reiksme; ++perkelimu; return *this; }
00049         ~Sekiklis() { --gyvu; }
00050
00051         bool operator==(const Sekiklis& o) const { return reiksme == o.reiksme; }
00052
00053         static void atstatyk() { gyvu = 0; kopiju = 0; perkelimu = 0; }
00054     };
00055
00056     int Sekiklis::gyvu = 0;
00057     int Sekiklis::kopiju = 0;
00058     int Sekiklis::perkelimu = 0;
00059
00060     /// Palygina Vector ir std::vector turinį
00061     template <typename T>
00062     bool sutampa(const Vector<T>& v, const std::vector<T>& sv) {
00063         return v.size() == sv.size()
00064             && std::equal(v.begin(), v.end(), sv.begin());
00065     }
00066
00067 } // anoniminis namespace
00068
00069 // =====
00070 // 1. KONSTRUKTORIAI
00071 // =====
00072
00073 TEST_CASE("Konstruktoriai", "[konstruktoriai]")
00074 {
00075     SECTION("numatygtasis -- tuscias vektorius") {
00076         Vector<int> v;
00077         std::vector<int> sv;
00078         REQUIRE(v.size() == sv.size());
00079         REQUIRE(v.capacity() == sv.capacity());
00080         REQUIRE(v.empty() == sv.empty());
00081         REQUIRE(v.data() == nullptr);
00082     }
00083
00084     SECTION("n elementu su reiksme") {
00085         Vector<int> v(5, 42);
00086         std::vector<int> sv(5, 42);
00087         REQUIRE(sutampa(v, sv));
00088         REQUIRE(v.size() == 5);
00089         REQUIRE(v[0] == 42);
00090         REQUIRE(v[4] == 42);
00091     }
00092
00093     SECTION("n numatygtai sukonstruotu elementu") {
00094         Vector<int> v(5);
00095         std::vector<int> sv(5);
00096         REQUIRE(sutampa(v, sv));
00097         REQUIRE(v[0] == 0); // int() = 0
00098     }
00099
00100     SECTION("is iteratoriu intervalo") {
00101         std::vector<int> saltinis = { 1, 2, 3, 4, 5 };
00102         Vector<int> v(saltinis.begin(), saltinis.end());
00103         REQUIRE(sutampa(v, saltinis));
00104     }
00105
00106     SECTION("is initializer_list") {
00107         Vector<int> v = { 1, 2, 3 };
00108         std::vector<int> sv = { 1, 2, 3 };
00109         REQUIRE(sutampa(v, sv));
00110     }
00111
00112     SECTION("Vector(5, 10) NEinterpretuojamas kaip intervalas") {
00113         // Be enable_if sios eilutes butu klaidingai parinktas
00114         // intervalo konstruktorius
00115         Vector<int> v(5, 10);
00116         REQUIRE(v.size() == 5);
00117         REQUIRE(v[0] == 10);
00118     }
00119
00120     SECTION("kopijavimo konstruktorius -- gili kopija") {
00121         Vector<int> originalas = { 1, 2, 3 };
00122         Vector<int> kopija(originalas);

```

```

00123
00124     REQUIRE(kopija == originalas);
00125     REQUIRE(kopija.data() != originalas.data()); // skirtinga atmintis
00126
00127     kopija[0] = 99;
00128     REQUIRE(originalas[0] == 1); // originalas nepakito
00129 }
00130
00131 SECTION("perkelimo konstruktorius") {
00132     Vector<int> saltinis = { 1, 2, 3 };
00133     const int* senasAdresas = saltinis.data();
00134
00135     Vector<int> tikslas(std::move(saltinis));
00136
00137     REQUIRE(tikslas.size() == 3);
00138     REQUIRE(tikslas.data() == senasAdresas); // ta pati atmintis
00139     REQUIRE(saltinis.size() == 0); // saltinis tuscias
00140     REQUIRE(saltinis.data() == nullptr);
00141 }
00142
00143 SECTION("konstruktoriai teisingai naikina objektus") {
00144     Sekiklis::atstatyk();
00145     {
00146         Vector<Sekiklis> v(10);
00147         REQUIRE(Sekiklis::gyvu == 10);
00148     }
00149     REQUIRE(Sekiklis::gyvu == 0);
00150 }
00151 }
00152
00153 // =====
00154 // 2. PRISKYRIMAS
00155 // =====
00156
00157 TEST_CASE("Priskyrimas", "[priskyrimas]")
00158 {
00159     SECTION("kopijavimo priskyrimas") {
00160         Vector<int> a = { 1, 2, 3 };
00161         Vector<int> b;
00162         b = a;
00163         REQUIRE(b == a);
00164         REQUIRE(b.data() != a.data());
00165     }
00166
00167     SECTION("priskyrimas sau nesugadina") {
00168         Vector<int> a = { 1, 2, 3 };
00169         a = a;
00170         REQUIRE(a.size() == 3);
00171         REQUIRE(a[0] == 1);
00172     }
00173
00174     SECTION("perkelimo priskyrimas") {
00175         Vector<int> a = { 1, 2, 3 };
00176         Vector<int> b = { 9 };
00177         b = std::move(a);
00178         REQUIRE(b.size() == 3);
00179         REQUIRE(a.size() == 0);
00180     }
00181
00182     SECTION("priskyrimas is initializer_list") {
00183         Vector<int> v;
00184         v = { 4, 5, 6 };
00185         REQUIRE(v.size() == 3);
00186         REQUIRE(v[1] == 5);
00187     }
00188
00189     SECTION("assign su n kopiju") {
00190         Vector<int> v = { 1, 2, 3 };
00191         std::vector<int> sv = { 1, 2, 3 };
00192         v.assign(5, 7);
00193         sv.assign(5, 7);
00194         REQUIRE(sutampa(v, sv));
00195     }
00196
00197     SECTION("assign su intervalu") {
00198         std::vector<int> saltinis = { 8, 9 };
00199         Vector<int> v = { 1, 2, 3 };
00200         v.assign(saltinis.begin(), saltinis.end());
00201         REQUIRE(sutampa(v, saltinis));
00202     }
00203 }
00204
00205 // =====
00206 // 3. ELEMENTŲ PRIEIGA
00207 // =====
00208
00209 TEST_CASE("Elementu prieiga", "[prieiga]")

```

```

00210 {
00211     Vector<int> v = { 10, 20, 30, 40 };
00212     std::vector<int> sv = { 10, 20, 30, 40 };
00213
00214     SECTION("operator[]") {
00215         REQUIRE(v[0] == sv[0]);
00216         REQUIRE(v[3] == sv[3]);
00217         v[1] = 99;
00218         REQUIRE(v[1] == 99);
00219     }
00220
00221     SECTION("at() su ribu tikrinimu") {
00222         REQUIRE(v.at(2) == sv.at(2));
00223         REQUIRE_THROWS_AS(v.at(4), std::out_of_range);
00224         REQUIRE_THROWS_AS(v.at(100), std::out_of_range);
00225
00226         // std::vector meta ta pacia isimti
00227         REQUIRE_THROWS_AS(sv.at(4), std::out_of_range);
00228     }
00229
00230     SECTION("front ir back") {
00231         REQUIRE(v.front() == sv.front());
00232         REQUIRE(v.back() == sv.back());
00233         v.front() = 1;
00234         v.back() = 4;
00235         REQUIRE(v[0] == 1);
00236         REQUIRE(v[3] == 4);
00237     }
00238
00239     SECTION("data() grazina rodykle i masyva") {
00240         int* p = v.data();
00241         REQUIRE(p[0] == 10);
00242         REQUIRE(p[3] == 40);
00243     }
00244
00245     SECTION("const prieiga") {
00246         const Vector<int>& cv = v;
00247         REQUIRE(cv[0] == 10);
00248         REQUIRE(cv.at(1) == 20);
00249         REQUIRE(cv.front() == 10);
00250         REQUIRE(cv.back() == 40);
00251         REQUIRE(cv.data() != nullptr);
00252     }
00253 }
00254
00255 // =====
00256 // 4. ITERATORIAI
00257 // =====
00258
00259 TEST_CASE("Iteratoriai", "[iteratoriai]")
00260 {
00261     Vector<int> v = { 1, 2, 3, 4, 5 };
00262     std::vector<int> sv = { 1, 2, 3, 4, 5 };
00263
00264     SECTION("begin ir end") {
00265         REQUIRE(*v.begin() == 1);
00266         REQUIRE(*(v.end() - 1) == 5);
00267         REQUIRE(v.end() - v.begin() == 5);
00268     }
00269
00270     SECTION("range-based for") {
00271         int suma = 0;
00272         for (int x : v) suma += x;
00273         REQUIRE(suma == 15);
00274     }
00275
00276     SECTION("atvirkstiniai iteratoriai") {
00277         std::vector<int> atvirksciai;
00278         for (auto it = v.rbegin(); it != v.rend(); ++it)
00279             atvirksciai.push_back(*it);
00280         REQUIRE(atvirksciai == std::vector<int>{ 5, 4, 3, 2, 1 });
00281     }
00282
00283     SECTION("const iteratoriai") {
00284         const Vector<int>& cv = v;
00285         REQUIRE(*cv.cbegin() == 1);
00286         REQUIRE(*(cv.cend() - 1) == 5);
00287         REQUIRE(*cv.crbegin() == 5);
00288     }
00289
00290     SECTION("veikia su STL algoritmais") {
00291         REQUIRE(std::accumulate(v.begin(), v.end(), 0) == 15);
00292         REQUIRE(*std::max_element(v.begin(), v.end()) == 5);
00293         REQUIRE(std::count(v.begin(), v.end(), 3) == 1);
00294
00295         std::sort(v.begin(), v.end(), std::greater<int>());
00296         REQUIRE(v[0] == 5);

```

```

00297
00298     REQUIRE(std::find(v.begin(), v.end(), 3) != v.end());
00299 }
00300 }
00301
00302 // =====
00303 // 5. TALPA -- svarbiausias skyrius
00304 // =====
00305
00306 TEST_CASE("Talpa", "[talpa]")
00307 {
00308     SECTION("empty ir size") {
00309         Vector<int> v;
00310         REQUIRE(v.empty());
00311         REQUIRE(v.size() == 0);
00312
00313         v.push_back(1);
00314         REQUIRE_FALSE(v.empty());
00315         REQUIRE(v.size() == 1);
00316     }
00317
00318     SECTION("capacity dvigubinama augant") {
00319         Vector<int> v;
00320         REQUIRE(v.capacity() == 0);
00321
00322         v.push_back(1);
00323         REQUIRE(v.capacity() == 1);
00324
00325         v.push_back(2);
00326         REQUIRE(v.capacity() == 2);
00327
00328         v.push_back(3);
00329         REQUIRE(v.capacity() == 4);
00330
00331         v.push_back(4);
00332         REQUIRE(v.capacity() == 4);
00333
00334         v.push_back(5);
00335         REQUIRE(v.capacity() == 8);
00336     }
00337
00338     SECTION("reserve paskiria atminti, bet nekuria elementu") {
00339         Vector<int> v;
00340         v.reserve(100);
00341         REQUIRE(v.capacity() >= 100);
00342         REQUIRE(v.size() == 0);
00343         REQUIRE(v.empty());
00344     }
00345
00346     SECTION("reserve nemazina talpos") {
00347         Vector<int> v;
00348         v.reserve(100);
00349         v.reserve(10);
00350         REQUIRE(v.capacity() >= 100);
00351     }
00352
00353     SECTION("reserve panaikina perskirstymus") {
00354         Vector<int> v;
00355         v.reserve(1000);
00356         const int* pries = v.data();
00357
00358         for (int i = 0; i < 1000; ++i)
00359             v.push_back(i);
00360
00361         REQUIRE(v.data() == pries); // atmintis nepersikele
00362     }
00363
00364     SECTION("shrink_to_fit atlaisvina nepanaudota atminti") {
00365         Vector<int> v;
00366         v.reserve(100);
00367         v.push_back(1);
00368         v.push_back(2);
00369         REQUIRE(v.capacity() >= 100);
00370
00371         v.shrink_to_fit();
00372         REQUIRE(v.capacity() == 2);
00373         REQUIRE(v.size() == 2);
00374         REQUIRE(v[0] == 1);
00375     }
00376
00377     SECTION("max_size grazina teigiama reiksme") {
00378         Vector<int> v;
00379         REQUIRE(v.max_size() > 0);
00380     }
00381
00382     SECTION("perskirstymu skaicius atitinka log2(n)") {
00383         Vector<int> v;

```

```

00384         int perskirstymu = 0;
00385         size_t senaTalpa = v.capacity();
00386
00387         for (int i = 0; i < 1000; ++i) {
00388             v.push_back(i);
00389             if (v.capacity() != senaTalpa) {
00390                 ++perskirstymu;
00391                 senaTalpa = v.capacity();
00392             }
00393         }
00394         // 1000 elementu + talpa 1,2,4,...,1024 + 11 perskirstymu
00395         REQUIRE(perskirstymu == 11);
00396     }
00397 }
00398
00399 // =====
00400 // 6. MODIFIKAVIMAS
00401 // =====
00402
00403 TEST_CASE("push_back ir pop_back", "[modifikavimas]")
00404 {
00405     SECTION("push_back prideda i gala") {
00406         Vector<int> v;
00407         std::vector<int> sv;
00408         for (int i = 1; i <= 10; ++i) { v.push_back(i); sv.push_back(i); }
00409         REQUIRE(sutampa(v, sv));
00410     }
00411
00412     SECTION("push_back su perkelimu") {
00413         Vector<std::string> v;
00414         std::string s = "labas";
00415         v.push_back(std::move(s));
00416         REQUIRE(v[0] == "labas");
00417         REQUIRE(s.empty()); // perkelta
00418     }
00419
00420     SECTION("pop_back pasalina paskutini") {
00421         Vector<int> v = { 1, 2, 3 };
00422         v.pop_back();
00423         REQUIRE(v.size() == 2);
00424         REQUIRE(v.back() == 2);
00425     }
00426
00427     SECTION("pop_back sunaikina objekta") {
00428         Sekiklis::atstatyk();
00429         Vector<Sekiklis> v;
00430         v.push_back(Sekiklis(1));
00431         v.push_back(Sekiklis(2));
00432         const int pries = Sekiklis::gyvu;
00433         v.pop_back();
00434         REQUIRE(Sekiklis::gyvu == pries - 1);
00435     }
00436 }
00437
00438 TEST_CASE("emplace_back", "[modifikavimas]")
00439 {
00440     SECTION("sukuria objekta vietoje") {
00441         Vector<std::string> v;
00442         v.emplace_back(5, 'a'); // std::string(5, 'a')
00443         REQUIRE(v[0] == "aaaaa");
00444     }
00445
00446     SECTION("grazina nuoroda i sukurta elementa") {
00447         Vector<int> v;
00448         int& r = v.emplace_back(42);
00449         REQUIRE(r == 42);
00450         r = 99;
00451         REQUIRE(v[0] == 99);
00452     }
00453
00454     SECTION("emplace_back nekopijuoja") {
00455         Sekiklis::atstatyk();
00456         Vector<Sekiklis> v;
00457         v.reserve(10);
00458         v.emplace_back(5);
00459         REQUIRE(Sekiklis::kopiju == 0); // joku kopiju
00460     }
00461 }
00462
00463 TEST_CASE("insert", "[modifikavimas]")
00464 {
00465     SECTION("insert i pradzia") {
00466         Vector<int> v = { 2, 3 };
00467         std::vector<int> sv = { 2, 3 };
00468         v.insert(v.begin(), 1);
00469         sv.insert(sv.begin(), 1);
00470         REQUIRE(sutampa(v, sv));
00471     }

```



```

00471     }
00472
00473     SECTION("insert i vidury") {
00474         Vector<int> v = { 1, 3 };
00475         std::vector<int> sv = { 1, 3 };
00476         v.insert(v.begin() + 1, 2);
00477         sv.insert(sv.begin() + 1, 2);
00478         REQUIRE(sutampa(v, sv));
00479     }
00480
00481     SECTION("insert i gala") {
00482         Vector<int> v = { 1, 2 };
00483         v.insert(v.end(), 3);
00484         REQUIRE(v.size() == 3);
00485         REQUIRE(v.back() == 3);
00486     }
00487
00488     SECTION("insert n kopiju") {
00489         Vector<int> v = { 1, 5 };
00490         std::vector<int> sv = { 1, 5 };
00491         v.insert(v.begin() + 1, 3, 9);
00492         sv.insert(sv.begin() + 1, 3, 9);
00493         REQUIRE(sutampa(v, sv));
00494     }
00495
00496     SECTION("insert intervala") {
00497         std::vector<int> saltinis = { 2, 3, 4 };
00498         Vector<int> v = { 1, 5 };
00499         std::vector<int> sv = { 1, 5 };
00500         v.insert(v.begin() + 1, saltinis.begin(), saltinis.end());
00501         sv.insert(sv.begin() + 1, saltinis.begin(), saltinis.end());
00502         REQUIRE(sutampa(v, sv));
00503     }
00504
00505     SECTION("insert initializer_list") {
00506         Vector<int> v = { 1, 4 };
00507         v.insert(v.begin() + 1, { 2, 3 });
00508         REQUIRE(v.size() == 4);
00509         REQUIRE(v[1] == 2);
00510         REQUIRE(v[2] == 3);
00511     }
00512
00513     SECTION("insert grazina iteratoriu i iterpta elementa") {
00514         Vector<int> v = { 1, 3 };
00515         auto it = v.insert(v.begin() + 1, 2);
00516         REQUIRE(*it == 2);
00517         REQUIRE(it == v.begin() + 1);
00518     }
00519 }
00520
00521 TEST_CASE("erase", "[modifikavimas]")
00522 {
00523     SECTION("erase viena elementa") {
00524         Vector<int> v = { 1, 2, 3, 4 };
00525         std::vector<int> sv = { 1, 2, 3, 4 };
00526         v.erase(v.begin() + 1);
00527         sv.erase(sv.begin() + 1);
00528         REQUIRE(sutampa(v, sv));
00529     }
00530
00531     SECTION("erase intervala") {
00532         Vector<int> v = { 1, 2, 3, 4, 5 };
00533         std::vector<int> sv = { 1, 2, 3, 4, 5 };
00534         v.erase(v.begin() + 1, v.begin() + 4);
00535         sv.erase(sv.begin() + 1, sv.begin() + 4);
00536         REQUIRE(sutampa(v, sv));
00537     }
00538
00539     SECTION("erase grazina iteratoriu i kita elementa") {
00540         Vector<int> v = { 1, 2, 3 };
00541         auto it = v.erase(v.begin());
00542         REQUIRE(*it == 2);
00543     }
00544
00545     SECTION("erase sunaikina objektus") {
00546         Sekiklis::atstatyk();
00547         Vector<Sekiklis> v;
00548         for (int i = 0; i < 5; ++i) v.emplace_back(i);
00549         const int pries = Sekiklis::gyvu;
00550         v.erase(v.begin(), v.begin() + 2);
00551         REQUIRE(Sekiklis::gyvu == pries - 2);
00552     }
00553 }
00554
00555 TEST_CASE("clear ir resize", "[modifikavimas]")
00556 {
00557     SECTION("clear pasalina elementus, bet ne talpa") {

```

```

00558     Vector<int> v = { 1, 2, 3 };
00559     const size_t talpa = v.capacity();
00560     v.clear();
00561     REQUIRE(v.size() == 0);
00562     REQUIRE(v.empty());
00563     REQUIRE(v.capacity() == talpa);    // talpa nepakito
00564 }
00565
00566 SECTION("clear sunaikina objektus") {
00567     Sekiklis::atstatyk();
00568     Vector<Sekiklis> v;
00569     for (int i = 0; i < 5; ++i) v.emplace_back(i);
00570     v.clear();
00571     REQUIRE(Sekiklis::gyvu == 0);
00572 }
00573
00574 SECTION("resize didinant") {
00575     Vector<int> v = { 1, 2 };
00576     std::vector<int> sv = { 1, 2 };
00577     v.resize(5);
00578     sv.resize(5);
00579     REQUIRE(sutampa(v, sv));
00580     REQUIRE(v[4] == 0);
00581 }
00582
00583 SECTION("resize didinant su reiksme") {
00584     Vector<int> v = { 1, 2 };
00585     std::vector<int> sv = { 1, 2 };
00586     v.resize(5, 9);
00587     sv.resize(5, 9);
00588     REQUIRE(sutampa(v, sv));
00589     REQUIRE(v[4] == 9);
00590 }
00591
00592 SECTION("resize mazinant") {
00593     Vector<int> v = { 1, 2, 3, 4, 5 };
00594     v.resize(2);
00595     REQUIRE(v.size() == 2);
00596     REQUIRE(v.back() == 2);
00597 }
00598 }
00599
00600 TEST_CASE("swap", "[modifikavimas]")
00601 {
00602     SECTION("nario swap") {
00603         Vector<int> a = { 1, 2 };
00604         Vector<int> b = { 3, 4, 5 };
00605         const int* aData = a.data();
00606         const int* bData = b.data();
00607
00608         a.swap(b);
00609
00610         REQUIRE(a.size() == 3);
00611         REQUIRE(b.size() == 2);
00612         REQUIRE(a.data() == bData);    // sukeistos rodykles, ne elementai
00613         REQUIRE(b.data() == aData);
00614     }
00615
00616     SECTION("ne-nare swap funkcija") {
00617         Vector<int> a = { 1 };
00618         Vector<int> b = { 2 };
00619         swap(a, b);
00620         REQUIRE(a[0] == 2);
00621         REQUIRE(b[0] == 1);
00622     }
00623 }
00624
00625 // =====
00626 // 7. NE-NARIAI OPERATORIAI
00627 // =====
00628
00629 TEST_CASE("Lyginimo operatoriai", "[operatoriai]")
00630 {
00631     Vector<int> a = { 1, 2, 3 };
00632     Vector<int> b = { 1, 2, 3 };
00633     Vector<int> c = { 1, 2, 4 };
00634     Vector<int> trumpas = { 1, 2 };
00635
00636     SECTION("== ir !=") {
00637         REQUIRE(a == b);
00638         REQUIRE(a != c);
00639         REQUIRE(a != trumpas);
00640     }
00641
00642     SECTION("leksikografinis palyginimas") {
00643         REQUIRE(a < c);
00644         REQUIRE(c > a);
00645     }
00646 }

```

```

00645         REQUIRE(trumpas < a);
00646         REQUIRE(a <= b);
00647         REQUIRE(a >= b);
00648     }
00649
00650     SECTION("rezultatai sutampa su std::vector") {
00651         std::vector<int> sa = { 1, 2, 3 };
00652         std::vector<int> sc = { 1, 2, 4 };
00653         REQUIRE((a < c) == (sa < sc));
00654         REQUIRE((a == b) == (sa == std::vector<int>{ 1, 2, 3 }));
00655     }
00656 }
00657
00658 TEST_CASE("erase ir erase_if (ne-nares)", "[operatoriai]")
00659 {
00660     SECTION("erase pasalina visas reikšmes kopijas") {
00661         Vector<int> v = { 1, 2, 3, 4, 5, 2 };
00662         const auto n = erase(v, 2);
00663         REQUIRE(n == 3);
00664         REQUIRE(v.size() == 3);
00665         REQUIRE(v == Vector<int>{ 1, 3, 5 });
00666     }
00667
00668     SECTION("erase_if pagal predikata") {
00669         Vector<int> v = { 1, 2, 3, 4, 5, 6 };
00670         const auto n = erase_if(v, [](int x) { return x % 2 == 0; });
00671         REQUIRE(n == 3);
00672         REQUIRE(v == Vector<int>{ 1, 3, 5 });
00673     }
00674 }
00675
00676 // =====
00677 // 8. SUDĖTINGI TIPAI
00678 // =====
00679
00680 TEST_CASE("Veikimas su sudetingais tipais", "[tipai]")
00681 {
00682     SECTION("Vector<std::string>") {
00683         Vector<std::string> v;
00684         v.push_back("pirmas");
00685         v.push_back("antras");
00686         v.emplace_back("trečias");
00687
00688         REQUIRE(v.size() == 3);
00689         REQUIRE(v[0] == "pirmas");
00690         REQUIRE(v.back() == "trečias");
00691     }
00692
00693     SECTION("Vector<Vector<int> -- vektorius vektoriuje") {
00694         Vector<Vector<int>> matrica;
00695         matrica.push_back(Vector<int>{ 1, 2 });
00696         matrica.push_back(Vector<int>{ 3, 4 });
00697
00698         REQUIRE(matrica.size() == 2);
00699         REQUIRE(matrica[0][1] == 2);
00700         REQUIRE(matrica[1][0] == 3);
00701     }
00702
00703     SECTION("perskirstymas nesugadina objektu") {
00704         Sekiklis::atstatyk();
00705         {
00706             Vector<Sekiklis> v;
00707             for (int i = 0; i < 100; ++i)
00708                 v.emplace_back(i);
00709
00710             REQUIRE(Sekiklis::gyvu == 100);
00711             REQUIRE(v[0].reiksme == 0);
00712             REQUIRE(v[99].reiksme == 99);
00713         }
00714         REQUIRE(Sekiklis::gyvu == 0); // nera nutekejimo
00715     }
00716
00717     SECTION("perskirstymas naudoja move, ne copy") {
00718         Sekiklis::atstatyk();
00719         Vector<Sekiklis> v;
00720         v.reserve(2);
00721         v.emplace_back(1);
00722         v.emplace_back(2);
00723
00724         const int kopijuPries = Sekiklis::kopiju;
00725         v.emplace_back(3); // sukelia perskirstyma
00726
00727         // Sekiklis turi noexcept move, todėl turi būti naudojamas
00728         REQUIRE(Sekiklis::kopiju == kopijuPries);
00729         REQUIRE(Sekiklis::perkelimu > 0);
00730     }
00731 }

```

```
00732
00733 // =====
00734 // 9. KRAŠTINIAI ATVEJAI
00735 // =====
00736
00737 TEST_CASE("Krastiniai atvejai", "[krastiniai]")
00738 {
00739     SECTION("operacijos su tusciu vektoriumi") {
00740         Vector<int> v;
00741         REQUIRE(v.begin() == v.end());
00742         REQUIRE(v.size() == 0);
00743         REQUIRE_NOTHROW(v.clear());
00744         REQUIRE_NOTHROW(v.shrink_to_fit());
00745         REQUIRE_THROWS_AS(v.at(0), std::out_of_range);
00746     }
00747
00748     SECTION("insert i tuscia vektoriu") {
00749         Vector<int> v;
00750         v.insert(v.begin(), 1);
00751         REQUIRE(v.size() == 1);
00752         REQUIRE(v[0] == 1);
00753     }
00754
00755     SECTION("erase visus elementus") {
00756         Vector<int> v = { 1, 2, 3 };
00757         v.erase(v.begin(), v.end());
00758         REQUIRE(v.empty());
00759     }
00760
00761     SECTION("insert nulis elementu") {
00762         Vector<int> v = { 1, 2 };
00763         v.insert(v.begin(), 0, 9);
00764         REQUIRE(v.size() == 2);
00765     }
00766
00767     SECTION("resize i ta pati dydi") {
00768         Vector<int> v = { 1, 2, 3 };
00769         v.resize(3);
00770         REQUIRE(v.size() == 3);
00771         REQUIRE(v[2] == 3);
00772     }
00773
00774     SECTION("didelis reserve") {
00775         Vector<int> v;
00776         REQUIRE_NOTHROW(v.reserve(100000));
00777         REQUIRE(v.capacity() >= 100000);
00778     }
00779 }
```